



Dissertation on

**“Tinker RL Lab:
Post-Training of Large Language Models”**

A multi-framework training harness and studies of GRPO signal starvation

Submitted in partial fulfilment of the requirements for the award of the degree of

Master of Technology

in

Data Science and Machine Learning

UE20CS972 – Project Phase – 2

Submitted by:

Arvind C R PES2PGE24DS140

Under the guidance of

Ramesh Prakash Guledgudd

May 2026 – October 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled “**Tinker RL Lab: Post-Training of Large Language Models — A multi-framework training harness and studies of GRPO signal starvation**” is a bonafide record of the project work carried out by **Arvind C R** (SRN: **PES2PGE24DS140**), in partial fulfilment of the requirements for the award of the degree of Master of Technology in Data Science and Machine Learning of PES University, during the period May 2026 – October 2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report.

The candidate’s individual contribution, the inherited Semester-3 group infrastructure, and the boundary between them are stated in the Declaration and in Chapter 1 §1.6.

.....	
Ramesh Prakash Guledgudd	Head of the Department
Project Guide	Department of CSE
.....	
External Examiner	

October 2026

Declaration

I declare that this project report is my own work. Where the work of others has been used, it is cited and acknowledged. The matter embodied in this report has not been submitted, in part or in full, to this or any other university or institution for the award of any other degree or diploma.

The multi-framework benchmark infrastructure, the baseline framework integrations, and the initial GRPO/PPO/DPO experimentation were produced by the Semester-3 project group, of which I was a member, and are inherited here as the shared foundation. The research direction, the experimental expansion, the statistical analyses, the P1–P8 internal reports, the reporting and registry artefacts, the adaptive group-size controller, the applied fraud study, and the E1–E14 evaluation campaign reported in this document are my individual contribution, carried out under the guidance of my project guide.

Every quantitative result in this report is bound to an evidence file in the project repository. Where a result is incomplete, blocked, or negative, it is reported as such rather than omitted. The boundary between inherited and individual work is stated precisely in `platform_hybrid/sem 4 work/PROVENANCE.md`.



.....
Arvind C R

SRN: PES2PGE24DS140

Declaration on the Use of AI Tools

The following AI tools were used in this work, as recorded in the project repository.

- **Coding and writing assistants.** Anthropic Claude models and OpenAI Codex were used to write and revise code, analysis scripts and documents, including drafts of this report. Commits they contributed carry a Co-Authored-By trailer in the git history.
- **Frontier-model synthesis.** ChatGPT Pro Extended and Gemini Deep Think produced proposed derivations and reasoning recorded in `platform_hybrid/paper/sections/frontier_synthesis_*.tex`, which those documents mark as not being experiments. Where this report uses such a result, it states its own derivation (Chapter 6 §6.3.3).
- **Independent verification.** An LLM verification pass (Kimi, 6 July 2026) reviewed two single-seed results and flagged both as `SUSPECT`; both were then re-tested and reported as nulls (Chapter 6 §6.4.3).
- **Evaluation triage.** The Jev (TypeSafe) judgment harness was used in the E1–E14 ledger only for classification, faithfulness and value ordering, never for arithmetic (Chapter 4 §4.5).
- **Review.** The pre-defence self-review of 27 September 2026 and the revision of this report that followed it were AI-assisted.

No AI assistant generated experimental data: every reported measurement comes from an executed run or evaluation recorded in the repository, and simulated values are labelled as simulations. The research direction, the experimental design, the choice of claims and their final wording are the author’s responsibility.

Acknowledgement

I thank my project guide, **Ramesh Prakash Guledgudd**, for direction and sustained feedback throughout this work, and for the freedom to pursue the conservative and negative results that form a significant part of it.

I thank the Semester-3 project group for the shared multi-framework benchmark and the baseline integrations on which the Semester-4 studies build. I thank the maintainers of the benchmark suites used in the E1–E14 campaign, whose native evaluators made an independently checkable evaluation possible, and the platforms — Tinker, Modal, and Google Cloud — on which the experiments ran.

Finally, I thank the reviewers whose questions at earlier reviews shaped the reporting discipline this report depends on: the insistence on stating what a number does *not* establish has improved the work materially.

Arvind C R

Abstract

This report studies group-relative reinforcement-learning post-training of large language models under one methodological commitment: a comparative claim is made only when the whole training stack is held fixed except the factor under test. It makes four contributions.

(1) A recomputable signal-starvation diagnostic and a loss-form audit. The Zero-Variance Fraction (ZVF) is logged per step and recomputed from stored reward tensors, with a closed-form Bernoulli null. Reading the runners’ code rather than the internal reports shows that the “GRPO” runners used here are REINFORCE with a group-mean baseline, without KL or clipping.

(2) Stack-conditioned reporting. A minimum reporting standard and a machine-readable stack registry with a `stackdiff` tool that flags label-flip risk between two runs from their records alone.

(3) Calibrated negative results. No reliable scaling law is recovered from reward curves across twelve fitted anchors. A same-stack PPO/GRPO contrast (0.992 against 0.990, $p = 0.374$, five seeds) and a group-size sweep find no detectable difference, but both run on a saturated task and establish neither equivalence nor an optimum. ZVF does not predict held-out accuracy ($\rho = 0.27$, 95% CI $[-0.37, 0.88]$). An adaptive group-size controller derived from ZVF saves no prompts in counterfactual simulation, because 92% of its escalations fall on groups that are already all-correct.

(4) An evaluation-governance protocol. One lightly trained actor (Qwen3.6-35B-A3B with a LoRA adapter, 40 training steps) is evaluated on fourteen held-out suites under native graders at pinned revisions, with receipts, a no-pooling rule and a per-lane serving audit that traced a zero score to a serving defect. Complete scopes include VerilogEval (129/312 = 41.35%), LAB-Bench (0.2288) and AgentDojo benign utility (88/97); six replacement scopes completed on 27 September 2026 (Chapter 8, Table 8.C); seven original contracts remain externally blocked. A paired trained-versus-base rerun on 5–151 identical items per lane finds no significant difference in any lane, which at those sizes is inconclusive. No claim of improvement over any baseline is made.

Keywords: GRPO, RLVR, zero-variance fraction, signal starvation, stack-conditioned results, benchmark evaluation, reproducibility, held-out evaluation, attribution.

Contents

Declaration	i
Declaration on the Use of AI Tools	ii
Acknowledgement	iii
Abstract	iv
I Foundations	1
1 Introduction	2
1.1 Background: critic-free RL post-training and the group-relative estimator	2
1.2 A Motivating Example	5
1.3 Problem Statement	8
1.4 Objectives	9
1.5 Scope and Non-Goals	12
1.6 Contribution of the Candidate	13
1.7 Organisation of the Report	15
1.8 Canonical Numbers	16
2 Literature Survey	18
2.1 From RLHF and PPO to Group-Relative Methods	18
2.2 Direct Preference Optimisation as a Contrast	22
2.3 Fixing GRPO’s Biases and Collapse	23
2.4 Verifiable Rewards, Frameworks and Reporting Practice	25
2.5 Scaling and Layer-Adaptation Context	27

2.6	Process-, Tree- and Stability-Aware Variants	29
2.7	Positioning the Portfolio	30
3	System Requirements Specification	35
3.1	Overview and Actors	35
3.2	Functional Requirements	36
3.3	Non-Functional Requirements	41
3.4	Test-Case Specification	43
3.5	Requirements Traceability Matrix	45
3.6	Hardware Requirements	47
3.7	Software Requirements	49
3.8	Constraints and Assumptions	49
II	Design and Implementation	51
4	Proposed Methodology	52
4.1	The Design Commitment to Attribution	52
4.2	Per-Step Telemetry	56
4.3	Formalising the Zero-Variance Fraction	58
4.4	The Eight-Study Portfolio	65
4.5	The Design of the E1–E14 Held-Out Evaluation Campaign	68
5	Implementation Details	74
5.1	Two Rosters, One Dispatch Surface	74
5.2	The Adapter Architecture: What versus Where	76
5.3	The Tinker GRPO Loop	79
5.4	The Parallel Runner and a Real Logging Defect	80
5.5	The Colab White-Box Path (Per-Layer Gradient Profile)	81
5.6	Software Modules	82
5.7	The E1–E14 Evaluation Harness	83
5.8	The Bridge and Sampler Path	85
5.9	Recovery Re-implementations under zvf-program/	86

III	Results	89
6	Results I: Core GRPO Studies (P1–P4)	90
6.1	Scope, Sources and Evidence Boundary	90
6.2	P1 — Cross-Library and Cross-Scale GRPO Behaviour	92
6.2.1	The anchor pool and the fitted saturation forms	93
6.2.2	The cross-scale test and its nulls	94
6.2.3	Where the geometric template fails	94
6.2.4	A withdrawn architectural contrast	95
6.3	P2 — The Zero-Variance Fraction	96
6.3.1	Definition and measurement	97
6.3.2	Measured magnitudes across libraries	97
6.3.3	The all-correct wall and the mastery–incapacity aliasing	98
6.3.4	Prognostic value	100
6.4	P3 — Group Size and Contrast Density	101
6.4.1	The measured sweep	102
6.4.2	The same-stack four-axis audit and the reconstructed budgets	104
6.4.3	The curriculum and matched-token null results	105
6.5	P4 — Length Bias and Held-Out Generalisation	106
6.5.1	The controlled null	106
6.5.2	The roster-level verbosity-trap audit	108
6.5.3	Elevated analyses	110
6.5.4	Held-out generalisation and the scale extension	112
6.6	Cross-Pillar Observations and Limitations	113
7	Results II: Reporting, Registry and Controller (P5–P7)	115
7.1	A Minimum Reporting Standard for Stack-Conditioned Results (P5)	115
7.2	The GRPO-Registry (P6)	118
7.3	From Diagnostic to Controller (P7)	121
8	The E1–E14 Held-Out Evaluation Campaign	126
8.1	The evaluated actor and the one-actor design	128

8.1.1	How the actor was trained	128
8.1.2	Serving conditions per lane	131
8.2	Suites with strictly complete scopes	134
8.3	Original-contract results	136
8.4	Replacement-scope lanes completed on 27 September 2026	137
8.5	The E4 rerun as a diagnostic	140
8.6	Externally blocked lanes and terminal states	140
8.7	Small-scale reruns of all fourteen lanes	141
8.8	Summary of the campaign at the point of writing	145
IV	Synthesis	146
9	Cross-Study Synthesis, Threats to Validity, Conclusions and Future Work	147
9.1	Cross-Study Synthesis	147
9.2	Rigour and Threats to Validity	151
9.3	Conclusions	156
9.4	Future Work	158
	References	161
V	Appendices	170
A	Complete Run Registry	171
A.1	Composition and temporal split	172
A.2	Semester-3 shared benchmark runs	173
A.3	Semester-4 P1–P8 study runs	175
A.3.1	Opening and null-result runs	180
A.4	E1–E14 evaluation campaign lanes	184
A.5	Completeness limitations	187
B	Reproducibility and Provenance	190
B.1	What this appendix establishes	190

B.2	The pinned runtime	190
B.3	The reviewer’s entry point	191
B.4	Where the documentation has drifted from the code	192
B.5	The audit apparatus	192
B.6	The deterministic ledger check	193
B.7	The receipt model	195
B.8	Evidence loss in the finish era	196
B.9	The Semester 3 and Semester 4 provenance boundary	196
B.10	Residual non-determinism and the limits of reproduction	197
C	Notation, Symbols and Abbreviations	199
C.1	Scope and convention	199
C.2	Symbols	199
C.3	Four collisions worth flagging	201
C.4	Abbreviations	201
C.5	The E1–E14 lane identifiers, scopes and statuses	204
D	Campaign Lane Detail	207
D.1	Actor identity, inference conditions and scoping notes	207
D.2	Training-run provenance	208
D.3	Serving conditions: format limits and the scope of the stop-sequence mechanism	209
D.4	Strictly complete scopes: lane detail	209
D.5	Original-contract results: lane detail	211
D.6	Replacement-scope lane narratives, 27 September 2026	212
D.7	E4 rerun: closure and full audit	215
D.8	Externally blocked lanes: access requests and verification	216
D.9	Small-scale reruns: detail and per-lane caveats	216
E	Corrections to Internal Source Documents	218
F	Applied Side Study: Credit-Card Fraud Detection (P8)	222
F.1	Dataset, splits, and the two arms	222

F.2	The head-to-head result	223
F.3	Why the scoring seat stays with the tree	224
F.4	Supporting measurements	224
F.5	Proposed roles for the language model	225
F.6	Limitations	226
G	Evidence Map	227
G.1	Chapter 1: Introduction	227
G.2	Chapter 2: Literature Survey	228
G.3	Chapter 3: System Requirements Specification	230
G.4	Chapter 4: Proposed Methodology	231
G.5	Chapter 5: Implementation Details	233
G.6	Chapter 6: Results I: Core GRPO Studies (P1–P4)	234
G.7	Chapter 7: Results II: Reporting, Registry and Controller (P5–P7)	238
G.8	Chapter 8: The E1–E14 Held-Out Evaluation Campaign	239
G.9	Chapter 9: Cross-Study Synthesis, Threats to Validity, Conclusions and Future Work	241
G.10	Appendix D: Campaign Lane Detail	243
G.11	Appendix F: Applied Side Study: Credit-Card Fraud Detection (P8)	245

List of Figures

1.1	The GRPO training loop	3
1.2	The confounding problem this work addresses	6
1.3	Boundary between the inherited Semester-3 group infrastructure and the individual Semester-4 contribution reported in this document	13
2.1	Algorithmic lineage from RLHF through PPO to GRPO, with DPO as an orthogonal branch	20
2.2	The eight studies P1–P8 grouped by theme, with the question each one addresses and the state of its evidence	33
4.1	The four de-confound pillars	53
4.2	The attribution design	53
4.3	Per-step telemetry recorded by the unified harness, and the destinations each record flows to	57
4.4	Definition of the Zero-Variance Fraction	63
4.5	Gradient utilisation, $1 - \text{ZVF}$, as a function of group size G	64
4.6	Design of the E1–E14 native-benchmark evaluation campaign	69
5.1	The evidence chain from sealed request through execution and native grading to a deterministic ledger check	75
5.2	Six RL frameworks behind one Tinker-style API	77
5.3	Evaluation harness internals	84
6.1	Cross-scale behaviour of reward across the studied model range: no reliable monotone scaling trend is recovered	91
6.2	Mean Zero-Variance Fraction by method in a simulation projection (a) and by measured experiment family (b); neither panel compares backend libraries	96

6.3	Group size G : one single-seed sweep, one three-seed sweep and one reconstruction	102
6.4	The post-hoc peak-then-decay rule (peak before 65% of logged steps and terminal reward below $0.90\times$ peak) on the roster and on the controlled comparison	107
8.1	Terminal status of all fourteen E1–E14 lanes: the 2026-09-19 ledger of record, updated with the six replacement scopes scored on 2026-09-27 (double border, ★)	127
8.2	LAB-Bench public split, per-category accuracy, all 1,967 questions graded across eight categories	134
8.3	VerilogEval decomposition: two native framings of 156 tasks, 67/156 code-completion and 62/156 spec-to-RTL, summing to $129/312 = 41.35\%$ pass@1	135
8.4	The six replacement-scope results scored on 2026-09-27, each with its 95% confidence interval	137
8.5	The E4 rerun diagnostic	141
8.6	Terminal-state taxonomy used to close every lane	142
9.1	Threats to validity grouped as internal, construct and external, each with the mitigation applied in this work	152

List of Tables

1.1	Canonical numbers. “results/” is platform_hybrid/experiments/results/; “sections/”, “archive/” and “neurips_2026_variants/” are under platform_hybrid/paper/. The Qwen3-8B GSM8K GRPO training-reward rows use different summaries (last-10 mean, trace mean, pooled) and the sources do not establish that they come from the same run, so they are not interchangeable.	16
2.1	Positioning of the eight portfolio studies (P1–P8) against closest prior art and the narrowed axis each claims.	30
3.1	Functional requirements. Sources: the Phase-1 report is platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex; other files are under platform_hybrid/, platform_local/ or zvf-program/flagship/.	37
3.2	Non-functional requirements.	42
3.3	Requirements traceability matrix: each functional requirement mapped to the artefacts that implement it and the audit check that verifies it. . . .	45
3.4	Hardware requirements and their status.	48
4.1	The four de-confound pillars, the single factor each isolates, and the design that holds the remainder fixed.	54
4.2	Loss form by study, read from the runner code. M = platform_hybrid/experiments/modal/, T = platform_hybrid/experiments/tinker-runs/, C = zvf-program/colab-experiments/. “n/a” means the study does no training; “not recoverable” means the repository does not hold the code or configuration that ran.	59
5.1	The four recovery re-implementations. Schedule offsets, hashes, reservation identifiers and HMAC domain strings are recorded in the cited source files rather than repeated here.	86

8.1	Training record of the E1–E14 actor, from the surviving run artefacts.	129
8.2	Serving conditions of the original campaign runs, by lane.	132
8.C	Replacement-scope results, 27 September 2026	138
8.D	Deviations from the fail-closed protocol, 27 September 2026	139
8.A	Base model on Tinker, small-scale subsets	143
8.B	Trained actor versus base model on the same vLLM engine, identical items	144
A.1	Registry composition by logging project, with temporal split and distinct base models.	172
A.2	Semester-3 cross-framework shared benchmark runs.	173
A.3	Semester-3 Modal campaign runs.	174
A.4	Semester-3 group-size sweep runs.	175
A.5	Semester-4 P1-P8 study runs: each claim mapped to its linked runs, stack, model and algorithm.	176
A.6	Semester-4 opening and null-result runs: the curriculum, token-budget, hard-prompt, loss-formulation and layer-freeze blocks.	181
A.7	E1-E14 evaluation campaign lanes with their run identifiers, stacks, platforms, models, algorithms and group sizes.	184
A.8	Field coverage across the 1,662-run ledger: rows carrying a value against rows blank or UNKNOWN.	187
A.9	Run totals quoted in this report and the corpus each one counts.	188
B.1	The deterministic ledger check by class, listing each specific check and what it establishes.	194
C.1	Notation: symbols used in this report, their meaning, and any divergence in how the underlying sources use them.	199
C.2	Abbreviations and their expansions, with names that no source expands recorded as identities rather than glossed.	202
C.3	The E1–E14 lanes: original contract, replacement scope, headline figure, terminal state and receipt.	204
E.1	Corrections applied to internal source documents. Each row’s corrected value is the one used in the chapter named in the first column.	218

F.1	The two arms of P8.	223
F.2	Head-to-head result on the synthetic card-fraud set, by arm and evaluation split.	223
F.3	Supporting P8 measurements. Bracketed ranges are paired-bootstrap 95% intervals as reported by the source. Source files are under platform_hybrid/experiments/results/ and platform_hybrid/paper/archive/absorbed/P08_fraud/sections/.	224

Part I

Foundations

Chapter 1

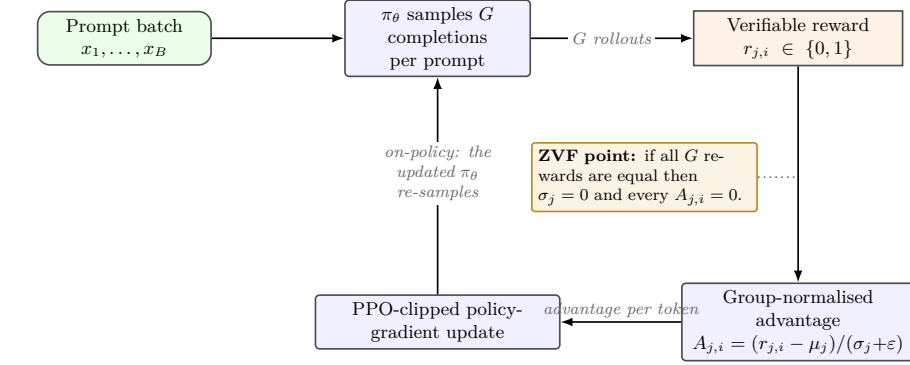
Introduction

1.1 Background: critic-free RL post-training and the group-relative estimator

Post-training is now the stage at which a base language model is turned into something that reliably follows instructions and solves problems. For several years the dominant recipe was reinforcement learning from human feedback (RLHF), in which a learned reward model is trained on human preference comparisons and then used as the reward signal for a policy-optimisation loop. RLHF works, but it introduces a second learned model whose errors the policy can exploit, and it makes reward construction expensive and hard to audit. The alternative that has taken hold more recently is reinforcement learning with verifiable rewards (RLVR): instead of a learned reward model, the reward comes from a programmatic checker — exact-match against a reference answer for grade-school mathematics, unit tests for code, a formal or semi-formal validator for structured output. The checker is deterministic, transparent, and free to run. What it gives up in generality it gains in measurability: every reward is a fact about a generated string rather than a prediction from a model (Figure 1.1).

The workhorse optimiser for both settings has been proximal policy optimisation (PPO), which optimises a clipped surrogate objective against a value function critic. The critic is a second network, usually of comparable size to the policy, trained to predict the expected return of the current state; its predictions supply the baseline that the advantage estimator subtracts. This works, but it is expensive and fragile. The critic consumes memory that could hold policy parameters, it has its own learning rate and its own failure modes, and its errors propagate directly into the policy gradient as biased advantages.

Group Relative Policy Optimization (GRPO) removes the critic entirely. Rather than learn a value function, GRPO samples a group of G completions for the same prompt,



The ZVF point, in detail

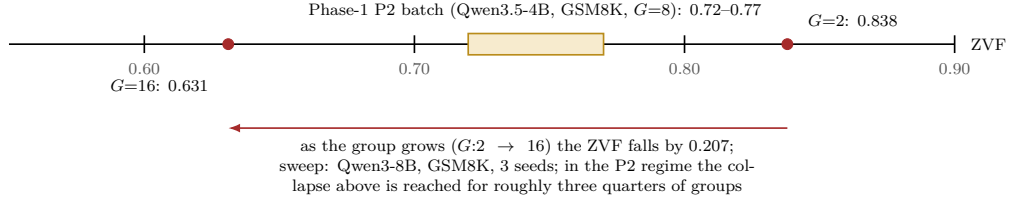
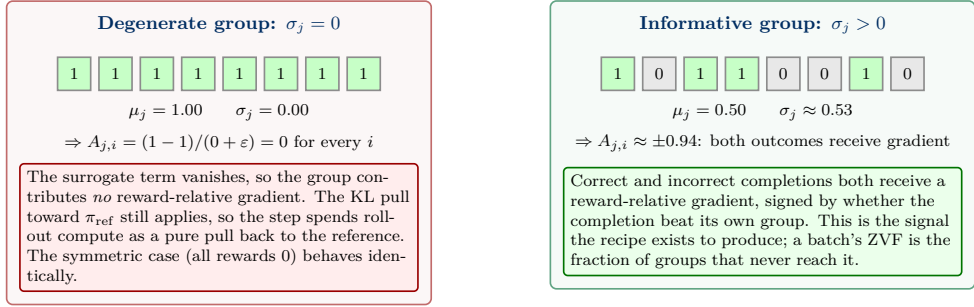


Figure 1.1: The GRPO training loop. A prompt is sampled G times, each completion is scored by a reward function, advantages are formed relative to the group, and the policy is updated. When all G rewards coincide the group-relative advantage collapses to zero and no reward-relative gradient flows, although the rollouts are still paid for. Number line: the Phase-1 P2 batch ZVF of 0.72–0.77 (four methods, 40 steps, 16 prompts $\times$ 8 samples, GSM8K, seed 0, Qwen3.5-4B) and the group-size sweep on Qwen2.5-0.5B two-digit addition over three seeds, where mean ZVF falls from 0.838 at $G = 2$ to 0.631 at $G = 16$ (a fall of 0.207).

scores all of them with the verifier, and derives the advantage of each completion from the group’s own reward statistics. Concretely, for a group with rewards $r_1, \dots, r_G$, the advantage assigned to completion i is a within-group standardisation,

$$A_i = \frac{r_i - \mu_g}{\sigma_g + \varepsilon}, \quad \mu_g = \frac{1}{G} \sum_j r_j, \quad \sigma_g^2 = \frac{1}{G} \sum_j (r_j - \mu_g)^2,$$

which in the original GRPO formulation is then used in a clipped, KL-regularised policy-gradient update. The runners used in this project are simpler: none has a KL term or a reference policy, and the Tinker runners, including the one that trained the campaign actor, have no ratio or clipping either, so they are REINFORCE with a group-mean baseline (Table 4.2 gives the loss form of every study). The empirical baseline is the group mean; there is no value network to train, no value loss to weight, and no critic to keep in sync with the policy. The trade is a much simpler optimiser for a much smaller effective batch, since G samples of the same prompt now carry one learning signal rather than G independent ones.

That trade has a sharp consequence, and it is the observation this project is organised around. Because the advantage is a *within-group* standardisation, its numerator is a deviation from the group mean and its denominator is the group’s own spread. When every completion in a group receives the same verifier outcome — all correct, or all incorrect — the group contributes nothing: $\sigma_g = 0$, every numerator is zero, and every advantage A_i is exactly zero. The group has been fully sampled and fully scored, at full token cost, and the reward-relative part of the update is identically zero. In an objective with a KL penalty, that penalty would still pull the policy toward the reference; in this project’s runners there is no such term, so the group contributes exactly zero gradient. We call the fraction of a training batch that falls into this state the **Zero-Variance Fraction (ZVF)**, and it is the single measurement that most of this report’s diagnostics are built around.

ZVF is not a rare edge case, but its size depends on the regime. The Phase-1 main configuration (Qwen3.5-4B, GSM8K, 40 steps, $G = 8$) records a ZVF of approximately 0.72–0.77, meaning that roughly three quarters of that rollout batch contributes no reward-relative gradient. Other measured runs span 0.16 (Qwen3-8B on held-out GSM8K at $G = 8$) to 0.84 (Qwen2.5-0.5B arithmetic at $G = 2$), so no single value is typical; the measured values are collected in the canonical-numbers table (Table 1.1). The fraction responds to the one knob that most directly controls it: on the Qwen2.5-0.5B arithmetic sweep, mean ZVF falls from 0.838 at $G = 2$ to 0.631 at $G = 16$, a decline of 0.207 across that range. Larger groups buy more contrast, with diminishing returns that saturate well before the group is small enough to be cheap.

There is a second and less comfortable observation in the background, and it is about attribution rather than mechanism. “GRPO” is not a single algorithm. It is a label that

covers implementations in Tinker, TRL, SkyRL, verl, OpenRLHF and a growing number of research forks, each of which chooses its own answer parser, sampling temperature, top- p , KL coefficient, clip range, LoRA target set, precision and evaluation harness. Two runs that both say “GRPO” in their titles may differ in a dozen ways that plausibly move the outcome, and the reported headline number does not distinguish between them. The project’s own stack-sensitivity probe makes this concrete: a nominally matched configuration (Qwen3-8B, seed 42, $G = 8$, learning rate 1×10^{-5} , GSM8K-500, 30 steps, visible hyperparameters held equal, although the managed Tinker run used the Qwen3-8B-Base checkpoint and TRL the instruction-tuned Qwen3-8B) was run across four framework paths, two of which completed and two of which are dry-run placeholders, with results recorded in `experiments/results/framework_comparison.json`. Four separate marginal one-way ANOVAs over 42 experiments give $\eta^2 = 0.546$ for library or framework, $\eta^2 = 0.558$ for training algorithm, $\eta^2 = 0.471$ for model family and $\eta^2 = 0.322$ for model scale — with the important caveat, stated by the source itself, that these factors are confounded with one another and with backend, reward implementation, hardware and task. Because each value is a marginal per-factor η^2 on a collinear corpus, the four sum to 1.897 and are not a variance decomposition; none estimates a factor’s independent contribution (Chapter 6 §6.1; source: `platform_hybrid/paper/sections/_shared_methods.tex`). The framework factor is nonetheless not a nuisance parameter to be wished away: marginally, outcomes vary with it about as strongly as with the algorithm label.

1.2 A Motivating Example

Consider a single training step with a group size of $G = 8$ on an easy GSM8K prompt that the current policy already solves. All eight sampled completions produce the correct final answer, so each receives reward $r_i = 1$. The group mean is $\mu_g = 1$ and the group standard deviation is $\sigma_g = 0$, so every advantage $A_i = 0$. The compute spent on that group is real — eight full forward-decode passes at whatever temperature the sampler is configured with — and the group supplies no task-learning signal. In this project’s runners, which carry no KL term, that group contributes exactly zero gradient. Now imagine a batch of 16 such prompts in which 12 are already solved and 4 are not: $12/16 = 0.75$ of the batch is “dark” compute. What the Semester 4 work adds to this picture is that 0.75 is not a worst case: it is close to the 0.72–0.77 measured on the Phase-1 main configuration. It is not a universal value either, since the measured runs range from 0.16 to 0.84 depending on model, task and G (Table 1.1), so any claim that GRPO is learning efficiently has to be read against the ZVF of its own regime (Figure 1.2).

Now suppose, having noticed this, that one wants to answer the obvious follow-up ques-

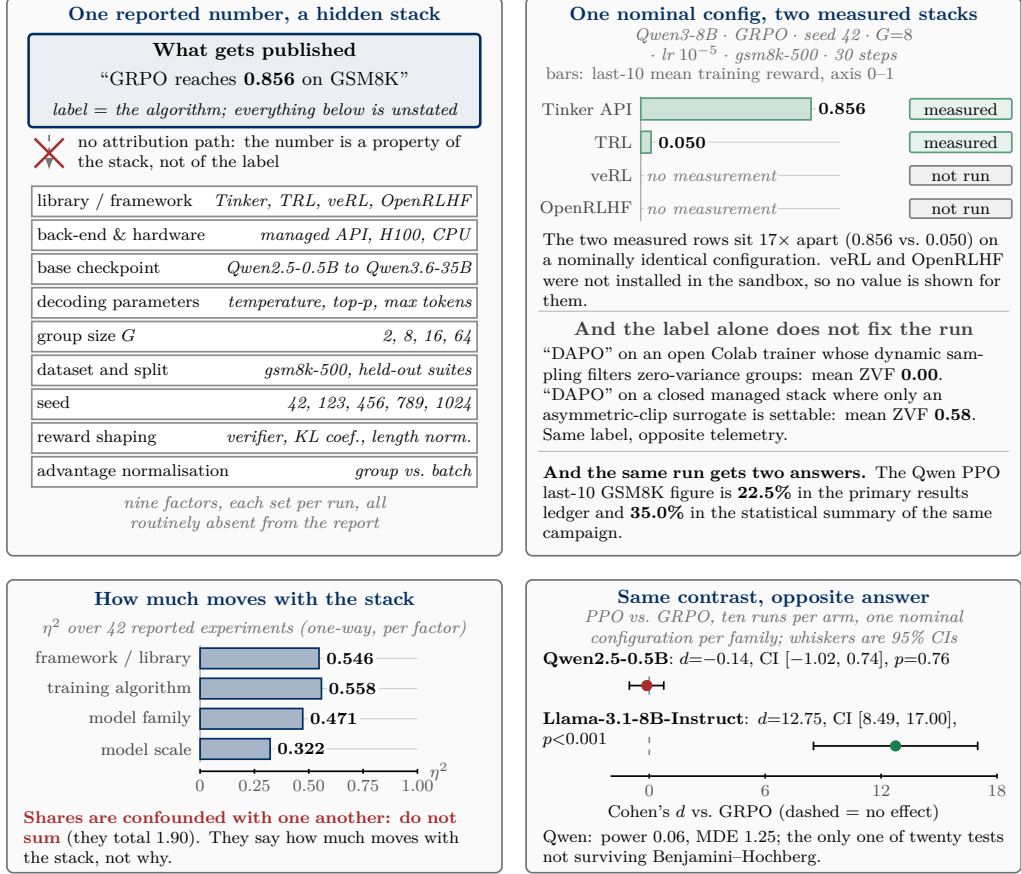


Figure 1.2: The confounding problem this work addresses. A single reported reward figure sits on top of an unstated stack; without fixing and reporting that stack the number is not attributable to the method under test. Top right: of the four frameworks in the nominal configuration only Tinker and TRL were run; veRL and OpenRLHF were not installed in the sandbox and have no measurement. The $17\times$ gap between the two measured stacks is an under-specification exhibit, not a clean back-end-only effect, because the managed stack also pinned a different base checkpoint (Qwen3-8B-Base vs. Qwen3-8B). Bottom left: one-way η^2 shares over 42 reported experiments; they are confounded with one another and with back-end, reward implementation, hardware and task, so they do not sum to one. Bottom right: Cohen’s d with 95% CIs from the Phase-1 statistical summary; the Qwen arms did not share a back-end (GRPO on the managed API, PPO on a Modal H100 cluster), so no directional Qwen algorithm claim is made.

tion: is the group-relative estimator actually a better choice than a learned critic? The natural experiment is a matched comparison — same model, same training prompts, same verifier, same step budget, same evaluation, with PPO and GRPO as the only difference. The project’s first attempt at that contrast is recorded in a historical internal report under the label “Qwen3-8B”, with a Welch t -test of $p = 0.7605$ computed over the final ten steps of a single run per arm. Those ten steps are autocorrelated observations of one run, not ten independent runs, and the project’s later methods note re-examined the row and marks it “provenance conflict; not estimable”. No effect size, confidence interval or power figure is therefore quoted for it here, and the Welch value is carried only as a provenance record (Chapter 6 §6.2.3). The project’s own rule is that single-seed Tinker experiments have “zero statistical power” and are “treated as descriptive only and are not subject to significance testing”.

Two further observations from the same historical source make the example sharper rather than cleaner. First, the same table reports the PPO-versus-GRPO contrast on Llama-3.1-8B-Instruct in the opposite direction; it is also one run per arm, so again no inferential statistic is quoted, but the direction alone shows that the apparent answer is not stable across model families. Second, the Qwen arms were not executed on a common back-end — the GRPO arm ran through Tinker-managed infrastructure and the PPO arm on a Modal H100 cluster — and the source states this explicitly and declines to draw the algorithmic conclusion it invites: “We therefore treat the PPO/GRPO rows as stack-conditioned evidence, not as an algorithm diagnostic baseline”, and “We therefore withhold a directional Qwen algorithm claim”. The same section records an aggregation gap that illustrates why such care is warranted: the Qwen PPO last-10 GSM8K figure is 22.5% in the primary results ledger but 35.0% in the statistical summary, and the source names the ledger value as primary and treats the summary value as evidence of an aggregation defect rather than as a second measurement.

The genuinely same-stack evidence in the repository is also a null, and its scope has to be stated with it. The same-stack PPO/GRPO run (Qwen2.5-0.5B, 40 steps, five paired seeds) is a two-digit addition task on which both arms are at ceiling: evaluation accuracy is 0.990 for GRPO and 0.992 for PPO, the paired difference is -0.002 with $p = 0.374$, and the 200 evaluation prompts are fresh random draws from the same generator used in training rather than a disjoint split. At that ceiling a null cannot separate “no effect” from “no headroom”, so the result is that no difference was detectable on a saturated task, not that the estimators are equivalent; a paired TOST at a ± 0.005 margin gives $p = 0.10$ (Table 4.2 gives the exact loss form of both arms). The variant audit (not to be confused with campaign lane E1) takes four GRPO-family variants (DAPO, GSPO, Dr. GRPO, AERO) plus the GRPO reference, runs five arms across eight paired seeds for 40 units at a frozen stack fingerprint, and evaluates on 500 held-

out GSM8K examples per unit. Every one of the four contrasts is inconclusive. The figures below are paired- t 95% intervals and paired- t p -values over the eight seeds, recomputed from `zvf-program/audit/results/full/*.json`: DAPO +0.00100, CI $[-0.00632, +0.00832]$, $p = 0.756$; GSPO +0.00500, CI $[-0.00357, +0.01357]$, $p = 0.210$; Dr. GRPO -0.00200 , CI $[-0.01272, +0.00872]$, $p = 0.673$; AERO -0.00075 , CI $[-0.01029, +0.00879]$, $p = 0.858$. These paired- t intervals replace the audit’s own intervals (corrected; see Table E.1). After Benjamini–Hochberg correction every adjusted p is at least 0.84. The pre-audit aggregator’s verdict that the differences “disappear” is itself declared invalid, because the locked exact noncentral paired- t minimum detectable effect of 0.010115893 exceeds the 0.010 margin the study was designed around. The audit’s permitted wording is deliberately narrow: across eight paired seeds no arm produced a BH-significant held-out difference from GRPO, and none met the locked exact-power equivalence rule — which is not the same as saying the methods are the same, nor that they fail to improve. That audit runs on Qwen3-8B GSM8K with 500 evaluation examples per unit (Appendix A, P11).

Taken together, these two halves of the example are the whole motivation for what follows. The mechanism half shows that a large fraction of the compute in a GRPO run is not doing what a casual reading of the loss curve suggests it is doing. The attribution half shows that the most natural comparative question one would ask about that mechanism — is this estimator better than the alternative? — cannot be answered from the label “GRPO” and “PPO” alone, and that the answer on the project’s own evidence is a non-significant contrast that is either back-end-confounded or measured at a performance ceiling, rather than a ranking. Neither observation is a defect of the GRPO estimator; both are defects of what gets measured and what gets reported.

1.3 Problem Statement

The project is organised around four problems, in decreasing order of how often the literature acknowledges them.

Attribution is missing. Comparative claims about RL post-training are routinely made across frameworks that differ in parser, sampler, KL coefficient, clip range, LoRA targets, precision and evaluator. A reported gain from “GRPO” over “PPO” is, on most published evidence, a gain from one complete stack over another complete stack, with no way to say which component produced it. The project’s own marginal per-factor η^2 values put the framework factor at a magnitude comparable to the algorithm factor, although they are not a partition of the variance, and its same-stack PPO/GRPO contrast, run on a saturated 0.5B addition task (evaluation accuracy 0.990 GRPO, 0.992 PPO),

returns a paired difference of -0.002 with $p = 0.374$. The problem is not that these numbers are surprising; it is that no paper in this area is obliged to report the information that would let a reader separate them.

Signal starvation is invisible. Standard training logs report reward, loss, gradient norm, entropy and completion length. None of these surfaces the fraction of a batch whose group advantage is exactly zero. A run can show a rising reward curve and a falling ZVF, or a rising reward curve with a ZVF pinned near 0.8, and the two look identical in a loss plot. The fraction of sampling budget buying no reward-relative gradient is, in the project’s own measurements, the largest measured source of zero-advantage sampling on this fixed setup, and it is not part of any standard telemetry.

There is no reporting or provenance standard. Even where authors are willing to describe their stack, there is no machine-readable form in which to do it, no validator that checks whether a claim’s recorded configuration is internally consistent, and no accepted minimum set of fields that a stack-conditioned result must carry. Consequently, results cannot be pooled, cannot be compared across papers, and cannot be audited after the fact.

Evaluation results are rarely reported with their terminal states. Benchmark suites in the agentic and reasoning evaluation space are frequently multi-scope (a public split versus a private original contract, a benign-utility scope versus an adversarial scope), quota-limited, or gated behind private payloads. Reports tend either to omit such suites or to collapse them into a single number, which conceals how much of the suite was actually graded. The project treats this as a first-class concern rather than a presentational one: a result that is genuinely partial should be recorded as partial, with its coverage denominator and its reopen condition, and a lane that cannot be run at all should be recorded as externally blocked rather than quietly dropped.

1.4 Objectives

The project pursues eight objectives. They are stated as they were pursued, not retrofitted to the results.

O1 — Build a single harness that can dispatch the same experiment to multiple back-ends. Provide one control surface over Tinker, TRL, SkyRL, verl and OpenRLHF paths so that a configuration can be held fixed while the framework varies. The harness is the deliverable; it is not a completed multi-framework benchmark. Only Tinker and TRL produced completed runs, and those two used different base checkpoints (Qwen3-8B-Base and Qwen3-8B Instruct), so no framework comparison is claimed (Chapter 5 §5.2).

O2 — Instrument the training loop so that signal starvation is directly observable.

Emit per-step telemetry carrying ZVF, gradient utilisation, policy entropy, completion length and KL divergence, and persist the raw per-group reward tensors so that any derived statistic can be deterministically recomputed after the fact rather than trusted from a log line.

O3 — Measure the zero-variance regime on real reward tensors and test the levers that might move it.

Characterise ZVF empirically on genuine GRPO reward tensors, and investigate re-baselining and curriculum interventions as ways of converting dark compute into contrast.

O4 — Establish and hold a scientific protocol.

Multi-seed runs, matched baselines, held-out evaluation with explicit denominators, deterministic recomputation, declared (not pre-registered) effect-size and power accounting, and reporting of negatives and nulls — including the cases where the answer is that the study was underpowered to answer.

O5 — Scope the most original direction in the field.

A literature scan for white space, which in this project identified stack provenance and reporting standards as the unclaimed ground and produced the minimum reporting standard (P5) and the machine-readable stack registry (P6).

O6 — Turn the diagnostic into an intervention.

Derive an adaptive group-size controller from the ZVF measurement and evaluate it as a training-time policy rather than a post-hoc observation (P7).

O7 — Build and exercise an evaluation-governance protocol on one trained actor.

Run a single trained checkpoint against fourteen agentic and reasoning suites that were not part of its training data, each with its own native evaluator at a pinned revision, under receipt, budget and decontamination controls, and record every lane’s terminal state. The contribution is the governance protocol and the receipt-bound record it produces. The campaign is not evidence about GRPO: the actor’s training run was short (40 steps, Chapter 8 §8.1.1), and the only trained-versus-base comparison is the small-sample Table 8.B.

O8 — Report under a deliberate asymmetry.

Where the evidence supports a structural or descriptive claim, state it; where it does not, withhold it. No cross-suite aggregate, no pooling of original-contract and replacement-scope numbers, and no claim of improvement over any baseline anywhere in the work. Baseline comparisons are reported where they exist, and none is claimed as an improvement. They are the paired trained-versus-base small-scale comparison of Chapter 8 (Table 8.B), which finds no significant difference in any lane, and the pre-to-post held-out comparisons of the controlled studies in Chapter 6.

The objectives are what the project set out to build. The results chapters answer four research questions that the objectives make testable. Each is mapped to the table that answers it, and §9.3 gives a one-line answer to each.

- **RQ1.** Is the Zero-Variance Fraction recomputable from stored reward tensors, and does it carry information about training outcome? Answered by the ZVF rows of Table 1.1 and §6.3.
- **RQ2.** With the training stack held fixed, do algorithm labels (PPO vs GRPO, GRPO vs Dr. GRPO) or group size separate on evaluation accuracy? Addressed by the same-stack rows of Table 1.1, the loss-form table in §4.3, and §6.2.3, §6.4 and §6.5. The experiments reported here can bound this question but not answer it: the PPO/GRPO, group-size and arithmetic GRPO/Dr. GRPO contrasts were all run on a saturated task at evaluation accuracy near 0.99, and the two unsaturated GRPO/Dr. GRPO comparisons (§1.2, §6.5.4) are inconclusive and establish no equivalence (§9.3).
- **RQ3.** Can a machine-readable stack manifest flag label-flip risk between two runs from their records alone? Answered by §7.2 (registry health and `stackdiff`).
- **RQ4.** What does a governance-first evaluation of one trained actor yield, and what can it not establish? Answered by Table C.3 and Tables 9.A and 9.B.

Contributions. Measured against these objectives, the thesis makes four contributions. Everything else in it is supporting evidence for one of them.

1. **A recomputable signal-starvation diagnostic and a loss-form audit.** ZVF is logged per step and recomputed from stored reward tensors against a closed-form Bernoulli null (§4.3, §6.3). Reading the runner code rather than the internal reports shows that the “GRPO” runners used here are REINFORCE with a group-mean baseline, without KL or clipping (Table 4.2).
2. **Stack-conditioned reporting.** A minimum reporting standard (P5) and a machine-readable stack registry with `stackdiff`, which flags label-flip risk between two runs from their records alone (Chapter 7).
3. **Calibrated negative results.** No reliable scaling law is recovered from reward curves; the same-stack PPO/GRPO and group-size contrasts are nulls on a saturated task; ZVF does not predict held-out accuracy; and the ZVF-derived controller saves no prompts in counterfactual simulation. Each is reported with the power limitation that bounds it (Chapters 6–7).
4. **An evaluation-governance protocol.** Native graders at pinned revisions, per-lane receipts, a no-pooling rule, a per-lane serving audit and a paired same-engine trained-versus-base design, exercised on one lightly trained actor across fourteen

held-out suites (Chapter 8). The contribution is the protocol and the record it produces, not the actor’s scores.

1.5 Scope and Non-Goals

The scope of this report is measurement and diagnostics, not leaderboard performance. The controlled studies and the campaign actor use small-to-mid open models — the Qwen2.5, Qwen3, Qwen3.5 and Qwen3.6 families, from roughly 0.5B parameters up to the 35B sparse mixture-of-experts configuration used for the campaign actor. The P1 cross-scale roster is wider, 0.6B to 1T total parameters, but its frontier-scale anchors are single-seed runs through a managed API and are read descriptively (Chapter 6 §6.1; source: `platform_hybrid/paper/sections/p1_abstract.tex`). Verifiable rewards come from mathematics and code checkers, principally GSM8K exact-match and HumanEval-style unit tests. Held-out evaluation uses suites’ own native evaluators at pinned revisions.

Several things are explicitly out of scope. This is not an attempt to set a state-of-the-art result on any benchmark, and no such claim is made anywhere in the report. It is not a study of frontier reasoning models. The evaluation sets used in the controlled comparisons are small, and their sizes differ by study: 200 fresh prompts for the 0.5B arithmetic runs, 200 per seed for the 1.5B GSM8K chain-of-thought runs, 500 for the variant audit, and only 8 to 20 in several Phase-1 openings (the layer-freeze result rests on $n = 8$). Table 1.1 gives the n behind each headline number. Claims about novel algorithms are not made: the length-weighting intervention (P4) and the adaptive controller (P7) were designed during the work and are reported as designed and simulated systems with their evaluation limits attached, not as demonstrated improvements.

The report states negative, null and withdrawn results alongside positive ones; this is its one general statement about reporting practice, and later chapters report the results without repeating it. Three evidentiary rules apply throughout and are worth stating here because they constrain what the later chapters are allowed to say. First, every number in this report is traceable to a repository file that was actually read; where a figure could not be sourced, the finding is described qualitatively and the figure is attributed to its source rather than reconstructed. Second, original-contract and replacement-scope benchmark results are never pooled and never averaged across suites — the two are different measurements of different things under different licences, and treating them as one column would manufacture a number that no suite reported. Third, no improvement over a baseline or over a base model is claimed unless the source states it with a test; where the source reports a difference without a test supporting it, or reports an inconclusive test, that is what is written here.

1.6 Contribution of the Candidate

This report covers work that spans two semesters and two very different modes of authorship, and the distinction matters for how the contribution should be assessed. Semester 3 was a group project (Group 6). The multi-framework repository, the baseline integrations, the initial Tinker, SkyRL and GRPO experiments, the literature foundation, and the early benchmark, evaluation and reproducibility scaffolding were produced by that group, of which the candidate was a member. Semester 4 is the candidate’s individual continuation, and the new research direction, the experiment expansion, the analysis, the report writing and the P1–P8 internal reports are the candidate’s solo work (Figure 1.3).

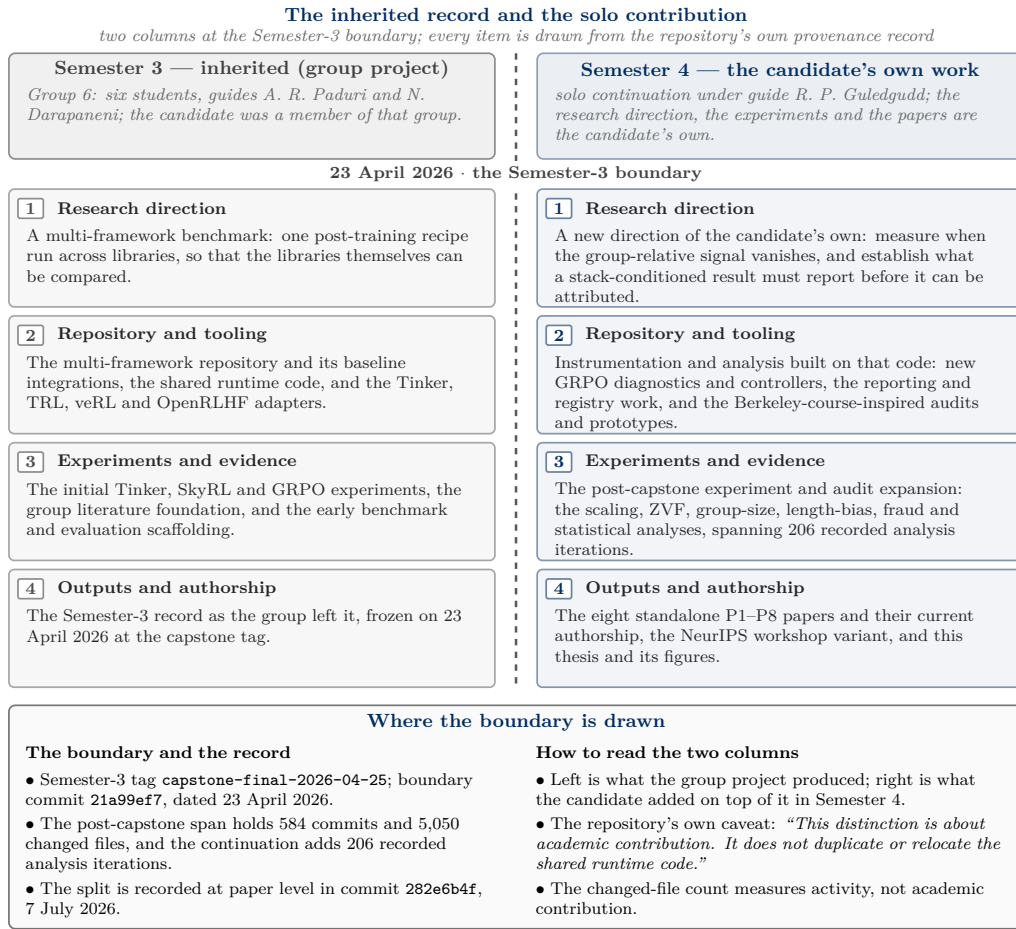


Figure 1.3: Boundary between the inherited Semester-3 group infrastructure and the individual Semester-4 contribution reported in this document. Both columns are drawn from the repository’s own provenance record (the Semester-3 tag and boundary commit, the post-boundary commit and file counts, the solo-authorship commit, and the Semester-4 README with its paper-to-evidence map); where the record does not settle a claim, the claim is left out rather than inferred.

The boundary is recorded precisely rather than loosely. The inherited state is frozen at the Git tag `capstone-final-2026-04-25`, corresponding to boundary commit `21a99`

ef766543a1ca86bfbad188445d3596ea73c of 23 April 2026. At the time the separation was documented, 584 commits had been made after that boundary, touching 5,050 changed files — a figure the provenance record explicitly declines to present as a measure of academic contribution, since it counts mechanical and generated changes alongside substantive ones. The transition to explicit solo authorship is marked by commit 282e6b4f0f158de11b3f5bb4656e2d871f68f5c0 of 7 July 2026.

What was added after that boundary is the substance of this report. It includes the eight unpublished internal research reports: P1, a cross-library and cross-scale study of GRPO behaviour; P2, the Zero-Variance Fraction as a descriptive diagnostic; P3, group size, contrast density and the relationship to DPO; P4, length bias and held-out generalisation; P5, the minimum reporting standard for stack-conditioned results; P6, a machine-readable GRPO stack registry; P7, an adaptive group-size controller derived from the signal-starvation diagnostic; and P8, a side study comparing a language model against gradient-boosted trees in credit-card fraud detection. It includes the post-capstone experimental, audit, synthesis and report iterations — 206 recorded analysis iterations — the Berkeley-course-derived research audits and prototypes, the expanded statistical analyses, and the new GRPO diagnostics and controllers. And it includes the E1–E14 benchmark campaign, in which a single Tinker-trained actor (base Qwen/Qwen3.6-35B-A3B at commit 995ad96e..., with the arvindcr4/pavlov-portfolio-qwen36-seed809-stepfinal-tinker-cf0ad8c1... adapter at commit 64444133..., in bfloat16) was evaluated against fourteen suites under receipt, budget and decontamination governance. Its results are reported per suite in Chapter 8 and never pooled; Table C.3 is the single source for each lane’s scope and terminal state.

Two further contributions deserve naming because they are contributions of restraint rather than of result. The first is that the campaign’s negative findings were kept rather than discarded. When BankerToolBench was re-run on the base model under a separate authorisation — 100/100 trials, 3 hours 29 minutes, \$15.07 charged — the mean reward was 0.0 across every trial. The cause was the serving bridge, which passed no stop sequences: generation ran past the end-of-turn token, agents terminated after 2 to 14 steps with no deliverables, and 27 or more trajectories ended in role-token degeneration of the form `user user assistant ....` The zero is therefore retained as an infrastructure finding, not a model result (Chapter 8 §8.5). The second is the discipline applied to the project’s own claims: the same-stack audit that invalidated the pre-audit aggregator’s “disappears” verdict did so on the strength of the study’s own locked minimum detectable effect, and the permitted wording it left in place is narrower than the finding it refuted.

The report does not reallocate the shared runtime code. The inherited repository remains where it was; the Semester 4 internal reports cite their canonical LaTeX sources and

evidence directories rather than duplicating them, and the distinction recorded here is a distinction about academic contribution, not about file ownership.

1.7 Organisation of the Report

The report is organised in four parts.

Part I — Foundations contains the present chapter, followed by Chapter 2, the literature survey, which situates the work against the RLHF and RLVR literature, the critic-free line of estimators, the group-size and advantage-estimation literature, and the reproducibility and reporting literature on which the provenance argument depends; and Chapter 3, the system requirements specification, which states the functional and non-functional requirements the harness, the telemetry layer and the campaign infrastructure were built to satisfy.

Part II — Design and Implementation contains Chapter 4, the proposed methodology, which sets out the measurement design, the statistical protocol, the evidence rules and the study-to-evidence mapping; and Chapter 5, the implementation details, covering the multi-framework harness, the per-step telemetry and raw tensor persistence, the campaign launcher and its receipt and budget governance, and the registry and validator components.

Part III — Results contains three chapters. Chapter 6 reports the core GRPO studies — the cross-library and cross-scale scaling work, the ZVF diagnostic, the group-size and contrast-density sweep, and the length-bias and held-out generalisation study, corresponding to P1 through P4. Chapter 7 reports the reporting, registry and controller work, corresponding to P5 through P7, including the minimum reporting standard, the machine-readable stack registry, and the simulated behaviour of the adaptive group-size controller. Chapter 8 reports the E1–E14 benchmark campaign: the actor, the governance model, the strictly completed scopes, the original-contract scores, the incomplete and blocked lanes, the verification results and the campaign’s terminal states.

Part IV — Synthesis contains Chapter 9, which draws the cross-study synthesis, states the threats to validity, gives the conclusions the evidence will bear, and sets out future work and the reopen conditions under which the blocked lanes could be completed.

Seven appendices follow. Appendix A gives the complete run registry. Appendix B documents reproducibility and provenance, including the inherited-versus-individual boundary and the environment records needed to reproduce the reported results. Appendix C gives the notation, symbols and abbreviations used throughout, and its Table C.3 is the single record of each E1–E14 lane’s scope, status and receipt. Appendix D holds the lane-by-lane campaign detail moved out of Chapter 8. Appendix E lists ev-

ery correction applied to an internal source document. Appendix F reports the applied fraud-detection side study (P8), which uses synthetic data and supervised fine-tuning and so sits outside the RL contribution. Appendix G is the evidence map: the source file behind every claim, section by section. Section 1.8, below, fixes the canonical value of every quantity that the sources report more than once.

1.8 Canonical Numbers

Several quantities in this report were measured more than once, in different regimes, and appear in the sources with different values. Table 1.1 fixes one row per measurement, with the model, task and group size it belongs to, the number of seeds and evaluation items behind it, and the file it comes from. Where the text quotes one of these quantities, this table is the reference, and a value that differs from its row is a different measurement, not a correction. Per-lane E1–E14 figures are kept in Table C.3.

Canonical numbers.

Table 1.1: Canonical numbers. “results/” is `platform_hybrid/experiments/results/`; “sections/”, “archive/” and “neurips_2026_variants/” are under `platform_hybrid/paper/`. The Qwen3-8B GSM8K GRPO training-reward rows use different summaries (last-10 mean, trace mean, pooled) and the sources do not establish that they come from the same run, so they are not interchangeable.

Quantity	Value	Model / task / G	n	Source
Phase-1 batch ZVF	0.72–0.77 (all-correct 0.65–0.71)	Qwen3.5-4B, GSM8K, G = 8; four methods × 40 steps, 16 prompts per step	1 seed	Phase1_Project_ Report_ZVF.tex
ZVF, sampling only (P2 recomputation)	0.130 / 0.190 / 0.155; pooled 0.158	Qwen3-8B, GSM8K test problems, G = 8, T = 1.0, no training	3 seeds × 200 problems	results/tinker_gsm8k_ zvf_s{42,123,456}.json
ZVF by group size	0.838 / 0.764 / 0.691 / 0.631 at G = 2 / 4 / 8 / 16 (fall of 0.207)	Qwen2.5-0.5B, two-digit addition	3 seeds × 40 steps per G	results/groupsize_zvf_ sweep.tsv
ZVF, nine-row mitigation panel (“vanilla GRPO” 0.481)	declared simulation projection, not a measurement	synthetic, K = 8	—	sections/zvf.tex
ZVF vs final held-out outcome	$\rho \approx 0.27$, 95% CI [−0.37, 0.88]	pooled local analysis cells, GSM8K	n = 23 cells	sections/p2_abstract.tex; Table A.5 (P2–C2)
ZVF vs performance, pooled across task types	r = −0.769 (confounded by task type)	5 model families, tool-use and GSM8K	N = 15 experiments	neurips_2026_ variants/paper_P8_ workshop.tex
ZVF vs final accuracy	r ≈ 0.09	instrumented runs	n = 12	sections/appendix_zvf_ formalization.tex
Same-stack PPO vs GRPO	GRPO 0.990, PPO 0.992; paired Δ = −0.002, p = 0.374	Qwen2.5-0.5B, two-digit addition, G = 8 (PPO: 128 prompts per step)	5 paired seeds × 200 fresh draws from the training generator	results/samestack_ppo_ grp.json
Group-size sweep, evaluation accuracy	0.982 / 0.988 / 0.990 / 0.978 at G = 2 / 4 / 8 / 16	Qwen2.5-0.5B, two-digit addition	3 seeds × 200 fresh draws	results/groupsize_zvf_ sweep.tsv

Quantity	Value	Model / task / G	n	Source
GRPO vs Dr. GRPO, arithmetic	0.987 vs 0.992, $p = 0.35$; completions 4.7 tokens	Qwen2.5-0.5B, $G = 8$, 40 steps	5 seeds $\times$ 200	results/drgrepo_vs_grpo.json
GRPO / Dr. GRPO pre $\rightarrow$ post	0.202 $\rightarrow$ 0.263 / 0.205 $\rightarrow$ 0.255	Qwen2.5-1.5B-Instruct, GSM8K-CoT, 200-token cap, $G = 8$	3 seeds $\times$ 200	results/drgrepo_gsm8k_cot.json
Qwen3-8B pre $\rightarrow$ post (GRPO)	82.0% $\rightarrow$ 83.3%, $p = 0.26$	Qwen3-8B-Instruct, GSM8K; 82.0% is the same checkpoint before RL	5 seeds	archive/absorbed/U01_main_compendium/main.tex
Variant audit, DAPO – GRPO	$\Delta = +0.001$, paired-t 95% CI $[-0.00632, +0.00832]$; no BH rejection	Qwen3-8B, GSM8K, five arms	8 seeds $\times$ 500	zvf-program/audit/results/full/statistical_refutation.md
Tinker vs TRL, Qwen3-8B GSM8K	last-10 training reward 0.856 (Tinker, Qwen3-8B-Base) vs 0.050 (TRL, Qwen3-8B Instruct)	GSM8K-500, $G = 8$, 30 steps	1 seed each	results/framework_comparison.json
Qwen3-8B GSM8K GRPO, reproduction headline	last-10 training reward 34.4%, peak 62.5%	Qwen3-8B, LoRA 32, $G = 8$, 30 steps	seed 42	REPRODUCE.md
Qwen3-8B GSM8K GRPO, P1 anchor fit	mean training reward 0.285, peak 0.625	Qwen3-8B, GSM8K	1 seed	sections/scaling_laws.tex
Pooled Tinker vs TRL training reward	99.9% vs 73.4%	Tinker runs pooled over 4B–235B models vs TRL on Qwen2.5-0.5B	pooled	LIMITATIONS_AND_IMPACT.md §6.2
Layer-freeze held-out gain	+0.0625 (full) vs +0.000 (frozen)	Qwen2.5-1.5B-Instruct	2 seeds $\times$ 8 items	results/p1_layerfreeze/FREEZE_FINDINGS.m
Curriculum opening held-out gain	+0.05 in both arms	Qwen3.5-4B, GSM8K, $G = 4$	1 seed $\times$ 20 items	results/curriculum_opening/FINDINGS.md
E1–E14 lane figures	see Table C.3	Qwen3.6-35B-A3B + seed809 adapter	per-lane native denominators	outputs/E1_E14_FINAL_RESULTS_2026-09-19.md

Chapter 2

Literature Survey

This chapter surveys the prior work on which the project is built and the gap it occupies. It revises the survey of the Phase-1 report, retaining that survey’s seven-part structure and updating each part with the material the Semester 4 work added: the P1–P8 internal reports and the E1–E14 evaluation campaign. Material on RLHF, PPO, DPO, GRPO, DeepSeek-R1, parameter-efficient fine-tuning and evaluation benchmarks also draws on the Semester-3 Group 6 capstone literature survey [1], a group work of six authors of whom the present author was one; where this chapter reuses that survey’s findings they are attributed to it here. Two rules of construction carry over from Phase 1. The first concerns proportion. The contribution of this project is overwhelmingly a contribution of measurement and reporting practice rather than of a new training algorithm, so the literature that establishes measurement practice receives more room here than the literature that establishes methods. The second concerns evidence. A figure is quoted only where a repository source actually states it; where a source states a finding without giving a number, the finding is described qualitatively and the chapter says so. Where the project’s own documents state the same quantity differently, the divergence is reported rather than silently reconciled, and Section 2.5 records one such divergence in full.

2.1 From RLHF and PPO to Group-Relative Methods

Policy-gradient post-training of language models begins with the RLHF pipeline. Christiano et al. [2] established the pattern of reward-model training on human pairwise preferences followed by reinforcement learning against that reward model, on Atari and MuJoCo tasks starting from a randomly initialised policy; the supervised fine-tuning stage that precedes both in the language-model pipeline was added in later work [3, 4]. Ziegler et al. [5] adapted the pattern to language, collecting approximately 60,000 comparisons and, following Jaques et al. [6, 7], adopting the per-token KL penalty against a frozen

reference policy that has been standard ever since. Stiennon et al. [3] scaled the recipe to a 6.7B summarisation model with 64,832 comparisons, demonstrated transfer to an out-of-distribution summarisation set (CNN/DailyMail), and produced early empirical evidence of reward over-optimisation: continued optimisation against a frozen reward model eventually degrades the true objective. Ouyang et al. [4] then built InstructGPT, whose three-stage pipeline used roughly 40 contractors to produce about 13,000 supervised fine-tuning prompts, about 33,000 reward-model prompts, and about 31,000 PPO prompts. Two details of that work matter for this project. First, the reward model was 6B rather than 175B because the larger reward model proved unstable during training — a reminder that the reward model is itself a trained component with its own failure modes. Second, the preference data were collected as rankings of $K \in \{4, \dots, 9\}$ responses, which yields $C(K, 2)$ pairwise comparisons from a single annotation batch; this is precisely the within-prompt comparative structure that group-relative methods would later exploit directly. InstructGPT’s headline results were that a 1.3B InstructGPT was preferred to the 175B GPT-3 base model, that the 175B InstructGPT was preferred to GPT-3 in 85% of evaluations, that hallucination on a closed-domain task fell from 41% to 21%, that toxicity was reduced by roughly 25%, and that TruthfulQA performance roughly doubled; inter-annotator agreement was approximately 73%. The same source records four limitations — annotation cost, reward-model over-optimisation, the difficulty of aligning to user intent rather than to annotator preference, and the residual capability costs of alignment (Figure 2.1).

The optimiser for that pipeline was PPO [8]. PPO’s clipping mechanism, with $\varepsilon \approx 0.2$, restricts each update’s policy ratio, and generalised advantage estimation [9] with its standard defaults ($\lambda = 0.95$, $\gamma = 0.99$) provides the advantage estimate. Recasting language generation as an RL problem makes the state the prompt plus generated prefix, the action the next token, the policy the language model, and the reward sparse and terminal. The practical consequence is a four-model architecture — policy, reference, reward, and value — whose mixed-precision memory footprint is roughly four times that of supervised fine-tuning alone; PPO-ptx mitigates capability regression by mixing pretraining gradients into the update. Zheng et al.’s “Secrets of RLHF” study [10] found that policy constraints, not algorithmic choice, are the single most critical factor in whether RLHF succeeds: without them, training exhibits reward hacking, length explosion, and catastrophic pattern collapse. That finding is the intellectual ancestor of the zero-variance analysis in this project, because it locates the failure mode in the constraint structure rather than in the objective.

Group Relative Policy Optimization (GRPO), introduced with DeepSeekMath [11], removes the learned value network altogether. For each prompt it samples a group of G completions, scores them with a reward function, and standardises each completion’s

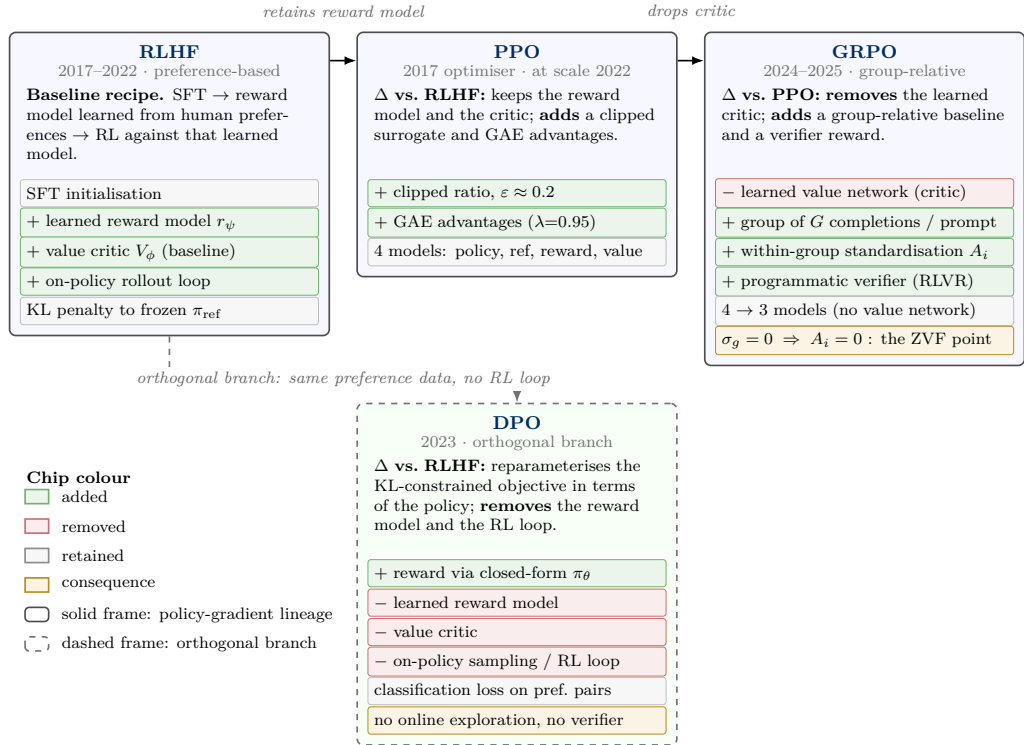


Figure 2.1: Algorithmic lineage from RLHF through PPO to GRPO, with DPO as an orthogonal branch. Each transition removes a component or substitutes a cheaper estimator; GRPO removes the critic and replaces it with a group-relative baseline. DPO (dashed frame) removes the on-policy loop and the reward model rather than simplifying the critic, trading online exploration for stability. Dates are first publication or first use at scale. GRPO variants (DAPO, Dr. GRPO and others) are discussed in the text and not drawn.

reward against the group mean and standard deviation; the resulting z-score serves as the advantage for every token of that completion. In the DeepSeekMath configuration $G = 64$ and $\beta = 0.04$. Eliminating the value model reduces the number of trained and served models from four to three; the paper motivates this as reducing PPO’s memory usage but does not quantify the saving. DeepSeekMath-RL 7B reached 51.7% top-1 accuracy on MATH against a 46.8% supervised fine-tuning baseline, 60.9% with self-consistency over 64 samples, GSM8K improved from 82.9% to 88.2%, and CMATH improved from 84.6% to 88.8% despite no Chinese data in the RL training set. The paper’s iterative variant retains 10% of historical data in a replay buffer to prevent catastrophic forgetting in the co-evolving reward model. RLOO [12] is the closely related leave-one-out estimator that baselines each sample against the mean of the *other* samples in the group; it already performs a within-group baseline, and is therefore directly relevant to the signal-starvation analysis this project conducts.

DeepSeek-R1 [13] then demonstrated that pure outcome-reward RL can elicit chain-of-thought behaviour from a base model without supervised cold-start data. R1-Zero was trained on DeepSeek-V3-Base, a 671B mixture-of-experts model, using rule-based rewards only and no neural reward model. AIME 2024 pass@1 rose from 15.6% to 71.0% over training in the original preprint — the *Nature* version cited here reports 77.9% after extended training — with consensus decoding over 64 samples (cons@64) reaching 86.7% and MATH-500 reaching 95.9%. The source states the training configuration as 16 sampled outputs per question (the group size), a training batch of 512 outputs (32 questions $\times$ 16 samples), a learning rate of 3×10^{-6} , a KL coefficient of 0.001, and approximately 10,400 policy update steps. The paper documents spontaneous chain-of-thought, self-verification, and the widely reported “aha moment” as emergent rather than taught behaviours. The full R1 model, trained with a four-stage pipeline that includes a language-consistency reward, reports 79.8 on AIME, 97.3 on MATH-500, and 71.5 on GPQA, against 79.2 / 96.4 / 75.7 for o1-1217. Distillation into six students from 1.5B to 70B produced Distill-Qwen-1.5B at 28.9% AIME and 83.9% MATH-500, and Distill-Qwen-32B at 72.6% and 94.3%, against 47.0% and 91.6% for Qwen2.5-32B-Zero trained from scratch; the 7B and 16B base models failed to show meaningful reasoning gains from direct RL.

For this project, the significance of that lineage is narrower than the headline numbers suggest. GRPO’s defining operation is a within-group standardisation, and a standardisation is undefined when the group’s rewards are all identical. That degenerate case is not an edge case: it is the modal case early in training on a hard task and late in training on an easy one. The rest of this chapter and the empirical chapters that follow are concerned with what that degeneracy does to the training signal, and with how honestly the field currently reports it.

2.2 Direct Preference Optimisation as a Contrast

An orthogonal branch of post-training removes on-policy sampling entirely. Direct Preference Optimisation (DPO) [14] exploits the fact that the RLHF objective with a KL constraint admits a closed-form optimal policy, which permits the reward to be reparameterised in terms of the policy itself. The result is a simple classification loss over a fixed dataset of preference pairs, with no reward model, no value model, and no sampling loop. Reported savings are 40–60% in both GPU memory and wall-clock time relative to PPO. Four limitations are documented in the same source: the training data are static and offline, so the implicit reward goes stale as the policy drifts; there is no on-line exploration beyond the dataset’s support; the loss can lower the absolute likelihood of preferred completions while widening their margin over dispreferred ones (likelihood displacement); and the method has weaker credit assignment for long-horizon reasoning, where correctness of later steps depends on earlier ones.

The contrast is structurally instructive for this project. DPO cannot suffer zero-variance collapse, because it never standardises within a sampled group — its contrast is supplied by the dataset rather than constructed at training time. But it equally cannot exploit a verifier that is available in the loop, which is the property that makes RLVR attractive on mathematics. GRPO’s zero-variance pathology is therefore the price of its on-policy, verifier-in-the-loop design, and measuring that price is exactly what makes the pathology worth quantifying. The DPO family has since diversified into inverse-preference and non-paired variants — IPO [15], KTO [16], ORPO [17], SimPO [18], and SLiC [19] among them — which the project’s cross-library protocol treats as no-rollout baselines.

The boundary between the two families is also less sharp than it first appears. Wu et al. [20] argue that GRPO is “secretly DPO” — that its efficacy stems from an implicit contrastive objective, which makes it structurally related to DPO — and report that 2-GRPO retains 97.6% of the performance of 16-GRPO at 12.5% of the rollouts (arXiv v3; the earlier v2 reported 98.1%). That reduction is the theoretical counterpart of the PCD statistic this project develops: if a group is exactly as informative as the correct/incorrect pairs it contains, then the pairwise-contrast density $PCD = ((G - 1)/G) \cdot E[p(1 - p)]$ is the natural magnitude-aware summary of a group’s usable signal, and the group-size question becomes a question about how much contrast a given sampling budget buys. The present project is careful about what it inherits from that argument: the reduction is cited as prior theory, while the empirical claims resting on it are scoped to this project’s own measurements, reported in Chapter 6.

2.3 Fixing GRPO’s Biases and Collapse

A dense and fast-growing literature targets specific GRPO pathologies. Dr. GRPO [21] removes two normalisation terms from the standard objective — the per-response $1/L$ factor and the question-difficulty amplification — restoring the sequence-level gradient up to a global scale, and is motivated as a fix for an optimisation bias in GRPO that artificially inflates response length, especially of incorrect responses. DAPO [22], from ByteDance, changes four things in one release: asymmetric Clip-Higher clipping, dynamic sampling that resamples until the batch carries signal, token-level loss normalisation, and over-long filtering; the reported result is 50 points on AIME 2024 with a 32B model. GSPO [23] (Qwen team, 2025) moves the importance ratio from the token level to the sequence level and was credited with stabilising the Qwen3 series, particularly its mixture-of-experts variants. Production-side variants include VAPO [24], a value-based PPO framework for long chain-of-thought reasoning.

A third thread attacks the zero-variance failure mode directly, and this is the thread the project’s own diagnostic sits in. GRESO [25] pre-screens prompts before sampling so that uninformative prompts are never paid for. CPPO [26] prunes completions after rollout, reporting up to $7.98\times$ speedup on GSM8K. NGRPO [27] adds advantage calibration and asymmetric clipping so that all-incorrect groups still carry gradient. Scaf-GRPO [28] injects tiered scaffolding hints when learning stagnates, reporting a 44.3% relative improvement on AIME 2024. MO-GRPO [29] addresses vanilla GRPO’s collapse to a single reward component under multiple objectives through variance-based reward re-weighting. SSPO [30] operates at subsentence granularity to balance GSPO’s stability against token-level GRPO’s expressiveness, raising GRPO’s five-benchmark mathematics average from 43.01 to 46.72 on Qwen2.5-Math-1.5B. RL-ZVP [31] takes the opposite approach to filtering: it extracts signal *from* zero-variance prompts through entropy-guided advantage shaping. AERO [32] instead adapts the rollout allocation and prunes rollouts to avoid zero-advantage dead zones. GVPO [33] re-weights by within-group variance. The tail continues with λ -GRPO [34], MC-GRPO [35] (median-centred advantage), G^2 RPO-A [36], ConSPO [37], and Goldilocks RL [38].

One act of bookkeeping in this literature deserves emphasis, because it is a claim about what may *not* be inferred. AVSPO [39], query recycling, ConSPO, and Goldilocks RL are described in the project’s own related-work source as prior or concurrent interventions, not as evidence for this project’s runner, and the internal report explicitly claims neither priority nor an intervention effect over them. The present thesis inherits that disclaimer without qualification.

Against this background, the project’s own contribution to the area is deliberately meagre in algorithmic ambition and deliberately demanding in evidential standard. The Zero-

Variance Fraction (ZVF) — the fraction of groups whose completions are all correct or all incorrect — was measured across libraries, model scales, and training phases in the companion ZVF report (P2), and that report stopped at description: ZVF tracks collapse, drifts upward, and is mechanically entangled with accuracy and group size, so it declined to promote ZVF to a predictive or causal statistic. A minimal binomial model of group degeneracy was then proposed, in which the expected ZVF indicator is $h_G(p) = p^G + (1 - p)^G$ and the usable signal is $S = p(1 - p)(1 - h_G(p))$; three regularities follow in closed form, each paired with an empirical check drawn from a 368-run Weights & Biases audit that counts all logged runs and is therefore a superset of the benchmark’s 70+ core training runs. Because $h_G(p)$ is symmetric under $p \leftrightarrow 1 - p$, ZVF assigns the same value to a mastered prompt and to a hopeless one, producing the U-shape measured across seven models; and because ZVF is an existence statistic, adding reward jitter of order 10^{-4} drives measured ZVF to zero while shifting the measured within-group contrast by less than 10^{-6} . The pairwise-contrast density is proposed as the magnitude-aware replacement, and the toy-scale validation reports a gradient-norm correlation of $\text{corr} = +0.71$, marked directional only. The word *controller* in that work names the implemented policy, not an established improvement: the decisive static-G = 16 matched-token bakeoff remains unrun.

The one genuinely measured GRPO-variant comparison in the repository is a Dr. GRPO versus GRPO control. On Qwen2.5-0.5B two-digit arithmetic at group size 8, 40 steps, and 5 seeds, both arms were at ceiling and indistinguishable: GRPO 0.987 ± 0.003 against Dr. GRPO 0.992 ± 0.003 (mean $\pm$ SE over 5 seeds; SDs 0.006 and 0.008), paired difference +0.5 percentage points, $t = 1.05$, $df = 4$, $p = 0.35$; mean completion length was 4.74 against 4.71 tokens. At that ceiling, and with completions under five tokens, the task cannot show either an accuracy or a length difference. In a second regime — Qwen2.5-1.5B-Instruct on GSM8K with a 200-token chain-of-thought cap and 3 seeds — GRPO moved from 20.2% to 26.3% and Dr. GRPO from 20.5% to 25.5%; per-seed McNemar tests were significant in 2 of 3 seeds for GRPO ($p = 0.071, 0.036, 0.016$) and 1 of 3 for Dr. GRPO ($p = 0.122, 0.017, 0.122$), and the pooled $p < 10^{-3}$ is expressly rejected as overstating significance; with the seed as the unit, one-sample t-tests on the three per-seed gains give $p = 0.005$ (GRPO) and $p = 0.010$ (Dr. GRPO) (recomputed from `drgrpо_gsm8k_cot.json`). Neither arm inflated length over 30 steps (GRPO 195 $\rightarrow$ 180 tokens; Dr. GRPO 194 $\rightarrow$ 187). The same source contains head-to-head and rank-correlation tables whose values are *projections*, not measurements: the GRPO baseline row is a synthetic baseline produced by a dry run of `platform_hybrid/experiments/variance_mitigation_integration.py`, the file states that it makes no measurement claim for any row of that table, the correlation coefficients (0.78 / 0.66 / 0.63 / 0.31) carry the same caveat, and the configuration proxies under `platform_local/unified/` are explicitly

not faithful reimplementations. Those numbers are reported here only to mark them as unusable as evidence. The nearest thing to a headline cross-library effect in the project is also the one that must be read most carefully: holding the visible group size, learning rate, data, seed, and label fixed while changing stacks moved final GSM8K reward from 85.6% to 5.0%, a $17\times$ swing; the audit that followed found the managed stack had also pinned a different base checkpoint (Qwen3-8B-Base on Tinker against the instruction-tuned Qwen3-8B on TRL), so the source describes this as an under-specification exhibit rather than a backend-only causal effect. The complementary telemetry fact is that running “DAPO” on an open Colab trainer whose dynamic sampling demonstrably filters zero-variance groups produced mean ZVF 0.00, while running “DAPO” on a closed managed stack where only an asymmetric-clip surrogate is user-settable produced mean ZVF 0.58 — the same label, opposite telemetry.

2.4 Verifiable Rewards, Frameworks and Reporting Practice

RLVR — reinforcement learning from verifiable rewards — was formalised for open post-training in TŪLU 3 [40], which is the source this project follows in treating a programmatic checker rather than a learned preference model as the reward. The implementation landscape the project unifies is crowded and heterogeneous. On the training side it includes TRL, OpenRLHF [41], veRL (HybridFlow) [42], NeMo-RL, DeepSpeed-Chat [43], trlX, Axolotl, and Tinker as a managed back-end; each makes different default choices about advantage normalisation, KL estimators, tokenisation loss masking, rollout batching, and importance-sampling granularity, and the project identifies those differences as the principal source of the cross-library gap it quantifies. On the inference side, vLLM [44] and SGLang [45] supply the rollout backends on which modern frameworks depend, with FlashAttention [46] and LoRA [47] as load-bearing kernels. Tinker is adopted here through its `message_with_tool` interface as the neutral substrate for tool-use extensions, and the same source explicitly cites tool-use and agentic RL settings as concurrent environments in which this project’s ZVF measurements will need to be re-evaluated rather than assumed to transfer.

The evaluation benchmarks are equally well established and equally awkward. GSM8K [48] comprises 8,500 problems split 7,473 / 1,319, on which a fine-tuned GPT-3 verifier reached 35% at release while current systems exceed 95%; GSM8K-Platinum [49] has since documented label noise in the original set, which is why the source describes the benchmark as necessary but not sufficient. MATH [50] comprises 12,500 problems across seven subjects and five difficulty levels, on which the original results reached

6.9%; MATH-500 is the 500-problem subset introduced with the Lightman process-supervision work [51], on which current systems sit at roughly 99%. AIME comprises 15 problems per year with integer answers in 0–999; reported AIME 2024 accuracies include GPT-5 at 95.7% and Grok 4 at 94.3%. Because these benchmarks are close to saturation, the source frames chain-of-thought as the action space that makes them interesting at all, and identifies process reward models as the bridge from terminal outcome signals to step-level credit.

Reporting practice is where the project’s own contribution is anchored, and its lineage is explicit. Model Cards [52] introduced documentation of intended use, out-of-scope use, and per-slice evaluation results; Datasheets for Datasets [53] extended the idea to datasets, covering composition, preprocessing, annotation process, and discriminatory-impact mitigation; Data Statements [54] added the speaker-demographic axis; and Data Cards [55] bridged research and industry practice with schema, provenance, sampling-distribution, and audit-transparency fields. A cross-coupling audit at iteration 109 maps these four works onto the project’s legacy 18-field experimental manifest, showing that they cover 12 distinct fields, that the remaining six are either RL-specific (per-step ZVF, group-size schedule) or stack-coverage items with no direct analogue in the prior literature, and that items 9–12 — annotation process and annotator demographics — have the lowest evidence density in the live manifests. The same iteration verified all four entries against CrossRef metadata (4/4 pass on year, five-gram title, and author-family overlap ≥ 0.6) and retrofitted two entries that had been present in the bibliography but never cited. The resulting standard, MIN-REPORT-RL, has eight items: seven run-manifest fields and one evaluation item (held-out pass@k), which belongs to the evaluation report rather than to run-start stack identity. The count was aligned at eight on 2026-07-11, and this thesis uses “the eight-item standard” and “the seven manifest fields” in that sense throughout.

The reproducibility literature to which this standard is a deliberate echo is older and better established. Henderson et al. [56] showed that deep-RL results depend on codebase, hyperparameters, and even the choice of random seeds, to the point that the same algorithm from two repositories behaves as two algorithms. Colas et al. [57] and Agarwal et al. [58] (`rliable`) supplied the statistical repair kit — rigorous comparison protocols and small-sample-aware aggregate metrics with interquartile-mean reporting and performance profiles. Jordan et al. [59] argued that rigorous benchmarking in RL is computationally prohibitive at the scale it would require, and called for an additional experimentation paradigm to complement it. Dodge et al. [60] proposed reporting expected validation performance as a function of compute budget; the project’s run manifest is the RL-for-LLM analogue, replacing *budget* with *stack*. Pineau et al. [61] formalised the NeurIPS reproducibility criteria, and the project draws the explicit lesson that self-

attested checklists have limits, which motivates an artifact-computed badge rather than a declared one. Statistical treatments of evaluation noise (Miller [62]; Zhang et al.) quantify the uncertainty floor against which the project’s own within-noise findings should be read, and the evaluation-stack sensitivity documented by the LM Evaluation Harness effort [63], by audits of reasoning evaluations [64], and by contamination studies of the data side of the same coin, together establish that RL *training* inherits all of this — the verifier is an evaluator in the loop — plus the additional vectors unique to on-policy optimisation.

Three existing resources come closest to the project’s registry and are the reason it exists in its present form. HELM [65] aggregates evaluation across a large model and scenario matrix, but its unit is the (model, scenario) cell, not the training stack. The LM Evaluation Harness [63] standardises evaluation by *executing* implementations, whereas the registry here standardises by *describing* stacks it cannot execute — many of them closed — which is precisely why the project required a null-versus-absent convention and an R5 rung that records un-auditability rather than converting it into a false value. Papers with Code catalogued paper-code-leaderboard linkage at scale, stopped at the repository boundary, and was discontinued in July 2025; the source treats its fate as a governance cautionary tale. The project’s own claim to novelty is narrow and stated negatively: no existing resource provides the executed loss form at GRPO-variant granularity, per-component implementation status, or an unauditability marker.

2.5 Scaling and Layer-Adaptation Context

Compute-optimal pretraining is governed by well-characterised scaling laws (Kaplan et al. [66]; Hoffmann et al., Chinchilla [67]), but RL post-training is much less well characterised, and the sources the project builds on separate cleanly into those that predict and those that describe. Hilton et al. [68] derived compute-efficiency frontiers for single-agent RL across Procgen games, Dota 2 and a toy MNIST-based task. For RLHF, Gao et al. [69] characterised the over-optimisation curve of reward models — the point at which additional optimisation against a frozen reward model buys a decreasing true objective — and Rafailov et al. [70] extended over-optimisation scaling laws to direct alignment algorithms (DPO, IPO and SLiC), which have no separate proxy reward model. Nimmatouri et al. [71] derived an empirical scaling law specifically for GRPO, identifying three training phases (slow start, rapid improvement, plateau) and reporting that most of the benefit is exhausted by roughly 80% of a single epoch. Tan et al. [72] treat group size as a budget knob in an empirical scaling study of GRPO post-training, in an appendix analysis showing that the optimal rollout size shifts with the training budget. Snell et al. [73] treat inference-time compute as a scaling axis in its own right, and the project’s

own related-work source cites these as the works against which its measurements should be read.

The project’s scaling study covers 70+ runs across seven RL libraries and five model families. This thesis uses one model range throughout: 0.6B–1T, with the anchors above 235B single-seed and descriptive. The rule is that the range spans every model with at least one completed run in the P1 roster. The source documents state narrower ranges because each covers a subset: 0.6B–~671B in the P2, P3 and P4 introductions, and 0.6B–235B for the multi-family data in the related-work source. The principal empirical result of the scaling work is a null: across that range, no reliable scaling law is recovered from reward curves. The companion group-size study adds a second bounded negative: a three-seed sweep does not establish an optimum, since the measured arms are statistically indistinguishable, and the measured signal-to-noise ratio grows at only about 52% of the $\sqrt{G}$ ideal, with $G \in \{2, 4, 8, 16\}$ measured and $G \in \{4, 8, 16, 32, 64\}$ explicitly reconstructed rather than run.

Layer adaptation supplies the second half of this section’s context, and connects the project’s white-box investigation to the parameter-efficient fine-tuning literature. LoRA [47] freezes the pretrained weights and injects trainable low-rank matrices into the attention projections, so that a $d \times d$ projection at $d = 4096$ with rank $r = 16$ trains $2dr$ rather than d^2 parameters, a 128 $\times$ reduction (this chapter’s arithmetic; the paper itself reports up to 10,000 $\times$ fewer trainable parameters for GPT-3 175B). QLoRA [74] extends this by quantising the frozen base to NF4 and adding double quantisation, which saves about 0.37 bits per parameter — approximately 3 GB for a 65B model — and permits 65B fine-tuning on a single 48 GB GPU. The motivating arithmetic is that Adam at mixed precision costs roughly 16N bytes, exceeding 100 GB for a 7B model. Prefix tuning, prompt tuning, and adapter layers are the earlier members of the same family, and LoRA’s advantages for RL specifically are that the frozen base doubles as the reference policy and that the trainable parameter count is small enough for modest hardware. The project’s layer-adaptation pillar asks a narrower and orthogonal question: where inside the network GRPO-style adaptation concentrates, and whether the layers that will matter can be identified early enough to freeze the rest. The reported finding is moderate concentration of per-layer adaptation, with the important negative that step-1 predictability does *not* survive scaling. This connects directly to Mukherjee et al. [75], who found that RL fine-tuning updates only 5–30% of model parameters across seven algorithms and ten model families, suggesting that RL activates a sparse subnetwork. The length-bias pillar completes the scaling picture with its own bounded null: under a 200-token GSM8K generation cap, GRPO and Dr. GRPO produce held-out gains that are indistinguishable at the project’s seed budget, and neither arm shows length inflation; the source is careful to state that the 200-token cap is part of the estimand rather than a nui-

sance detail, so the null bounds the capped, short-horizon regime and cannot adjudicate uncapped reasoning or long agent trajectories.

2.6 Process-, Tree- and Stability-Aware Variants

Beyond the group-baseline fixes, a parallel line of work changes *what* signal drives the update. Process reward models supply denser, step-level feedback instead of a single terminal reward, which reduces the chance that a whole group collapses to identical terminal rewards. Let’s Verify Step by Step [51] showed that step-level PRMs substantially outperform outcome reward models on MATH; Math-Shepherd [76] derives process rewards from automatically generated process annotations, avoiding costly human labels; PRM800K [51] and PRMBench [77] supply step-level evaluation data; OmegaPRM [78] uses Monte-Carlo tree search to supervise process rewards without human-in-the-loop; ReasonEval [79] scores reasoning steps for validity and redundancy with trained LLM evaluators; and Implicit PRM [80] learns process rewards implicitly from outcome labels, with VersaPRM [81] extending process rewards to multi-domain tasks. Tree-of-thought search [82], self-consistency [83], and critic-style post-hoc verifiers all either benefit from or substitute for process supervision. The same source documents the counter-current that matters most here: VerifyBench and “A Sober Look” [64] warn that reward hacking and evaluation artefacts can inflate perceived gains on reasoning benchmarks, which is the project’s stated motivation for insisting on binary verifiable rewards in its own mathematics work.

Stability-aware variants constrain the update itself. GSPO’s sequence-level importance ratio, discussed above, is the clearest instance, credited with stabilising the Qwen3 mixture-of-experts series. SSPO’s subsentence-granularity objective is explicitly framed as balancing GSPO’s stability against token-level GRPO’s expressiveness. Entropy-guided advantage shaping in RL-ZVP regulates the update when the group carries no reward contrast, and GRESO’s pre-screening is a sampling-side stability control. On the reasoning-RL side, GRPO-Lead [84] and LCPO [85] tune reasoning length through length-aware rewards, LIMR [86] selects a small high-impact subset of RL training data, Search-R1 [87] trains reasoning agents to interleave search with chain-of-thought, and TTRL [88] performs test-time RL on unlabelled data using majority-vote rewards.

The Phase-1 survey’s judgement on this body of work is retained without amendment, because the Semester 4 evidence has not changed it. These methods propose *levers*; this project first builds the *instrumentation* needed to tell whether any such lever converts wasted rollouts into held-out accuracy — ZVF, group utilisation, pairwise-contrast

density, and the collapse decomposition. The nulls reported in the results chapters are exactly the kind of evidence that instrumentation is designed to produce, and they constitute a standing caution against adopting a variant on the strength of a single-seed improvement. The project applies that caution to its own proposals: the adaptive group-size controller is described as an implemented policy rather than an established improvement, and its decisive matched-token comparison remains unrun.

2.7 Positioning the Portfolio

The portfolio’s positioning against prior art is a per-pillar statement of what is claimed and, more importantly, what is not. Table 2.1 reproduces the Phase-1 positioning and updates it with the Semester 4 evidence (Figure 2.2).

Positioning of the P1–P8 studies against closest prior art.

Table 2.1: Positioning of the eight portfolio studies (P1–P8) against closest prior art and the narrowed axis each claims.

Pillar	Closest prior art	Narrowed axis claimed
P1 (scaling)	whole-model scaling and saturation laws; Mukherjee et al. [75] on sparse RL subnetworks	Per-layer adaptation is moderately concentrated; step-1 predictability does not survive scaling; no reliable reward-curve scaling law recovered
P2 (ZVF)	DAPO dynamic sampling, GVPO, AERO, RL-ZVP, GRESO, CPPO, NGRPO	<i>Measure</i> ZVF as an existence statistic; show it is entangled with accuracy and group size; propose PCD as the magnitude-aware replacement

Pillar	Closest prior art	Narrowed axis claimed
P3 (group size)	integer-G sweeps; MC-GRPO; Tan et al. [72] budget-knob treatment	Trainability is non-monotone in G ; a three-seed sweep finds its arms statistically indistinguishable and does not establish an optimum; measured SNR grows at $\sim 52\%$ of the $\sqrt{G}$ ideal
P4 (length bias)	Dr. GRPO and λ -GRPO length fixes	A bounded null: under a 200-token cap, GRPO and Dr. GRPO held-out gains are indistinguishable with no length inflation
P5 (reporting standard)	Datasheets, Model Cards, Data Statements, Data Cards	An eight-item RL-run reporting standard (seven manifest fields and one evaluation item) targeting the stack rather than the label
P6 (registry)	static run-config registries; HELM; LM Evaluation Harness; Papers with Code	Machine-readable stack records with per-component implementation status and an unauditability marker
P7 (controller)	adaptive training-signal control (G^2 RPO-A adaptive guidance [36]); DAPO dynamic sampling	Closed-form $h_G(p)$ prediction of ZVF, with a controller described as implemented, not established

Pillar	Closest prior art	Narrowed axis claimed
P8 (fraud study)	tuned gradient-boosted trees as the tabular fraud-detection baseline	A measurement-discipline side study comparing a fine-tuned LLM with a gradient-boosted tree on synthetic credit-card fraud, with no RL claim (Appendix F); P8 was archived and absorbed into the portfolio

The two crowded pillars are P2 and P4, where the literature is dense and the project accordingly narrows its claim to measurement rather than to a new algorithm. The widest open ground remains the systems and reporting side, where no GRPO-specific machine-readable reporting standard exists — the gap Section 2.4 establishes from three directions: HELM’s unit is the (model, scenario) cell rather than the training stack, the LM Evaluation Harness standardises evaluation by executing code while the registry must describe stacks it cannot execute, and Papers with Code stopped at the repository boundary and was discontinued in July 2025.

The Semester 4 evaluation campaign extends this positioning from method comparison to benchmark practice. Rather than adding one more leaderboard number, it evaluates a single trained actor against fourteen held-out agentic and reasoning suites, each through its own native grader at a pinned revision, under receipt, budget and decontamination controls, and it reports every lane’s terminal state rather than a cross-suite aggregate. Its results are in Chapter 8; Table C.3 gives each lane’s scope and status. One finding bears directly on the benchmark-practice literature: E4’s mean reward of 0.0 across 100 base-model trials was caused not by the model but by a serving bridge that passed no stop sequences (Section 8.5), which is exactly the kind of unreported stack detail this positioning argues must be recorded.

Two gaps in this survey should be stated plainly, since the chapter’s purpose is to locate the work rather than to flatter it. First, the primary papers for the fourteen evaluation suites are not surveyed here at the depth the algorithmic literature receives; the suites are discussed as instruments, sourced to the campaign’s own results document, and their native protocols are documented in Chapter 8 and its receipt trail rather than in this chapter. Second, the survey’s treatment of tool-use and agentic RL is inherited from the project’s own related-work source and is explicitly flagged there as a concurrent setting in which the ZVF measurements will need to be re-evaluated rather than assumed to transfer.

The eight-study portfolio: the question each study asks		
grouped by theme; the question is the study's own and the status is the state of the evidence in this thesis		
Scaling <i>does the recipe follow a law?</i> P1 · P3	P1 Scaling laws for GRPO post-training <i>Does held-out GRPO gain follow a predictable cross-scale slope, or does the recipe resist a scaling law?</i> complete · null recovered: no reliable cross-scale slope, and no identifiable saturation exponent	P3 Group size, contrast, DPO bridge <i>How do collapse, reward, held-out gain and token cost move with group size G — and what does G buy?</i> complete · under saturation, retention $G=2$ vs. $G=16$ is 1.00; SNR is 52% of the $\sqrt{G}$ ideal
Diagnostic <i>what is the signal doing?</i> P2	P2 The Zero-Variance Fraction as a descriptive diagnostic <i>What fraction of each rollout batch supplies no reward-relative gradient, and can a collapsed group be re-baselined?</i> complete · recomputable from stored reward tensors (0.130, 0.190, 0.155; pooled 0.158); link to outcome $\rho \approx 0.27$	
Mechanism <i>why does the signal vanish?</i> P4 · P7	P4 Length bias and held-out generalisation <i>Do length penalties change how long a model answers — and does that change held-out accuracy at all?</i> complete · held-out gains indistinguishable; pooled length 194.4 → 183.5 tokens under a 200-token cap	P7 From diagnostic to controller <i>Can the ZVF diagnostic be closed into a controller that adapts group size from measured starvation?</i> partial · model and 368-run audit complete; the controller is not joined to P5
Reporting <i>what must be stated?</i> P5 · P6	P5 Report the stack, not the label <i>What minimum set of fields must a stack-conditioned result report before it can be attributed at all?</i> complete · flagship: the seven-field minimum-report manifest; badge 43–96 over twenty seed entries	P6 GRPO-Registry: a stack catalogue <i>Can evolving run state be recorded as a machine-readable registry, and diffed to predict a label flip?</i> complete · stackdiff flags the open-vs-closed DAPO pair as an R5 label-flip risk from manifests alone
Applied <i>does it transfer?</i> P8	P8 LLM vs. gradient-boosted trees in credit-card fraud <i>When two models are measured on different splits, what can the comparison actually support?</i> partial · archived side study absorbed as a measurement-discipline case study; the two AUCs are not like-for-like	
complete — the study answers its question, including when the answer is a null partial — scope narrowed or a limb deliberately left unclaimed		
Reading the map		
Established - The ZVF is measurable from stored reward tensors and reproduces: 0.130, 0.190, 0.155. - Under saturation, retention is flat in G: $G=2$ and $G=16$ retain identically. - A seven-field manifest alone is enough to flag a label-flip risk.	Declared null - P1 recovers no reliable cross-scale slope and no identifiable saturation exponent. - P4 finds GRPO and Dr. GRPO held-out gains indistinguishable. - Its error-bar audit returns three null verdicts and four decisive, none suggestive.	Not claimed - P7's controller is audited but not joined to P5's attribution programme. - P8's two AUCs are not a like-for-like ranking: the splits differ by construction. - P2's diagnostic-to-outcome association is real but weak.

Figure 2.2: The eight studies P1–P8 grouped by theme, with the question each one addresses and the state of its evidence. Every question is answered from the repository's own record; two of the eight answer with a declared null, and a study whose scope narrowed between the group semester and this one is marked partial rather than complete.

What the chapter does establish is that the project's contribution is a measurement and reporting contribution, that the literature it sits in is dense enough that claim-narrowing rather than claim-extension is the appropriate move, and that the specific gap the portfolio occupies — machine-readable, per-component, executed-loss-form disclosure for group-relative RL runs — is not filled by any of the resources surveyed above.

Chapter 3

System Requirements Specification

This chapter states what the Tinker RL Lab system must do, what qualities it must hold while doing it, and how each requirement is verified. It inherits the specification of the Phase-1 report and extends it across the Semester-4 additions: the multi-framework benchmark rosters, the Zero-Variance Fraction (ZVF) diagnostic and the adaptive group-size controller derived from it, the minimum reporting standard and the machine-readable GRPO stack registry, and the E1–E14 evaluation campaign with its fail-closed completion gate. It is written as a specification rather than as a retrospective description, but every requirement below corresponds to a component that exists in the repository, and every functional requirement is paired with the concrete test that checks it. Where a requirement is not satisfied — the cloud capacity for two evaluation lanes, the white-box gradient path at the largest model scale — the shortfall is named in §3.6 and §3.8 rather than written around.

3.1 Overview and Actors

The central object of the Phase-1 specification was the **run**: one configuration, one seed, one framework, one training budget, producing per-step telemetry and a persisted set of raw group reward tensors. Semester 4 introduces two further first-class objects. The first is the **lane**: a single benchmark suite evaluated under a named scope — an original contract with the suite’s owner, or a declared replacement scope — carrying its own provider, payload, licence, quota and terminal state. The second is the **receipt**: an immutable, hash-bound record that a declared step actually happened, against declared inputs, at a declared revision. Requirements that concern runs, lanes and receipts are distinguished below where the distinction matters.

Four actors interact with the system, and the fourth is new in Semester 4.

The experimenter declares work. Under the original design this meant declaring a run through a single configuration object and nothing else: the model, the dataset, the reward function, the decoding parameters (temperature, top- p , maximum tokens), the group size G , the learning rate, the KL coefficient β and the seed all live in one file, and the framework is a switch rather than a rewrite. Under the campaign design the same actor declares a lane through a sealed resource request: the lane specification fixes an immutable task bundle, a pinned native runtime, a native grader, a budget and an applicable licence in advance, and those declared inputs are then checked for internal consistency before any launch is permitted.

The back-end executes the training loop and, under the managed path, is the Tinker service; under the white-box path it is a local or rented GPU. For evaluation, the corresponding actor is the **provider grader**: the benchmark owner’s own native evaluator at a pinned revision. The system is permitted to record a suite score only when that evaluator produced it, or when a declared replacement harness reproduces it under a scope that is labelled as a replacement.

The auditor is a downstream script or an independent reader who recomputes published diagnostics from stored raw artefacts. In Semester 4 this role acquires force: the audit suite of §3.4 is executed as a gate with a non-zero exit status rather than as advisory tooling, and the completion gate for the campaign lanes is fail-closed by construction, in that a missing or malformed declaration stops the procedure instead of degrading it.

The campaign operator holds launch authority for paid work. Spend on any lane requires a bound authorization receipt issued by the operator; the system will not mint one for itself, and lanes are recorded as blocked when the receipt or the technical chain behind it is absent.

A fifth party, the **external access holder**, is an actor only in the negative sense: several lanes depend on a private payload, a hosted evaluation service or a quota grant that only the provider can issue. The specification requires that such a lane be recorded with an explicit terminal state and a named reopen condition rather than silently dropped, which is why the campaign’s terminal-state vocabulary exists as a first-class output.

3.2 Functional Requirements

Table 3.1 lists the functional requirements. Each row states the requirement and names the source that defines it; §3.5 maps each to the artefact that implements it and the test that verifies it.

Functional requirements.

Table 3.1: Functional requirements. Sources: the Phase-1 report is `platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex`; other files are under `platform_hybrid/`, `platform_local/` or `zvf-program/flagship/`.

ID	Requirement	Source
FR-1	One declarative file fully determines a run (model, dataset, reward, decoding, G, learning rate, KL coefficient, seed, output); no framework-specific parameters are needed on any supported back-end	Phase-1 report

ID	Requirement	Source
FR-2	A configuration is executable on multiple frameworks through thin adapters that share the reward grader and decoding configuration. Two rosters are in scope and are kept apart: the cross-RL-library roster (TRL, Stable-Baselines3, CleanRL, Tianshou, PufferLib, rl_games, d3rlpy) and the cross-launcher LLM-RL roster (Tinker, TRL, SkyRL, veRL, OpenRLHF, Atropos, HF reference launchers). Completed-run parity is not met: only Tinker and TRL produced completed runs; the veRL and OpenRLHF entries are dry-run placeholders	<code>_shared_methods.tex</code>; <code>framework_comparison.json</code>
FR-3	Every step computes ZVF and its all-correct / all-wrong decomposition from the reward tensors	Phase-1 report

ID	Requirement	Source
FR-4	Every step emits a structured telemetry record (reward, ZVF, gradient utilisation, collapse counts, entropy, completion length, KL) to a log and to the tracker; the controller work adds a per-step counterfactual against a matched fixed-G control	Phase-1 report; controller_cf_per_step.tsv
FR-5	Raw per-group reward tensors are persisted for every run, so diagnostics can be recomputed	Phase-1 report
FR-6	Sweeps over G, seed and baseline-versus-intervention arms run under a fixed wall-clock or token budget	Phase-1 report
FR-7	For single-GPU models, per-layer LoRA gradient norms are recorded; not required at managed scale, where they are not exposed	Phase-1 report
FR-8	Every run emits a provenance record binding configuration, grader version and rollout hashes; a campaign lane additionally declares its provider package and grant documents against published schema versions, each with a 64-character hash	Phase-1 report; e1_e14_completion_gate.py

ID	Requirement	Source
FR-9	A stack-conditioned result carries the seven manifest fields of the eight-item standard (loss form, reference KL, sampler/backend with base-checkpoint revision and hash, telemetry, group-size schedule, held-out split, decontamination) plus its eighth, evaluation item (held-out pass@k); a 0–100 badge is emitted, and an unreported field is distinguished from one reported as absent	<code>paper_P6_registry.tex</code> ; <code>minreport_audit_summary.json</code>
FR-10	The stack registry is queryable by stack, field or status	<code>registry/schema.json</code> ; <code>registry/query.py</code>
FR-11	Each named variant stores an explicit delta against the GRPO reference, and <code>stackdiff</code> returns an R0–R5 flip-risk verdict for same-label entries	<code>paper_P6_registry.tex</code> ; <code>registry/provenance/</code>
FR-12	An adaptive group-size controller with escalation asymmetry and hysteresis records its counterfactual against the best static recipe at matched and unequal rollout counts; it measures the trade and does not claim a win	<code>paper_P7_zvf_controller.tex</code> ; <code>controller_conf_summary.json</code>

ID	Requirement	Source
FR-13	Each lane records results against a declared evidence class (exact complete, partial exact, partial recovery, externally blocked, local-setup-ready-provider-input-required) tied to the source status	e1_e14_completion_gate.py
FR-14	The completion gate validates, and never executes, a lane package; a valid package proves only that its inputs are consistent and available; signatures are checked against provider trust roots, and the gate refuses rather than warns	e1_e14_completion_gate.py
FR-15	Original-contract and replacement-scope results are never pooled, no cross-suite aggregate is computed, and every accuracy carries its denominator	E1_E14_FINAL_RESULTS_2026-09-19.md
FR-16	The export path builds an anonymised review package whose build runs the audit suite, and refuses a package built with audits skipped	run_all_audits.py; export_guard_audit.py

3.3 Non-Functional Requirements

Non-functional requirements.

Table 3.2: Non-functional requirements.

ID	Requirement	Source
NFR-1 Attribution	Comparative claims hold the stack fixed except the factor under test; confounded factors are reported as confounded	<code>_shared_methods.tex</code>
NFR-2 Reproducibility	Multi-arm claims rest on at least three seeds; single-seed results are descriptive only	Phase-1 report
NFR-3 Recomputability	Every derived diagnostic is reproducible from stored raw artefacts, independent of logged summaries	Phase-1 report
NFR-4 Denominator discipline	Held-out figures carry their n; negative, null and underpowered results are reported as such	<code>main_eai_body.tex</code>
NFR-5 Isolation	Concurrent experiments run in separate processes, not threads (after the tracker thread-safety defect)	Phase-1 report
NFR-6 Budget	Spend is recorded per run against a declared envelope; per-run bounds and lead-issued authorisation receipts apply even under an unlimited total	<code>Pending_Experiments.md</code>
NFR-7 Fail-closed	Validation defaults to refusal; an unverifiable package is not scored	<code>e1_e14_completion_gate.py</code>

ID	Requirement	Source
NFR-8 Least privilege	Evidence records hold no key material; credential checks record only liveness and scope	Pending_Experiments.md
NFR-9 Lost-source auditability	Execution sources for sealed artefacts live in tracked paths; sealed requests and receipts suffice to pin a faithful re-implementation	E1_E14_FINAL_RESULTS_2026-09-19.md
NFR-10 External traceability	External payloads are bound by hash and validated against trust roots and schema versions; an unreported field is null, not zero or false	e1_e14_completion_gate.py; paper_P6_registry.tex

3.4 Test-Case Specification

The test cases below are the concrete checks attached to the requirements. TC-1 through TC-5 are inherited from the Phase-1 specification and are restated here because later chapters depend on them; TC-6 onward are the Semester-4 additions, and TC-6 is the suite that carries the largest share of the functional requirements.

TC-1 — Recompute ZVF and the collapse decomposition from stored tensors. Recompute the published ZVF and collapse figures from the persisted per-group reward tensors of the main configuration, using the documented analysis script. The published values are a ZVF of approximately 0.72–0.77 and an all-correct fraction of 0.65–0.71; recomputation must match. Verifies FR-3 and FR-5.

TC-2 — Recompute the detector metric from raw tensors. Recompute the Phase-3 detector’s area under the ROC curve from raw tensors under five-fold stratified cross-validation. The published figure is an AUROC of 0.84 ± 0.01 against 0.43 for the reward-only comparator. Verifies FR-5 and NFR-3.

TC-3 — Re-run a matched-budget arm comparison. Re-run the baseline and the curriculum arm at a matched token budget over three seeds; the Phase-1 result is that

both arms average +0.028 on the held-out set with no curriculum advantage. Verifies FR-1, FR-2 and FR-6.

TC-4 — Recompute headline diagnostics. Recompute each headline diagnostic quoted in a results chapter directly from stored artefacts, with the recomputation recorded next to the published value. Verifies NFR-3.

TC-5 — Provenance verification. Verify a run’s provenance record end to end, from configuration to rollout hashes to the grader version. Verifies FR-8.

TC-6 — Execute the submission audit suite as a gate. Run `python platform_local/run_all_audits.py`. The suite executes nine audits — `claim_issues`, `sync_issues`, `anon_issues`, `strength_issues`, `package_issues`, `workflow_issues`, `export_guard_issues`, `caveat_issues` and `scientific_issues` — emits one METRIC `<name>=<count>` line per audit, and returns exit status 0 only when every audit reports zero issues. Individual checks that this test enforces include the presence of scope and caveat language in the review documents (`readme_missing_key_results_scope_header`, `readme_missing_heldout_language`, `checklist_missing_preliminary_key_results_label`), the absence of overstated checkpoint claims (`misleading_checkpoint_availability_claim`, `submission_overstates_checkpoint_release`), the presence of every reviewer caveat (`heldout_scope`, `tool_eval_protocol`, `codegen_subset`, `capacity_confound`, `moe_diagnostics`, `budget_and_splits`, `replication_release`), the absence of banned marketing formulations, the survival of the technical caveats into the anonymous build (`anonymous_missing:<label>` for the seven needles), and the presence of the audit call and its skip flag in the export script. Verifies FR-16, NFR-4 and NFR-7.

TC-7 — Structural checks on the evaluation script and its result files. For each held-out evaluation script, assert by parsing the source — not by reading its documentation — that it exposes a seed argument, locks the dataset split to the test split, samples with a non-zero temperature, consumes a checkpoint path, and writes the declared metadata keys including the split, the sampling flag, the seed and the model source. For each result file, assert that the schema version is 2, that the evaluation status is one of completed or failed, that the required configuration and summary fields are present, that the recorded split is the test split, that the attempted count equals the sum of correct and incorrect, and that a failed evaluation carries a failure reason and a completed evaluation carries non-zero attempts. Verifies FR-8 and NFR-4.

TC-8 — Build the report from source. Compile the main report and its supplementary through the full LaTeX chain and flag an empty journal field in the bibliography. Verifies FR-16.

TC-9 — Dry-run the completion gate against a declared lane. Present a provider

package to the gate and require it to validate the declaration without executing anything: the expected outcomes are acceptance with the declared evidence class, or a specific refusal naming the missing element. A package must never be reported as a benchmark result by this procedure. Verifies FR-13, FR-14 and NFR-7.

TC-10 — Registry query and null discipline. Query the registry for a labelled entry and its delta record, and confirm that a field absent from the source is returned as unreported rather than as a zero value, and that a same-label pair with divergent behaviour triggers the flip-risk verdict. Verifies FR-9, FR-10, FR-11 and NFR-10.

3.5 Requirements Traceability Matrix

Requirements traceability matrix.

Table 3.3: Requirements traceability matrix: each functional requirement mapped to the artefacts that implement it and the audit check that verifies it.

Requirement	Implemented in	Verified by
FR-1 Configuration object	platform_local/unified/launcher.py; run configurations under platform_hybrid/experiments/	TC-3
FR-2 Back-end-agnostic execution	platform_local/trl_integrations/; platform_hybrid/experiments/results/framework_comparison.json	TC-3; parity shortfall recorded in §3.2
FR-3 Per-step ZVF	Training-loop telemetry (Phase-1 design)	TC-1
FR-4 Per-step telemetry	Telemetry writer; platform_hybrid/experiments/results/p5p8/controller_cf_per_step.tv	TC-1, TC-4
FR-5 Raw tensor persistence	Persisted per-group reward tensors	TC-1, TC-2
FR-6 Sweeps	Sweep runner	TC-3

Requirement	Implemented in	Verified by
FR-7 White-box gradients	Single-GPU training path	Not automatable in the audit suite; checks in Chapter 5 §5.5 (per-layer gradient profile)
FR-8 Provenance record	zvf-program/flagship/e1_e14_completion_gate.py; run provenance records	TC-5, TC-7, TC-9
FR-9 Minimum-report manifest	platform_hybrid/experiments/results/p5p8/minreport_audit_summary.json	TC-10
FR-10 Registry query	platform_hybrid/registry/schema.json, query.py, entries/	TC-10
FR-11 Variant deltas, stack-diff	platform_hybrid/registry/provenance/	TC-10
FR-12 Adaptive controller	platform_hybrid/experiments/results/p5p8/controller_cf_summary.json	TC-4
FR-13 Lane harness	zvf-program/e1_wave10/, zvf-program/e5_successor27/, zvf-program/e2_core/, zvf-program/e13_balrog/, zvf-program/e6e9/check_quota_status.py; runners actually used on 2026-09-27: outputs/finish_pending_2026-09-27/<lane>/code/	TC-9
FR-14 Completion gate	zvf-program/flagship/e1_e14_completion_gate.py	TC-9

Requirement	Implemented in	Verified by
FR-15 Scope separation	outputs/E1_E14_FINAL_RESULTS_2026-09-19. md	TC-6, TC-9
FR-16 Blind-review packaging	platform_local/run_all_audits.py, export_guard_audit.py	TC-6, TC-8
NFR-1 Attribution	Method sections and comparison artefacts	TC-3, TC-10
NFR-2 Reproducibility	Seed management in the runner	TC-3
NFR-3 Recomputability	Documented analysis procedures	TC-1, TC-2, TC-4
NFR-4 Denominator discipline	Caveat and claim-strength checks	TC-6, TC-7
NFR-5 Isolation	Process-level execution	Operationally checked; not in the audit suite
NFR-6 Budget	Spend receipts	TC-9
NFR-7 Fail-closed behaviour	Gate constants and suite exit status	TC-6, TC-9
NFR-8 Least privilege	Credential-gate receipts	TC-9
NFR-9 Auditability	Tracked execution paths	TC-9
NFR-10 External traceability	Hash equality and trust-root validation	TC-9, TC-10

Two rows deserve comment. FR-7 has no automated check because the white-box gradient path is a measurement procedure rather than an invariant; its evidence is the result data itself. NFR-5 is likewise checked operationally rather than by the suite, because process isolation is a property of how runs are launched and is not visible in any post-hoc artefact.

3.6 Hardware Requirements

Hardware requirements and their status.

Table 3.4: Hardware requirements and their status.

Compute surface	Requirement	Status
Managed training back-end (Tinker)	Sampling, training and checkpointing for the ~35B-total, ~3B-active MoE actor in bfloat16 with a pinned LoRA; per-layer gradients are not exposed, which is why FR-7 is scoped to small models	Met
White-box GPU	One CUDA device with at least 24 GB (an L4-class cloud notebook), enough for the 1.5B configuration and a reduced-length 3B re-test	Met
Serverless GPU (Modal)	Large scoring sweeps and the hosted sampler bridge; the bridge is a precondition for the lanes that use it	Met
Evaluation-lane cloud capacity	E9 needs a documented minimum of 4 vCPUs (8 requested) and E6 needs 16, against a live quota of 1 in each region, with increase requests open; a local E9 container build needs at least 30 GiB of free disk against 31 GiB available; E2 needs a GCP binding that only the project owner can create	Not met at 2026-09-21
Orchestration workstation	Orchestration, analysis, figures, the audit suite and the report toolchain	Met

Sources: *outputs/E1_E14_FINAL_RESULTS_2026-09-19.md*; *platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex*; *outputs/UNBLOCK_CARRYOUT_2026-09-21.md*; *outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md*; *outputs/PES_Phase2_Review_2026-09-12/finish/e9_completion/status_2026-09-19.json*.

3.7 Software Requirements

The runtime is Python 3.11 with PyTorch and the Hugging Face stack (Transformers, PEFT, Datasets) for model handling, tokenisation and adapters; the RL back-ends are TRL, veRL through its HybridFlow path, and OpenRLHF, driven through the shared launcher, with the Tinker client SDK for the managed path. Experiment tracking is a weights-and-biases project; telemetry is stored as JSONL and as persisted tensors; analysis is carried out by the documented scripts named in the Phase-1 specification.

Semester 4 adds five software components to the requirement set. First, the audit suite: `platform_local/run_all_audits.py` together with the nine audit modules it imports and the shared data model in `utils/audit_utils.py`, which defines the issue, result and suite types and the METRIC line format; the suite requires a Python interpreter with no third-party dependencies beyond the standard library for its structural checks. Second, the completion gate and the per-lane harnesses under `zvf-program/`, which require Ed25519 signature verification and JSON Schema validation against the published provider-package, provider-grant and trust-root schema versions. Third, the registry: a schema, an entry store and a query interface, together with the provenance records under `platform_hybrid/registry/`. Fourth, the report toolchain: a LaTeX distribution capable of the multi-pass build with BibTeX, plus a markdown assembler that concatenates the chapter files and, when available, renders the document through pandoc. Fifth, the judgement-layer harness under `zvf-program/jev_lab/`, which records its own receipts and is used only for classification, faithfulness checking and value ordering.

3.8 Constraints and Assumptions

Rewards are binary and verifiable. The specification assumes a programmatic checker — exact match for the mathematics tasks and unit tests for the code subset. Nothing in the requirement set addresses learned reward models, and none of the diagnostics should be read as applying to them.

Held-out sets are small. Several Phase-1 evaluation sets have n between 8 and 20, and the controlled Semester-4 comparisons use 200 to 500 items (Table 1.1 gives the n

behind each headline), with the consequence that learning-gain magnitudes are noise-limited by construction. This is a constraint on what the results can mean, not a defect of the analysis: the small- n figures are reported with their denominators and are not used to support comparative claims.

The managed back-end is a black box at the layer level. Per-layer gradients are not exposed on the managed path, so claims about the relationship between reward spread and gradient magnitude are confined to the small-model white-box configuration and are directional only.

Group-size differences must not be attributed to sampling noise. The comparison protocols share a deterministic prompt schedule, so that method-to-method differences in ZVF reflect training dynamics rather than independent draws of prompts.

Framework parity is incomplete. The requirement in FR-2 covers two seven-library rosters, but only two of the seven cross-launcher entries produced completed runs in this release, and the remaining comparison entries are dry-run placeholders. Comparative framework claims are therefore constrained to the rosters and the completed runs actually recorded, and the repository states the dry-run status explicitly.

The scale study reports descriptive limits rather than a scaling law. The cross-scale analysis spans more than seventy runs across seven libraries and five model families, and its central result is a negative one: no reliable positive cross-scale slope is recovered and no saturation exponent is identifiable. The specification inherits that boundary — the requirement is to report identifiability limits, not to assert a law.

External dependencies are assumed unavailable until proven otherwise. Payload grants, hosted evaluations and quota increases are treated as absent until a response arrives; a lane with no provider response is closed as externally blocked with a reopen condition rather than left pending indefinitely.

Arithmetic and string lookups stay in code. Whatever judgement layer is used for triage, classification or value ordering, verification-type checks are performed by deterministic code. This rule was banked after the judgement layer returned an incorrect answer to a parity probe in a healthy window, and it constrains the architecture: no requirement in this chapter may be verified by a language model where a deterministic check is possible.

Named variants are labels, not specifications. The assumption that a method name identifies a method is false on the available evidence: entries sharing a label diverge substantially in measured behaviour, which is why the registry’s delta records and flip-risk verdicts are specified as requirements rather than as documentation niceties.

Part II

Design and Implementation

Chapter 4

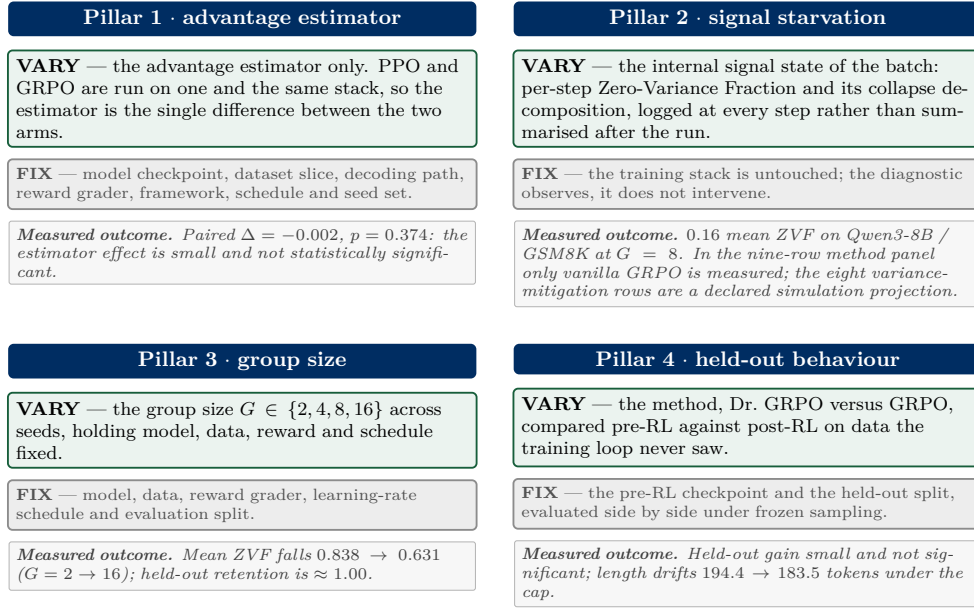
Proposed Methodology

This chapter sets out the methodology of the work rather than its results. It states four things in order: the design commitment to attribution and the four de-confound pillars that operationalise it (§4.1); the per-step telemetry that makes a run’s internal state observable at the granularity at which the mechanism acts (§4.2); the formalisation of the Zero-Variance Fraction as a descriptive diagnostic rather than a metric (§4.3); the eight-study portfolio P1–P8 and the question each study asks (§4.4); and the design of the E1–E14 evaluation campaign, including its single trained actor, pinned native evaluators, frozen sampling, receipt model, terminal-state taxonomy and decontamination handling (§4.5). Throughout, the reporting discipline is part of the method: figures are given only where a repository file supports them, and where a lane produced no admissible result, it is recorded as blocked rather than written around (Figure 4.1).

4.1 The Design Commitment to Attribution

The premise of the project is that an RL-for-LLM result is not identified by the name of its algorithm. Two runs both labelled GRPO may differ in loss form, reference-policy handling, sampler, precision, base-checkpoint identity, data order and group-size schedule, and the label alone does not say which of those differences produced the outcome. The project’s response is a design commitment rather than a post-hoc covariate adjustment: hold the whole experimental stack fixed and vary exactly one factor at a time, so that an observed difference is attributable to the factor that was varied rather than to the several things that changed alongside it. This commitment is the methodological spine of the whole thesis, and it has a practical corollary that shapes every experiment reported later: a study is defined by the single axis it varies and is only interpretable to the extent that every other axis was genuinely pinned (Figure 4.2).

The commitment is implemented in a unified launcher, `platform_local/unified/la`



Vary exactly one factor; hold every other factor fixed.

Headline configuration (varies by study; Table 4.2): LoRA rank 32; Adam with $\beta_1 = 0.9$, $\beta_2 = 0.95$; learning rate 1×10^{-4} ; seeds {42, 123, 456, 789, 1024}; one shared reward grader and one shared decoding path across back-ends; Welch / Mann–Whitney, Cohen’s d with Hedges–Olkin intervals, 95% bootstrap CIs (10,000 resamples) and a declared multiple-comparison family, all fixed in advance of the comparisons they support.

Figure 4.1: The four de-confound pillars. Each holds the whole stack fixed and varies exactly one factor. In Pillar 2 the nine-row method panel measures vanilla GRPO only; its eight variance-mitigation rows are a declared simulation projection (Chapter 6).

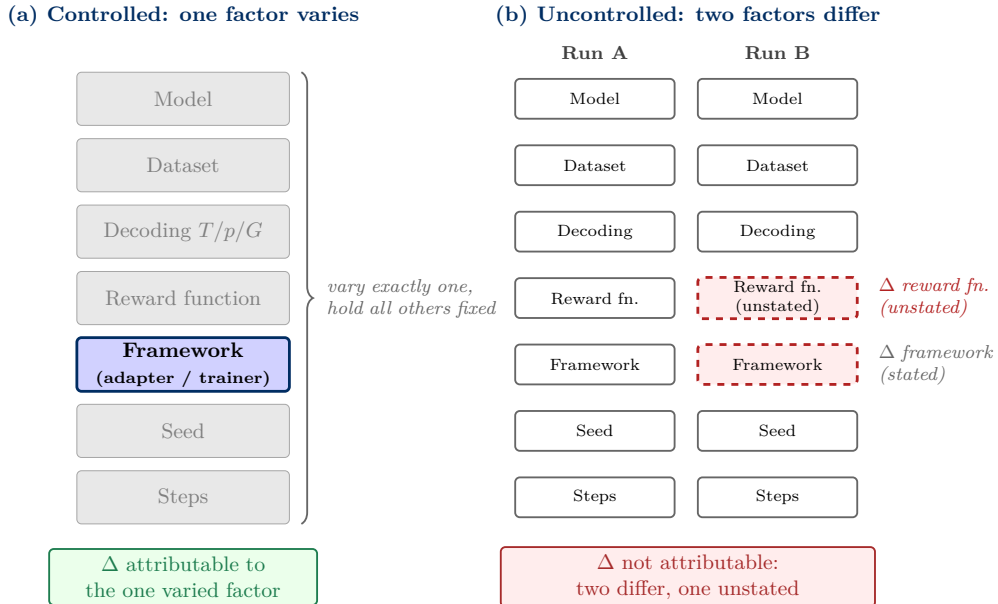


Figure 4.2: The attribution design. Exactly one factor of the training stack is varied while the remainder is held fixed, so a measured difference cannot be attributed to an unstated framework difference.

`uncher.py`, which consumes a single configuration object and dispatches it to thin per-framework adapters — `trl_integrations/`, `verl/`, `platform_modal/openrlhf/`, `platform_hybrid/skylr/` and `platform_tinker/atropos/` — with the reference GRPO loop implemented once in `platform_tinker/tinkerrl/grpo.py`. The reward grader and the decoding path are shared code. That sharing is the load-bearing part of the design: because the grader and the sampler are the same objects across back-ends, a difference in outcome between two frameworks is attributable to the framework, and not to a divergent reward parser or a differently configured decoder masquerading as an algorithmic difference. The commitment also bounds what a result may claim. A completed run supports a statement about that stack, at that revision, with that grader; it does not license a statement about “GRPO” in the abstract, and the project’s reporting standard (§4.4, P5) is built around that distinction.

Four pillars carry the commitment, each holding the remainder of the stack fixed while varying one factor:

The four de-confound pillars.

Table 4.1: The four de-confound pillars, the single factor each isolates, and the design that holds the remainder fixed.

Pillar	Factor isolated	Design
1 · PPO versus GRPO, same stack	Advantage estimator: group baseline versus learned value head	Same model, runner, clipped surrogate and step budget on a saturated 0.5B addition task; the arms also differ in prompts per step (16×8 versus 128)
2 · Zero-Variance Fraction	Signal starvation and its confounders	Per-step ZVF logged on every run; the nine-row method panel, vanilla GRPO row included, is a declared simulation projection (§6.3.2)
3 · Trainability and group size	Contrast density against the variance knob	$G \in \{2, 4, 8, 16\} \times \text{seeds}$, held out
4 · Held-out generalisation	Whether GRPO adds real capability	Dr. GRPO versus GRPO, pre-to-post, McNemar

Pillar 1 asks whether a reported GRPO advantage survives when the advantage estimator

is the main thing that changes. The comparison is designed on Qwen/Qwen2.5-0.5B two-digit addition over 40 steps and five paired seeds, evaluated on 200 prompts drawn fresh from the training generator rather than from a disjoint split, and the exact loss form of both arms is in Table 4.2. Because that task is near ceiling, the design can bound an estimator effect but cannot separate “no effect” from “no headroom”; the result is reported in Chapter 6 §6.2.3. The Qwen3-8B row in the project’s own effect-size table is excluded from Pillar 1 as a provenance conflict and is not treated as an estimate. Pillar 2 turns a run’s internal signal state into a first-class measurement; it is formalised in §4.3. Pillar 3 varies the group size G while holding model, data, reward and schedule fixed, and is the axis on which the preference-density argument of P3 rests. Pillar 4 asks the question the other three cannot answer — whether any of this changes held-out behaviour — by comparing a pre-RL checkpoint with its post-RL successor on data the training loop never saw.

The headline configuration of the internal report sources is LoRA rank 32, Adam with $\beta_1 = 0.9$ and $\beta_2 = 0.95$, learning rate 1×10^{-4} , and the seed set {42, 123, 456, 789, 1024}. It is not held fixed across the programme. The canonical dispatch matrix uses rank 16 and learning rate 1×10^{-6} , the stack probe uses 1×10^{-5} , the P4 scale extension uses rank 4, and the Tinker runners default to 3×10^{-5} ; Table 4.2 lists the values each study actually used. The instrumented 79-run roster described in that source spans the Qwen3, Qwen3.5, Llama-3.x, DeepSeek-V3.1 and Nemotron-120B families, from 0.6B to approximately 671B total parameters, including mixture-of-experts anchors at the frontier end of that range; the wider P1 cross-scale roster adds a 1T anchor (Kimi-K2) and so spans 0.6B to 1T (Chapter 6 §6.1; source: `platform_hybrid/paper/sections/p1_abstract.tex`). The statistical apparatus is declared in the shared methods source but was not pre-registered (Chapter 9 §9.2; the analysis log records 206 iterations). It consists of Welch t-tests and Mann–Whitney U tests for two-sample contrasts; Cohen’s d with Hedges–Olkin intervals; 95% bootstrap confidence intervals with 10,000 resamples; reliable aggregate metrics (interquartile mean, median and optimality gap); and Bonferroni and Benjamini–Hochberg corrections over a declared test family. Power was computed rather than assumed: at 80% power the minimum detectable effect for an equal- n comparison of five seeds against five is $d_{\min} = 2.024$, improving to $d_{\min} = 1.530$ when a five-seed group is compared against a pooled group of fifteen. Those figures are a standing reminder of how coarse a five-seed design is, and the project reports several null results that are null precisely because the design could not have detected anything smaller.

Two limitations belong in this section rather than in a later discussion. First, the framework roster is broader than the framework evidence: although seven RL libraries are named in the study design, only Tinker and TRL produced completed runs, and the

veRL and OpenRLHF entries in `framework_comparison.json` are dry-run placeholders rather than measurements. Cross-library claims are therefore narrower than the roster implies. Second, the variance decomposition that motivates the whole attribution commitment is itself four separate marginal one-way ANOVAs — library/framework, training algorithm, model family and model scale — each run against the same 42 experiments with valid performance observations. On a corpus where the factors are collinear a marginal η^2 absorbs the variance of each factor’s correlates, so the design reads the decomposition only as evidence that framework and algorithm vary a great deal across this corpus, which is what motivates holding them fixed — not as an estimate of any factor’s independent contribution, which a marginal one-way analysis on a non-orthogonal design cannot identify. The values are reported in Chapter 6 §6.1. The historical twenty-test multiple-comparison family that accompanies it pre-dates the Tier-A/B/C partition and is retained as a reproducibility audit trail, explicitly not as a corrected survival claim.

4.2 Per-Step Telemetry

Attribution is only as good as the run’s observability. Every training step therefore emits a record containing the Zero-Variance Fraction; gradient utilisation; the collapse decomposition that separates all-correct from all-wrong zero-variance groups; and, alongside them, reward, loss, entropy, completion length and the KL divergence to the reference policy. Records are written to JSONL and mirrored to Weights & Biases, so a run can be analysed locally without a network dependency and reviewed remotely without a local environment. The reason ZVF and its decomposition are logged as separate fields rather than collapsed into a single starvation number is that the same aggregate fraction can describe opposite situations: a group of completions that are all correct and a group that are all wrong both contribute zero within-group variance, and the remedy for each is different. Logging the decomposition preserves that distinction at the point of collection instead of trying to recover it afterwards (Figure 4.3).

The quantity being logged is the group-relative advantage actually used by the runner:

$$A_i = \frac{r_i - \mu_g}{\sigma_g + \varepsilon}, \quad \mu_g = \frac{1}{G} \sum_j r_j, \quad \sigma_g = \sqrt{\frac{1}{G} \sum_j (r_j - \mu_g)^2}$$

This form is taken from the Phase-1 report. Throughout this thesis σ_g is the population standard deviation, with $1/G$ as written above. Every GRPO-style runner in Table 4.2 computes it that way. Two places use the unbiased $1/(G - 1)$ form instead: the reference `zvf()` of §4.3, whose `ddof=1` variance does not change a $\leq \varepsilon$ zero-variance test, and the batch-wide whitening of the PPO arm in the same-stack run. The ZVF-definition figure

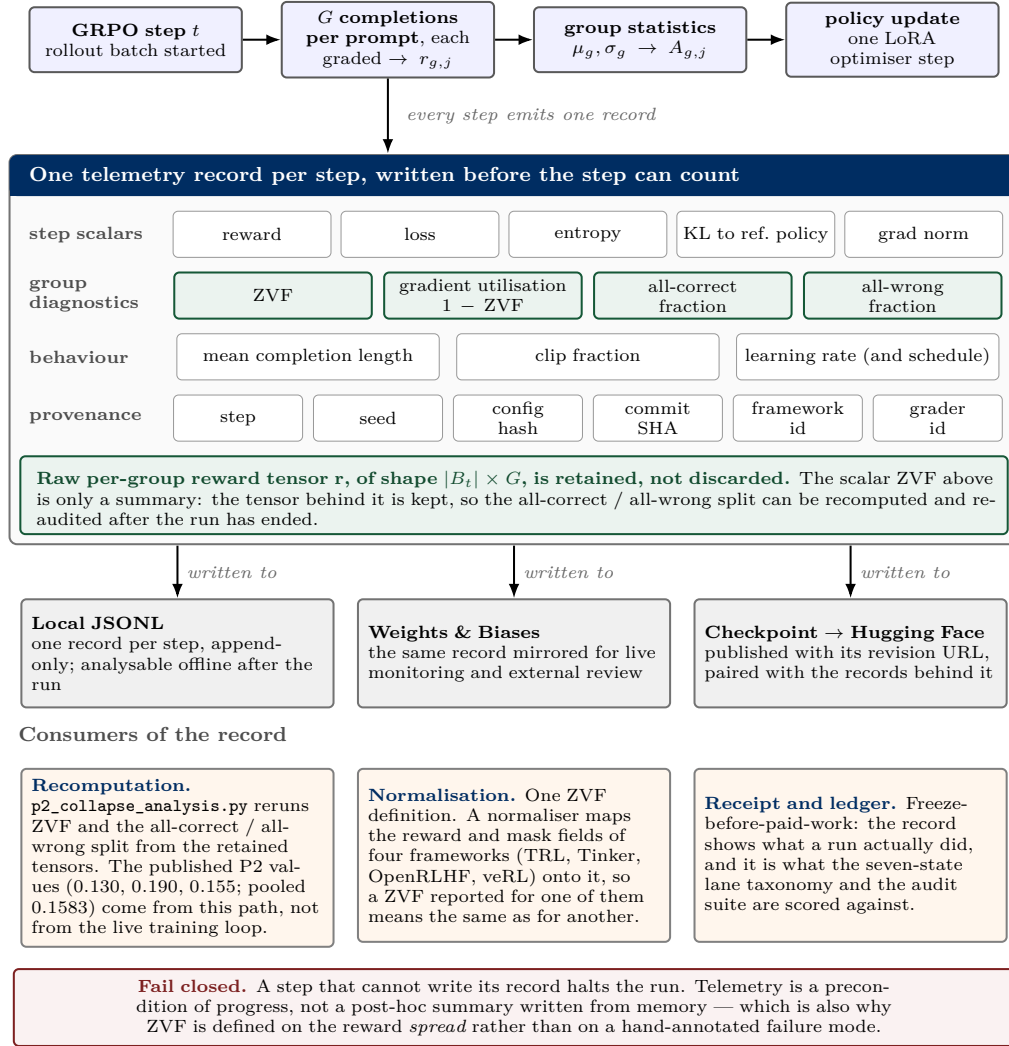


Figure 4.3: Per-step telemetry recorded by the unified harness, and the destinations each record flows to.

(Figure 4.4) also shows the unbiased form, which gives $\sigma_g \approx 0.535$ for its example group where the population form gives 0.5. When $\sigma_g = 0$ every A_i is zero, so the whole group contributes no gradient through the advantage term. Whether anything else acts depends on the loss form: a KL penalty, auxiliary loss or clipping term would be unaffected, but none of the runners in Table 4.2 has a KL penalty or a reference policy, so for them a zero-variance group contributes exactly zero gradient.

Two further design choices distinguish this telemetry from conventional training curves. The first is that the raw per-group reward tensors are retained, not only the summaries computed from them. This makes a published number auditable by recomputation: any downstream diagnostic, such as `p2_collapse_analysis.py`, can reproduce a reported statistic from the stored tensors rather than trusting a logged summary. The convention has been exercised: study P2 recomputes three stored GSM8K group tensors (Qwen3-8B sampling runs, $G = 8$) from their raw values rather than reading the figures back from a log line, and the recomputed values are reported in Chapter 6 §6.3.1. The second is that the logging layer is treated as part of the apparatus and is expected to fail. An early Weights & Biases integration used the library’s thread-unsafe global run object from multiple worker threads, which produced repeated “Run is finished” summary-write failures; the fix was to give each run its own process in `campaign.py`. The defect is recorded because telemetry that silently drops writes is indistinguishable from telemetry showing a healthy run, and the remedy — process-level isolation per run — is a structural property of the campaign rather than a patch.

4.3 Formalising the Zero-Variance Fraction

The Zero-Variance Fraction is the share of prompt groups in a batch whose reward variance is zero. Formally, for a batch B_t of prompt groups at step t , each group g contributing K sampled completions with rewards $r_{g,1}, \dots, r_{g,K}$,

$$\text{ZVF}_t = \frac{1}{|B_t|} \sum_{g \in B_t} \mathbf{1}[\widehat{\text{Var}}(r_{g,1}, \dots, r_{g,K}) \leq \varepsilon]$$

where $\widehat{\text{Var}}$ is the unbiased sample variance and $\varepsilon = 10^{-6}$ for rewards normalised to $[0, 1]$. A prompt-set formulation used elsewhere in the internal report sources writes the same quantity as $\text{ZVF}_t = \frac{1}{|P_t|} \sum_{p \in P_t} \mathbf{1}[\text{Var}_{c \sim C(p)} r(c) = 0]$, with $C(p)$ the set of sampled completions for prompt p . The reference implementation is deliberately small, so that the diagnostic cannot acquire hidden behaviour:

```
def zvf(rewards_2d: np.ndarray, eps: float = 1e-6) -> float:
    # rewards_2d shape: [num_groups, K]
```

```

var_per_group = rewards_2d.var(axis=-1, ddof=1)
return float((var_per_group <= eps).mean())

```

Gradient utilisation is then defined as its complement, $GU_t = 1 - ZVF_t$. The complement is a descriptive accounting identity, not a claim about optimiser behaviour. Its scope must be read against the runner actually used, so the loss form of every study is listed in Table 4.2, read from the runner code rather than from the internal reports’ prose. Three facts in the table govern how ZVF is interpreted. First, no runner in the programme has a KL penalty or a frozen reference policy, so for every study a zero-variance group contributes exactly zero gradient through the policy term; the only escapes are optimiser momentum from earlier steps and the per-batch normalisers. Second, the Tinker runners, including the one that trained the E1–E14 actor, are REINFORCE with a group-mean baseline: the loss is $-A_i$ times the summed token log-probability, averaged over completions, with no importance ratio and no clipping, and this thesis calls them that rather than GRPO in the strict sense. Third, the runners labelled GRPO are not one loss: the same-stack and Dr. GRPO studies use a clipped ratio with two inner epochs, the Tinker runners do not, and the Phase-1 four-method run divides by no σ even in its “grpo” arm. The quoted sentence from paper_P8_workshop.tex, which disclaims objectives with KL, auxiliary losses, clipping or token masking, therefore applies to none of the measured configurations (Figures 4.4 and 4.5).

Loss form by study, read from the runner code.

Table 4.2: Loss form by study, read from the runner code. M = platform_hybrid/experiments/modal/, T = platform_hybrid/experiments/tinker-runs/, C = zvf-program/colab-experiments/. “n/a” means the study does no training; “not recoverable” means the repository does not hold the code or configuration that ran.

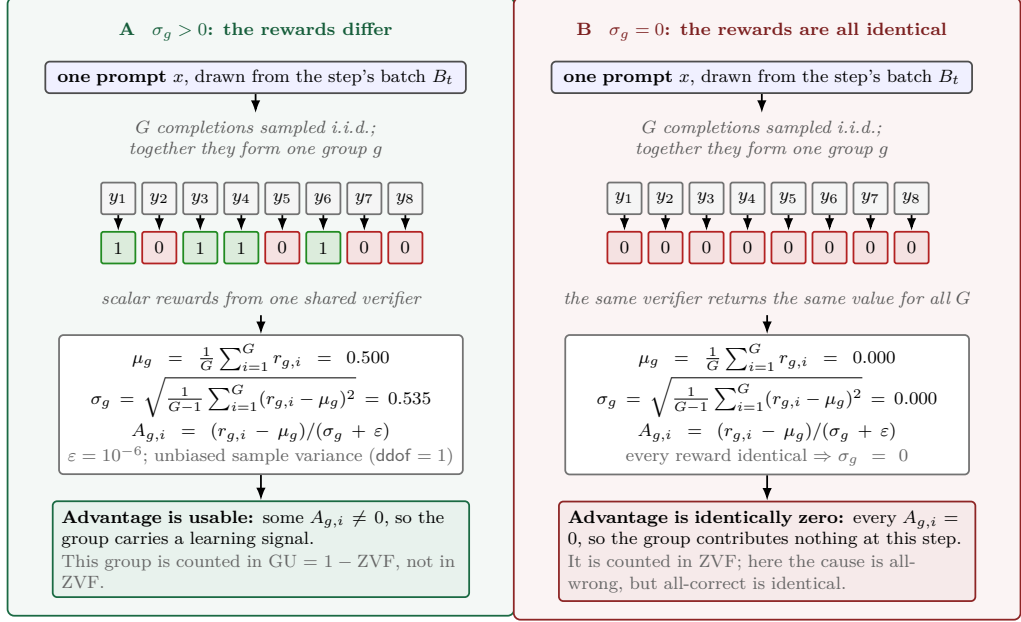
Study / run family	Runner	Ratio / clip	KL / reference	Token mask	Advantage	Critic	LoRA r	lr	G	Steps
Tinker reference loop; E1–E14 actor (pa vlov_po rtfolio preset)	platfor m_tinke r/tinke rrl/grp o.py	none (REIN- FORCE, group- mean baseline)	none	full sequence	(r – μ)/(σ + 10^{-8}), 1/G	none	32	1e-5 (actor); 3e-5 default	4 (actor); 8 default	40 (actor)

Study / run family	Runner	Ratio / clip	KL / reference	Token mask	Advantage	Critic	LoRA r	lr	G	Steps
P1 cross- scale Tinker anchors	T/scripts/tink er_param l1el_ru nner.py, T/campa ign_v2. py, T/sc ripts/w orld_cl ass_exp eriment s.py; per- anchor lineage for the frontier anchors not re- coverable	none	none	full sequence	$(r - \mu)/\sigma$, 1/G	none	32	3e-5	8	20–30 logged
P2 re- computed tensors (0.130 / 0.190 / 0.155)	platform_hybrid/experiments/tinker_direct_eval.py (sampling only, Qwen3-8B)	n/a	n/a	n/a	ZVF from popula- tion variance	n/a	n/a	n/a	8, T = 1.0	0
P2 Phase-1 four- method batch (0.72– 0.77)	T/scripts/n2_reward_t ensor_2 0260704 .py	Tinker importance_sampling ratio against sampler log-probs; no clip	none	completion only	$r - \mu$, no σ division (all arms)	none	32	1e-5	8	40
P3 Qwen2.5- 0.5B arith- metic sweep	M/modal_groups_ize_zvf_sweep.py	none	none	completion only	$(r - \mu)/(\sigma + 10^{-6})$, 1/G	none	16 (q, v)	1e-4	2, 4, 8, 16	40
P3 Qwen3- 8B matched- budget panel	T/live_zvf_probe.py (schema match; no launcher link)	none	none	full sequence	$(r - \mu)/(\sigma + 10^{-8})$, 1/G	none	4	3e-5	2 / 16	160 / 20

Study / run family	Runner	Ratio / clip	KL / reference	Token mask	Advantage	Critic	LoRA r	lr	G	Steps
P3 Qwen3.5- 4B openings	platform_hybrid/experiments/openings/groups/size_zvf.py	none	none	full sequence	$(r - \mu)/\sigma$, 1/G; zero- variance groups skipped	none	8	3e-5	2-16	8
P4 arith- metic, GRPO vs Dr. GRPO	M/modal_drgrpo_vs_grpo.py	token- level ratio, clip 0.2, 2 epochs	none	completion only	GRPO ($r - \mu)/(\sigma + 10^{-6})$, 1/G; Dr. GRPO $r - \mu$	none	16	1e-4	8	40
P4 GSM8K- CoT, Qwen2.5- 1.5B- Instruct	M/modal_drgrpo_gsm8k_cot.py	as above	none	completion only	as above	none	16	1e-5	8	30
P4 Qwen3- 8B uncapped extension	T/live_zvf_probe.py (-loss drgrpo)	none	none	full sequence	GRPO 1/G; Dr. GRPO $r - \mu$	none	4	3e-5	8	30
Same- stack PPO vs GRPO	M/modal_samestack_ack_ppo_grpo.py	both arms: sequence- level ratio on length- normalised log-prob, clip 0.2, 2 epochs	none	completion only	GRPO ($r - \mu_g)/(\sigma_g + 10^{-6})$, 1/G; PPO $r - V$, whitened over the batch with $1/(n - 1)$	PPO: linear value head on the prompt- end hidden state, 0.5 · MSE loss; no GAE (one- step)	16 (q, v)	1e-4	GRPO 16 prompts × 8; PPO 128 prompts × 1	40
P7 Colab four-arm audit	C/e3_open_audit.py	sequence- level ratio, clip 0.2 (DAPO 0.2 / 0.28), 2 epochs	none	completion only	population variance; Dr. GRPO $r - \mu$; zero- variance groups dropped	none	full fine-tune	2e-6	4, adaptive up to 10	10
P7 Tinker adaptive- G	T/scripts/qp7_adaptive_g_20260704.py	Tinker im- portance_sam- pling ratio; no clip	none	completion only	$r - \mu$	none	32	1e-5	4 → 6 → 8	16

Study / run family	Runner	Ratio / clip	KL / reference	Token mask	Advantage	Critic	LoRA r	lr	G	Steps
Tinker side of the Tin- ker/TRL pair	launcher not recover- able; config dump fr amework _config _dumps/ tinker_ qwen3_8 b_gsm8k .yaml	not re- coverable	managed (dump: β null)	not re- coverable	not re- coverable	none	16 (dump)	1e-5	8	30
TRL side of the Tin- ker/TRL pair	M/modal _grp_o_t rl.py (TRL GR POTrain er)	TRL defaults, not set in code	β not set in code (dump: 0.04)	TRL comple- tion mask	TRL default scaling	none	32 in code, 16 in dump	1e-5	8	30
P1-roster PPO baselines	M/modal _ppo_ca mpaign. py (TRL PPOTrai ner)	TRL PPO defaults (clip, GAE, adaptive KL)	TRL default	TRL default	TRL whiten- ing	value head	32	1e-5	n/a	not recorded

Three properties make the definition more than a restatement of the training loop. First, it is non-tautological under a simple null: if completions were independent Bernoulli draws with per-prompt success probability p_g , then $\mathbb{E}[ZVF_t] = \mathbb{E}_{p_g}[p_g^K + (1 - p_g)^K]$, a quantity that depends on the model’s accuracy distribution and the group size rather than on the observed batch. The identity $\text{pass}@G - p^G = 1 - ZVF$ follows algebraically under the same i.i.d.-Bernoulli model and is therefore reported as an algebraic remark rather than as evidence. Second, it is checked against that null on instrumented runs: a Qwen2.5-0.5B sweep over $G \in \{2, 4, 8, 16\}$, three seeds and 40 steps — twelve runs and 480 logged steps — compares per-step observed ZVF with the Bernoulli-null prediction $p^G + (1 - p)^G$. Because the null for each step is computed from the same step’s mean reward, a high correlation is partly mechanical, so the comparison is designed as a shape check rather than a validation, and the baseline used is the per-step average of the closed form (results in Chapter 6 §6.3.3). Third, at larger scale the same instrument is run on Qwen3-8B against GSM8K test problems through the Tinker managed API at $G = 8$, temperature 1.0, over 200 problems and three seeds (sampling only, no training; these are the same three stored tensors whose ZVF P2 recomputes), with the zero-variance groups decomposed into all-correct and all-wrong and compared with the same null. Deviations from the null in either direction are not read as evidence about dependence between completions.



Definitions. $\text{ZVF}_t = \frac{1}{|B_t|} \sum_{g \in B_t} \mathbf{1}[r_{g,1} = r_{g,2} = \dots = r_{g,G}] = \frac{\# \text{ groups whose } G \text{ rewards are identical}}{\# \text{ groups in the step's batch } B_t}$

$\text{GU}_t = 1 - \text{ZVF}_t$ (gradient utilisation: the share of groups that carry within-group contrast)

A group's rewards are identical exactly when its reward variance is zero, which is how the implementation tests it: $\text{Var}(r_{g,1}, \dots, r_{g,G}) \leq \epsilon$. Both quantities are shares of *groups*, not of *tokens*.

Measured composition. Qwen3-8B on a held-out GSM8K slice via the Tinker API, $G = 8$, $T = 1.0$, 200 problems $\times$ 3 seeds: mean ZVF = 0.16, hence GU = 0.84.

$$\text{ZVF} = 12.7\% + 3.2\% = 0.16$$

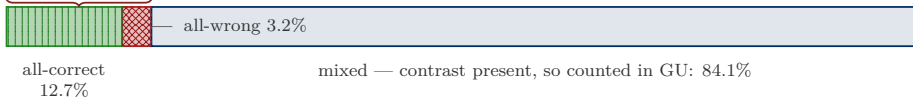


Figure 4.4: Definition of the Zero-Variance Fraction. One prompt produces G completions with scalar rewards; when the within-group reward standard deviation is zero the advantage is identically zero and the sample contributes no gradient. ZVF is the fraction of such groups. The composition bar is measured on Qwen3-8B, held-out GSM8K slice, $G = 8$, $T = 1.0$, 200 problems $\times$ 3 seeds; all-correct (vertical hatch) and all-wrong (cross-hatch) together make up the ZVF.

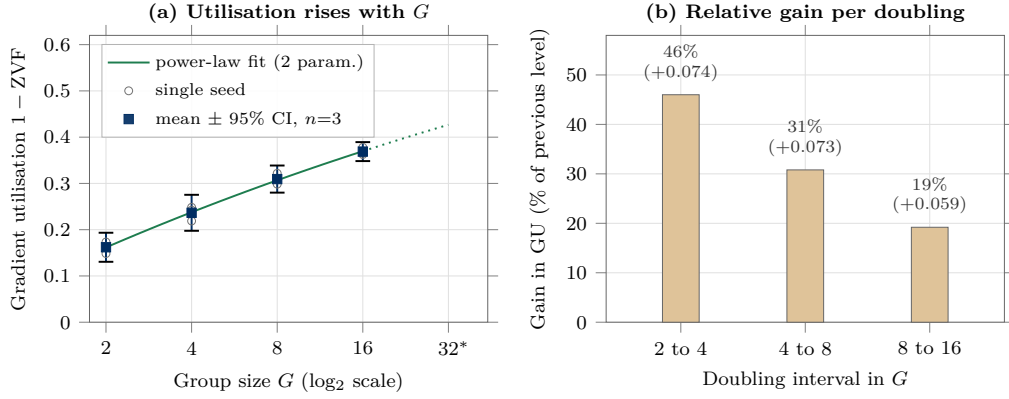


Figure 4.5: Gradient utilisation, $1 - \text{ZVF}$, as a function of group size G . Qwen2.5-0.5B, arithmetic-correctness task, 40 steps, 16 prompts, $G \in \{2, 4, 8, 16\}$, seeds 42/123/456 ($n = 3$ per G ; `groupsize_zvf_sweep.json`). (a) Open circles are single seeds; squares are seed means with 95% t -intervals ($df = 2$). The line is a two-parameter power law, $\text{ZVF} = 0.922 G^{-0.137}$, fitted to the four means (2 residual degrees of freedom), so it is descriptive rather than a validated law; $G = 32^*$ was not trained and the dotted segment is extrapolation. (b) The absolute gain per doubling is nearly constant (+0.074, +0.073, +0.059), so the diminishing return is relative: 46% $\rightarrow$ 31% $\rightarrow$ 19% of the previous level. Utilisation stays below 0.37 at $G = 16$.

That the quantity is portable across libraries is a separate, weaker claim, and the project states its limits explicitly. A normaliser at `platform_modal/scripts/zvf_compute_cross_framework.py` maps each framework’s reward and mask fields onto the common definition — TRL’s rewards with `batch_size / group_size` and `completion_mask`; Tinker’s `rollout.reward` with `rollout.group_id` and `rollout.eos_mask`; OpenRLHF’s `reward_score` with `prompt_index` and `attention_mask`; veRL’s `data_source/reward` with `uid` and `response_mask` — and emits JSONL records of the form `{step, zvf, batch_reward_mean, n_groups, K}`. The scope correction attached to that parser is that it normalises reward logs and does not make its runner canonical GRPO; because the project does not bundle raw per-group rollout JSONL for every framework, no fresh four-way pointwise ZVF overlay is claimed, only the narrower implementation-level portability claim. The threshold ε is fixed in advance, and its sensitivity is reported with the P2 results in Chapter 6 §6.3.1. Boundary conditions are declared where the diagnostic does not apply: dense or continuous reward signals, $K = 1$, SFT-saturated baselines and non-outcome-reward RL all fall outside the setting in which the definition is meaningful.

The methodological point of this section is that ZVF is a diagnostic and not a metric. The design commits to that position before the results rather than inferring it from them, and it takes four precautions. First, because the statistic is unsigned, a high value is read

as consistent with either a solved or an unsolved task and never as a score in one direction; the 368-run audit that shows both extremes is reported in Chapter 7 §7.1 and §7.3. Second, a pooled association between ZVF and performance is always broken down by task type before it is read, because the task mix of a pool can set the sign of the association (results in Chapter 6 §6.3.4). Third, the association with final accuracy and with final held-out outcome is tested, but ZVF is not promoted to a causal or incrementally predictive statistic whatever that test returns. Fourth, no step-level p-value is computed on the 480 autocorrelated steps of the instrumented sweep, because treating correlated within-run steps as independent samples would manufacture significance. One further boundary is recorded for the same reason: a variance-mitigation table in the internal report sources is a declared simulation projection and not evidence, and its rows are read as synthetic rather than as independent library deployments. The result of these choices is a diagnostic that is useful for describing where within a batch the learning signal is and is not, and that is explicitly not proposed as a quantity to be maximised, ranked or reported as a headline score.

4.4 The Eight-Study Portfolio

Semester 3 delivered the shared code and the initial experimental foundation; the Semester 4 programme — the research direction, the expanded experiments, the analysis and the P1–P8 internal reports — is the present author’s solo continuation, and the roster records 206 analysis iterations behind the numbers reported in this thesis. The portfolio is organised as eight studies, each written up as an unpublished internal report, each asking one question and each carrying its own claim boundary; the titles quoted below are the internal reports’ working titles. This section states the questions and designs; the results are in Chapters 6–7 and, for P8, Appendix F. The questions and boundaries are stated here because they are methodological commitments: a study’s question determines which comparisons it may make, and the boundary states what it declines to conclude.

P1, “Scaling Laws for GRPO Post-Training of Language Models: A Cross-Library, Cross-Scale Study”, asks whether GRPO post-training outcomes scale with model scale in a way that can be fitted and extrapolated. Its evidence base is more than seventy runs across seven RL libraries and five model families from 0.6B to 1T parameters, fitted with saturating forms and tested for a cross-scale slope. The frontier anchors — Qwen3-235B-MoE, DeepSeek-V3.1, Kimi-K2 and Nemotron-120B — were run through the Tinker managed API in a single-seed, descriptive-only regime, so the report’s claim boundary is a descriptive limits-and-identifiability audit rather than a positive scaling law; matched multi-seed cross-scale runs are an explicit non-claim. Results are in Chap-

ter 6 §6.2.

P2, “The Zero-Variance Fraction: A Descriptive Diagnostic for Signal Starvation in GRPO”, asks how much of a training batch carries usable group-relative signal, and whether that fraction predicts the final outcome. The study characterises ZVF descriptively, treats the identity of §4.3 as algebraic rather than empirical, tests the association with final held-out outcome without promoting ZVF to a predictive statistic, and marks the variance-mitigation table as a simulation projection; the retired roster item that previously carried a causal framing now lives under `archive/absorbed/R02_main_zvf/`. Results are in Chapter 6 §6.3.

P3, “Group Size in GRPO: Contrast Density and the Bridge to DPO”, asks how trainability varies with the group size G and whether GRPO admits an all-pairs preference interpretation. Its measured sweep covers $G \in \{2, 4, 8, 16\}$ on the near-ceiling arithmetic task, with seed-level equivalence tests against $G = 2$ and a one-way test of held-out accuracy across G . No directly trained matched-budget $G = 32$ cell exists, so every $G = 32$ value comes from an explicitly labelled reconstructed token-budget grid and is held as a hypothesis rather than a measurement. The claim boundary is that the data may not be read as a universal optimum, and the all-pairs preference reading resembling DPO is offered as an interpretation and not as a proven equivalence. Results, including a conditional utility audit over 505 tasks, are in Chapter 6 §6.4.

P4, “Length Bias and Held-Out Generalization in GRPO and Dr. GRPO”, asks whether the verbosity trap associated with GRPO appears in the measured regime and whether either method changes held-out accuracy. Its controlled comparison runs GRPO against Dr. GRPO on a near-ceiling arithmetic task and in a short-horizon GSM8K chain-of-thought regime with a 200-token generation cap, alongside a multi-seed Qwen3-8B pre-RL versus post-RL held-out control. The 200-token cap bounds the claim: under a cap that already controls length, absence of the verbosity signature would be the expected outcome rather than a positive finding. Results are in Chapter 6 §6.5.

P5, “Report the Stack, Not the Label: RL-for-LLM Results Are Stack-Conditioned”, asks how much of a reported result is attributable to the stack rather than to the algorithm label, and what a reportable stack must contain. Three exhibits carry the argument: a nominally matched-configuration backend audit, a same-label “DAPO” comparison between true dynamic sampling on an open trainer and an asymmetric-clip surrogate on a closed stack, and a twelve-cell head-to-head of four labelled algorithms on one fixed stack; their results are in Chapter 7 §7.1. The study proposes an eight-item minimum reportable stack (seven run-manifest fields and one evaluation item) — loss form; reference-policy and KL handling; sampler, backend and precision including base-checkpoint identity by revision or hash; per-step ZVF and gradient-utilisation trajectory; group-size schedule; a disjoint held-out split; decontamination plus a parser-

robustness probe; and held-out pass@k curves alongside pass@1 — and formalises an RL-reproducibility threat model with flip-risk grades R0–R5. This is the standard this thesis is written to.

P6, “GRPO-Registry: A Machine-Readable Catalog of Group-Relative RL Stacks and Their Variant Deltas”, asks whether stacks can be compared mechanically rather than through prose. It defines stack records carrying the seven manifest fields of the MIN-REPORT-RL standard, a per-component variant-delta status, three variant-delta records verified against the DAPO, Dr. GRPO and GSPO papers, twenty seed stack entries, and a reference CLI with a 0–100 MIN-REPORT auditor badge and a `stackdiff` command that maps a pair of entries to an R0–R5 flip-risk verdict. The study is explicitly a resource study rather than a second ranking, and it cites P5’s seventeen-fold exhibit only to motivate the schema. Results are in Chapter 7 §7.2.

P7, “From Diagnostic to Controller: A Signal-Starvation Theory of GRPO and an Adaptive Group-Size Intervention”, asks whether the diagnostic of §4.3 can be turned into a control signal. It models usable learning signal per prompt as $S = p(1 - p)(1 - h_G(p))$ with $h_G(p) = p^G + (1 - p)^G$, yielding three testable consequences: that the expected ZVF indicator equals $p^G + (1 - p)^G$; that larger G lowers ZVF; and that gradient magnitude tracks $p(1 - p)$. Each consequence is paired with a measurement that could falsify it: the 368-run audit, a model-by-group-size grid, and a toy-scale open-backward-pass experiment whose result is admitted as directional only. The study promotes pairwise-contrast density, $PCD = (G - 1)/G \cdot E[p(1 - p)]$, as the candidate control signal, subject to a micro-jitter falsification test against ZVF, and then audits an adaptive group-size controller against fixed recipes and DAPO-style dynamic sampling on a four-arm open trainer and in a counterfactual controller trace. Results are in Chapter 7 §7.3. The methodological commitment that governs this study is stated at its head: adaptive group size is not promoted, and no result establishes prospective superiority over static $G = 16$, Dr. GRPO or dynamic sampling.

P8 occupies the eighth slot with a study outside the RL programme, “LLM vs. XGBoost in Credit-Card Fraud: Sensor and Scribe, Not Scorer”. Its question is whether a large language model should be used as the fraud scorer or as a component around one, and it is designed as a comparison of a boosted-tree model with a Qwen3.5-4B scorer on held-out splits, with the scorer’s cost as a triage component recorded alongside its accuracy. Results are in Appendix F. The roster evolved after the semester-4 title list was fixed, and the present position is recorded as it stands: the fraud study is parked outside the ZVF thesis, contributes no RL evidence, and is retained for its measurement discipline; the active P8 slot in the current report roster is an exploratory workshop artifact. No result from P8 is used anywhere in this thesis to support an RL claim.

4.5 The Design of the E1–E14 Held-Out Evaluation Campaign

The evaluation campaign is the applied half of the programme and is deliberately narrow in a way that makes its results interpretable: it evaluates one actor against fourteen benchmark suites that were not part of its training data under a receipt, budget and decontamination regime, instead of training many actors and comparing them. The single actor is the base model Qwen/Qwen3.6–35B–A3B at commit 995ad96e..., with adapter arvindcr4/pavlov-portfolio-qwen36-seed809-stepfinal-tinker-cf0ad8c1... at commit 64444133..., in bfloat16. How that adapter was trained is reconstructed in Chapter 8 §8.1.1. “Held-out” is used here in a narrow sense: the actor was trained only on API-Bank-RLVR and SWE-Gym, and none of the fourteen suites is in that training data. It does not mean that every suite was decontamination-checked. The content check below covers LAB-Bench, AgentDojo and Omni-MATH, the E11 receipt records its own verified decontamination, and the E10 receipt itself declares `heldout_claim: false`. One actor fixes the object of study: every reported per-suite figure describes the same frozen policy, so a difference between two suites is a property of the suite and not of a model swap. The design consequence is that the campaign cannot support model-comparison claims at all, and it makes none (Figure 4.6).

Evaluators are the suites’ own scorers, used natively rather than reimplemented, and they are pinned by revision. The surviving native runners live under `zvf-program/flagship/`, including `e1_swe_bench_pro_full_eval.py`, `e11_verilog_eval_decontamination.py`, `eval_pavlov_sdab.py`, `eval_pavlov_xlam.py`, `mle_bench_eval.py` and `eval_apex_agents.py`. The evaluation-side revisions pinned for the decontamination audit are LAB-Bench 998a8e0a40cf116c80e1b0e7a805ebb5fb9fa838, AgentDojo 089ed468cf3ed0322acc66b0211f26d9d90dbf60 and Omni-MATH 40ba231d8f16e29ecd40e6407e2c8640145a8f62, with the training-side envelope pinned at API-Bank-RLVR bf67c42626f02c305514b1df16dcabc5fc616333 (597 rows, MIT) and SWE-Gym bb94ed9e39bbeb96a7fcbfb533b80f25a7fd59cb (2,438 train rows, MIT). Using the native scorer is a substantive choice and not a convenience: when the Omni-MATH lane produced two rows whose judged outputs were truncated before any equivalence verdict, the native scorer’s omission of missing-verdict rows was adjudicated as correct official behaviour rather than a parser bug, and the lane closed with no re-judge because re-judging would deviate from the official scorer and generate new evidence with no effect on the score.

For the training-selection protocol, sampling is frozen rather than merely recorded: temperature 0.1, top-p 0.95, one sample per row, a maximum of 128 response tokens, with the service seed fixed where the provider supports it and otherwise recorded in the re-

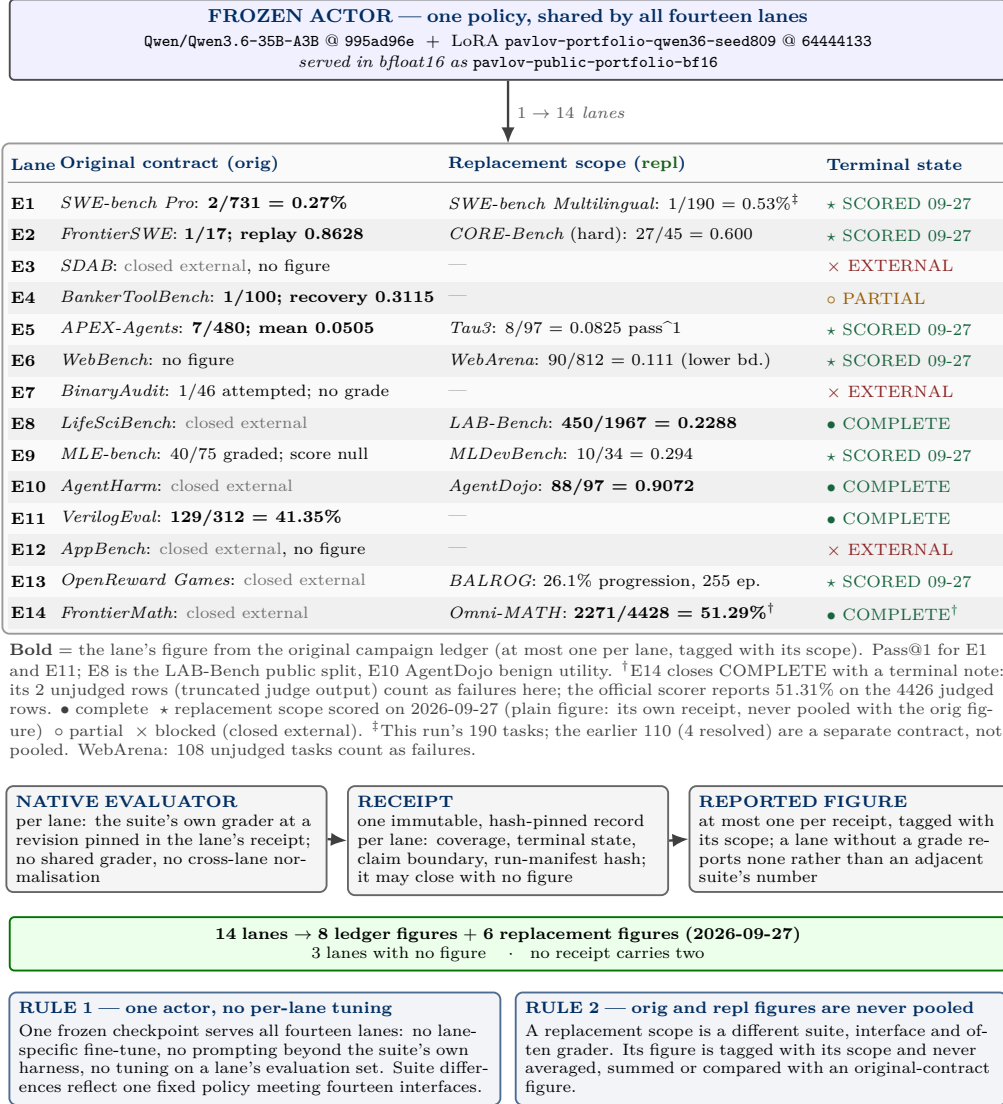


Figure 4.6: Design of the E1–E14 native-benchmark evaluation campaign. One frozen actor serves all fourteen lanes, and each suite is graded by its own native evaluator at a pinned revision. Each lane lists its original-contract suite and, where one exists, its replacement scope side by side; Figure 8.1 names the replacement scope where one exists. Figures and terminal states come from the 2026-09-19 ledger (E1_E14_FINAL_RES ULTS_2026-09-19.md and finish/Pending_Experiments.md; 11/11 deterministic checks pass), updated with the six replacement scopes scored on 2026-09-27 (★; finish_pending_2026-09-27/<lane>/result.json), which are separate receipts and are never pooled with a lane's original-contract figure. E14's 2271/4428 = 51.29% counts the 2 unjudged rows as failures; the official scorer reports 51.31% = 2271/4426 on the judged rows.

ceipt. Prompt and response limits are 1,200 and 128 tokens respectively, and the same source requires the strict verifier implementation to be hashed before paid work, with a perfect call defined as a verifier score of exactly 1.0 and no parser, target, split or verifier change permitted mid-campaign. Freezing the sampler addresses the failure mode in which a benchmark score quietly improves because decoding became more favourable between runs; it also fixes the comparison between the base reference and the adapter, which is evaluated side by side on untouched manifests. The 128-token setting belongs to that protocol only. The E1–E14 lanes were decoded under each suite’s own settings, which range from 1,024 to 8,192 output tokens at temperatures 0 to 0.2, and are recorded per lane in the serving table of Chapter 8 §8.1.2. The actor’s own training run also departed from the protocol in several fields (§8.1.1).

The receipt model is freeze-before-paid-work, and it is the part of the campaign design that carries the most methodological weight. Before any paid phase a run must record an immutable base-model revision, tokenizer revision, container or environment digest and adapter revision; the selection slice and the disjoint holdout are re-created from a pinned dataset revision and sealed, are write-once, are not mounted in training, and the word “holdout” is reserved for a suite or manifest only once its split, task-ID, license, runtime and decontamination receipts are complete. The selection slice is used for arm decisions and is therefore unsuitable for confirmatory claims; the final evaluation manifests are never used to choose an arm, alter a prompt, or re-run a failed suite with a more favourable split. Tracking is verified before spend rather than after: an online Weights & Biases run must be initialised and its Tinker run identifier recorded before the first paid step, every step records loss, reward, configuration, seed, split identity and cumulative cost, and every checkpoint is published to Hugging Face with a unique repository or revision URL, with a failure of any of these stopping the run fail-closed. If a receipt is missing, the phase is blocked rather than silently shortened. At handoff the protocol requires four separate evidence classes to be reported apart: completed receipts; operational and provenance receipts; prospective gates and budget projections; and admissible final results, the last existing only if every gate passed.

That last distinction has teeth in practice, and the campaign’s disposition record shows it. The portfolio-level protocol defines a predeclared equal-domain macro average over sixteen domains, a conjunctive coverage gate, an integrity gate, a global improvement gate requiring the lower bound of a one-sided 95% interval for the portfolio difference to exceed zero, a domain no-regression guard with a predeclared five-percentage-point tolerance, and a safety guard bounding any violation increase by two percentage points — and it states that a cost cap may not be met by dropping domains or suites, and that a suite or domain which is missing, underpowered, contaminated or outside the cost cap must be labelled incomplete or blocked, “not negative and not positive”. The portfolio

training phase was never launched, so the campaign does not report a portfolio result; the recorded outcome is a set of per-lane evaluations of the one adapter, and the protocol's own instruction that a partial subset is not a portfolio result and may not be relabelled as one is honoured throughout.

Lane dispositions are recorded in a seven-state taxonomy rather than as pass or fail: COMPLETE, for lanes with verified full-coverage results and no open gate (E8 public, E10 benign, E11); COMPLETE_TERMINAL_NOTE, for a lane complete under the native protocol with a recorded terminal note (E14 replacement); CLOSED_EXTERNAL, for lanes with no local or paid path, closed by written closure records (E3, E7, E8-private, E10-private, E12, E14-private); PENDING_QUOTA, for lanes gated on cloud quota resolution alone (E6, E9); AMENDMENT_ACCEPTED_LAUNCH_PENDING (E2, E13); REBUILD_READY_LAUNCH_PENDING (E1 wave10, E5 successor27); and CLOSED_PARTIAL (E4). Those are the states of the 19 September ledger. The three pending states were emptied on 27 September, when the E1, E2, E5, E6, E9 and E13 replacement scopes were run to completion (§8.4). This report assigns them COMPLETE on coverage, with two qualifications recorded against the label: E6 still has 108 tasks awaiting a judge, and E1's coverage spans two contracts reported separately. The taxonomy exists so that a lane which has not been measured is not silently absorbed into a percentage. Carried alongside it is the project's evidence rule, stated in the ledger of record and repeated in every results artefact: original-contract and replacement-scope numbers are never pooled. The rule is not cosmetic. The original-contract E1 lane and its replacement scope are different suites with different task populations, and a table that added them would produce a number that corresponds to no benchmark.

Decontamination is handled as a declared deterministic procedure with its own stated limits. The check hashes complete UTF-8 fields and then normalised fields with SHA-256; for fields of at least eight tokens it hashes each consecutive eight-token sequence and computes exact set Jaccard similarity for every training and evaluation pair sharing a shingle; and it tests whether an entire normalised evaluation field is a token-bounded contiguous substring of a training field. Against the pinned training envelope, the per-suite inventories were 1,967 LAB-Bench tasks over 13,113 text fields with 5,105 eligible for eight-token near checks, 97 AgentDojo tasks over 71,966 fields with 9,382 eligible, and 4,428 Omni-MATH tasks over 13,284 fields with 9,394 eligible; the count of near pairs at or above 0.80 was zero in all three suites, and the maximum eight-gram Jaccard similarity was 0 for LAB-Bench and AgentDojo and 0.13414634146341464 for Omni-MATH, below the declared 0.80 threshold, with all 1,338 candidate pairs sharing an eight-gram examined. Short exact matches of zero to three normalised tokens were retained and enumerated rather than dropped — the longest examples being seven characters in the LAB and Omni inventories and at most seventeen raw characters in

AgentDojo, such as “Paris” and “Conference Room A” — and the audit states plainly that these collisions do not demonstrate duplication of a task but do make an unqualified “zero overlap” claim incorrect. The audit’s scope limits are recorded with equal care: it does not reconstruct which training records were consumed: 512 is the size of the training pool recorded in the E11 decontamination receipt (`selected_training_row_count`), from which the 40-step run drew 80 prompts at two per step, and the transform and consumed-row inventory that would identify them were not recorded; it establishes a content comparison against the verified historical source envelope rather than proving global pretraining cleanliness, semantic disjointness, character-edit disjointness or image cleanliness; ASCII normalisation loses mathematical punctuation and non-ASCII text; and fields shorter than eight tokens fall outside the near-match and containment rule. The audit itself is receipted: it re-verifies Weights & Biases run `bsv8vx04` against `training_identity_verified.json`, matches the 23,733 original source-field document hashes to the historical E11 receipt, adds 2,507 decoded nested API-Bank strings to reach 26,240 source documents, and records 43,977,748 bytes downloaded against a 500 MiB cap, with the checker not having enumerated the roughly 2.58 billion possible document pairs.

Two features of the campaign’s execution are methodological rather than incidental. The first is that a lane stops when its receipts are incomplete, and the record says why. A single inventory pass on 2026-09-21 checked every blocked lane live and found concrete, named failures: E1 lacked W&B authentication, a sealed reservation and a live actor endpoint; E2 awaited an owner IAM binding that only the project owner can create; E5 lacked both a bound authorization receipt and any real native runtime adapter in the repository; E13’s request remained a draft with launch authorisation false; E6 and E9 were refused by cloud quota in both regions. Those blockers were cleared on 27 September by a single lead authorisation and a move from AWS to Google Cloud and Modal, and each lane recorded the change as a deviation (§8.4). Recording a lane as blocked-with-reason, rather than reporting a shortened run, is the same principle as the terminal-state taxonomy applied to execution. The second is that negative outcomes are reported with their full denominators and their mechanisms. Generation failures and artifact losses stay in a lane’s denominator rather than being excluded, as in the E1 SWE-bench Pro lane, and a zero score is audited trajectory by trajectory before it is read, as in the base-model E4 rerun whose zero was traced to the serving bridge rather than to the model (results in Chapter 8 and Section 8.5; sources: `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md`; `outputs/e1_e14_small_scale_2026-09-26/E4/BRIDGE_STOP_TOKEN_FINDING.md`). Reporting outcomes with the mechanism identified is the behaviour the whole apparatus is designed to produce.

The campaign also carries its integrity accounting in the same register. Eleven of eleven deterministic checks passed on 2026-09-19, covering exact divisions, receipt presence, sealed-artifact hash equality and lost-source absence probes; the recorded spend was \$5.6301 against the \$50 additional cap counted at that date, with later budget-mode records moving to an unlimited mode whose per-run resource and timeout bounds remain in force. A judgment layer was added for classification, faithfulness and value-ordering questions and then partly invalidated by an external service outage: both blocker-classification batteries returned degenerate distributions after the incident, their receipts are retained as incident evidence, and the per-lane blocker column therefore rests on document evidence and not on model judgment. A design rule was banked from the healthy window and is followed in this thesis: arithmetic and string lookups stay in code, and model judgment is reserved for classification, faithfulness and value ordering, because a parity probe was answered incorrectly even while the service was healthy.

What the campaign does not claim follows directly from the design. It does not claim improvement over any baseline anywhere, because no lane carries a test establishing it. The one trained-versus-base comparison is the paired small-scale rerun of Table 8.B, with 5 to 151 identical items per lane, and its outcome is reported in Chapter 8; the campaign's role in this thesis is to fix the evaluation protocol, the receipt discipline and the disposition taxonomy that any future improvement claim would have to satisfy. It does not claim that the fourteen suites constitute a portfolio result. It does not pool original-contract with replacement-scope figures, and it does not average across suites. Its one trained actor means it says nothing about model comparisons. And where the record is incomplete — a lost execution source, an unanswered external access request, a run whose budget exceeded the authorised envelope — the incompleteness is written into the ledger rather than resolved by inference.

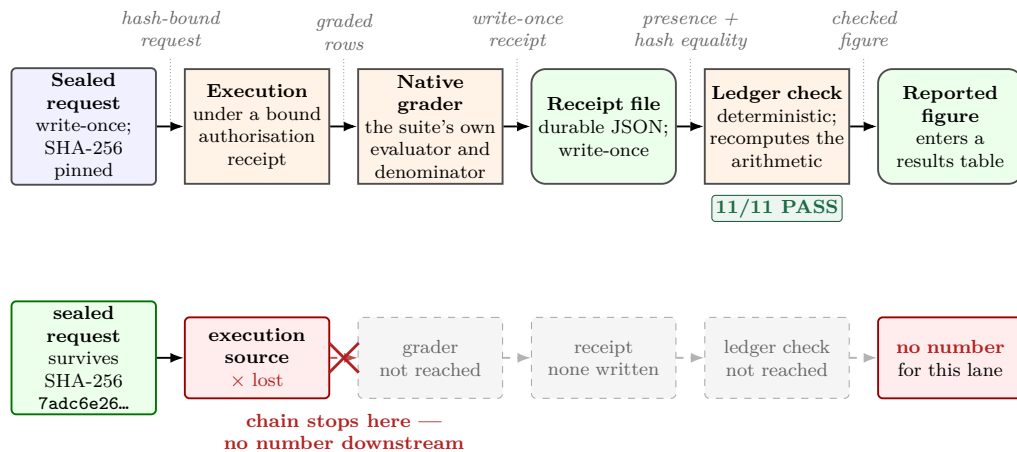
Chapter 5

Implementation Details

This chapter describes what was built. Its organising claim is that the engineering contribution of the project is a **dispatch and evidence layer** rather than a training algorithm: the algorithmic content of GRPO is inherited from the literature, but the work of making six reinforcement-learning frameworks and six compute backends addressable through one Tinker-style interface — and of making every number that comes out of them traceable to a sealed, hash-pinned artefact — is where the implementation effort sits. The chapter proceeds from the abstraction (the adapter architecture), through the concrete training loop, to the two engineering episodes that most shaped the codebase (the parallel logging defect and the Colab white-box path), and finally to the evaluation harness and the recovery re-implementations undertaken after an execution-source loss in the final phase of the project (Figure 5.1).

5.1 Two Rosters, One Dispatch Surface

A reader approaching the project’s comparison material will encounter two different framework rosters, and conflating them is the single most likely misreading of the implementation. The internal reports’ shared methods note states the distinction explicitly: the cross-launcher LLM-RL scaffold referenced in the framework-gap discussion is a seven-library roster — Tinker, TRL, SkyRL, veRL, OpenRLHF, Atropos, and HF reference launchers — that targets LLM post-training launchers specifically, while the cross-RL-library evidence reported elsewhere uses a different roster. Critically for this chapter, the same note records that “only Tinker and TRL produced completed runs in this release; veRL and OpenRLHF entries in the `framework_comparison.json` artefact are dry-run placeholders”. The veRL and OpenRLHF entries in that artefact carry peak and last-ten-average reward values, but they are the output of the harness resolving a launch plan, not the output of a training run, and they must never be pooled with real



Example lane: the E1 wave-10 recovery (16 tasks, \$16) was sealed and locally validated, then the `.codex-run/` deletion removed its execution source — so no number is reported for that lane.

Fail-closed. A break at any link means the lane reports **no number**: the chain is never repaired by inference, and the gap is carried in the ledger of record with a named reopen condition rather than filled in. The break above is itself receipted — the deterministic check `codex_run_execution_sources_absent` records the four lost paths as duly absent (probed 4, still present: none) — and the lane stays at `REBUILD_READY_LAUNCH_PENDING` with its values in no results table. The same rule runs in the opposite direction at E9 MLE-bench, which reports coverage of 40/75 (53.33%) and keeps the suite score null, because coverage without complete grading is not a score.

checks: `LEDGER_CODE_CHECK_2026-09-19.json` (11 of 11 PASS, 0 failed, schema `ledger-code-check-v1`)

Figure 5.1: The evidence chain from sealed request through execution and native grading to a deterministic ledger check. A break anywhere in the chain means no number is reported for that lane. Upper row: the chain; the 11/11 PASS badge is the 2026-09-19 deterministic ledger check (`LEDGER_CODE_CHECK_2026-09-19.json`). Lower row: the E1 wave-10 recovery, where the lost execution source stops the chain.

results or read as framework comparison evidence.

Atropos occupies a different position again. Rather than being a cell in the dispatch matrix, it is integrated through a dedicated package under `platform_tinker/atropos/`, a `tinker_atropos` module containing `config.py`, `api.py`, `trainer.py`, `types.py`, `dataset.py`, `stats_utils.py`, and an `environments/` directory with per-task environment servers. Its integration model is the Atropos managed-server pattern: a three-terminal workflow in which `run-api` starts the Atropos API, `launch_training.py --config configs/default.yaml` starts Tinker training against a `TINKER_API_KEY`, and an environment server — a built-in one or any external Atropos environment pointed at the same config file — serves rollouts. The README’s quickstart example runs a fifty-step training run on Llama-3.1-8B-Instruct against a GSM8K environment, and the environment contract is that the `env` and `openai` sections are consumed by the environment server while the `tinker` section is consumed by the trainer and ignored by the environment. This separation is what allows an arbitrary existing Atropos environment to be reused against the Tinker backend without modification to the environment.

5.2 The Adapter Architecture: What versus Where

The dispatch surface is built on one orthogonal decomposition, stated in the backends module docstring: “A Backend knows **where** to run one experiment cell (local GPU, Modal, Colab, vast.ai, GCP, HF Spaces); the framework dimension (`trl/tinker/verl/openrlhf/skylr`) is the **what**. `plan()` resolves a cell into a concrete `LaunchPlan` without spending compute, so `--dry-run` and the matrix test work uniformly”. The `LaunchPlan` dataclass carries backend, framework, command, `driver_file`, output, env, and notes, with a `format()` method for printing; the default `_execute` runs `subprocess.run(plan.command, shell=True, check=True)`. Because plan resolution is separated from execution, the entire matrix can be validated without a GPU, an API key, or a dollar of spend — a property that the dry-run placeholders discussed above depend on, and which is also the reason those placeholders must be labelled as such (Figure 5.2).

The matrix itself ships as data. `platform_local/unified/matrix.json` declares schema `tinkerrl-framework-backend-matrix-v1` with thirty cells — five frameworks (`trl`, `tinker`, `verl`, `openrlhf`, `skylr`) crossed with six backends (`local`, `modal`, `colab`, `vast`, `gcp`, `hfspace`), five cells per row and per column — and a `canonical_experiment` block that pins the single configuration every cell is meant to instantiate: Qwen/Qwen3-8B on GSM8K with GRPO, thirty training steps, `num_generations` 8, per-device batch size 2, gradient accumulation 8, learning rate

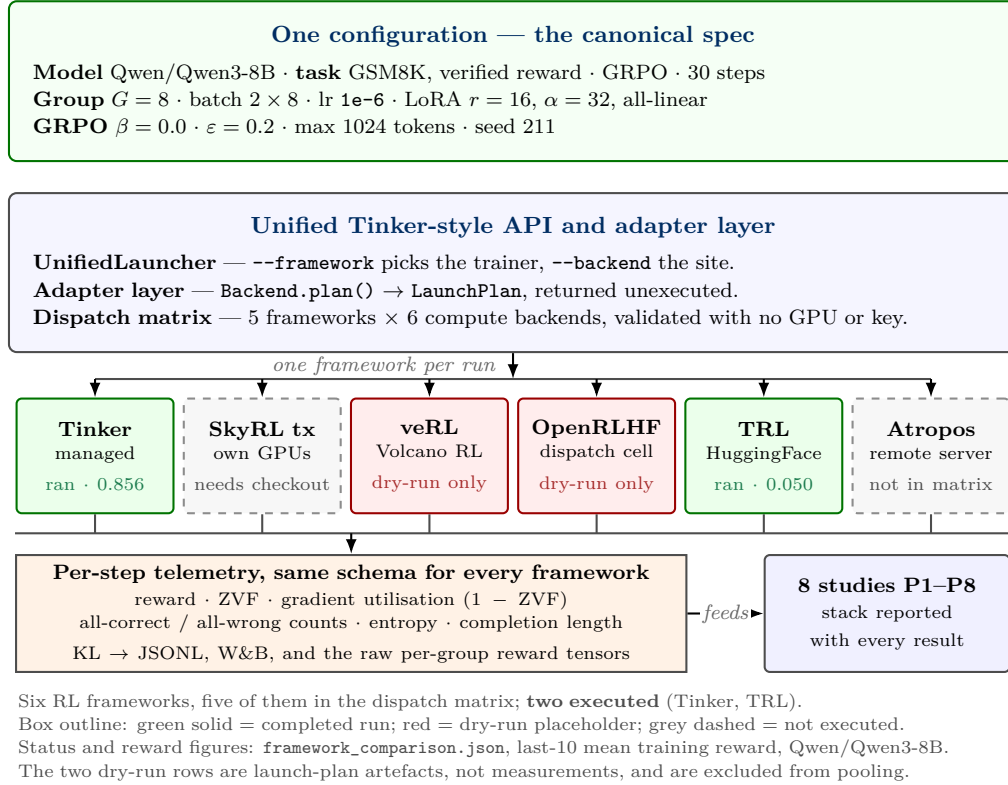


Figure 5.2: Six RL frameworks behind one Tinker-style API. Five of the six are in the dispatch matrix (5 frameworks × 6 compute backends); only two executed (Tinker, TRL: last-10 mean training reward on Qwen/Qwen3-8B GSM8K, from `framework_comparison.json`), two are dry-run launch plans and two were not executed. One configuration drives every framework, but the executed comparison bundled a different base checkpoint (Chapter 7), so outcome differences are reported with the stack rather than attributed to it.

1e-06, LoRA rank 16 and alpha 32 over all linear modules, beta 0.0, epsilon 0.2, maximum completion length 1024, and seed 211. Each cell is a concrete plan naming a framework, a backend, the launch command, the driver file and an output location; for example, the `trl/local` cell runs `python -m platform_local.unified --backend local --framework trl` and dispatches in-process to `platform_local/trl_integrations/trainer.py`.

The launcher implements the dispatch. `UnifiedLauncher` holds three registries: `FRAMEWORKS` mapping `skyrl`, `tinker`, `verl`, `openrlhf`, and `trl` to human-readable labels; `BACKENDS` mapping `local`, `modal`, `colab`, `vast`, `gcp`, and `hfspaces` to labels; and `ALGORITHMS` covering `grpo`, `ppo`, `reinforce`, `dapo`, and `dpo`. Dispatch is outer-on-backend and inner-on-framework, which is why the per-framework handlers can each be written against the framework’s own idioms. `_run_tinker` shells to `python -m platform_tinker.tinkerrl.grpo_cli --preset <spec.task>`; `_run_openrlhf` delegates to `platform_modal/openrlhf/trainer.py:run_openrlhf_training({}, output_dir="./checkpoints/openrlhf")`, which drives `openrlhf.cli.train_ppo_ray --advantage_estimator group_norm`; `_run_trl` generates a resumable GRPO training script through `trl_integrations.trainer.generate_trl_train_script` into a temporary directory and executes it; `_run_verl` imports `verl.config` and `verl.trainer` directly; and `_run_skyrl` requires a `SKYRL_CHECKOUT` environment variable and emits `uv run --extra gpu --extra tinker -m skyrl.tinker.api`. For the `local` and `colab` backends the launcher runs `dispatch_framework()` in-process, a path the code comments identify as the one that breaks the Colab entry’s self-recursion — an implementation detail worth recording because it indicates that the backend and framework dimensions are not perfectly orthogonal in practice, and the launcher carries framework-specific knowledge precisely at the points where they intersect.

Fairness across frameworks is addressed by configuration auditing rather than by assertion. The `platform_hybrid/experiments/framework_config_dumps/` supplements one exact hyperparameter dump per framework for the canonical Qwen3-8B + GSM8K matched-configuration run, and the accompanying README recomputes the field partition from those dumps: of 44 tracked leaf fields, 25 are identical across all four frameworks (LoRA rank, alpha, target modules and dropout; GRPO group size, rollouts per group and max new tokens; sampling temperature, top-p, top-k and do-sample; optimizer name, learning rate, weight decay, betas and epsilon; reward type; and the task, seed and source-git-sha metadata), while 11 are Tinker-managed — serialised as `null # managed_by_tinker` in the Tinker dump — covering `kl.beta`, `kl.surrogate`, importance-sampling granularity, reference-model offload and recompute, tokens per optimizer step, reward broadcast precision, gradient checkpointing, micro-batch size,

gradient accumulation, and tensor-parallel size. The same README records two disclaimers that matter more than the identical count: `model` and `model_sha256` are *not* identical, because the Tinker run uses Qwen/Qwen3-8B-Base while TRL, veRL and OpenRLHF use the instruction-tuned Qwen/Qwen3-8B, and `model_sha256` is a placeholder rather than a verified checksum. Remaining fields are framework-specific and documented rather than harmonised — KL beta is 0.04 for TRL and veRL against 0.02 for OpenRLHF, importance-sampling granularity is token-level for TRL and veRL against sequence-level for OpenRLHF, and reference-model placement differs across all four (`on_device`, `managed`, `separate_process`, and `sharded_ray`). The appendix’s original “byte-identical” framing is retracted on the strength of this partition; the accurate statement is that a majority of tracked fields are matched, a known Base-versus-Instruct confound survives, and eleven fields are not under user control at the Tinker API level at all.

5.3 The Tinker GRPO Loop

The reference implementation of the training loop lives in `platform_tinker/tinker_rl/grpo.py` and is exposed through `grpo_cli.py`. The loop follows the standard four-stage shape recorded in the Phase-1 report under “The Tinker GRPO Loop”: sample a group of G completions per prompt, compute group-relative advantages, form the loss, and take an optimiser step. Advantages are normalised within the group to mean zero and unit standard deviation, using the population σ (`normalize_rewards`, $\varepsilon = 10^{-8}$). The loss is not the clipped, KL-regularised surrogate of the original GRPO paper. It is REINFORCE with a group-mean baseline,

$$L(\theta) = -\frac{1}{N} \sum_i A_i \sum_t \log \pi_{\theta}(y_{i,t} \mid x, y_{i,<t})$$

averaged over the N completions in the batch, with no importance ratio, no clipping, no KL term and no reference policy. The sampler is refreshed only at checkpoint boundaries (every `save_every` steps), so rollouts between refreshes come from slightly stale weights with no importance correction. The implementation uses the Tinker SDK’s `ServiceClient` with `create_lora_training_client`, `forward_backward_custom` with `loss_type_input="logprobs"` so that per-token log-probabilities flow into a custom loss, `optim_step` with `AdamParams`, and `sample()` for rollouts; resumption uses `create_training_client_from_state_with_optimizer`.

The single most consequential property of this loop for the project’s research questions is the degenerate case. When the group’s reward standard deviation σ_g is zero, every

advantage A_i is zero, and because the loss has no KL or auxiliary term, the group contributes exactly zero gradient. This is the mechanism behind the zero-variance-fraction diagnostic that runs through the project; it is recorded in the Phase-1 report as the observation that motivated the collapse analysis, and it is the reason `zero_loss` and `zero_reward` are first-class outputs of the training CLI rather than derived quantities. Zero reward is counted as an average reward equal to zero; zero loss is counted as a step loss below $1e-6$.

The CLI exposes eight presets — `tooluse_synth`, `tooluse_baseline`, `tooluse_heldout`, `tooluse_xlam`, `gsm8k`, `math100`, `pavlov_xlam`, and `pavlov_portfolio` — each resolving a dataset factory and a reward function through a `DATASET_FACTORIES` registry and a `REWARD_MAP` (`ExactMathReward` for `GSM8K`, `StrictToolCallReward` for `pavlov_xlam`, `PavlovNonXLAMReward` for `pavlov_portfolio`). Tracking and campaign gates are enforced before a run starts, through `config.validate_tracking()` and `config.validate_campaign_gate()`, and the CLI refuses to launch without a `TINKER_API_KEY`. Each seed’s result is printed as a structured record containing `run_id`, `sampler`, `avg_first5`, `avg_last10`, `peak_reward`, `zero_loss`, `zero_reward`, `heldout_reward`, and `reward_trace`, so that a run’s evidence is emitted at the point of execution rather than reconstructed afterwards. The eight presets replaced an earlier collection of standalone `grpo_*.py` scripts, with `legacy_main` retained to translate the old flag surface.

5.4 The Parallel Runner and a Real Logging Defect

Sweep execution is handled by `parallel_sweep.py` and `campaign.py`, with the runner’s design shaped directly by a defect found during development. The defect is documented in the group-size sweep findings and is worth quoting in full, because it is the clearest illustration in the project of why logging infrastructure deserves the same scrutiny as training code:

The threaded runner trains all G in parallel correctly, BUT wandb uses a thread-unsafe global run — with 4 threads calling `wandb.init(reinit=True)`, the G8/G16 **summary** writes failed (“Run is finished”) after another thread finished its run. Per-step `wandb.log` mostly survived; the summary dict didn’t. **Fix for next parallel batch:** isolate each run in its own `PROCESS` (multiprocessing / subprocess per config), not a thread — or use `wandb-core` service mode with explicit per-run objects. Training itself was unaffected (all step data is in the run log).

Two aspects of this record are instructive. First, the failure was silent with respect to training: gradients were correct, steps completed, and per-step metrics were written. Only the run summary — the aggregated dictionary from which a results table would be built — was lost, which is precisely the class of failure that corrupts downstream reporting while leaving the primary artefact intact. Second, the remedy adopted was structural rather than retry-based: isolating each run in its own process removes the shared mutable global entirely, whereas retrying the summary write would have left the race in place. The finding also notes the alternative of wandb-core service mode with explicit per-run objects, which would preserve in-process execution at the cost of depending on service-mode semantics that the project chose not to rely on.

The same findings file documents the group-size sweep that exposed the defect, and its numbers are reported here with the qualifications the source attaches to them. The sweep ran $G \in \{2, 4, 8, 16\}$ on Qwen3.5-4B for six steps on GSM8K at seed 0 under the W&B group p3-groupsweep, with total token counts of 9,663, 19,964, 34,084, and 68,055 respectively. At $G = 2$ the run showed a 50% batch-level collapse with a +0.000 gain; at $G = 4$ a 17% collapse with a +0.125 gain. The source flags the $G = 4$ gain explicitly as supported by one example at $n = 8$ on a single seed and therefore *suggestive rather than proven*, and records that the independent verification of the sweep returned the verdict SUSPECT. These are not results this thesis claims; they are the observations that motivated the group-size investigation and that the later controlled work supersedes, and they are reported in the register in which they were recorded.

5.5 The Colab White-Box Path (Per-Layer Gradient Profile)

The per-layer gradient profile — numbered P1 in the Phase-1 series, and distinct from the P1 cross-scale study of Chapter 6 — asks a question the managed Tinker backend cannot answer. Per-layer gradient norms require access to the model’s internals during the backward pass, and the managed service exposes no such interface. The implementation therefore leaves the managed path entirely and runs on a Colab L4 GPU in `platform_hybrid/experiments/openings/p1_layer_profile.py`, using Qwen2.5-1.5B-Instruct with LoRA adapters on the q/k/v/o projections, five GRPO-style advantage-weighted steps, and per-layer LoRA gradient norms recorded at each step. The white-box path is thus a deliberate architectural exception: the project accepts the loss of managed-scale training in exchange for gradient visibility, and confines the exception to a question that only gradient visibility can address.

The first-pass result appeared strong. Across 28 layers, the top-k layer set identified at step 1 overlapped the final top-k set perfectly, with a measured overlap of 1.0; the top 25% of layers accounted for a 0.476 share of gradient mass; layer 0 carried a gradient norm of 63 against 6–17 elsewhere, and a secondary mid-late band appeared at layers 17–23. A scaled re-test disciplined this result. The re-run used `p1_layer_profile_scaled.py` on Qwen2.5-3B-Instruct, with 24 real GSM8K problems, ten steps, $G = 4$, 36 layers, and seeds $\{0, 1\}$. The step-1-to-final top-k overlap fell to 0.111 — chance level — and the top-25% concentration measured 0.39 with a standard deviation of 0.013. The source’s verdict is unambiguous: the original predictability finding “was an artifact of the toy 1.5B/hardcoded-arithmetic setup”. A later four-seed pass at the same configuration (seeds $\{0, 1, 2, 3\}$) supersedes the two-seed figure and gives a mean overlap of 0.083 ± 0.048 , again approximately chance, with concentration 0.39.

A companion freeze test asked whether the layer profile could be exploited: if a small band of layers dominates the gradient, freezing the rest should preserve the benefit at lower cost. It did not. The full 28-layer configuration trained 2.18M parameters and produced a mean held-out gain of $+0.0625$ (seed 0 $+0.125$, seed 1 0.0). Freezing to a hot band comprising layer 0 plus layers 12–27 — 17 layers, 1.32M parameters, or 0.61 times the full count — produced $+0.000$. The source concludes that “aggressive freezing loses the benefit... The freeze lever does not work at this scale”, and records that the comparison is underpowered at $n = 8$ held-out examples over 2 seeds. Both P1 outcomes are negatives: the apparent structure did not survive scaling, and the intervention it suggested did not survive testing. They are reported as findings rather than suppressed, because a layer-importance claim that collapses under a scaled re-test is exactly the kind of result the project’s evidence discipline exists to surface.

5.6 Software Modules

The module inventory recorded in the Phase-1 report remains the accurate map of the codebase. The unified launcher, `platform_local/unified/launcher.py`, maps one configuration onto a chosen backend; `platform_tinker/tinkerrl/grpo.py` is the reference GRPO loop against the managed backend; `platform_local/trl_integrations/` and the `verl/` integration hold the per-framework adapters, with `platform_modal/openrlhf/` and `platform_hybrid/skyrl/` completing the roster, and `platform_tinker/atropos/` carrying the Atropos integration described above. Analysis code is separated from training code by intent: `p2_collapse_analysis.py` recomputes zero-variance fraction and collapse statistics from stored reward tensors, so the diagnostic is reproducible from evidence rather than from a live run; `p1_layer_profile.py` implements the white-box gradient probe; `parallel_sweep.py` and

`campaign.py` drive sweeps; and `p8_detector.py` is an exploratory method-trace classifier evaluated by row-wise cross-validated AUROC.

The p8 detector deserves an explicit caveat in an implementation chapter. It is an exploratory artefact evaluated by row-wise cross-validation, and its AUROC is a measure of separability on the traces it was given, not a benchmark result against an external suite; it should not be read alongside the E-lane scores reported later in this thesis. Beyond these modules, the `platform_local/unified/` package carries `canonical.py` for the canonical configuration and `peft_utils.py` for adapter handling, and the `backends` package implements one module per backend (`local.py`, `modal.py`, `colab.py`, `vast.py`, `gcp.py`, `hfspaces.py`) behind the common `base.py` interface.

5.7 The E1–E14 Evaluation Harness

Evaluation is governed by a completion gate, `zvf-program/flagship/e1_e14_completion_gate.py`, schema `e1-e14-completion-gate-v1`. The gate maintains five evidence classes — `COMPLETE_EXACT`, `PARTIAL_EXACT`, `PARTIAL_RECOVERY`, `BLOCKED_EXTERNAL`, and `LOCAL_SETUP_READY_PROVIDER_INPUT_REQUIRED` — and maps lane source statuses onto them, with unknown states failing closed rather than defaulting to a classification. Its `LANE_SPECS` table binds each lane to a replacement scope, a task count, and the runner or adapter that produces its evidence: E2 FrontierSWE at 17 tasks (`pavlov_frontier_swe_eval_adapter.py`, `frontier_swe_eval.py`), E3 SDAB at 80, E4 BankerToolBench at 100 (`run_harbor_via_tinker_bridge.py`), E5 APEX-Agents at 480, E7 BinaryAudit at 46 (`modal_non_e11_readiness.py`), E9 MLE-bench at 75 (`e9_mle_bench_streaming.py`), E12 AppBench at 6, and lanes E6, E8, E10, E13 and E14 with their respective runners. The gate’s own header states its scope precisely: it neither invokes an adapter nor accepts a package as a benchmark result (Figure 5.3).

The results the gate classifies are recorded in `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md`, verified by eleven of eleven deterministic checks on 2026-09-19 and bound to surviving receipts. The single Tinker-trained actor is base Qwen/Qwen3.6-35B-A3B at commit 995ad96e... with adapter arvindcr4/pavlov-portfolio-qwen36-seed-809-stepfinal-tinker-cf0ad8c1... at commit 64444133..., in `bfloat16`. The document’s opening constraint governs how everything below may be read: original-contract and replacement-scope numbers are never pooled, and per the project evidence rule no cross-suite average is formed at any point.

The per-lane results the gate classified are reported once, in Chapter 8 (Tables 8.A–8.C), with each lane’s scope and terminal state in Table C.3; they are not repeated here.

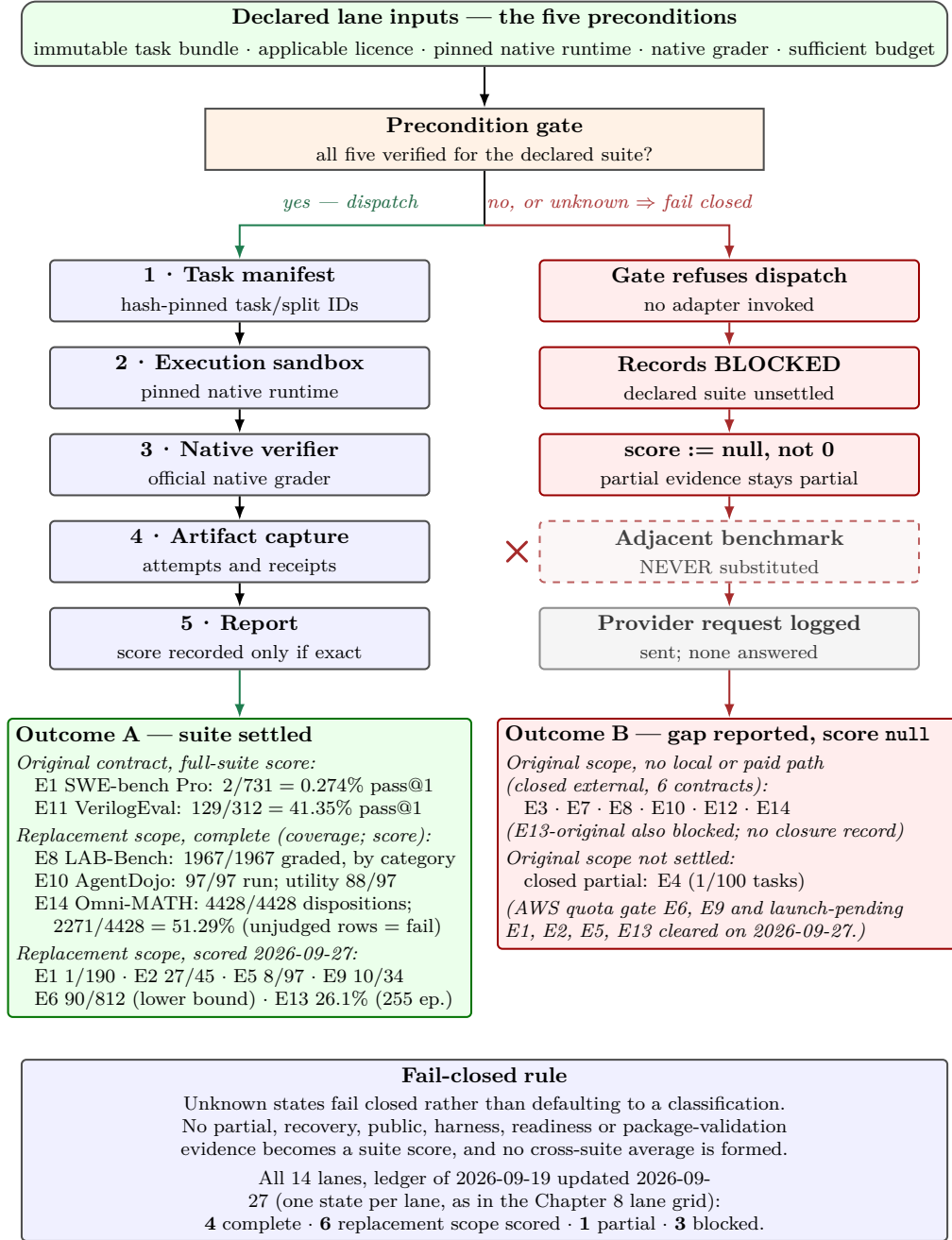


Figure 5.3: Evaluation harness internals. The harness is fail-closed: when a precondition cannot be satisfied the lane records **BLOCKED** rather than substituting an adjacent benchmark. Outcomes and the lane tally are taken from the 2026-09-19 ledger of record (Pending_Experiments.md), updated with the six replacement scopes scored on 2026-09-27. Outcome A separates original-contract scores from replacement-scope results and gives coverage and score separately; Outcome B lists original-scope suites that are closed external, then the one lane whose scope is not settled. E14 is reported as $2271/4428 = 51.29\%$ with the two unjudged rows counted as failures (the official scorer reports 51.31% over 4426 judged rows).

One further result from this phase belongs in the implementation record because of what it demonstrates about the harness. Under separate authorization, the E4 Banker-ToolBench suite was re-run from scratch on the base model through the Modal bridge’s `/v1/responses` path at `--n-concurrent 2` under a cap of \$55.91445263. All 100 of 100 trials completed, with one `NonZeroAgentExitCodeError`, and the mean reward was 0.0 — every trial in the zero bucket. The audited mechanism is that agents terminated after 2–14 steps (median about 6) without producing output in `deliverables/`, causing the verifier to short-circuit to zero on a “No deliverables found” condition, and that 27 or more trajectories ended in role-token degeneration of the form `user user assistant ... repeated`. This bridge path set no stop sequences, which produces exactly this degeneration, so the zero is attributed to the bridge rather than to the model (Section 8.5); the run cost \$15.07 of the cap over 3h29m, against 16.35M prompt and 2.18M completion tokens. Reporting a mean reward of 0.0 with its mechanism attached, rather than reporting a low score and moving on, is the standard this chapter’s evidence rules are meant to enforce.

5.8 The Bridge and Sampler Path

Runners reach a Tinker-trained model through `zvf-program/flagship/modal_tinker_openai_bridge.py`, a Modal-deployed OpenAI-compatible bridge whose own header states that “the deployed bridge is infrastructure, not a benchmark result. Its default paid-inference budget is zero”. The bridge declares app name `pavlov-tinker-openai-bridge`, model alias `pavlov-qwen36-tinker` over model id `Qwen/Qwen3.6-35B-A3B`, and pins its sampler at `tinker://cf0ad8c1-1f1b-5ff3-8bd7-2a0bf232657b:train:0/sampler_weights/seed809_final`, with the Hugging Face side pinned by commit `64444133c55d88c3f1bf0df8a2f5d7ac646125c8`. Runtime limits are 16,384 maximum completion tokens against a 65,536-token model context, with an 8,192-token context prefix reserved; `DEFAULT_MAX_USD` is `Decimal("0.00")`, so every paid path must be opted into explicitly at launch. The pinned dependency set is `fastapi 0.116.1`, `huggingface-hub 1.27.0`, `tinker 0.30.0`, `transformers 5.5.4`, and `wandb 0.21.0`.

The architectural point of the bridge is that it converts a Tinker sampler into an OpenAI-compatible endpoint. A harness written against the OpenAI chat-completions API — which most agentic benchmark harnesses are — can therefore be pointed at a Tinker-trained actor without modification, and the E4 rerun described above used exactly this path in its `/v1/responses` form. The same pattern appears in the actor-side configuration of the E1 recovery wave, where the actor is addressed as an OpenAI-compatible chat-completions service at `http://127.0.0.1:18015/v1` (container port 8000) un-

der model name pavlov-public-portfolio-bf16, with max_tokens 8192, temperature 0, seed 809, top-p 0.95, and chat_template_kwargs {"enable_thinking": false}. The bridge and the local actor endpoint are two deployments of the same interface contract, and keeping that contract stable across deployment modes is what allows one harness to serve both.

5.9 Recovery Re-implementations under zvf-program/

In the final phase of the project an execution-source loss — deletion of the .codex-run directory — removed the finish-era recovery and build code, confined in scope to the E1 wave10 controller, the E9 restored dependencies, and the E6 validator, while the replacement-lane native runners survived under zvf-program/flagship/. The project's lost-source policy is that sealed requests, validation receipts and test logs pin any faithful re-implementation. Four such re-implementations are worth describing, because together they show what the evidence discipline looks like when it is expressed as code rather than as convention. None of them produced a benchmark number; their value is that their safety properties are testable offline.

The four recovery re-implementations.

Table 5.1: The four recovery re-implementations. Schedule offsets, hashes, reservation identifiers and HMAC domain strings are recorded in the cited source files rather than repeated here.

Artefact	Scope	Safety properties enforced in code	Offline tests	Status
E1 wave10 (zvf-program/e1_wave10/)	Rebuilds the sealed v6 controller for 16 SWE-bench Multilingual tasks (9 gson, 5 terraform, 2 immutable-js)	Acts only on the sealed request hash and its embedded task list; requests built only through the lane's native request builder; v6 process-group contract with unconditional finalisation; GENERATION_FAILED reserved for completed responses without a valid diff; write-once receipts; no retry, resume or replay	10/10 (test_process_group.py 4/4, test_cleanup_regression.py 6/6), matching the lost suite	Reservation recorded RESERVED_NOT_DISPATCHED
E2 driver (zvf-program/e2_core/driver.py)	Schedules the CORE-Bench target-helper GPU chain on GCP (one n1-standard-4 with a T4) under a 3 USD + 1 USD hold	Validation-only by default; refuses --execute without a fresh owner IAM binding receipt and a sealed request; exact early deletes; no retry	12/12 (schedule_arithmetic, deletes, cutoff, UNVERIFIED/LATE reporting, IAM gate)	Blocked on the owner IAM binding

Artefact	Scope	Safety properties enforced in code	Offline tests	Status
E5 successor27 (zvf-program/e5_successor27/controller.py)	Rebuilds the expired sealed packet for 27 never-started Tau3 banking tasks (97 in scope)	Pinned model revision and fresh run identifiers, with the earlier namespace barred; concurrency 3; fixed host, actor and dispatch deadlines; offline <code>verify</code> , <code>plan</code> and <code>gate</code> subcommands	52/52; <code>verify</code> and <code>plan</code> return <code>PLAN_READY_NOT_EXECUTED</code> , <code>gate</code> and <code>launch</code> refuse	Needs a bound authorisation receipt and a native runtime adapter
E13 supervisor (zvf-program/e13_balrog/hosted_supervisor.py)	Hosted supervisor for the BALROG lane; design artefact, not deployed	Absolute deadline $D = t_0 + 3600$ s that is never rebased; short HMAC-authenticated leases that reject replays; renewal and receipt signatures in separate domains; live execution unimplemented and guarded by <code>NeverExecutedError</code>	17/17	Not deployed

Sources: *zvf-program/e1_wave10/README.md*; *zvf-program/e2_core/driver.py*; *zvf-program/e5_successor27/controller.py*; *zvf-program/e13_balrog/hosted_supervisor.py*; *outputs/E1_E14_FINAL_RESULTS_2026-09-19.md*.

The companion E2 `launch_gate.py` is deliberately trivial: it checks only that the amendment acceptance record and the surviving CORE-Bench harness entry points exist, which keeps the gate’s claim small enough to be true.

Read together, these four artefacts are the implementation answer to a question the results chapters raise: how a project maintains evidential integrity when its execution environment is partly lost and partly unreachable. The answer taken here is to re-derive the controllers from the sealed requests and the surviving test records, to keep every re-implementation offline-testable, and to make the no-retry, no-replay, hash-pinned, fail-closed properties explicit in code so that they can be demonstrated rather than asserted. The cost of that discipline is visible: four controllers, three of them complete and one a deliberate stub, produced no new benchmark numbers in this phase. That is the intended trade.

None of the four controllers ran the lanes in the end. On 27 September 2026 a single lead authorisation superseded the sealed-reservation and owner-binding gates they enforce, and each lane was run with a thin runner that calls the benchmark’s native loop and grader directly. The runners live under *outputs/finish_pending_2026-09-27/<lane>/code/*, and each lane’s *result.json* lists the bypass as a deviation (§8.4). The fail-closed properties above therefore describe how the lanes were held, not how they

were measured.

Part III

Results

Chapter 6

Results I: Core GRPO Studies (P1–P4)

This chapter reports the four core GRPO studies that make up the first half of the P1–P8 internal reports: the cross-library and cross-scale behaviour of GRPO training reward (P1), the Zero-Variance Fraction as a diagnostic of signal starvation (P2), the effect of group size on trainability (P3), and the length-bias and held-out-generalisation behaviour of GRPO against its Dr. GRPO variant (P4). Each study is reported here in the form the evidence supports, which in every case is more conservative than the hypothesis that motivated it. Three of the four pillars return a negative or an explicitly bounded result, and the fourth returns a diagnostic whose measured association with the outcome it is meant to predict is weaker than the literature would suggest (Figure 6.1).

6.1 Scope, Sources and Evidence Boundary

The four pillars share one methodological commitment, which is the organising principle of the whole project: every comparative claim is made with the entire training stack held fixed except the one factor under test. Where that was not possible, the result is reported as an association rather than a contrast, and the confound is named. The reader should treat this chapter as the primary result surface for P1–P4; the corresponding LaTeX sources are the canonical artefacts, and every figure quoted below is taken from one of them.

Each study draws on a different evidence base, and the differences matter. P1 pools 70+ runs across seven reinforcement-learning libraries and five model families spanning 0.6B to 1T parameters, but the frontier-scale anchors within that pool are single-seed runs executed through a managed training and inference API, so the cross-scale analysis is descriptive and its uncertainty is wide. P2’s measured magnitudes come from a controlled five-seed panel plus a matched-budget rollout budget check; a portion of its comparative table is a declared simulation projection rather than a set of indepen-

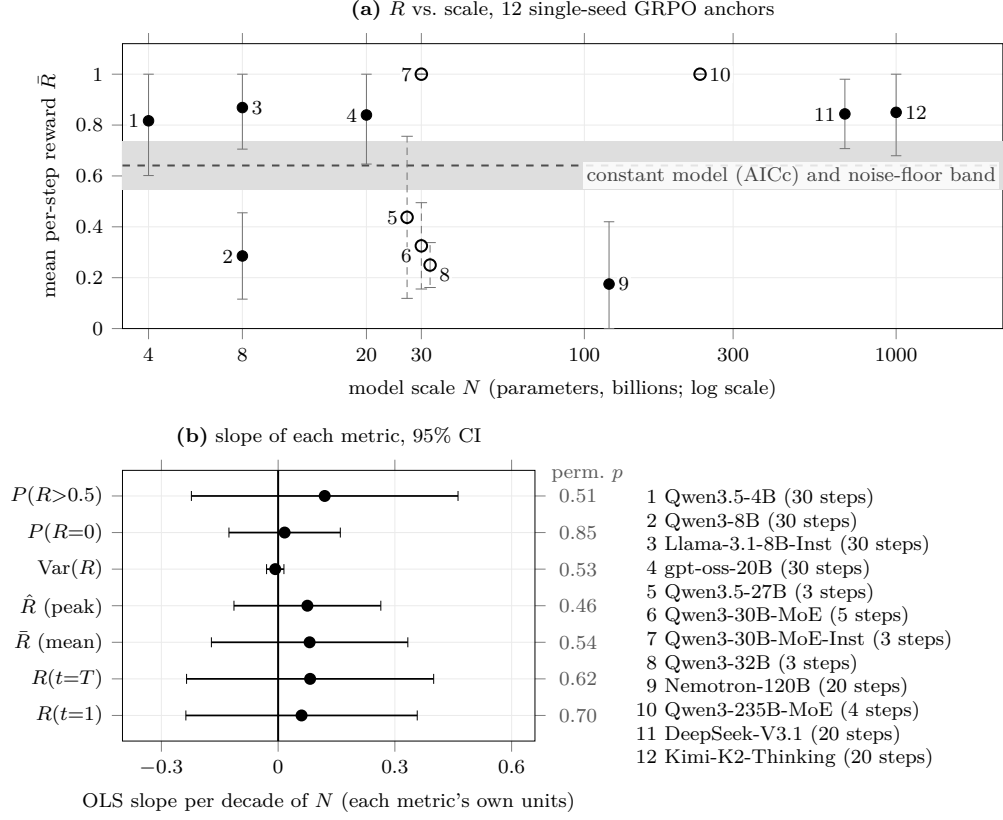


Figure 6.1: Cross-scale behaviour of reward across the studied model range: no reliable monotone scaling trend is recovered. (a) Twelve GRPO anchors from 4B to 1T parameters, one seed each, listed in the key with their number of logged steps. $\bar{R}$ is the mean over a run’s logged steps of the per-step training reward (the fraction of rollouts rewarded, on a 0–1 scale; no further normalisation). Whiskers are ± 1 SD of the per-step reward within the run, clipped to $[0, 1]$; they describe step-to-step noise, not a confidence interval. Open markers have fewer than ten logged steps. The dashed line and grey band are the constant model selected by AICc (pooled mean 0.641) and the $\pm \sigma_{\text{step}}$ noise floor (0.096). OLS slope $+0.081 \pm 0.129$ per decade ($R^2 = 0.038$, permutation $p = 0.54$); Spearman $\rho(\log_{10} N, \bar{R}) = +0.149$ ($p = 0.64$). (b) OLS slope per decade of N for seven summary metrics, with 95% CIs (± 1.96 SE) and permutation p over the twelve anchors. Each slope is in its own metric’s units (reward, squared reward or probability), so slopes are not comparable across rows; every interval includes zero. Sources: experiments/results/scaling_law_extended_frontier.tsv and scaling_law_power_law.tsv.

dent library deployments, and that distinction is preserved below. P3’s measured sweep covers $G \in \{2, 4, 8, 16\}$; every $G = 32$ and above number in the pillar is a joint-fit reconstruction, not a trained arm. P4’s controlled comparison is sixteen runs across two algorithms and two tasks; its roster-level audit is drawn from a separate, larger pool that mixes models, tasks and libraries.

The corpus-level motivation for the fixed-stack commitment (Chapter 4 §4.1) is a variance decomposition computed as four separate marginal one-way ANOVAs, each run against the same 42 experiments with valid performance observations: library/framework $\eta^2 = 0.546$, training algorithm $\eta^2 = 0.558$, model family $\eta^2 = 0.471$, model scale $\eta^2 = 0.322$. These four values sum to 1.897. A set of shares that sums above one is not a partition of a common variance budget, and on a corpus where the factors are collinear a marginal η^2 absorbs the variance of each factor’s correlates, so the values show that framework and algorithm vary a great deal across this corpus and do not estimate either factor’s independent contribution. The historical twenty-test multiple-comparison family that accompanies the decomposition silently includes single-seed rows whose Welch statistics are mathematically undefined, and reduces to a single surviving comparison under the restricted family; it is retained as a reproducibility audit trail and not as a corrected survival claim.

Throughout this chapter, “statistically indistinguishable” means that a formal equivalence or difference test failed to separate the arms at the stated level, not that the arms were demonstrated to be equivalent. Where an equivalence test was run and returned a null, that is stated explicitly.

6.2 P1 — Cross-Library and Cross-Scale GRPO Behaviour

The question P1 sets out to answer is whether the benefit of GRPO post-training scales with model size. The framing is deliberate: the hypothesis of interest is a Chinchilla-style power law in parameter count N , and the P1 report’s declared claim boundary is that it reports a descriptive limits and identifiability audit rather than a positive scaling law. The short answer is that no reliable scaling law is recovered from the reward curves, and that the evidence base is too small and too heterogeneous to falsify a plausible scaling hypothesis even if one were present.

6.2.1 The anchor pool and the fitted saturation forms

The fitted anchor pool spans roughly 2.4 orders of magnitude in parameter count. This is a subset of the full 0.6B–1T roster: for the cross-scale fits, only anchors with a usable per-step reward trace and a known parameter count were retained. Two properties of that pool limit what the fits can support, and both are stated here rather than left to the reader. First, the anchors are not measured on a common footing: five of the twelve are interrupted runs whose recorded values are means over three to five logged steps, against twenty to thirty for the remaining seven, so the dependent variable is a short-horizon snapshot for part of the pool. Second, scale is collinear with training recipe across these anchors rather than independent of it: the learning rate, LoRA rank and related settings change together and only at the scale boundary, so a fit through the pool cannot separate an absent scale effect from recipe changes offsetting scale changes. The null reported below is therefore a statement about this pool as configured, and not an estimate of how reward scales with parameter count in general. The frontier-scale anchors — Qwen3-235B-MoE, DeepSeek-V3.1 at the 685B class, Kimi-K2 at 1T, and Nemotron-120B — were obtained through the Tinker managed training and inference API in a single-seed, descriptive-only regime, since full base weights are not released for those anchors and the project does not own frontier hardware.

Each trace was fitted with a saturating-exponential form $R(t) = R_{\max}(1 - e^{-\lambda t})$, with the saturation rate λ constrained by the optimiser to the interval $[0, 10]$. The fitted results are the first place the negative result becomes visible. Across the five characterised anchors, the ordinary-least-squares trend slope on the raw reward trace, β_{OLS} , satisfies $|\beta_{\text{OLS}}| \leq 0.0037$ in every case — that is, the traces are flat to within a few thousandths of a reward unit per step. Qwen3.5-4B fits a mean reward of 0.817 with a peak of 1.000 and λ pinned at the bound of 10.0; Qwen3-8B fits 0.285 with a peak of 0.625; Llama-3.1-8B-Instruct fits 0.869 with a peak of 1.000; DeepSeek-V3.1 fits 0.844 with a peak of 1.000. Four of these five traces have λ pinned at the fit boundary — that is, the optimiser wanted a faster saturation than the constraint permitted — and the P1 report’s verdict is that for those traces no single saturation exponent is identifiable at all.

The exception is Nemotron-120B, which fits $R_{\max} = 0.182$ with $\lambda = 0.99$, well inside the bound, and which the pipeline classifies as a collapse rather than a saturation. The autopsy table records a peak at step 3, a fitted $R_{\max}$ of 0.875, a late-phase mean R_{late} of 0.208, a drop of -0.667 from peak to late phase, a zero-reward step fraction of 0.55, and a post-peak OLS slope of -0.0036 per step. Across the rest of the pool, the zero-reward step fraction is at most 0.067. The Nemotron run is therefore better described as a distinct collapse phase than as a point on a smooth trend, and this is the reason the pillar’s headline object is a local, stack-conditioned taxonomy of endpoint ceilings rather than a functional form in N .

6.2.2 The cross-scale test and its nulls

Three cross-scale statistics were computed against $\log_{10} N$: the slope of the mean reward, the slope of the peak reward, and the slope of the reward variance. None is distinguishable from zero. The mean-reward slope is -0.024 per decade with a bootstrap mean of -0.117 and a 95% interval of $[-0.796, +0.313]$, Spearman $\rho = -0.07$; the peak-reward slope is $+0.037$ with a bootstrap mean of -0.010 and an interval of $[-0.623, +0.197]$, $\rho = +0.22$; the variance slope is -0.002 with an interval of $[-0.063, +0.027]$, $\rho = -0.14$. A residual-resampling bootstrap with 1,000 draws applied to the Nemotron anchor returns $R_{\max} = 0.310$, $\lambda = 3.26$, $P(\lambda \geq 9.5) = 0.277$ and a t_{80} of 17.87 steps with an interval of $[0.16, 161.4]$ — an interval wide enough to contain almost any behavioural claim. A 70/30 anchor holdout returned slope estimates of $+0.0000$, $+0.0000$, -0.0000 , -0.0000 and $+0.0016$ across the five holdout configurations. Fitting a power law over the full twelve-anchor set from 4B to 1T returns $p \geq 0.46$ for every exponent tested.

Two model-selection checks were run to see whether the saturation story was at least the best available description of the traces. Fitting constant, linear and saturating forms to each trace, AIC and BIC select the constant model on every single trace, with a ΔAIC of at most 2 over the runner-up. That is, a flat line describes these reward curves at least as well as any saturating curve does. A changepoint test with a permutation null estimated change locations $\hat{\tau}$ of 27, 28, 5, 15 and 18 steps across five traces, with permutation p-values of 0.493, 0.789, 0.742, 0.909 and 0.137; none is significant. An effective-plateau-horizon analysis at tolerance $\varepsilon = 0.20$ found that only two of the five traces — Qwen3.5-4B and Llama-3.1-8B-Instruct — reach a horizon at all, both at 27 steps, because the others never satisfy the tolerance.

A compute-adjusted proxy, which folds an estimated token budget into the scale axis, returns a slope of $+482 \pm 490$ per decade. The point estimate is positive and the interval is an order of magnitude wider than the estimate, so the compute axis does not rescue the scaling story either.

6.2.3 Where the geometric template fails

The three-phase learning template used throughout the project — a slow start, a period of rapid improvement, and a plateau, with collapse as a degenerate tail — was tested directly against the anchor pool as an internal consistency check. Only 2 of the 12 anchors match the template’s geometric prediction. The template itself survives as a descriptive vocabulary: it names the failure modes the pipeline detects, and the collapse criteria (peak ≥ 0.4 , late mean below 40% of peak, zero-reward fraction ≥ 0.30) are used in the pillar’s flagging throughout. What does not survive is the claim that these traces realise

the template as a progression.

One further contrast deserves note because it is our own measurement rather than a synthesis. Under a same-stack comparison of PPO and GRPO on a saturated Qwen2.5-0.5B two-digit addition task, evaluation accuracy is 0.990 for GRPO and 0.992 for PPO, and the paired algorithm difference is -0.002 with $p = 0.374$ (paired t on $n = 5$ seeds, each seed’s held-out accuracy on 200 items; source: `platform_hybrid/paper/sections/frontier_synthesis_scaling.tex`; recomputed from `platform_hybrid/experiments/results/samestack_ppo_grpo.json`). The 200 evaluation items are fresh random draws from the training generator, not a disjoint split. This is the project’s closest de-confound: the runner, clipped surrogate and step budget are shared, but the arms also differ in prompts per step (16 prompts $\times$ 8 completions for GRPO against 128 distinct prompts for PPO; Table 4.2). The two estimators are statistically indistinguishable, and because both sit at the ceiling the null cannot separate no effect from no headroom. They are not shown to be equivalent. The five per-seed differences are -0.005 , $+0.005$, -0.005 , 0.000 and -0.005 , each one item of 200, at a 0.98–1.00 accuracy ceiling. A paired TOST with a ± 0.005 margin gives $p = 0.104$, so equivalence at that margin is not established, although it is at ± 0.010 ($p = 0.008$). The exact sign-flip permutation p on the same five pairs is 0.625. It is reported here rather than in P2 or P3 because it is the algorithmic counterpart of the P1 null — the cross-scale story is not the only axis on which GRPO’s advantage fails to separate from its alternatives. The project’s historical effect-size table also carries a Qwen3-8B PPO-versus-GRPO row. It is marked a provenance conflict and not estimable, and its Welch test, recorded in the historical twenty-test family at $p = 0.7605$, is carried as a provenance record rather than as an estimate, so no statistic is quoted for it here.

6.2.4 A withdrawn architectural contrast

An earlier version of this analysis reported a MoE-versus-dense gap across six pairs of anchors, with a mean gap of $+0.338$ and a permutation $p = 0.023$. That claim has been withdrawn. The defect is architectural: the Nemotron-120B anchor is a Latent-MoE, not a dense model, so the original six-versus-six allocation misclassified it. With the classification corrected, the same test returns a one-sided permutation $p \approx 0.178$. A leave-Nemotron-out recomputation returns a gap of $+0.278$ with $p \approx 0.049$, and this too is withdrawn, because removing the misclassified anchor from one side of the comparison does not repair the design. The withdrawal is recorded here because the project’s convention is that retired claims remain visible with the reason for retirement attached.

A related cross-anchor statistic is retained, with its own caveat. Across the twelve anchors, seven have $|\Delta_{1T}| \leq 0.2$, where Δ_{1T} measures how far the anchor’s endpoint

deviates from the three-phase template’s terminal prediction; the largest deviation is Qwen3.5-27B at +0.562 and the largest negative is Nemotron at -0.50 . A Cohen’s κ analysis of the categorical phase assignments is also reported in the source: across three labelling methods, the heuristic and AIC labels agree at $\kappa = +1.000$ on the five-anchor set (both label every trace “plateau”) and $\kappa = +0.625$ on the twelve-anchor set, while the changepoint labels agree with neither (κ between -0.029 and $+0.077$). The supportable summary is that categorical capability structure carries more signal than $\log_{10} N$, which is what the P1 report’s abstract says and what the numbers support.

6.3 P2 — The Zero-Variance Fraction

The second pillar asks a mechanistic question: when GRPO stops learning, is the cause visible in the reward statistics of the groups it samples? The diagnostic proposed is the Zero-Variance Fraction (Figure 6.2).

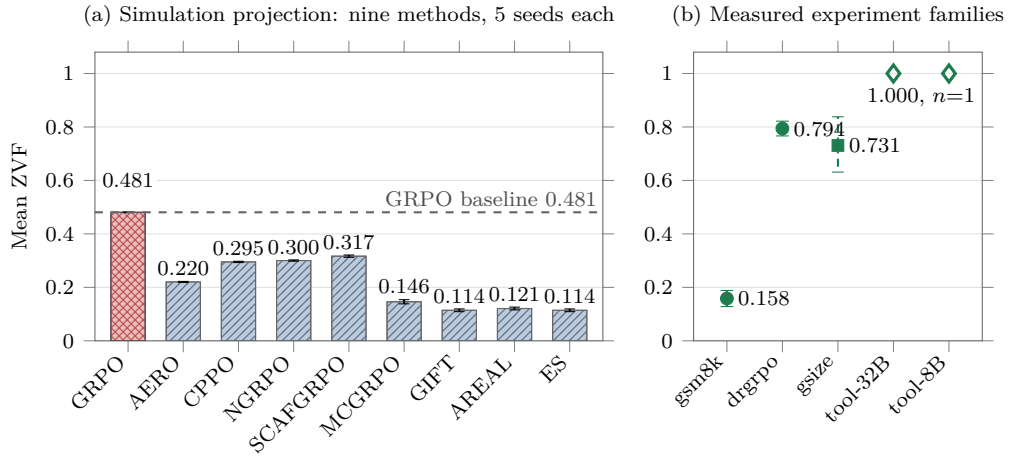


Figure 6.2: Mean Zero-Variance Fraction by method in a simulation projection (a) and by measured experiment family (b); neither panel compares backend libraries. (a) Nine variance-mitigation methods in a simulation projection ($G = 8$, math-verifiable task, 5 seeds each; hatched bars mark projected, not model-training, values): bar = mean of per-seed mean ZVF, whisker = ± 1 SD across seeds (≤ 0.008). The projection ranks no methods; GRPO (cross-hatched) is the reference at 0.481, and in 3 of its 5 seeds the per-step collapse flag fires. (b) Measured runs: gsm8k = Qwen3-8B, three seeds of 200 GSM8K prompts at $G = 8$ (mean ± 1 SD); drgrpo = Qwen2.5-0.5B arithmetic, GRPO and Dr. GRPO, 5 seeds each (10 runs, mean ± 1 SD); gsize = Qwen2.5-0.5B group-size sweep, mean over $G \in \{2, 4, 8, 16\}$ with a dashed whisker spanning the per- G values (a range over G , not seed dispersion); tool-32B (Qwen3-32B) and tool-8B (Llama-3.1-8B-Instruct) are single runs (open diamonds, no interval) in which every group has zero reward variance. Source: platform_hybrid/experiments/results/zvf_by_library.tsv and the per-seed files it cites.

6.3.1 Definition and measurement

For a prompt x sampled from the training distribution, GRPO emits G completions and scores each with a verifier. Centring those rewards within the group produces the advantage signal. If all G completions receive the same reward — all correct, or all incorrect — every centred reward is zero and the group contributes no contrastive signal to that step’s update. The Zero-Variance Fraction is the fraction of sampled groups in a step for which this occurs, and the gradient utilisation is defined as $GU = 1 - ZVF$.

What a zero-variance group does to the update depends on the loss form. The source states the general case: in an objective with a KL penalty or auxiliary loss, a zero-variance group does not make the total gradient zero, and ZVF measures only the fraction of groups contributing zero centred reward contrast. None of the runners used in this project has a KL term or a reference policy (Table 4.2), so for every measured study a zero-variance group contributes exactly zero gradient through the policy term; only optimiser momentum from earlier steps and the batch normalisers carry over. ZVF is therefore a direct measure of wasted rollouts on these runners, and a statement about the group-relative term only on objectives that carry other terms.

The statistic is validated by recomputation. Three stored GSM8K group tensors (Qwen3-8B, $G = 8$, sampling only, 200 problems per seed) were re-read from disk and their ZVF recomputed from the raw per-group rewards; the recomputed values are 0.130, 0.190 and 0.155, pooled 0.1583, matching the reported values exactly. This is the mechanical check that the quantity in the tables is the quantity the code computes. The statistic is also insensitive to the zero-variance threshold: the reported ε -sensitivity is at most 0.4%.

6.3.2 Measured magnitudes across libraries

The headline measurement is a nine-row panel run at $K = 8$ with five seeds and a 100-step rollout budget, spanning vanilla GRPO and eight named variance-mitigation methods. Vanilla GRPO records a mean ZVF of 0.481 against a mean last-10 accuracy of 0.379, with zero of five seeds collapsing, zero of five drifting, and five of five plateauing. The remaining rows are: AERO 0.220 ZVF with 0.399 accuracy; CPPO 0.295 with 0.392; NGRPO 0.300 with 0.387; SCAFRPO 0.317 with 0.409; MCGRPO 0.146 with 0.322; GIFT 0.114 with 0.326; AREAL 0.121 with 0.328; and ES 0.114 with 0.316. The four lowest-ZVF rows each record one of five seeds drifting and four of five plateauing.

The variance-mitigation rows require a caveat that the source states explicitly and that this chapter preserves: they are a declared simulation projection used for sensitivity analysis, and they “should be read as synthetic, not as independent real-library deployments”.

The caveat covers the whole panel. The vanilla GRPO row and its collapse/drift/plateau counts come from the same dry-run file, `experiments/results/variance_mitigation.tsv`, as the other eight rows, and Chapter 2 §2.3 already records that GRPO baseline row as synthetic. None of the nine rows is a measured deployment. Anyone citing the nine-row ordering as evidence that a named method mitigates failure would be citing a projection.

Two further results sharpen the picture, and the first mixes measured and projected rows. First, across 60 runs the late-phase ZVF of GRPO runs sits in the range 0.87–0.97, while early-phase ZVF for variance-mitigation GRPO and for the largest group size sits in the range 0.04–0.13; smaller group sizes start higher, between 0.25 and 0.61. The lag-1 autocorrelation of the ZVF trace is approximately 0.94, meaning the quantity moves smoothly rather than jittering, which is what makes it usable as a warning indicator at all. Of those 60 runs, 45 are the five-seed variance-mitigation projection rows above (corrected; see Table E.1). The measured part is the remainder, which includes the 12-run group-size sweep. The source does not report the ranges or the autocorrelation separately for the measured subset, so these figures describe a pool that is mostly synthetic.

Second, and more directly relevant to practice, the diagnostic rises *after* the reward plateau rather than before it. Measuring first-passage times, ZVF crosses its warning threshold only 1 to 3 steps ahead of the plateau. The recommended operating rule from the pillar is therefore not predictive but custodial: report ZVF alongside mean reward, and treat a rise of more than 0.3 sustained over ten consecutive rollout steps as a warning. The project reports this limitation against its own proposed diagnostic rather than in a footnote.

6.3.3 The all-correct wall and the mastery–incapacity aliasing

The mechanism behind large ZVF values on easy tasks is a saturation effect that the project calls the all-correct wall. A matched-budget panel was run at 2,560 rollouts per arm — $G = 2$ over 160 steps against $G = 16$ over 20 steps — on Qwen3-8B with GSM8K, LoRA rank 4 and batch size 8. The late $G = 2$ arms reach a mean training reward of approximately 1.0 with ZVF between 0.75 and 1.0; the $G = 16$ arms hold ZVF at or below 0.25 at the same rollout budget. The reading is that when the policy solves the task, nearly every group becomes all-correct, nearly every centred reward is zero, and the estimator loses its contrast even though the reward signal is at its maximum. Signal starvation in this regime is driven by ease, not by difficulty — which is the opposite of the intuition that motivates variance-mitigation work.

This produces the pillar’s most important structural caveat. ZVF is unsigned and sym-

metric: an all-wrong group ($K_x = 0$) and an all-correct group ($K_x = G$) score identically. The statistic therefore cannot distinguish mastery from incapacity, and this aliasing is the explanation the project gives for the weak association between ZVF and final held-out accuracy. Under an i.i.d.-Bernoulli model the closed form is $ZVF(p_x, G) = h_G(p_x) = p_x^G + (1 - p_x)^G$, and it follows in one line. If the G rewards of a group are independent Bernoulli(p) draws, the group has zero variance exactly when all G rewards are equal. That happens when all are 1, with probability p^G , or all are 0, with probability $(1 - p)^G$. The two events are disjoint, so $P(\text{zero variance}) = p^G + (1 - p)^G$. At $p = \frac{1}{2}$ this is $h_G = 2^{1-G}$, so $\ln h_G$ falls by $\ln 2$ for each unit increase in G . The project’s source document for this closed form and the aliasing analysis is headed as reasoning contributions and proposed theorems from ChatGPT Pro Extended and Gemini Deep Think, “NOT experiments this paper ran”. That provenance is kept on record, and AI assistance is declared in the front matter, but the result does not rest on it: the derivation above is elementary and is the argument this thesis relies on. What the project itself measured is the empirical ZVF against the closed form, evaluated at the per-step training reward rate because ZVF is a training-batch quantity (corrected; see Table E.1). Recomputed from the per-step training rewards in `platform_hybrid/experiments/results/groupsize_zvf_sweep.json` (3 seeds $\times$ 40 steps per arm), the per-step-average null is 0.828, 0.709, 0.597 and 0.522. The measured mean ZVF is 0.838, 0.764, 0.691 and 0.631, so the residuals are +0.010, +0.055, +0.094 and +0.109: measured ZVF sits *above* the null at every G . Gradient utilisation rises across the same range from 0.162 to 0.369. The direction of that gap depends entirely on which baseline is used, and the project’s own formalisation names the correct one. Evaluating the closed form at a single pooled or held-out rate overstates the Bernoulli prediction, because h_G is convex and the per-step reward rate varies; the right baseline is the per-step average of the closed form, which is 0.60 at $G = 8$ rather than the 0.33 obtained at the pooled mean. Against the per-step-average baseline the measured ZVF at $G = 8$ (0.69) *exceeds* the null by approximately 0.09, and the source is explicit that most of that apparent gap is the Jensen effect rather than correlated completions and that it therefore “do[es] not read it as strong evidence that the independence null is a lower bound”. This chapter uses only the per-step-average baseline. It is a weak consistency check on shape rather than evidence about dependence. The formalisation’s own summary of the same sweep, twelve runs and 480 logged steps, reports per-step observed ZVF correlating at $r = 0.92$ with the Bernoulli-null prediction $p^G + (1 - p)^G$, with mean ZVF of 0.84, 0.76, 0.69 and 0.63 at $G = 2, 4, 8$ and 16 respectively. That correlation is a shape check, not a validation: the null for each step is computed from the same step’s mean reward, so a high r is partly mechanical. At larger scale the same instrument was run on Qwen3-8B against GSM8K test problems through the Tinker managed API at $G = 8$, temperature 1.0, over 200 problems and three

seeds (sampling only, no training; these are the three stored tensors of §6.3.1), yielding a mean ZVF of 0.16 composed of 12.7% all-correct groups, 3.2% all-wrong groups and 84% mixed groups; the raw correlation with the mechanical null is approximately $r = 0.93$ with rank $\rho \approx 0.65$, and the i.i.d. null over-predicts (approximately 0.28 against the observed 0.16). The direction of the deviation from the null is therefore opposite in the two runs: above the null on the 0.5B sweep, below it on this Qwen3-8B sampling run. The source does not state which baseline produced the 0.28, and neither deviation is read as evidence about dependence between completions.

6.3.4 Prognostic value

The pillar’s central claim is not that ZVF is large but that it is informative, and here the results split cleanly. Pooling the measured runs, the naive correlation between mean ZVF and last-10 accuracy is $r = +0.22$, which is not significant. The residual table on disk pools $n = 17$ per-experiment rows, and at $n = 17$ the Fisher-z 95% CI is $[-0.29, +0.63]$ (corrected; see Table E.1). After mean reward is regressed out of last-10 accuracy, the pooled residual correlation with ZVF is $r = +0.205$ (Fisher 95% CI $[-0.31, +0.62]$, $p = 0.43$, $n = 17$ experiments), and its interval includes zero. The source’s own conclusion from that analysis is that “we therefore do not treat ZVF as an incremental predictor above reward mean”, and this chapter keeps that conclusion. Neither the raw nor the residualised diagnostic is shown to predict graded outcome on measured data.

Pooling across task types shows why a pooled association cannot be read as a score. Over $N = 15$ experiments spanning five model families, the association between ZVF and performance is Pearson $r = -0.769$ ($p = 0.0008$) with Spearman $\rho = -0.784$ ($p = 0.0005$), but it is confounded by task type: tool-use experiments reach ZVF = 100% at approximately 0% reward while GSM8K experiments average ZVF = 8.5%. Within GSM8K the association weakens to $r = +0.40$ ($p = 0.09$, $N = 19$), and then to $r = +0.13$ ($p = 0.79$) when restricted to the seven model-varying runs. The $N = 19$ is a separate and larger pool of Tinker GSM8K runs, not a subset of the $N = 15$ pooled experiments, so the two analyses are not nested. A statistic whose sign is set by which tasks are included in the pool is not a score. On the instrumented runs ZVF also does not track final accuracy ($r \approx 0.09$, $n = 12$).

Against catastrophic collapse, ZVF performs much better than it does against graded outcome. Mean ZVF correlates with catastrophic collapse at Spearman $\rho = 0.56$ (Fisher 95% CI $[0.19, 0.79]$, $n = 23$ pooled cells) and point-biserial 0.62. Its correlation with final held-out accuracy is $\rho = 0.27$, and that is not significant: the registry’s bootstrap 95% CI is $[-0.37, 0.88]$ and the Fisher CI is $[-0.16, 0.61]$, $p = 0.21$, $n = 23$. The 23 cells pool 80 heterogeneous local rows, and the source does not state how many of them are

variance-mitigation projection rows. A three-channel risk index built on the diagnostic — magnitude, critical slowing down via a rolling lag-1 autocorrelation with window 15, and drift slope, blended 0.3/0.5/0.2 with a max-fusion variant — was evaluated over a 52-row cross-experiment panel comprising 45 variance-mitigation seeds, 2 tool-use collapse anchors and 5 scaling-law collapse anchors, with 11 failures and 41 safe runs. Cross-experiment AUROCs are 0.663 for magnitude, 0.758 for critical slowing down, 0.579 for drift, 0.763 for the blended risk index and 0.929 for the max-fusion index; within-methods, the same values fall to 0.073, 0.335, 0.293, 0.396 and 0.805 respectively. None of these AUROCs is a measurement on real training runs, and this chapter reports them as a synthetic-panel exercise. All 45 variance-mitigation rows come from the declared simulation projection (§6.3.2). The seven collapse anchors enter with channel values fixed by the analysis script, not measured: mean ZVF 1.0 or an imputed 0.95, lag-1 0.95 or 0.85, and slope 0 or 0.015. The 41 safe rows are therefore all synthetic. The max-fusion index was selected after the per-axis results were seen. For the 0.929 figure, a Hanley–McNeil 95% CI with 11 positives and 41 negatives is [0.82, 1.00]; that interval describes this constructed panel only. No AUROC is computed on the measured rows alone, because they contain too few failures to estimate one. The gap between the cross-experiment and within-method columns is a warning about the index’s transportability: most of the cross-experiment discrimination is coming from the fact that collapse clusters by experimental family, not from a within-family predictive signal. The declared thresholds are a risk above 0.55 to treat as failure-positive, below 0.30 as safe, and the interval between as requiring investigation.

The pillar’s own conclusion names three load-bearing limitations: the diagnostic is mechanically coupled to reward sparsity, group size and baseline accuracy; its strong association holds only for catastrophic collapse and not for the graded held-out outcome; and the unsigned statistic aliases mastery with incapacity. A proposed replacement, the pairwise contrast density, is introduced in the synthesis document with a target association of $\rho \gtrsim 0.45$; the project states that this bar is not yet met by the replacements and that the head-to-head protocol remains future work.

6.4 P3 — Group Size and Contrast Density

The third pillar studies G , which plays two roles simultaneously in GRPO: it is a variance-reduction knob, like a Monte-Carlo baseline, and it is a preference-density knob, since centring rewards within a group induces implicit pairwise contrasts — nonzero only for mixed-reward groups. Practitioners often treat larger G as strictly better by analogy to variance reduction, which conflates the two roles and ignores cost. The pillar’s conclusion is that the measured evidence does not justify a group-size optimum

and that the dependence of trainability on G is non-monotone (Figure 6.3).

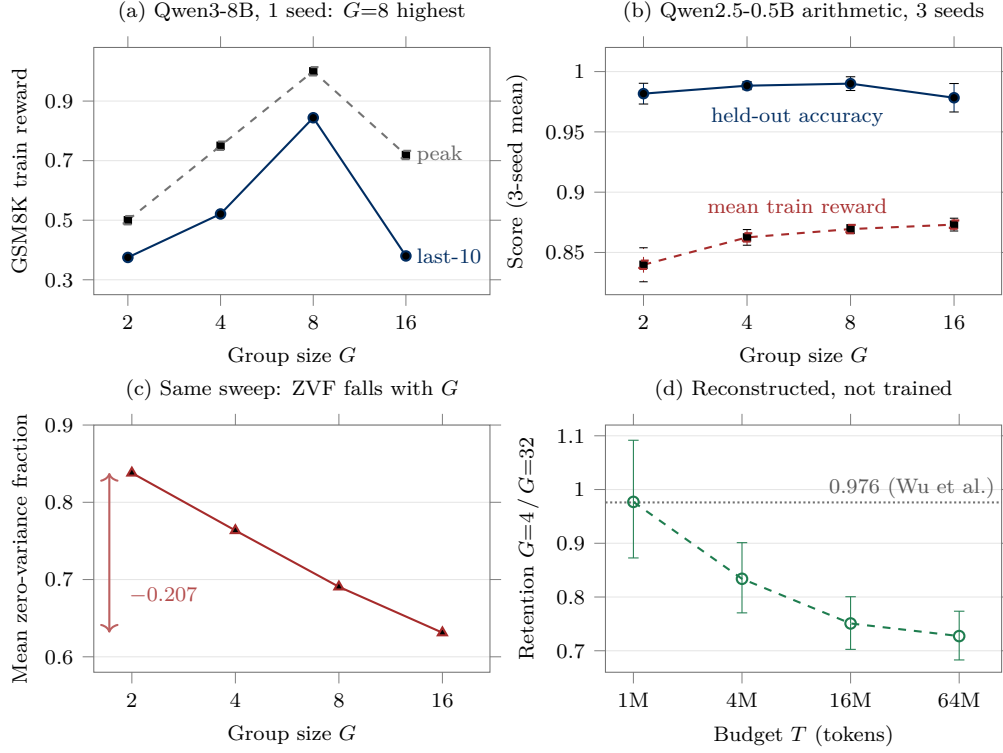


Figure 6.3: Group size G : one single-seed sweep, one three-seed sweep and one reconstruction. (a) Qwen3-8B on GSM8K through Tinker, one seed, 30 steps: peak and last-10 in-training reward (not held-out); $G = 8$ is highest, but with no seed replication no optimum can be established. (b) Qwen2.5-0.5B on arithmetic, 3 seeds, 40 steps: held-out accuracy and mean train reward, bars ± 1.96 SE (approximate 95% CI). The held-out apex at $G = 8$ (0.990) overlaps $G = 4$ (0.988) and $G = 16$ (0.978); overlap is descriptive, not a test, and the arms are statistically indistinguishable. (c) Mean zero-variance fraction on the same sweep, falling by 0.207 from $G = 2$ to $G = 16$. (d) Reconstructed retention of $G = 4$ relative to $G = 32$ against token budget (dashed line and open markers: reconstructed values, not matched-budget training runs; bars are the file’s conditional reconstruction intervals). The point estimate reaches the Wu et al. 0.976 retention level at 1 of 4 budgets ($T = 1$ M), but the equivalence test at that margin is not passed at any budget. Sources: paper/expected_results.json (a); experiments/results/group_size_effect.tsv (b–d).

6.4.1 The measured sweep

The primary sweep measures $G \in \{2, 4, 8, 16\}$ on a near-ceiling two-digit addition task with Qwen2.5-0.5B, three seeds and 40 steps, evaluated on 200 fresh draws from the training generator. Held-out accuracy (200 fresh problems per run) traces an inverted U with its apex at $G = 8$. Values are mean $\pm$ SE over $n = 3$ seeds: 0.9900 ± 0.0029 (SD 0.0050) at $G = 8$, 0.9817 ± 0.0044 (SD 0.0076) at $G = 2$, 0.9883 ± 0.0017 (SD

0.0029) at $G = 4$ and 0.9783 ± 0.0060 (SD 0.0104) at $G = 16$. At $n = 3$ seeds the arms are statistically indistinguishable: a one-way ANOVA across the four arms gives $F(3, 8) = 1.82$, $p = 0.22$, recomputed from `platform_hybrid/experiments/results/gro_upsize_zvf_sweep.json`. The minimum detectable effect at 3 versus 3 seeds is $d = 3.07$, so only very large differences could have been detected. A separate check run at $B = 4000$ with seed 20260702 records $G = 2$ and $G = 16$ at 0.9817 and 0.9783, a difference of +0.0033 ($G = 2$ minus $G = 16$) and an interval of $[-0.008, +0.015]$; $G = 4$ and $G = 8$ add +0.010 and +0.012 respectively, and $G = 4$ versus $G = 32$ was not measured on that sweep.

The retention figures are the cleanest way to state the same result. $G = 2$ retains 100.3% of $G = 16$. A later unified synthesis at the same configuration reports “retention” ratios and signal-to-noise ratios with intervals. The “retention” is a ratio of last-10 *training* reward, not of held-out accuracy, so this chapter calls it a training-reward ratio. The point values are $G = 2$ 1.0000, $G = 4$ 0.9941, $G = 8$ 0.9960 and $G = 16$ 1.0068, with SNR 1.46, 1.76, 1.94 and 2.16. All four ratios sit above the Wu et al. [20] 97.6% reference bar. At the seed level (paired TOST on per-seed last-10 training reward against $G = 2$, margin ± 0.02 , $n = 3$ seeds), the TOST p-values are 0.012 at $G = 4$, 0.043 at $G = 8$ and 0.056 at $G = 16$ (corrected; see Table E.1). Equivalence to $G = 2$ at that margin is therefore supported, narrowly, for $G = 4$ and $G = 8$, and is **not** established for $G = 16$.

The signal-to-noise ratio grows sublinearly with G : it reaches about 52% of the $\sqrt{G}$ ideal, and about 48% above a flat baseline. Reward itself is statistically flat across the sweep, in the range 0.971–0.984. The marginal ZVF gain from $G = 8$ to $G = 16$ is $0.6906 \rightarrow 0.6312$, a fall of 0.059, which is smaller than the $G = 2 \rightarrow G = 4$ fall of $0.8380 \rightarrow 0.7635$, a fall of 0.075. Diminishing returns on the contrast axis are visible in the measured data, which is what the reconstruction later formalises.

The pillar’s own framing of this regime is important and is stated in its abstract: near the ceiling, variance is compressed, so the finding is “no detectable G effect under saturation”, not evidence of equivalence on harder, unsaturated tasks. That is, the flat retention curve is a property of an easy task, and the pillar explicitly declines to generalise it. The measured equivalence tests were run up to $G = 16$, and at the seed level they establish equivalence only up to $G = 8$; there is no directly trained, matched-budget $G = 32$ cell in the primary sweep. The retention-versus-budget axis is where the picture changes: $R(1M) = 0.976$, $R(4M) = 0.833$, $R(16M) = 0.750$ and $R(64M) = 0.727$, fitted by $R(T) = 0.729 + 0.247 e^{-T/5.76M}$ with $R^2 = 0.948$ and an asymptote of 0.729. The four points are themselves reconstructed (§6.4.2), and a three-parameter curve through four points leaves one residual degree of freedom. The fit is therefore illustrative only, and no confidence interval is reported for its asymptote (corrected; see Table E.1). The Wu reference line and this reconstructed curve cross at $T = 1M$ and diverge by 25 percentage

points at $T = 64M$. The operative conclusion is scoped to the 0.5B arithmetic task: no G is distinguishable there, so $G = 2$ is the cheapest setting at statistically indistinguishable accuracy. It is not a recommendation for harder tasks, and it does not conflict with the single-seed Qwen3.5-4B observation of §6.4.3 that $G = 2$ under-trains with a 50% batch collapse, which is a different model and task.

6.4.2 The same-stack four-axis audit and the reconstructed budgets

The strongest de-confound for P3 is a four-axis factorization audit of the same training stack. On the algorithm axis, the residual effect is small: $\eta^2 = 0.023$, from the same five-seed same-stack contrast as §6.2.3. The paired GRPO–PPO difference is -0.002 . The factorization file reports a Monte-Carlo permutation p of 0.62; the exact sign-flip p over the five pairs is 0.625, and the paired t p of 0.374 in §6.2.3 is the primary test for this contrast. Equivalence on the algorithm axis is not established, because the TOST at ± 0.005 in §6.2.3 does not establish it (corrected; see Table E.1). On the group-size axis the file reports $\eta^2 = 0.54$ and 0.63, with Cohen’s $d = -1.47$ for the $G = 2 \rightarrow 16$ comparison. The 0.54 is `eta2_G_last10`, a share of last-10 *training reward*, not of terminal held-out accuracy; 0.63 is for gradient norm. Recomputed on the 12 sweep runs (3 seeds per G), the training-reward share gives $F(3, 8) = 3.08$, $p = 0.090$. The corresponding held-out share is $\eta^2 = 0.41$, $F(3, 8) = 1.82$, $p = 0.22$. The source itself grades the G -axis result only SUGGESTIVE. The supportable reading is that the group-size axis shows a large but not statistically significant share of training-reward variance at $n = 3$ seeds per arm, and that the algorithm label is not shown to matter on this stack.

A Miller-style [62] error-bars audit covered the pillar’s four headline contrasts. Two of them are held here at hypothesis-generator status rather than reported as precise (corrected; see Table E.1): the gradient-utilisation ratio at $T = 1M$, with a point estimate of 5.03 and no interval that $n = 4$ budgets can support, and the negative retention-versus-training-token slope, which is fitted to the four *reconstructed* budgets of the grid below, not to trained arms. The SNR-versus- $\sqrt{G}$ slope has an interval of $[+0.148, +0.583]$, which contains the ideal value of $+0.500$ and is therefore graded NULL. The native Wu-style equivalence claim that $G = 2 \approx G = 16$ is also graded NULL. This chapter therefore reports none of the four P3 headlines as precise, and the pillar’s own summary is that the data do not justify a universal optimum.

Everything at or above $G = 32$ in the pillar comes from an explicitly reconstructed token-budget grid, retained only as a hypothesis generator. The reconstruction table gives, for $G = 4$ versus $G = 32$, a $T = 1M$ comparison of 0.41 against 0.42 with a difference of -0.01 , a 97.6% retention ratio and a “generalizes” verdict of yes, a point-estimate criterion of retention at least 90% — the equivalence test at the 97.6% margin, with a reconstruction

interval of $[0.873, 1.092]$ at $T = 1\text{M}$, is not passed at any of the four budgets; at $T = 4\text{M}$, 0.55 against 0.66 with -0.11 , 83.3% and no; at $T = 16\text{M}$, 0.63 against 0.84 with -0.21 , 75.0% and no; at $T = 64\text{M}$, 0.64 against 0.88 with -0.24 , 72.7% and no. The source labels these as conditional reconstruction intervals, not fresh-run confidence intervals. The $T = 64\text{M}$ accuracy row across $G = 4, 8, 16, 32$ and 64 reads 0.64, 0.80, 0.85, 0.88 and 0.87 — the apex moves with budget, which is precisely the pattern that makes a universal G optimum incoherent, and precisely the pattern the project declines to claim as measured. A fit of the critical-budget form $|\Delta(T)| = a(1 - e^{-T/\tau})$ gives $\hat{a} = 0.30$ and $\hat{\tau} \approx 12.1\text{M}$ tokens, with a 5% critical budget of 2.21M tokens and an interval of $[1.29\text{M}, 4.77\text{M}]$. A difficulty-gated adaptive rule derived from this analysis reports a mean selected group size of 7.0 and a projected 56.2% rollout saving against always using $G = 16$, with 5 of 20 cells under-predicting by more than 5 percentage points. These are projections, and the chapter marks them as such.

A separate conditional utility audit over 505 tasks, scoring each group size by $(1 - \text{ZVF})/\sqrt{G}$, selects $G = 4$ over $G = 5$ by 0.00177 with a 95% bootstrap interval of $[0.00082, 0.00270]$. The margin is positive but small, and it is conditional on cohort and objective.

6.4.3 The curriculum and matched-token null results

Two interventions that P3’s design space naturally suggests were run and both returned nulls. They are reported here rather than elsewhere because they are group-size interventions, and because their negative results are load-bearing for the chapter’s overall argument.

The first is a difficulty curriculum that filters collapsed groups during training. The intervention works as designed on its own terms: it drives the zero-gradient waste fraction from 0.50 to 0.00, eliminating wasted rollouts completely. It does so at approximately 4.81 times the sampling cost, and its held-out gain is identical to the baseline’s — both arms move from 0.80 to 0.85 on a held-out set of $n = 20$ (one seed, eight steps), a gain of $+0.05$ in each case, which is one item of 20. At a matched 30k-token budget the curriculum ties the baseline at $+0.028$ against $+0.028$, and at five times the cost it loses. The verdict recorded is Null. An important provenance note attaches to this result: an independent verification pass conducted on 6 July 2026 flagged the curriculum claim, along with a single-seed $G = 4$ “win” of $+0.125$, as SUSPECT and overclaimed, before multi-seed re-testing confirmed both as noise. The group-size sweet-spot result is itself a null for the same reason — the $+0.125$ advantage at $G = 4$ rested on a single held-out example and did not survive multi-seed re-testing, and $G = 2$ under-trains with a 50% batch collapse and a $+0.0$ gain.

The second null family concerns loss formulations intended to remove the length bias.

On the held-out metric, the standard sum formulation is nominally the best of the three tested: +0.125 against +0.000 for mean-normalisation and +0.042 for the surprise formulation. These are measured on held-out sets of $n = 12\text{--}20$ per arm, so each difference corresponds to about one or two items and none is distinguishable from noise. The length change under the standard arm is recorded as $\Delta\text{len} = -16$. The verdict is Null. A fourth null, on a step-1 predictive layer-freeze, is reported in the same table because it was proposed as a cheap way to stabilise group-relative training: it achieves a per-step overlap of 1.0 on a 1.5B hardcoded-arithmetic toy, but only 0.11 — approximately chance, on both seeds — at 3B on real GSM8K, while the concentration statistic still holds at approximately 0.39. A later four-seed pass at 3B supersedes that two-seed figure with a mean overlap of 0.083 ± 0.048 (mean $\pm$ SD, $n = 4$ seeds), also approximately chance. Verdict: Null, and a reversal of its own toy-scale positive under scaling.

The pattern across all four nulls is consistent and worth stating plainly: the interventions that target the mechanism the diagnostic identifies do not improve held-out outcomes at matched cost. That is a negative result about the diagnostic’s prescriptive power as much as about the interventions themselves, and it is one of the reasons P2 is reported as a monitoring tool rather than an optimisation objective.

6.5 P4 — Length Bias and Held-Out Generalisation

The fourth pillar tests the most widely repeated specific criticism of GRPO: that it rewards verbosity, inflating completion length in a way that Dr. GRPO corrects. The pillar’s headline is a scoped null, and the scope is the point (Figure 6.4).

6.5.1 The controlled null

Sixteen controlled runs were analysed: ten on an arithmetic task with Qwen2.5-0.5B over 40 steps, and six on GSM8K chain-of-thought with Qwen2.5-1.5B-Instruct over 30 steps, split evenly between GRPO and Dr. GRPO. The arithmetic half is saturated: evaluation accuracy is 0.987 for GRPO and 0.992 for Dr. GRPO (paired $p = 0.35$, five seeds), and completions average 4.7 tokens, so its accuracy and length nulls are nulls at a ceiling. Only the GSM8K-CoT half has headroom. At these 30–40-step horizons, neither algorithm exhibits the verbosity-trap signature on either task: the pillar’s length-bias flag, which fires when completion length grows while reward stays flat or falls, is zero on every one of the sixteen runs. That flag is not the roster audit’s peak-then-decay rule of §6.5.2. Recomputed from the per-step logs, the peak-then-decay rule (peak before 65% of training, last-10 mean reward below 90% of peak) fires on none of the ten arithmetic runs but on five of the six GSM8K-CoT runs, three GRPO and two Dr. GRPO,

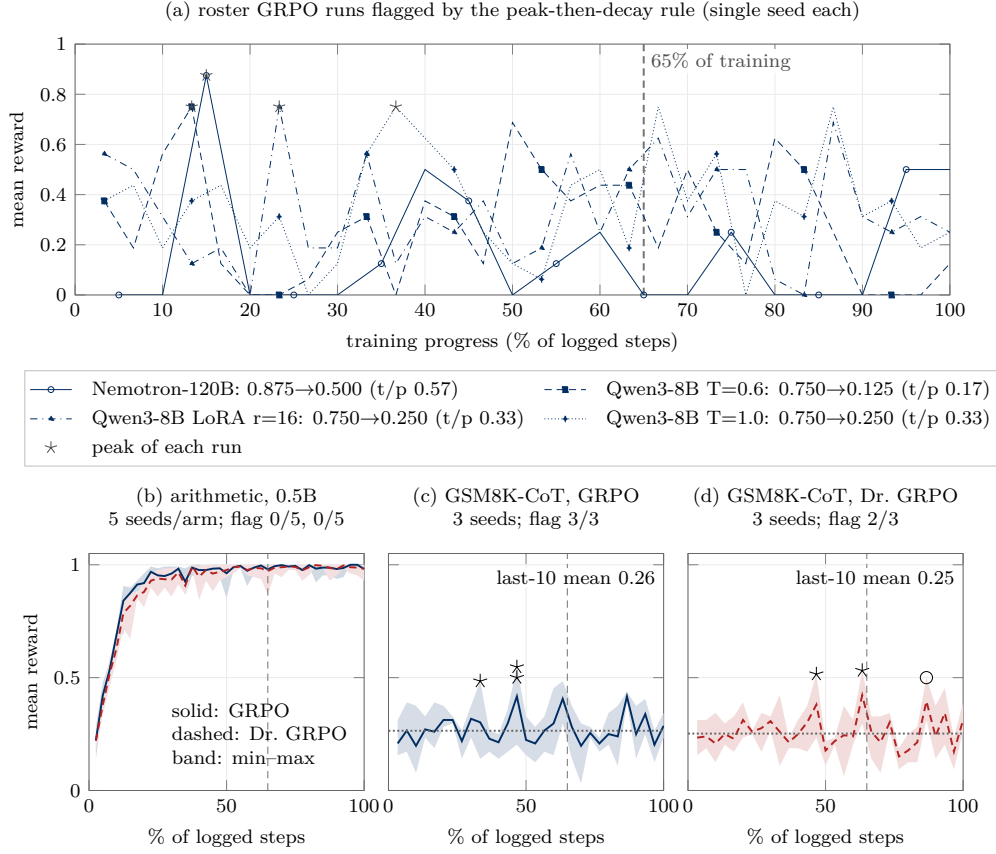


Figure 6.4: The post-hoc peak-then-decay rule (peak before 65% of logged steps and terminal reward below $0.90 \times$ peak) on the roster and on the controlled comparison. (a) The four of eleven roster GRPO runs that carry the flag (single seed each; legend gives peak→terminal reward and t/p = terminal÷peak; stars mark peaks). (b)–(d) The controlled sixteen-run comparison, seed mean (line) and min–max across seeds (band); dashed vertical line at 65%, dotted horizontal line at the arm’s last-10 mean. (b) Arithmetic, Qwen2.5-0.5B, 40 steps, 5 seeds per arm: every seed peaks at 1.0 and stays there, so the rule fires on 0 of 10 runs. (c, d) GSM8K-CoT, Qwen2.5-1.5B-Instruct, 30 steps, 3 seeds per arm: seed peaks (stars = flagged, open circle = not flagged) fall at 33–63% of training and the last-10 mean (0.26 GRPO, 0.25 Dr. GRPO) is far below every peak, so the rule fires on 3 of 3 GRPO and 2 of 3 Dr. GRPO runs. Completion length falls in all sixteen runs, so the pillar’s length-bias flag (length rising while reward stays flat or falls) is zero everywhere: the peak-then-decay rule marks noisy reward in both algorithms, not length inflation. Flags recomputed from the per-step logs in `drgppo_vs_grppo.json` and `drgppo_gsm8k_cot_full.json`.

because the reward trace on that task is noisy and peaks early; length still falls in all five. The summary correlations are as follows, given as mean $\pm$ SD across seeds ($n = 5$ arithmetic, $n = 3$ GSM8K-CoT). On arithmetic, the rank correlation between training step and completion length is -0.405 ± 0.176 for GRPO and -0.307 ± 0.181 for Dr. GRPO; on GSM8K-CoT it is -0.622 ± 0.301 and -0.355 ± 0.255 . The step-versus-reward correlations are $+0.709 \pm 0.082$ and $+0.753 \pm 0.074$ on arithmetic, and $+0.046 \pm 0.162$ and $+0.025 \pm 0.091$ on GSM8K-CoT. The length-versus-reward correlations are -0.419 ± 0.155 and -0.293 ± 0.182 on arithmetic, and -0.501 ± 0.258 and -0.618 ± 0.118 on GSM8K-CoT. Mean reward over the last ten steps is 0.988 for both algorithms on arithmetic, and 0.265 for GRPO against 0.252 for Dr. GRPO on GSM8K-CoT. The direction of the length trend is negative everywhere: all sixteen length slopes are negative, with the arithmetic GRPO slope at -0.029 tokens per step and the GSM8K GRPO slope at -0.481 , and the count of runs with a positive horizon is zero. On the harder task, GRPO’s final-ten-step mean completion length is 184.4 tokens against Dr. GRPO’s 189.3 — a difference of $+2.7\%$ in the direction opposite to the critique.

The pillar’s abstract states the scope of this null explicitly. The claim is bounded by the 200-token cap used in that regime, and the summary sentence is that “the verbosity-trap signature is absent in this regime”. The pillar is not claiming that GRPO never inflates length; it is claiming that within a capped, short-horizon regime it does not, and it names the length-confounded regime in which a Dr. GRPO advantage should appear as future work.

6.5.2 The roster-level verbosity-trap audit

Set against the controlled null is a descriptive audit over the whole experiment roster, and here the picture differs. The audit defines four statistics per trace with at least five steps: an instability index $\sigma(r_{\text{tail}})/|\mu(r_{\text{tail}})|$ over the final 50% of training; a monotonicity score, the fraction of consecutive step-pairs with strictly improving reward; a regression severity, the largest normalised drop from peak; and a binary verbosity-trap flag, set when the peak occurs before 65% of training and the terminal reward falls below 90% of peak.

On that definition, the eleven GRPO runs in the roster have a mean instability index of 0.267 ± 0.335 , against 0.785 ± 0.297 (mean $\pm$ spread across runs; the source does not label the $\pm$, and its size relative to the mean implies an SD) for the seventeen PPO-labelled runs. Four of the eleven GRPO runs, 36% (Wilson 95% CI [15%, 65%], one run per configuration), carry the flag on this definition, including both Qwen3-8B runs and the Nemotron-120B run. That count is reported here as a description of the roster, not as evidence of length bias, because the flag does not isolate the behaviour it is named

for. Three properties of the rule prevent the stronger reading. Its thresholds are post hoc: neither the 65% peak position nor the 0.90 terminal-to-peak ratio is pre-registered anywhere in the repository, and the rule cannot be traced to a commit that predates the traces it judges. It is not specific to length: the metric it tests is in-training reward, and the project’s own source reads the flag as “a generic instability detector” rather than a length measure, so a run can carry it without any change in response length. And its base rate is high: applied to the wider trace pool rather than this eleven-run subset, the rule fires on the majority of runs — including PPO-labelled runs and the Dr. GRPO control in the one matched comparison — so 36% sits at or below the rate the rule produces on traces with no decay at all. The project’s own length data point the other way: under the 200-token cap, length drifts from 194.4 to 183.5 tokens, i.e. shorter. The run-level audit therefore does not establish length inflation, and the pillar’s negative finding rests on the controlled comparison below rather than on this roster count. The Nemotron run records the highest instability index in the pool at 1.194, with a regression severity of 1.0 and a terminal-to-peak reward ratio of 0.57. The two DeepSeek-V3.1 GRPO runs, by contrast, show markedly lower instability, with a mean of 0.143. The PPO rate of 12/17 has a Wilson 95% CI of [47%, 87%], which overlaps the GRPO interval, so the two rates are not distinguished either.

Two caveats on that comparison must travel with it. First, the PPO pool is dominated by classic-RL library runs (SB3, CleanRL, Tianshou) that failed to train at all, with last-10 rewards below 5%; a near-zero reward trajectory mechanically produces a high instability index. Restricted to LLM-native PPO, the PPO Llama instability is 0.105, lower than GRPO Llama’s 0.228, and the source therefore explicitly declines to claim that GRPO is more or less stable than PPO at the algorithmic level. Second, the audit’s reward metric is the in-training reward, not a held-out one, and the audit pool is single-seed per configuration: each Tinker configuration was run once and is reported descriptively. The only multi-seed matched base-versus-RL control on Qwen3-8B is the five-seed comparison discussed below.

An earlier draft of this report’s summary attributed to P4 the claim that Dr. GRPO is the only algorithm without the verbosity-trap pattern. That claim could not be located in any P4 source. What the sources state is that 36% of the audit’s GRPO runs carry the flag; the audit does not report a per-algorithm flag rate for Dr. GRPO, and in the controlled sixteen-run comparison the length-bias flag is zero for both algorithms while the peak-then-decay rule fires on both (three of three GRPO and two of three Dr. GRPO runs on GSM8K-CoT). The finding recorded here is therefore the sourced one: the peak-then-decay pattern is not specific to GRPO, it appears in both arms of the controlled comparison without any length inflation, and the project makes no claim that Dr. GRPO is uniquely free of it. The narrowest related statement the sources do support

is a length-drift one — that Dr. GRPO is the only algorithm whose length keeps drifting upward after the initial compression phase — which is a different statistic and is not the verbosity-trap flag.

6.5.3 Elevated analyses

Because the controlled 30–40-step null is a null, the pillar runs a sequence of progressively more sensitive analyses to establish whether the trap is present but invisible at that horizon. Four of these deserve report, and all four are exploratory. They were run across iterations 12, 20 and 24 of the analysis log after the controlled null was known. The seed counts are small: $n = 5$ arithmetic and $n = 3$ GSM8K seeds, and three seeds admit only 10 distinct bootstrap resamples. Several of the intervals resample autocorrelated steps or windows within runs rather than seeds. The p-values below are uncorrected unless stated, and roughly fifteen tests are reported across the three iterations.

A sliding-window trap-onset analysis with window $W = 10$ relaxes the flag from “peak before 65% of training” to “local peak followed by decay within a window”, and finds traps in 2 of 10 arithmetic runs (20%), 4 of 6 GSM8K runs (67%) and 6 of 16 runs overall (37.5%) — every one of which is a local re-expansion rather than a terminal divergence. This is the analysis that most nearly matches the roster-level trap rate. It is a point estimate of trap prevalence in the controlled data, not an upper bound (corrected; see Table E.1). The Wilson 95% CIs are [5.7%, 51%] for 2/10, [30%, 90%] for 4/6 and [18.5%, 61%] for 6/16, with runs as the unit.

A paired bootstrap on the Dr. GRPO minus GRPO difference in rank correlations gives, on arithmetic, $\Delta\rho(\text{step}, \text{len}) = +0.098$ with an interval of [+0.073, +0.119] — excluding zero, which suggests that Dr. GRPO compresses length relative to GRPO on the easy task (seed-paired bootstrap, $n = 5$); $\Delta\rho(\text{step}, \text{reward}) = +0.043$ with an interval of [−0.024, +0.112], which is not significant; and $\Delta\rho(\text{len}, \text{reward}) = +0.126$ with an interval of [+0.045, +0.201], again excluding zero. On GSM8K the same three quantities are +0.267 [+0.164, +0.419], −0.020 [−0.069, +0.065] and −0.117 [−0.265, +0.015]. The exploratory reading is that Dr. GRPO’s length attenuation is task-dependent, and that on the harder task it deepens rather than weakens the length–reward coupling.

A decile-binned coupling analysis on GSM8K shows the mechanism directly: completions in the first length decile, spanning 153–180 tokens, earn a mean reward of 0.351, while completions in the eighth decile, spanning 194–195 tokens, earn 0.233. Longer completions are rewarded less, which is the opposite of the verbosity-trap prediction. A 100-step cross-validation on the arithmetic metrics file returns $\rho(\text{step}, \text{len}) = +0.41$ and $\rho(\text{step}, \text{reward}) = +0.27$, but the absolute length range across that run is 0.08 tokens against a cap of 5 tokens — a positive correlation over a range too small to be a length

effect.

The length–ZVF coupling analysis at iteration 12 is the most mechanistically interesting of the elevated results, because its sign flips between tasks. On arithmetic, $\rho(\text{len}, \text{zvf})$ is -0.449 $[-0.584, -0.324]$ for GRPO and -0.291 $[-0.428, -0.139]$ for Dr. GRPO; on GSM8K it is $+0.286$ $[+0.231, +0.348]$ and $+0.187$ $[+0.148, +0.225]$. The Dr. GRPO minus GRPO difference is $+0.158$ $[+0.075, +0.239]$ on arithmetic and -0.099 $[-0.132, -0.041]$ on GSM8K. Conditioning ZVF on length bins makes the arithmetic structure concrete: the shortest bin has ZVF 0.92 and the longest 0.62, a drop of -0.30 $[-0.34, -0.26]$, whereas on GSM8K the corresponding values are 0.30 and 0.41, a rise of $+0.11$ $[+0.10, +0.13]$. On the easy task, shorter completions are more often all-same-reward; on the hard task, longer ones are.

The mechanism analysis at iteration 20 fits, per run, a regression of step-level mean length on step-level mean reward. On arithmetic, Dr. GRPO reduces the slope magnitude: GRPO -3.65 ± 0.08 against Dr. GRPO -3.22 ± 0.15 (mean $\pm$ SD over 5 seeds). The paired differences below are recomputed from `platform_hybrid/experiments/results/length_bias_mechanism_per_run.tsv` as Dr. GRPO minus GRPO, paired by seed, $n = 5$ (corrected; see Table E.1). Each is given with a paired-t 95% CI, the paired-t p and the exact sign-flip permutation p ; the permutation p cannot fall below 0.0625 at $n = 5$. The OLS slope difference is $+0.43$ $[+0.18, +0.68]$, paired t $p = 0.009$, permutation $p = 0.0625$. The Theil–Sen slope difference is $+1.11$ $[+0.33, +1.88]$, $p = 0.017$, permutation $p = 0.0625$. The $\rho(L, R)$ difference is $+0.10$ $[-0.02, +0.22]$, $p = 0.080$. The lag-1 autocorrelation is essentially unchanged, -0.01 $[-0.27, +0.26]$, $p = 0.94$. On GSM8K ($n = 3$) the OLS difference reverses sign, -4.41 $[-37.7, +28.9]$, $p = 0.63$, far too wide to interpret. Across the fourteen exploratory tests of iterations 20 and 24, Benjamini–Hochberg correction leaves the arithmetic OLS and Theil–Sen differences at $q = 0.051$ and 0.058 , just above 0.05. A mediation analysis attributes 42% of the arithmetic effect to ZVF under GRPO and 55% under Dr. GRPO, against 28% and 12% on GSM8K.

The drift analysis at iteration 24 closes the sequence. Windowed ρ drift on arithmetic is $+0.326$ for GRPO with $p = 0.035$, decomposing into an early -0.330 and a late -0.003 , against $+0.187$ for Dr. GRPO with $p = 0.399$; on GSM8K the corresponding values are $+0.019$ with $p = 0.905$ and -0.084 with $p = 0.769$. The sign-flip analysis of late windows gives fractions of 0.527 ($p = 0.468$), 0.382 ($p = 0.562$), 0.048 ($p = 0.011$) and 0.000 ($p < 0.001$) across the four algorithm-task cells. The $p < 0.001$ cell is the GSM8K Dr. GRPO arm, in which not a single late window flips sign. It is the only one of the fourteen exploratory tests that stays below 0.05 after Benjamini–Hochberg correction ($q = 0.014$). Its unit is the within-run window, not the seed, so it too describes three runs rather than a replicated effect.

6.5.4 Held-out generalisation and the scale extension

The pillar’s held-out results are the part of the chapter closest to a conventional capability claim, and they are reported with corresponding care. The only multi-seed matched base-versus-RL control on Qwen3-8B is a five-seed comparison on Qwen3-8B-Instruct, which moves from 82.0% to 83.3% with $p = 0.26$. Stated as a mean improvement over the pre-RL checkpoint, the gain is +0.013 with a paired $p = 0.256$ at $n = 5$ — small and not statistically significant, and a matched pre-RL control for the Qwen3-8B-Base checkpoint is not available. In summary, on this near-ceiling configuration, GRPO does not produce a held-out improvement that can be separated from noise at the available seed budget, and that GRPO and Dr. GRPO produce held-out gains that are indistinguishable from each other at that budget.

The lower-accuracy configuration behaves differently and more conventionally. On Qwen2.5-1.5B-Instruct with GSM8K-CoT, both algorithms do produce significant pre-to-post held-out gains: +6.2 percentage points for GRPO and +5.0 for Dr. GRPO ($n = 3$ seeds $\times$ 200 held-out items per arm). The unit of analysis is the seed. A one-sample t on the three per-seed deltas gives $p = 0.0051$ for GRPO (deltas +0.055, +0.060, +0.070) and $p = 0.0099$ for Dr. GRPO (+0.045, +0.060, +0.045). As supporting detail, per-seed McNemar exact p -values are 0.071, 0.036 and 0.016 for GRPO and 0.122, 0.017 and 0.122 for Dr. GRPO; no pooled McNemar p -value is used, because the same 200 items recur across seeds (corrected; see Table E.1). The two algorithms’ gains do not differ (paired by seed, $p = 0.25$) (recomputed from `platform_hybrid/experiments/results/drgro_gsm8k_cot.json`; source: `platform_hybrid/paper/sections/p4_results_intro.tex`). This is the project’s clearest evidence that the pipeline trains something real; it is reported alongside the near-ceiling null precisely because the contrast between the two configurations is what the pillar’s bounded-null interpretation rests on. Across the same comparison, the pooled mean completion length drifts from 194.4 to 183.5 tokens, that is downward.

A scale extension tests whether the null survives when the cap is lifted. Qwen3-8B with LoRA rank 4 was run on GSM8K for 30 steps at $G = 8$ and batch size 4, three seeds per algorithm, with the completion cap raised fivefold to 1,024 tokens. All six arms compress: GRPO moves 1004 $\rightarrow$ 905, 981 $\rightarrow$ 944 and 996 $\rightarrow$ 900 tokens, and Dr. GRPO moves 999 $\rightarrow$ 931, 972 $\rightarrow$ 902 and 1000 $\rightarrow$ 878 — declines of between 3.8% and 12.2% in every arm. Late ZVF is 0.45, 0.70 and 0.72 for Dr. GRPO against 0.47, 0.47 and 0.55 for GRPO, and step-0 completions are approximately 98% of the cap. The step-0 saturation is the caveat that limits this extension: with completions already at 98% of the ceiling at the first step, the run starts truncated, and length compression under a binding cap is not the same phenomenon as length control in an uncapped regime. The pillar reports the extension as consistent with the null, not as an independent confirmation of it.

Finally, the P4-facing headline in the Miller-style error-bars audit is the iteration-136 arithmetic late-training-efficiency contrast. Late efficiency is training-reward gain per unit of length change. The contrast is Dr. GRPO minus GRPO, paired by seed, $n = 5$, with mean difference $+0.020$. Recomputed from `length_bias_iter136_step_coupling.tsv`, the effect size is $d_z = +2.68$, the mean paired difference divided by the SD of the differences. The exact one-sided Wilcoxon signed-rank p is 0.031 , which is $1/32$, its floor at $n = 5$ (corrected; see Table E.1). The paired t that matches d_z gives $t(4) = 5.96$, two-sided $p = 0.004$. The two-sided exact sign-flip p is 0.0625 , again the $n = 5$ floor: all five pairs point the same way, yet the non-parametric test cannot reach 0.05 two-sided. The contrast was one of eight directional hypotheses in the iteration-136 family (four per task). Under Holm correction across the eight, the family’s own Wilcoxon p becomes 0.25 . This chapter therefore grades it *suggestive*, not decisive (corrected; see Table E.1): a consistent direction across five seeds on a training metric, with two-sided non-parametric $p = 0.0625$ at $n = 5$. The audit’s three NULL verdicts fall on Pillar-1 and Pillar-3 headlines rather than on this one. The pillar also records its own limitations plainly: a single-benchmark held-out evaluation, a single-seed top-10-checkpoint frontier comparison, weak length identifiability, and marginal paired tests.

6.6 Cross-Pillar Observations and Limitations

Three observations span the four pillars and are worth stating together, because the chapter’s overall contribution is better captured by their pattern than by any single number.

First, the project’s strongest results are its de-confounds, and its de-confounds mostly return nulls. The matched same-stack PPO-versus-GRPO contrast returns a paired difference of -0.002 with $p = 0.374$ ($n = 5$ seeds, at a near-ceiling accuracy, not shown equivalent by TOST at ± 0.005); the four-axis factorization audit attributes $\eta^2 = 0.023$ to the algorithm axis and $\eta^2 = 0.54$ of last-10 training-reward variance to the group-size axis ($F(3, 8) = 3.08$, $p = 0.09$, $n = 12$ runs); the controlled length-bias comparison returns zero length-bias flags across sixteen runs. In each case the stack was held fixed except the factor under test (and, in the PPO arm, the number of distinct prompts per step), and in each case the expected algorithmic advantage failed to appear. Each of these comparisons was run on the saturated 0.5B arithmetic task, at evaluation accuracy 0.98 – 0.99 . The group-size axis carries the largest variance share, but at $n = 3$ seeds per arm it does not reach significance on either training reward or held-out accuracy. It is also a hyperparameter rather than an algorithm choice. The chapter’s empirical conclusion is therefore conditional on saturation: on a 0.5B arithmetic task at evaluation accuracy near 0.99 , no algorithm-label effect was detectable, which cannot distinguish “no effect” from “no headroom”; the sampling configuration is the more plausible lever but was not shown to

matter at this seed budget. The unsaturated Qwen2.5-1.5B GSM8K-CoT configuration, where pre-to-post gains of 5 to 6 points were measured, is where these contrasts would need to be rerun (§9.4).

Second, the diagnostics and interventions are less prescriptive than their motivating literature suggests, and the project says so. ZVF measures signal starvation reliably. It associates with catastrophic collapse ($\rho = 0.56$, 95% CI [0.19, 0.79], $n = 23$ cells), and its association with graded held-out accuracy is not distinguishable from zero ($\rho = 0.27$, 95% CI [-0.37, 0.88], $n = 23$), because it is unsigned and therefore cannot distinguish mastery from incapacity. The four interventions built on it — the difficulty curriculum, the group-size sweet spot, the length-bias loss formulations and the step-1 layer freeze — all return nulls at matched cost, and two of them (the curriculum and the $G = 4$ win) were flagged as SUSPECT by an independent verification pass before multi-seed re-testing reclassified them as noise. Reporting an intervention as a null after an independent reviewer flagged it as overclaimed is, in this project’s convention, part of the result.

Third, the single-seed regime is the dominant shared limitation. The cross-scale anchors are single-seed and descriptive. The roster-level verbosity-trap audit is single-seed per configuration, and its in-training reward metric is not a held-out one. Only the P3 sweep, the P4 controlled comparison, the five-seed Qwen3-8B held-out control and the P4 scale extension operate at $n \geq 3$, and the Qwen3-8B control is the only multi-seed matched base-versus-RL comparison on Qwen3-8B; the Qwen2.5-1.5B GSM8K-CoT pre/post runs and the small-sample pairs of Table 8.B are the other base-versus-trained comparisons. Every headline in this chapter is therefore stated at the strength that its own evidence base supports, which is in several cases weaker than the strength at which it would need to be stated to generalise beyond the named stack. The reconstructions ($G = 32$ and above in P3, the $T = 64M$ grid, the difficulty-gated saving projection) are held at hypothesis-generator status for the same reason: they are joint-fit extrapolations, not trained arms, and the project declines to let them carry a recommendation.

Taken together, the four pillars describe a training method in which no algorithmic-identity effect was detected, and in which signal availability and sampling configuration are the more plausible levers, although they were not shown to be significant at the seed budgets used, and whose most widely repeated specific failure mode — length inflation — is absent in the regimes where it was tested directly and present in only a minority of the roster runs where it was measured descriptively. That is a narrower and more cautious set of conclusions than the P1–P4 internal reports set out to establish, and the chapter reports it as such.

Chapter 7

Results II: Reporting, Registry and Controller (P5–P7)

The three internal reports covered in this chapter form a chain rather than a set of independent results. P5 makes a claim about the epistemic status of reinforcement-learning results for language models and proposes a checkable reporting standard to match it. P6 turns the standard into a machine-readable artifact with a validator, a badge and a diff tool. P7 takes the diagnostic that the standard requires authors to publish — the zero-variance fraction (ZVF) — and asks whether it is mechanistically predictable and ready to control rollout allocation. Each report is explicit about where its claim stops, and this chapter reports their evidence in the same register.

7.1 A Minimum Reporting Standard for Stack-Conditioned Results (P5)

P5’s position is that results in this field are *stack-conditioned*, and that reporting the framework or algorithm label without the stack is therefore insufficient to support the claims that are routinely made from it. The evidence for that position is not a single demonstration but a family of levers, each showing that varying one component of the stack, with the label held fixed, moves or flips the reported outcome or its telemetry.

The cleanest lever is a managed 12-cell head-to-head: four group-relative labels (GRPO [11], DAPO [22], Dr. GRPO [21], GSPO [23]) run on one stack — Tinker-managed, Qwen3.5-4B, GSM8K, three seeds per arm. Mean last-10 training reward spans 0.710 to 0.744, a band of 0.034, with per-arm values of 0.744 (DAPO), 0.742 (Dr. GRPO), 0.723 (GRPO) and 0.710 (GSPO); mean ZVF over the same arms is 0.578, 0.500, 0.567 and 0.511. The labels move telemetry by more than they move outcome, and neither

movement is large.

The largest lever in the corpus is an outcome flip of 5.0% to 85.6% final training reward under a fixed label when the backend is changed. That figure must be read with its provenance caveat attached, because the same inspection that surfaced it also found that the nominal backend comparison bundled a different base checkpoint: Qwen3-8B-Base on the managed Tinker run against the instruction-tuned Qwen3-8B on TRL. The number is therefore evidence that item 3 of the standard must record backend *and* checkpoint identity; it is not a causal estimate of the backend alone. A second same-label lever points the other way: the label “DAPO” produced mean ZVF 0.00 on an open trainer running true dynamic sampling and 0.58 on a closed stack running an asymmetric-clip surrogate.

These exhibits are summarised by a variance decomposition, and its design has to be read with it. The algorithm term comes from the four-method same-stack N2 run (GRPO, AERO, GIFT, AREAL; one seed; 40 steps each, so 4 groups of 40 per-step observations, $n = 160$, $df = 3$ between and 156 within): $\eta^2 = 0.0454$ on ZVF, with a bootstrap CI of [0.0014, 0.0585], and $\eta^2 = 0.0075$ [0.0014, 0.0577] on mean reward. Because the steps are autocorrelated within a run, those 156 within-group degrees of freedom overstate the independent information. No group-size term is reported from this decomposition: the group-size file, `groupsize_zvf_sweep.tsv`, holds one row per G level (4 cells, one observation per cell, $df = 0$ within), on which any one-way decomposition returns $\eta^2 = 1$ by construction (corrected; see Table E.1). The 98-cell mega-campaign decomposition is an unbalanced partial grid (98 of 120 cells over model family, task slice, G, temperature and seed, with two seeds). Its four stack axes account for 72.9% to 92.7% of the variance in the five logged metrics; the seed axis rounds to 0.000, so no stack-to-seed ratio is quoted as an effect size (corrected; see Table E.1).

The standard that P5 proposes has eight items — seven run-manifest fields and one evaluation item — and each item earned its place by exhibiting a documented lever of this kind. Item 1 is the loss form, reported as a tuple of ratio level, clip low and high, token mask, advantage normalisation and dynamic-sampling status — the same label with and without dynamic sampling produced opposite ZVF telemetry (0.00 against 0.58). Item 2 is the reference policy and KL handling, which distinguishes family members that are otherwise conflated and whose omission is unrecoverable after the fact. Item 3 is the sampler, backend and precision, including base-checkpoint identity, motivated by the 17× exhibit above. Item 4 is the per-step ZVF and gradient-utilisation trajectory, because a single mean aliases opposite regimes: across a 368-run audit drawn from a 790-run corpus, mean ZVF is 0.97 and 0.95 for two low-reward models (Llama-3.2-1B and Llama-3.2-3B), falls to 0.25 and 0.29 at mid-range rewards, and rises again to 0.87 and 0.96 for two near-saturated models. Item 5 is the group-size schedule, since

group size is a signal-geometry knob rather than a throughput knob; measured mean ZVF for Qwen3-32B falls from 0.33 at $G=4$ to 0.18 at $G=8$ and 0.08 at $G=16$, and for Llama-3.1-8B from 0.76 to 0.65 to 0.47, while the two boundary cases compress the effect as the theory predicts. Item 6 requires a held-out split disjoint from the reward environment. Item 7 requires a contamination check and a parser-robustness probe: adding micro-jitter $\varepsilon \sim U(0, 10^{-4})$ to rewards collapsed batch ZVF from 0.158 to 0.000 while pairwise contrast density moved by 3×10^{-7} , which establishes that a diagnostic sensitive to verifier rounding is reporting a property of the stack rather than of the policy. Item 8 is deliberately different in kind: held-out pass@k curves at $k \in \{1, 8, 32\}$ belong to the evaluation report rather than to the run manifest. The lever here is a matched base-versus-post-RL panel — four adapters at $G \in \{2, 4, 16, 32\}$, one stack, the same problems — on which pass@1 deltas scatter from -0.5 to $+1.9$ points (under which reading the $G=2$ arm would be called a regression) while the pass@32 frontier improves by $+1.5$ to $+2.5$ points in all four arms. Neither reading reaches significance at a single training seed, which is itself the argument: without pass@k curves and stated noise bands the same panel supports any narrative the reader prefers.

P5 also ships the machinery that makes the standard enforceable rather than advisory. An auditor scores a manifest against the eight items and emits a 0–100 weighted badge; on an $n=103$ audit the weighted total was 55.0 with a badge span of $[35.8, 56.7]$ and standard deviation 3.1, awarding zero gold, zero silver, 99 bronze and 4 wood. Coverage, however, is where the negative result lies. On 98 manifests, every one of the seven manifest fields is present as a key 100% of the time, yet item 2 is *validated* in 0 of 98 cases (0.0%), and all twelve of its sub-fields are present at 0 in each case; nine of the sub-fields across items 1 and 2 — advantage normalisation, clip high, clip low, dynamic sampling, KL coefficient, KL estimator, ratio level, reference snapshot and token mask — sit at 0.0% with zero information content. Twenty-three sub-fields examined across the population show no field at a full null rate, but twenty of the twenty-three carry at most two distinct values, and the twenty entries collapse to ten distinct 23-bit fingerprints.

Two further analyses bound what the standard can be expected to buy. The first is a discriminative-entropy audit: four of the seven manifest fields are vacuous on this corpus — loss form, reference policy and KL, sampler/backend/precision, and decontamination notes each carry too little task-discriminative information to distinguish runs — while the per-step ZVF path and the group-size schedule carry the information. A field-sufficiency regression agrees: a random forest on the disclosed stack fields, scored by out-of-fold R^2 over the 98 cells (8-fold cross-validation, 8 fold seeds), predicts ZVF at $R^2 = 0.832 [0.772, 0.879]$ and mean reward at $R^2 = 0.993 [0.990, 0.996]$. The label-only comparator is $R^2 = 0.000$ by construction, because all 98 cells share one algorithm label, so it is a floor rather than a competing model. The second is a manifest self-sufficiency

test, and it splits sharply by metric. The manifest carries essentially all of the ZVF signal ($R^2 = 0.776$ [0.730, 0.846] against 0.773 [0.735, 0.834] for the cells, $\Delta R^2 = -0.003$ [-0.045, +0.034]) but only a minority of the reward signal ($R^2 = 0.280$ [0.219, 0.443] against 0.743 [0.688, 0.820], $\Delta R^2 = +0.463$ [+0.309, +0.564]); all 98 cells are therefore labelled `honest_but_vacuous` on the three-axis triangulation.

The most useful negative in P5 is the placebo-replacement bound. The four-of-seven vacuity pattern admits two readings: the schema chose the wrong items, or any stack-descriptor schema would be vacuous on a single-stack corpus. Projecting six candidate replacement items onto the live corpus refutes the first reading. Four of the six — KL coefficient, clip range low, advantage normalisation and mini-batch size — project to exactly one unique value on the 98-cell corpus with a bootstrap CI excluding any positive information gain, because the corpus never varies them; temperature schedule would add 0.995 bits and lift the total from 11.41 to 12.41 bits, and a model-family label is redundant with an existing column. The variance partition confirms it: stack axes explain $\eta^2 = 0.947$ of ZVF variance across 25 unique buckets, with 5.3% seed residual and an algorithm term of zero by construction. The schema is not the binding constraint; the corpus is. On the live corpus the fingerprint over the seven manifest fields carries 11.4 bits in total, four fields carry exactly zero, and Hamming distance over the fingerprint predicts the absolute difference in ZVF (Spearman $\rho = +0.529$, $p \leq 10^{-3}$), PCD ($\rho = +0.541$, $p \leq 10^{-3}$) and mean reward ($\rho = +0.251$, $p = 0.003$), while the badge itself is constant across cells and cannot discriminate seed-pair dispersion ($\rho = +0.000$, $n = 48$ pairs). The unit of these Hamming-distance correlations is the pair of cells. Pairs that share a cell are not independent, so the naive p-values overstate the evidence. A Mantel-type permutation test over cell labels would be needed, and none was run, so the ρ values are reported descriptively and their p-values are not relied on.

7.2 The GRPO-Registry (P6)

P6’s premise is that the deltas defining GRPO variants — DAPO’s asymmetric clip, dynamic sampling and token-level loss; Dr. GRPO’s dropped length and standard-deviation normalisation; GSPO’s sequence-level importance ratio [23] — live in paper appendices and framework defaults rather than in any machine-readable form. The registry is the response, and it ships as three artifacts at `registry/`: `schema.json`, `entries/*.json` and `query.py`.

The schema defines two record types under a single `oneOf`, both carrying `schema_version`. The stack record’s core is `min_report`, one sub-object per minimum-report item, with base-checkpoint identity required inside the

sampler_backend block precisely because the audit that motivated item 3 found a managed backend silently pinning a different base checkpoint. Two design decisions carry weight. The first is the distinction between an unreported field and a field reported as absent: a null means the author said nothing, whereas an explicit false, 0.0 or "none" means the author made a checkable claim — “a stack that says ‘we use no KL term’ is making a checkable claim; a stack that says nothing is not.” The second is that item 8, the held-out pass@k requirement, is attached under outcomes as an evaluation-report field rather than folded into the run manifest, preserving the eight-item standard’s split between what describes a run and what describes a result. Per-component delta status is recorded on the stack side in `variant_deltas_applied`, with each entry’s status drawn from {implemented, surrogate, absent, unknown}. The shipped entry for the managed DAPO run makes the point concretely: `clip_higher` is recorded as surrogate with the note that the asymmetric clip was set through the user-facing config while the underlying managed loss form is unverifiable, `dynamic_sampling` is recorded as absent, and `token_level_loss` is unknown because it is managed by the runtime. An optional `ci_method` object (method, bootstrap count, seed, confidence level, source) is populated on the seven derived entries with `n_boot` = 2000 and a percentile 95% interval, and left null on the remaining twenty-four.

Three variant-delta records were verified against their source papers: DAPO with five components (clip higher at $\varepsilon_{\text{low}} = 0.2$ and $\varepsilon_{\text{high}} = 0.28$, dynamic sampling, token-level loss, overlong reward shaping, KL removed); Dr. GRPO with two removals (the $1/|o_i|$ response-length normalisation in the loss and the division by the within-group reward standard deviation in the advantage); and GSPO with a length-normalised sequence-likelihood importance ratio and sequence-level clipping.

The seed population is twenty stack records in five families, all of which pass validation. Family A holds four framework config dumps (Qwen3-8B, GSM8K, seed 42) scoring badges 74, 60, 60 and 62; family B holds the four open Colab trainer arms at toy scale, scoring 96, 96, 96 and 96; family C is the managed head-to-head at 57, 62, 62 and 60; family D is a same-stack four-method run at 67 each; and family E is a five-seed risk-index batch at 43 each. The finding worth stating plainly is why the ordering looks the way it does. The config dumps were assembled for a reviewer-requested fairness audit and score only 57 to 74: they nail items 2, 3 and 5, expose item 1 only partially, and cover items 6 and 7 not at all. The open Colab arms score 96 not because the runs are better but because an open backward pass, coupled with explicit declarations that no KL term and no decontamination check were used, makes every field reportable. Reproducibility supplements of the kind commonly shipped with RL papers do not automatically satisfy MIN-REPORT.

The health audit is the registry’s own evidence that it works. Every shipped record

parses: 31 of 31 pass, comprising 20 stack records and 11 variant-delta records, with zero broken delta pointers. The delta-mix table is the sharpest single result in P6: the label “DAPO” is claimed by ten entries, of which four implement a delta, one records a surrogate, three record it as absent and two record it as unknown — the same label covering four distinct implementations (Table `tab:p6-delta-mix`). Per-item coverage on the twenty stack records is 100% for the held-out split (40/40), 95% for sampler/backend (76/80), 90% for telemetry (54/60), 68% for the group-size schedule (41/60), 48% for reference policy and KL (29/60), 32% for the loss form (38/120) and 20% for decontamination (8/40) (Table `tab:p6-coverage`). The per-openness breakdown inverts the conventional expectation: the open subset records sampler/backend at 91.7% and telemetry at 83.3% with reference KL at 58.3%, while the managed subset records sampler/backend and telemetry at 100% but reference KL at 33.3% and the loss form at only 22.9%.

The registry was stress-tested rather than asserted. A batch of adversarial perturbations with $B = 4000$ bootstrap resamples — $13 \text{ categories} \times 31 \text{ entries} \times 10 \text{ mutations} = 4,030$ generated, of which 800 were no-ops for their entry and were skipped, leaving 3,230 applied — moved the overall robustness statistic from 0.864 [0.756, 0.878] before the corresponding schema bumps to 1.000 [1.000, 1.000] after them, closing three silent-failure categories: a missing identifier pattern check (0.645 to 1.000), MIN-REPORT leaves declared in `properties` but not in `required` (0.000 to 1.000, fixed by promoting key presence to a schema invariant), and free-form delta references (0.000 to 1.000, fixed by replacing `delta_id` with an enum generated from the entry filenames). A separate consistency check cross-references the variant-delta records against the MIN-REPORT blocks: of 31 triples from 13 entries, 7 are MATCH, 0 are MISMATCH, 0 are MISSING_REPORT, 5 are SURROGATE_OBS and 19 are NOT_APPLICABLE, giving an implementation-honesty match rate of $7/7 = 1.000$ with a Wilson 95% CI of [0.65, 1.00] (a percentile bootstrap of 7/7 is degenerate at [1, 1]).

The empirical payoff of the schema is the same-stack measurement. Re-executing four GRPO-family methods (GRPO, AERO, GIFT, AREAL) on a single identical stack — managed sampler, $G = 8$, seed 0, 40 steps, 16 prompts per step — with the full per-prompt reward tensor logged at every step, produces reward deltas of magnitude ≤ 2 percentage points. Two of the three show within-run intervals that exclude zero on the side of a *decrease* (AERO at -0.014 [$-0.023, -0.005$] and AREAL at -0.020 [$-0.032, -0.008$]). Those intervals resample prompt-steps within a single seed-0 run per method, so they carry no seed variance and are not replication, and the three contrasts are uncorrected for multiplicity. The magnitude thesis survives. Whether any difference exceeds seed-to-seed noise was not tested, so neither the claim that all differences are within noise nor its negation is established. The largest same-stack effects are not on reward at all:

AERO and AREAL lengthen completions by approximately 31 tokens and tighten the length coefficient of variation, and GIFT’s loss shift of -1.67×10^4 is a baseline constant. A later registry extension measures the anti-herding residual $\delta_{\text{div}} = \text{ZVF}^{\text{iid}} - \text{ZVF}^{\text{obs}}$ on the same tensors, reporting a stack-mean of 0.049 with a range of [0.039, 0.053]; only GIFT shows a borderline-significant reduction against GRPO ($p = 0.066$, CI $[-0.021, +0.001]$), which resolves a synthesis range of [0.13, 0.23] downwards — the anti-herding diversity bonus is real but smaller than the qualitative synthesis had guessed.

The reference CLI, `registry/query.py`, implements four verbs using only the standard library: `list`; `query --item reference_kl` returning FULL, PARTIAL or UNREPORTED; `badge`; and `stackdiff`, which maps a pair of entries onto the R0–R5 ladder and reports the highest rung triggered — R0 identical manifests, R1 nuisance-only, R2 runtime differences, R3 loss-form or KL-handling differences, R4 differing variant-delta sets, R5 unauditable because a differing item is null on at least one side. The badge formula weights the seven items equally and is outcome-blind: $\text{badge}(s) = \lfloor 100 \cdot \frac{1}{7} \sum_i c_i(s) \rfloor$. `stackdiff` prints a warning at R4 and above that outcome differences must not be attributed to the shared algorithm label, and the P6 report is careful to describe the ladder as “a risk scale, not an effect-size scale.” Applied to the shipped entries, `stackdiff` flags the open-versus-closed “DAPO” pair as R5 from manifests alone, and across the twenty seed entries the MIN-REPORT badge spans 43–96. Two boundaries deserve restating. The registry reports stated provenance and conflicts; it does not reconcile unresolved exact-ID or model conflicts, nor does it validate nominal algorithm comparisons. And the badge is coarse relative to the corpus — twenty entries collapse to ten distinct fingerprints, and twenty of twenty-three sub-fields carry at most two values.

7.3 From Diagnostic to Controller (P7)

P7 asks a narrower question than P5 and P6. Given that the zero-variance fraction is worth reporting, is it mechanistically predictable, and is it ready to control rollout allocation? The P7 report’s own framing is that this does not overturn the descriptive restraint of the earlier work — it *tests* ZVF’s predictive value through falsifiable predictions rather than assuming it — and it flags at the outset that the weakest link in the chain, the causal connection from within-group contrast to gradient magnitude, rests on toy-scale directional evidence only.

The theory is compact. Fix a prompt with latent per-rollout success probability p and a group of G conditionally independent Bernoulli rewards. Write $h_G(p) = p^G + (1 - p)^G$ for the probability that the group is degenerate, and model the usable signal per prompt

as $S(p, G) = p(1-p)(1-h_G(p))$. The first factor is the contrast a non-degenerate group can carry; the second is the probability that any signal survives centring. Both factors vanish at the endpoints and peak at $p = 1/2$, so the reading is that GRPO learns fastest exactly where the policy is most uncertain and starves at both ends. Three consequences follow, each paired with the measurement that could have falsified it. T1 states that the expected ZVF indicator is exactly $h_G(p)$. T2 states that h_G is strictly decreasing in G , with exponential decay rate $-\ln \max(p, 1-p)$. T3 states that with unnormalised group-centred advantages the mean absolute advantage in a group is exactly $2\hat{p}(1-\hat{p})$.

The three checks behave as the theory predicts, with the deviations reported rather than smoothed over. For T1, binning the per-group reward tensors of the GSM8K runs by empirical success rate gives a ZVF indicator averaging 1.000 at $\hat{p} \in \{0, 1\}$ and 0.000 at every interior bin; at run level, measured ZVF sits *above* $h_G(\bar{p})$ at every G , which the P7 report attributes to the Jensen gap — the binomial model evaluates h_G at the mean p while the true prompt population mixes mastered and hopeless prompts. The Jensen gap explains only part of it. On the repository sweep the measured ZVF also sits above the per-step-average null, which already absorbs the across-step variation in p : 0.838 against 0.828 at $G = 2$, 0.764 against 0.709, 0.691 against 0.597 and 0.631 against 0.522 at $G = 16$, recomputed from `groupsize_zvf_sweep.json`. This is the same baseline used in Chapter 6 §6.3.3 and Chapter 4 §4.3. For T2, two independent grids agree. The 368-run audit shows monotone decline for every model with an interior success rate (Qwen3-32B $0.33 \rightarrow 0.18 \rightarrow 0.08$; Llama-3.1-8B $0.76 \rightarrow 0.65 \rightarrow 0.47$), with the two boundary rows behaving as the theory flags them: Llama-3.2-1B sits near $p \approx 0$ where $h_G \approx 1$ regardless of G ($0.98 \rightarrow 0.97$), and Qwen3-4B-Instruct sits near saturation where the decay rate is tiny and seed noise dominates, producing the single non-monotone cell ($0.89 \rightarrow 0.84 \rightarrow 0.91$). The repository-internal sweep at $G \in \{2, 4, 8, 16\}$ shows the same decline, with mean ZVF falling from 0.838 to 0.631 while held-out accuracy stays essentially flat near 0.98 throughout — raising G buys signal, not, on this near-saturated task, outcome. T3 cannot be checked on the closed stack at all, because it does not expose per-group gradients; the open-backward-pass measurement, $\text{corr}(\|\nabla\|, \hat{p}(1-\hat{p})) = +0.71$ on a 0.5B model with synthetic arithmetic, is reported strictly as directional evidence.

Because ZVF aliases mastery with incapacity, the P7 report promotes the pairwise contrast density $\text{PCD} = ((G-1)/G) \cdot E[p(1-p)]$ as the signal to control on. The falsification that motivates the promotion is the micro-jitter experiment already cited in §7.1: ZVF collapses from 0.158 to 0.000 while PCD is unmoved. The controller’s design follows from the theory: a `zvf-triage` callback watches per-step ZVF with PCD as an aliasing guard, and an adaptive- G controller escalates group size on ZVF spikes, changing only the sampler budget so that it composes with any group-relative objective.

The four-arm open-trainer audit is the first place where the intervention is measured

against alternatives on the same data, budget and seeds. Against a 120-rollout baseline, GRPO gains +0.500 held-out at mean ZVF 0.25; Dr. GRPO gains +0.575 at 0.27 on the same 120 rollouts; DAPO-style dynamic sampling gains +0.550 while driving ZVF to 0.00 at 174 rollouts, a +45% cost; and adaptive-G gains +0.575 while lowering mean ZVF only modestly to 0.23 at 186 rollouts. Three readings are stated. Telemetry behaves as the mechanisms dictate. Dynamic sampling buys perfect telemetry at a 45% rollout premium and does not exceed the cheaper fixed recipe. Adaptive-G spends comparable overhead and ties the best held-out result, concentrating its spend on flagged steps — and the tie is small-n and single-task, so the description is “competitive with, not superior to, the best fixed recipe.”

The controller simulation then asks a counterfactual question on the real reward tensors: for each of the 2,560 prompt-step decisions in the four-method run, when would each controller have fired, what group size would it have chosen, and how much contrast would it have restored? The answer is a negative result of some force. Across all four methods and all five thresholds of the point-estimate rule, the controllers save *zero* prompts. Of GRPO’s 640 prompt-step pairs, 461 are truly degenerate — 427 all-correct and 34 all-wrong — with $\hat{p}$ at exactly 0 or 1, and the counts are near-identical for the other methods (AERO 461, GIFT 493, AREAL 452). At $\hat{p} \in \{0, 1\}$ no group size recovers contrast, because $ZVF(G) = 1$ for every G . The empirical ZVF of roughly 0.72 is high because the prompts are easy, not because they are noisy at the decision boundary. Bootstrapping the observed binary rewards to size $G = 16$ confirms it directly: 0 of 1,867 boundary prompts would have been restored by escalation. The operating range where the trigger fires selectively is $\tau \in [0.70, 0.80]$, 12 to 19 fires per 40 steps at a cost ratio of 1.30 to 1.48, and the different thresholds differ only in how wastefully they fire.

In the same trace the escalation asymmetry is stark: 1,723 of 1,867 escalation events (92.3%) occur on all-correct groups and only 144 on all-wrong groups. A symmetric ZVF trigger mostly spends on solved prompts. The P7 report is explicit that deleting those fires is a counterfactual proposal only, because performance preservation has not been measured.

Two refinements sharpen the picture and then complicate it. The Bayesian controller replaces the point estimate with a Beta(1,1) posterior and a mid-range probability $m(k, G) = \Pr[0.05 \leq p \leq 0.95 \mid k, G]$, escalating when m exceeds a posterior threshold. At $\tau_{\text{post}} = 0.60$ it fires on the same 466.75 degenerate prompt-steps as the point-estimate rule at $\tau = 0.50$ — with a 95% bootstrap CI of [454.25, 485.00] — but at strictly lower cost, 8,854 rollouts against 10,240, a 14% saving; and all 466.75 of those fires are on saturated prompts, so it too saves zero prompts. Above $\tau_{\text{post}} = 0.65$ it never fires, because the maximum m over degenerate observations is 0.630.

The calibrated eight-controller comparison then produces a genuine metric reversal with

bootstrap CIs attached. On the cost axis the Pareto winner is Dualformer-Auto, an automatic difficulty-gated de-escalation rule, at a mean cost ratio of 0.657 [0.599, 0.738] against 1.000 for fixed $G = 8$ — a 34% rollout saving, achieved because the prompts it shrinks to $G = 2$ would have been degenerate at $G = 8$ anyway. On a different axis, contrast restored per 1,000 extra rollouts, the ordering reverses: `zvf-triage` at $\tau = 0.50$ restores 90.10 units per 1,000 extra rollouts [89.40, 90.77], against 86.68 for $\tau = 0.70$, 84.00 for Dualformer-Auto and 80.00 for the Bayesian rule, and all four pairwise differences have CIs excluding zero. The Bayesian controller — Pareto-dominant on the earlier axis — is dominated here, because its principled prior fires on the hardest-to-restore degenerate prompts. The regime decomposition explains the mechanism rather than leaving it as a ranking: `zvf-triage` at $\tau = 0.50$ fires on 100% of steps in every regime on this panel, so it is operationally equivalent to “escalate every step,” and its apparent advantage is mechanical exposure rather than a smarter rule. Its restoration decomposes 64.76% degenerate, 13.69% boundary and 21.56% mid-range; the non-degenerate 35.24% share is what the Bayesian rule sacrifices.

Two further analyses bound what this family of shrink-only rules can achieve. A per-prompt hindsight-optimal calculation scans G^* down from a base of 8 and finds a bimodal optimum: $G^* = 8$ for 72.9% of the 2,560 prompt-steps, which are fully saturated and admit no rescue by any group size, and $G^* = 2$ for the remaining 27.1%, which are already non-degenerate at $G = 8$ and are cheapest to shrink. Mean G^* is 6.376, giving a cost ratio of 0.797 [0.776, 0.818] — a 20.3% saving that is pure economy, since mean contrast restored per prompt is 0.000 and mean contrast lost to shrinking is 0.130 ZVF units. This hindsight optimum is not a ceiling on cost (corrected; see Table E.1): Dualformer-Auto’s observed 0.66 to 0.68 on the four-bin rule is *cheaper*. The two optimise different objectives. The hindsight calculation keeps every already-degenerate prompt at $G^* = 8$ and only shrinks non-degenerate ones, whereas Dualformer-Auto also shrinks prompts that would have been degenerate at $G = 8$. The 0.797 is therefore the best cost ratio under a no-shrinking-of-saturated-prompts constraint, not a lower bound on cost for any adaptive rule. A finite-pool correction then shows that even this bound is loose: because a finite pool never collides more than independent sampling, the exact hypergeometric contrast-preservation probability exceeds the binomial one everywhere, by +0.0494 [+0.0481, +0.0506] on the 693 relevant prompt-steps, which means the binomial model over-predicts starvation and the “zero contrast restored” accounting understates true preserved contrast by roughly 34.2 ZVF units.

Cross-base replication is where the retreat from “controller” to “audit” is earned. On a second panel of five independent seeds, the trigger fires 4.20 times per seed [3.00, 5.40] at $\tau = 0.70$, with the per-step rate falling from 0.48 to 0.28 as the panel’s overall ZVF drops; what replicates is the sign and the operating-point logic, not a fixed firing

magnitude. The Bayesian branch is entirely silent on this panel, firing 0.0 times per seed, because every step’s implied mid-range probability lies in $[0.95, 0.999]$ — a real negative finding that scopes the Bayesian controller to distributions containing boundary steps. And the aggregate firing rate is stable while the *location* is not: at $\tau = 0.70$ with a de-escalation threshold of 0.95, the cross-seed Jaccard overlap of the fire sets is 0.133, only 13.3% of steps see all five seeds choose the same group size, and the mean cross-seed Kendall- τ of per-step ZVF trajectories is -0.060 . The firing rate is seed-stable; the firing location is not, and the training-step index is a meaningless axis for the ZVF signal across seeds. The replayed unification confirms the direction: on this panel Dualformer-Auto is the only controller that saves compute (+14% $[0.10, 0.18]$) while *zvf-triage* spends more than baseline (-28% $[-0.36, -0.20]$), with disjoint CIs.

Five design rules close the P7 report, and they read as constraints discovered rather than principles asserted. Gate on PCD and alarm on ZVF: escalate only when ZVF is high *and* PCD indicates interior difficulty, because high ZVF with near-zero PCD and high reward is mastery — where $h_G(p) \approx 1 - G\varepsilon$ near the boundary and added rollouts buy order ε — while high ZVF with near-zero reward is incapacity, where the correct intervention is the data, not the sampler. Escalate multiplicatively in small steps, since at $p = \frac{1}{2}$ each additional rollout halves the degenerate fraction; the audited $4 \rightarrow 6 \rightarrow 8$ schedule at multipliers of 1.5 and 1.33 captured most of the achievable reduction at a total cost of +55% rollouts, whereas $G = 16$ costs four times the rollouts of $G = 4$ for a ZVF drop from 0.76 to 0.63 and no held-out gain on a near-saturated task. Trigger on spikes rather than levels, because ZVF is temporally sticky with a lag-1 autocorrelation of approximately 0.94 and drifts upward over training, so a level threshold fires either never or permanently. Never compare controllers across stacks; ship the seven manifest fields with item 5 carrying the escalation log. And treat $ZVF = 0$ as a purchase rather than a prize — dynamic sampling shows perfect telemetry is always available at a 45% rollout premium, and on this task it was not the best use of the marginal rollout.

The chapter’s summary of P7 is the report’s own. No result establishes prospective superiority over static $G = 16$, Dr. GRPO, or dynamic sampling; adaptive group size is explicitly *not* promoted to a recommended policy; and all intervention evidence is single-task, small-n, or retrospective. The report’s contribution is a retrospective controller audit together with a pre-registered prospective test plan, and the bakeoff that would close the question — a matched-budget, multi-seed held-out comparison against the strong static baseline — is future work rather than a claim. In the same spirit, the report flags its own August 2026 evidence boundary: unsigned ZVF alone cannot select between retiring, distilling, retrying, resampling or warm-starting a prompt, so any operator policy remains a prospective evaluation contract rather than a demonstrated benefit.

Chapter 8

The E1–E14 Held-Out Evaluation Campaign

This chapter reports the evaluation of a single Tinker-trained actor against fourteen benchmark suites that were not part of its training data, recorded lane by lane under sealed reservations, immutable receipts and explicit terminal states. Lanes that did not finish and lanes that could not start are reported too, because the methodological claim of this work is not that the actor performs well but that every figure attached to it can be traced to a surviving, hash-pinned artifact, and that the boundary between what was measured and what was not is drawn explicitly. Lane-by-lane detail that does not change any figure is collected in Appendix D (Figure 8.1).

Two reporting rules govern everything below, and they are worth stating before any number appears. First, original-contract results and replacement-scope results are never pooled and never averaged across suites; a replacement scope is a different suite, a different interface and often a different grader, and combining such figures would produce a number with no referent. Second, no figure in this chapter is quoted to a precision beyond what its source receipt records, and no comparison against a baseline, a sibling suite, or the untrained base model is asserted for the campaign lanes of Sections 8.2–8.6. Those lanes have no matched base-model arm. The one paired baseline comparison in this chapter is the small-scale trained-versus-base rerun of Section 8.7 (Table 8.B): 14 lanes, 5 to 151 identical items each, one vLLM engine for both arms. It finds no significant difference in any lane, and this chapter therefore claims no improvement of any kind over the base model. Where a lane’s record itself contains an unresolved discrepancy, the discrepancy is stated.

The consolidated ledger of record for the campaign is `outputs/E1_E14_FINAL_RESU LTS_2026-09-19.md`, which supersedes the working tables of 2026-09-05 and 2026-09-12; the per-lane terminal-state ledger is `outputs/PES_Phase2_Review_2026-0`

Lane terminal state, E1–E14 (ledger of 2026-09-19, updated with the runs of 2026-09-27)

E1 SWE-bench Multilingual ★ SCORED 1/190 [†]	E2 CORE-Bench (hard) ★ SCORED 27/45	E3 SDAB × CLOSED EXTERNAL	E4 Banker- ToolBench △ CLOSED PARTIAL	E5 Tau3 ★ SCORED 8/97	E6 WebArena ★ SCORED 90/812 [‡]	E7 BinaryAudit × CLOSED EXTERNAL
E8 LAB-Bench (public split) ● COMPLETE	E9 MLDevBench ★ SCORED 10/34	E10 AgentDojo (benign) ● COMPLETE	E11 VerilogEval (native) ● COMPLETE	E12 AppBench × CLOSED EXTERNAL	E13 BALROG ★ SCORED 26.1% (255 ep.)	E14 Omni-MATH* (replacement) ● COMPLETE

● **COMPLETE** (4) — full coverage, no open gate: E8, E10, E11; E14* terminal-complete

★ **SCORED** (6) — double border: replacement scope scored on 2026-09-27: E1, E2, E5, E6, E9, E13

△ **PARTIAL** (1) — closed with partial evidence: E4 (1/100 tasks)

× **BLOCKED** (3) — hatched: whole lane externally closed, no provider response: E3, E7, E12 (original contracts of E8, E10, E13, E14 are also blocked; those lanes report replacement scopes)

Source: finish/Pending_Experiments.md terminal-state table; E1_E14_FINAL_RESULTS_2026-09-19.md; finish_pending_2026-09-27/<lane>/result.json.

[†] This run's 190 tasks; the earlier 110 (4 resolved) are a separate contract and are not pooled.

[‡] Lower bound: 108 tasks could not be judged and count as failures.

Figure 8.1: Terminal status of all fourteen E1–E14 lanes: the 2026-09-19 ledger of record, updated with the six replacement scopes scored on 2026-09-27 (double border, ★). Each cell names the suite whose scope the state refers to, and status is carried by glyph, word, border style and hatching as well as colour. Cells show the replacement scope where one exists (E1 SWE-bench Multilingual, E2 CORE-Bench, E5 Tau3, E6 WebArena, E8 LAB-Bench public, E9 MLDevBench, E10 AgentDojo benign, E13 BALROG, E14 Omni-MATH) and the original contract otherwise; the original-contract suites and their results (e.g. E1 SWE-bench Pro 2/731, E2 FrontierSWE, E5 APEX-Agents, E9 MLE-bench) are shown in the campaign-design figure (Chapter 4) and are never pooled with replacement-scope numbers. *E14 closes terminal-complete: $2271/4428 = 51.29\%$ with the two unjudged rows counted as failures (official scorer: 51.31% over 4426 judged rows).

9-12/finish/Pending_Experiments.md; and the arithmetic of the ledger was re-verified by deterministic code checks recorded in outputs/verification/LEDGER_CODE_CHECK_2026-09-19.json, which reports eleven of eleven checks passing with zero failures.

8.1 The evaluated actor and the one-actor design

A single actor was evaluated across the whole campaign. The base model is Qwen/Qwen3.6-35B-A3B at revision 995ad96eacd98c81ed38be0c5b274b04031597b0, and the trained parameter set is the LoRA adapter arvindcr4/pavlov-portfolio-qwen36-seed809-stepfinal-tinker-cf0ad8c1-1f1b-5ff-9f777c4018b6 at commit 64444133c55d88c3f1bf0df8a2f5d7ac646125c8, served in bfloat16 under the identifier pavlov-public-portfolio-bf16. The Tinker training run behind the adapter has since been purged, so the actor can no longer be sampled on Tinker; the adapter itself is preserved on the Hugging Face branch checkpoint-seed809-stepfinal-9f777c4018b6 at the recorded commit, and a check on 26 September 2026 confirmed the 2.25 GB weight file is present there, so the actor can be re-served off-Tinker.

The one-actor design is deliberate. No lane received a separate checkpoint, per-suite prompt engineering beyond its native harness, or tuning against its own evaluation set: the actor’s only training suites were API-Bank-RLVR and SWE-Gym (§8.1.1), although the E10 receipt records its decontamination inventory as absent (TRAINING_INVENTORY_ABSENT). Differences between suites therefore reflect the interaction of one fixed policy with fourteen different interfaces, observation spaces, tool conventions and graders, and they are not evidence about what a specialised actor could achieve. The campaign also cannot support any statement about seed variance. Inference conditions were frozen per lane and recorded in that lane’s own receipt (§8.1.2).

8.1.1 How the actor was trained

The adapter was produced by a single Tinker training run (W&B run bsv8vx04), whose configuration survives in the local W&B run directory, the local run receipt and the Hugging Face checkpoint branches; where these files live and how their hashes are verified is recorded in Appendix D. Table 8.1 gives every field they record.

Training record of the E1–E14 actor. From the surviving run artefacts.

Table 8.1: Training record of the E1–E14 actor, from the surviving run artefacts.

Field	Value	Evidence
Launch	<code>python -m platform_tinker.tinkerrl.grpo_cli --preset pavlov_portfolio, no overrides</code>	<code>wandb-metadata.json</code>
Base model	Qwen/Qwen3.6-35B-A3B	<code>config.yaml</code> ; run receipt at revision 995ad96e...
Loss	REINFORCE with a group-mean baseline: $A_i = (r_i - \mu_g) / (\sigma_g + 10^{-8})$ with population σ ; loss $-A_i \sum_t \log \pi(y_{i,t})$, averaged over completions. No importance ratio, no clipping, no KL term, no reference model	<code>platform_tinker/tinkerrl/grpo.py</code> (<code>normalize_rewards, make_grpo_loss_fn</code>)
Sampler refresh	Only at checkpoints: steps 1–20 sampled from the step-0 weights and steps 21–40 from the step-20 weights, with no importance correction	<code>grpo.py</code>
Reward	PavlovNonXLAMReward: API-Bank rows scored by StrictToolCallReward, SWE-Gym rows by Patch Reward (1.0 for an exact diff, otherwise partial credit capped at 0.99)	<code>platform_tinker/tinkerrl/grpo_cli.py</code> , <code>grpo.py</code> (current code; see Appendix D)

Field	Value	Evidence
Training data	Suites api_bank_rlvr_train and swe_gym_train (API-Bank-RLVR at bf67c426..., SWE-Gym at bb94ed9e...); dataset revision bd1f0db6...; training pool of 512 rows	run receipt; config.yaml; E11 decontamination receipt
Declared evaluation suites	14 primary evaluation suites (the original E1–E14 contracts, including verilog_eval); heldout_suite_ids empty; evaluate_heldout false	run receipt
Group size, batch, steps	$G = 4$; 2 prompts per step; 40 steps; 80 prompt draws and 320 completions in total	config.yaml; run receipt
Optimiser	$\text{lr } 1 \times 10^{-5}$; Adam β_1 0.9, β_2 0.95, ϵ 10^{-8}	config.yaml
LoRA	rank 32, alpha 32, all linear layers	config.yaml; adapter config
Decoding during training	prompt 1,536 and response 384 tokens; temperature 0.7, top-p 0.95; seed 809	config.yaml
Reward trace	mean 0.025 over the first 5 steps and 0.2425 over the last 10; reward zero on 22 of 40 steps; loss zero on 35 of 40 steps	run receipt; output.log
Date and cost envelope	started 2026-08-09 08:37:44Z; runtime 1,934 s; \$16.50 authorised, \$18 maximum	wandb-metadata.json; run receipt

Field	Value	Evidence
Checkpoints	step 0, step 20, step 40 and final, each a separate Hugging Face repository whose weights sit on a checkpoint-* branch (main holds only .gitattributes); the evaluated adapter is branch checkpoint-seed809-stepfinal-9f777c4018b6 at 64444133..., saved immediately after step 40 with no optimiser step in between	run receipt; outputs/e1_e14_small_scale_2026-09-26/ADAPTER_AVAILABILITY_CHECK_2026-09-26.json
Tinker run	cf0ad8c1-1f1b-5ff3-8bd7-2a0bf232657b:train:0; all sampler paths since purged	run receipt; outputs/TINKER_SAMPLER_STATUS_2026-09-20.json

Four items cannot be recovered — the exact training code, which 80 training prompts were drawn, the Tinker sampler weights, and any held-out training-time evaluation — and the run departs from the planned Pavlov protocol; both are recorded in Appendix D so that no reader fills them in.

What this means for the chapter is direct. The actor received 40 steps of sparse-reward, critic-free policy-gradient training, with zero loss on 35 of them. The fourteen suite results are properties of this checkpoint, not evidence about what GRPO-style training contributes. The campaign is reported as an exercise in evaluation governance. The only trained-versus-base evidence is the small-sample paired comparison of Table 8.B.

8.1.2 Serving conditions per lane

The E4 zero (§8.5) was traced to a serving bridge that sent no stop sequences. The same bridge, `zvf-program/flagship/modal_tinker_openai_bridge.py`, served several other lanes, so Table 8.2 records for every lane how the actor was served and whether that defect could apply. Its `/v1/responses` route hard-codes `stop=None` and thinking on. Its `/v1/chat/completions` route sends stop sequences only if the client supplies them, and the agent clients used here (opencode, mini-swe-agent and the APEX client)

do not. The deployed bridge version (sha256 f59c08b9...) matches neither committed version of the file.

Serving conditions of the original campaign runs, by lane.

Table 8.2: Serving conditions of the original campaign runs, by lane.

Lane	Serving path	Stop sequences	Template / thinking	Max tokens	Temp.	Parse failure / truncation	Could the stop defect apply?
E1 SWE-bench Pro	Mixed: 476 Tinker sampler, 255 Modal vLLM (merged PEFT)	none; relies on EOS	chat template, thinking off	8,192	0.2	14 generation failures, 4 artefacts lost; 40/476 Tinker and 22/255 vLLM responses at the cap	Not through the bridge. The Tinker path also sent no stops, but no response contains role-token lines and cap rates match across backends
E2 FrontierSWE	bridge (Habor/opencode)	not recorded; client sets none	not recorded	not recorded	not recorded	not recorded	Yes
E3 SDAB	no model output	—	—	—	—	—	n/a
E4 Banker-ToolBench	bridge (opencode); base rerun via /v1/responses	none	template default (thinking on)	not recorded	bridge default 0.2	pass3 archive missing	Yes; confirmed for the base rerun
E5 APEX-Agents	bridge (actor and judge)	not recorded	not recorded	not recorded	not recorded	3 unscored failures, 1 interrupted	Yes
E6 WebArena	no model output	—	—	—	—	—	n/a
E7 BinaryAudit	bridge (mini-swe-agent)	not recorded	not recorded	16,384 (output limit)	not recorded	attempt ended on an HTTP 402 bridge-budget error	Yes, but the attempt ended on a budget error
E8 LAB-Bench	Modal vLLM, merged BF16	none; vLLM EOS	thinking off	1,024	0	1,259/1,967 unparsed (64.0%); 1,086/1,967 length-truncated (55.2%)	No
E9 MLE-bench	streaming arm via bridge /chat/completions; merged arm via vLLM	none in either arm	thinking off	4,096	0.2	not recorded	Yes for the streaming arm (single-turn); no for the merged arm

Lane	Serving path	Stop sequences	Template / thinking	Max tokens	Temp.	Parse failure / truncation	Could the stop defect apply?
E10 AgentDojo	Modal vLLM, merged BF16	none; vLLM EOS	thinking on	4,096	0	1 length finish in 418 recorded response bodies (407 main run, 11 continuation)	No
E11 VerilogEval	Tinker sampler, called directly	none	template default (thinking on)	1,024	0.2	150/312 extraction failures (48.1%); truncation not recorded (mean response $\approx$ 1,002 of 1,024 tokens)	Not through the bridge; unknown, because per-item outputs are not kept locally
E12 AppBench	no model output	—	—	—	—	—	n/a
E13 BALROG	Modal vLLM, merged BF16	none; vLLM EOS	thinking on	8,192	1.0	78 length finishes against 189 stop finishes	No
E14 Omni-MATH	Modal vLLM, merged BF16	none; vLLM EOS	thinking off	2,048	0	2,982/4,428 responses at the 2,048-token cap (67.3%); 2 unjudged	No

Sources: per-lane receipts under `outputs/modal_e1_e14/2026-08-16/`, `outputs/public_portfolio_2026-09-05/`, `outputs/e2_frontier_swe/`, `outputs/e5_apex_agents/`, `outputs/e7_binaryaudit/`, `outputs/e9_mle_bench/`, and `outputs/PES_Phase2_Review_2026-09-12/finish/e13_control/`; bridge bindings in `outputs/modal_e1_e14/2026-08-16/non_e11_readiness/`.

Two consequences follow. First, the lanes served through the bridge (E2, E4, E5, E7 and the E9 streaming arm) are exposed to the same defect that produced the E4 zero, and none of their receipts records the stop, thinking or token settings, so their figures may measure the harness as much as the model. Second, E8, E11 and E14 are dominated by format and length limits, so their headline figures are end-to-end harness scores; §8.2 gives each with its parseable-only sensitivity figure. A note on the scope of the stop-sequence mechanism is in Appendix D.

8.2 Suites with strictly complete scopes

Three suites reached strictly complete scopes, and the ledger’s own integrity check asserts exactly this set: E8, E10 and E11. E8 and E10 are replacement scopes. E11 is an original-contract full-suite result that also meets the strict-completeness check, because the suite evaluated is VerilogEval itself; the ledger lists it both among its replacement scopes and among its original-contract results, but it is one run and one result. A fourth, E14, closed terminal-complete under its native protocol with a recorded note. The six replacement scopes completed on 27 September postdate that check and are reported separately in §8.4 (Figures 8.2 and 8.3).

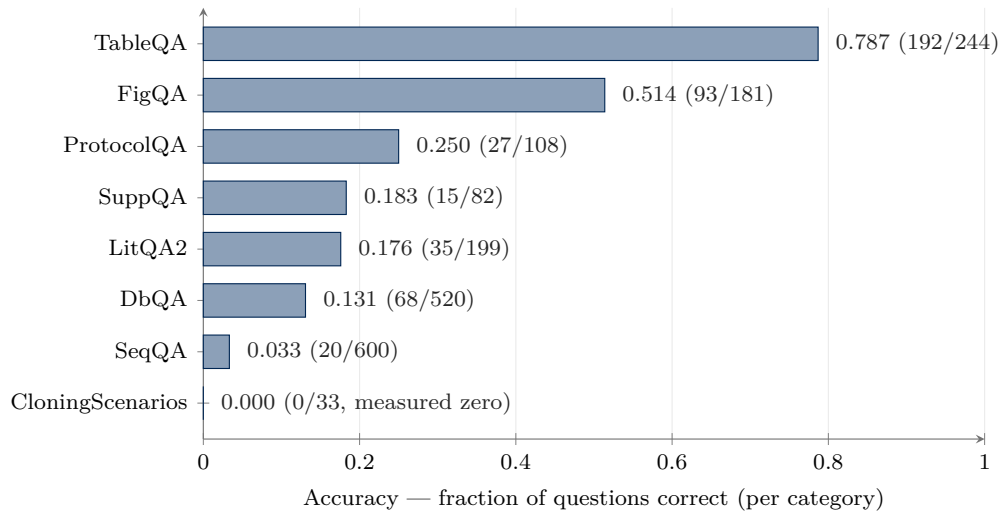


Figure 8.2: LAB-Bench public split, per-category accuracy, all 1,967 questions graded across eight categories. Each bar is correct/total for one category (labels give the counts). Accuracy is strongly category-dependent, so no pooled cross-category average is drawn or claimed. CloningScenarios is a measured zero (0/33), not a missing value.

E8 — LAB-Bench, public split. All 1,967 of 1,967 expected items were evaluated across eight categories. The receipt records 450 correct and 1,517 incorrect responses, a published overall accuracy of $450/1967 = 0.2288$ (Wilson 95% CI [0.211, 0.248], unit = question, $n = 1,967$); per-category figures are in Appendix D. For 1,259 of the 1,967 responses the native pipeline extracted no answer (`native_answer_parse_none`), and those are counted as incorrect, so the figure is an end-to-end harness score. As a sensitivity figure, not the headline, accuracy conditional on a parsed answer is $450/708 = 0.636$ (Wilson 95% CI [0.600, 0.670]). The headline therefore mostly measures the 64% of responses that yielded no answer, with length truncation (1,086 finishes) the dominant failure, and not domain knowledge.

E10 — AgentDojo, benign utility scope. Over the frozen AgentDojo v1.2.2 benchmark at commit 089ed468cf3ed0322acc66b0211f26d9d90dbf60, the terminal native re-

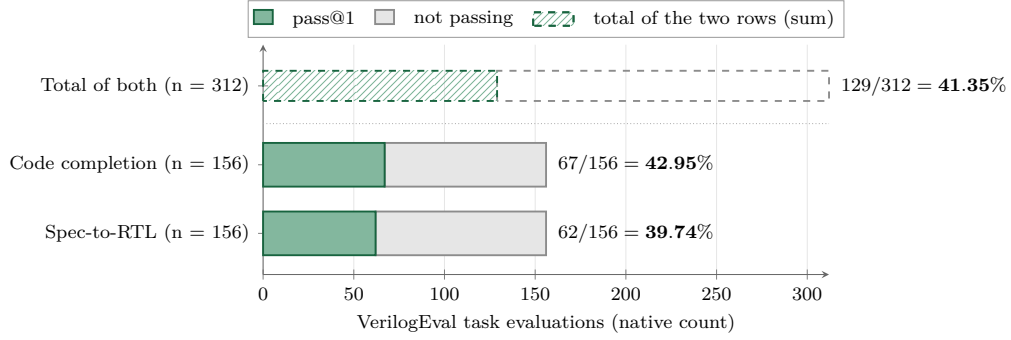


Figure 8.3: VerilogEval decomposition: two native framings of 156 tasks, 67/156 code-completion and 62/156 spec-to-RTL, summing to 129/312 = 41.35% pass@1. The dashed top bar is the sum of the two rows, not a third measurement. Graded by the suite’s native harness (receipt `e11_full_receipt.json`); the 129/311 = 41.48% value that excludes `Prob099_m2014_q6c` is a non-canonical sensitivity only.

ceipt records status `COMPLETE_NATIVE_BENIGN_UTILITY`, all 97 episodes completed, 88 utility passes and a score of 88/97 = 0.9072 (Wilson 95% CI [0.833, 0.950], unit = episode, n = 97). Its claim boundary is stated verbatim in the receipt as “Native default benign task utility only;no prompt-injection security or held-out claim”, and its decontamination status is `TRAINING_INVENTORY_ABSENT`. Completion and utility are distinct: the lane is complete in coverage and 0.9072 in benign utility, never “97/97 correct”, and it is not a security result of any kind. An earlier attempt stopped at 94 of 97 episodes because of a collector fault, not a model failure; the fault and its minimal repair are recorded in Appendix D.

E11 — VerilogEval, two native framings. All 312 pinned problems were evaluated — 156 code-completion (`code-complete-iccad2023`) and 156 specification-to-RTL — under the upstream NVlabs/verilog-eval harness. The result is 129 passes of 312, or 0.4135 (Wilson 95% CI [0.360, 0.469], unit = problem), decomposed into 67/156 = 0.4295 ([0.354, 0.508]) on code-completion and 62/156 = 0.3974 ([0.324, 0.476]) on specification-to-RTL. The receipt records 150 extraction failures, which appear as `FAIL` verdicts in the denominators rather than being dropped. 129/312 is therefore an end-to-end harness score: 150 of the 183 failures are format failures. As a sensitivity figure, not the headline, the pass rate among the 162 responses from which code was extracted is 129/162 = 0.796 (Wilson 95% CI [0.728, 0.851]). One problem, `verilog_eval/spec-to-rtl/Prob099_m2014_q6c`, fails against its own test bench under both simulators; excluding it yields 129/311 = 0.4148, but no immutable justification for the exclusion was captured, so the raw 129/312 is canonical and 129/311 is a noncanonical sensitivity only. Harness revisions, the verdict rule and the amendment are in Appendix D.

E14 — Omni-MATH replacement scope, terminal-complete. The scope accepted 4,426 of 4,428 rows and reproduced the official scorer’s output of 0.5131, which is

2,271/4,426: the scorer omits the two unjudged rows from its denominator. Under the campaign evidence rule those rows count as failures, so the reported accuracy is $2,271/4,428 = 51.29$ per cent, a difference of 0.02 percentage points. For both rows the Omni-Judge output is truncated before any verdict, so the official scorer’s omission is correct behaviour rather than a parser defect, and no re-judge was performed.

8.3 Original-contract results

Seven lanes have an original-contract figure, reported separately from any replacement scope with its coverage caveats; two are full-suite results. Lane detail is in Appendix D.

- **E1 — SWE-bench Pro.** 2 of 731 tasks resolved, a pass@1 of $2/731 = 0.00274$, or 0.274 per cent (Wilson 95% CI [0.08%, 0.99%], unit = task), over the complete 731-row test split of ScaleAI/SWE-bench_Pro. The fourteen generation failures and four lost artifacts stay in the denominator.
- **E2 — FrontierSWE.** 1 of 17 tasks evaluated, replay-normalised score 0.8628; partial, because revision-bound authorisation for the remaining sixteen was never obtained.
- **E4 — BankerToolBench.** 1 of 100 tasks evaluated, recovery metric 0.3115 — a verifier recovery metric on a single task, not the simple fraction 37/128 and not a suite score; the pass3/pass16 archive is missing.
- **E5 — APEX-Agents.** 7 of 480 tasks natively scored. The prefix mean of 0.050505 treats unscored attempts as zero and is not a native 480-task score; the seven scored tasks average 0.079365.
- **E7 — BinaryAudit.** One of 46 attempts recorded, verifier reward 0.0 after an agent error, no grade.
- **E9 — MLE-bench.** 40 of 75 competitions carry native grades, a coverage of 53.33 per cent, and the suite score is null; a separate merged-vLLM arm holds one valid grade (H&M, 0.02132) and is not combined.
- **E11 — VerilogEval** is reported in §8.2; it is the second original-contract full-suite result, alongside E1, and not an additional one.

No lane here has a matched base-model run, so none licenses a claim that the actor outperforms its untrained counterpart, a predecessor model, or a published number.

8.4 Replacement-scope lanes completed on 27 September 2026

At the 19 September ledger of record, six replacement scopes were started but incomplete: E1 (SWE-bench Multilingual, 35 of 300 graded), E2 (CORE-Bench, 45 capsule setups, no grades), E5 (Tau3, 20 of 97 cleaned) and E13 (BALROG, 13 of 255 episodes) were launch-pending, and E6 (WebArena, 0 of 812) and E9 (MLDevBench, 0 of 34) were PENDING_QUOTA because AWS granted one vCPU per region. All six were run to completion on 27 September (Figure 8.4).

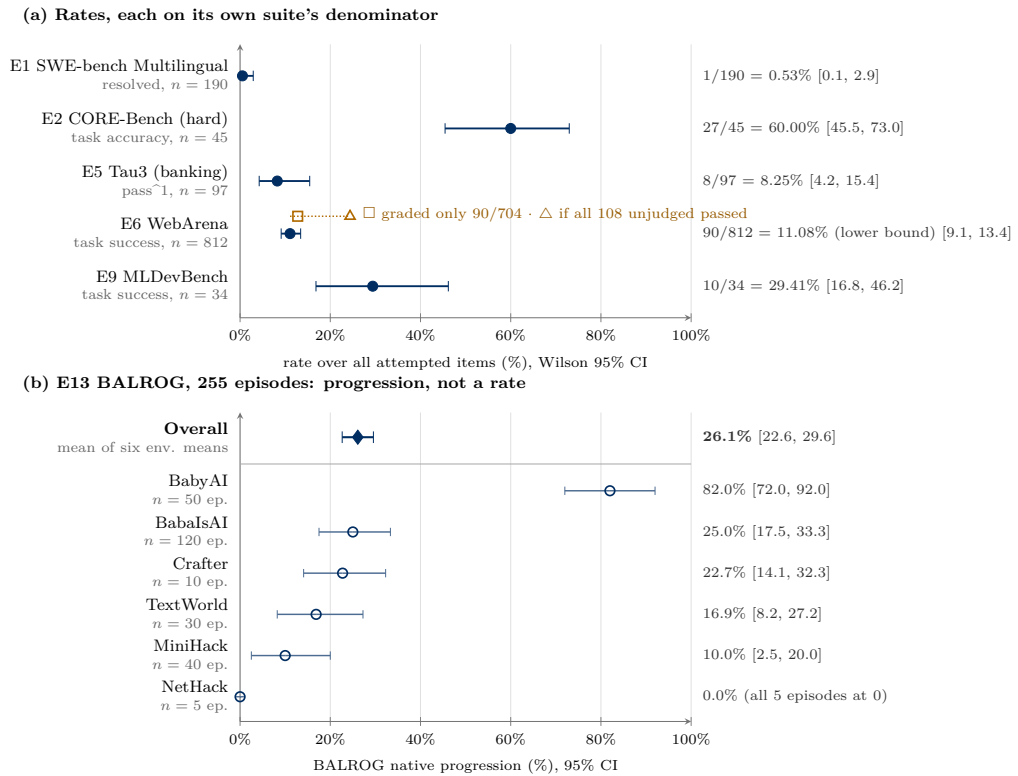


Figure 8.4: The six replacement-scope results scored on 2026-09-27, each with its 95% confidence interval. (a) Rates over all attempted items, with errors, timeouts and ungraded items counted as failures (Wilson intervals). For E6 WebArena the filled point is the lower bound 90/812; the square is the rate over the 704 judged tasks and the triangle the value if all 108 unjudged tasks had passed. (b) E13 BALROG reports native progression, not a success rate: per-environment means with percentile-bootstrap intervals ($B = 10,000$), and the BALROG overall score, the unweighted mean of the six environment means, with its normal-approximation interval. Each row is a different suite with its own metric and denominator, so no cross-lane average is drawn, and none of these values is pooled with an original-contract figure. Source: `outputs/finish_pending_2026-09-27/<lane>/result.json`.

The override was self-authorized. `outputs/finish_pending_2026-09-27/AUTHORIZATION.json` records its issuer as “lead (repository owner arvindcr4), via Claude

Code session”, its directive verbatim as “finish all pending lanes”, recorded at 2026-09-27T00:41Z, with per-lane spending caps, and states that it supersedes the lead-issued bound-authorisation receipt, the reservation seals and the GCP owner-IAM binding. The candidate, as project lead, therefore issued the directive that set aside gates he had himself put in place; no independent party approved it. The evaluation-governance claim of this thesis is scoped accordingly. It does not mean that fail-closed gates were never overridden. It means that every override and every deviation from the reference protocol is recorded in the lane’s own result file, named per lane (Table 8.D), and never pooled: no replacement-scope figure is combined with an original-contract one. All six lanes were served the same actor, pavlov-public-portfolio-bf16, through vLLM with the non-thinking chat template and not through the Tinker bridge of §8.1.2, although the result files do not record stop settings.

Table 8.C. Replacement-scope results, 27 September 2026. One run per item, native grader. Errors, timeouts and ungraded items count as failures in the denominator. Graded counts items with a native verdict. Intervals are Wilson 95% for rates; E13’s uses BALROG’s standard error, and its score is the mean of six per-environment means.

Lane	Benchmark (scope)	Attempted	Graded	Metric	Result	95% CI	Spend (USD)
E1	SWE-bench Multilingual (190 tasks not attempted before)	190	57	resolved rate	1/190 = 0.005	[0.001, 0.029]	1.87
E2	CORE-Bench hard (all 45 capsules)	45	45	task accuracy	27/45 = 0.600	[0.455, 0.730]	8.83
E5	Tau3 banking (all 97 tasks)	97	72	pass ¹	8/97 = 0.082	[0.042, 0.154]	18.42
E6	WebArena (all 812 tasks)	812	704	success rate, lower bound	90/812 = 0.111	[0.091, 0.134]	13.46
E9	ML-Dev-Bench (all 34 tasks)	34	34	success rate	10/34 = 0.294	[0.168, 0.462]	1.67
E13	BALROG (all 255 episodes, 6 environments)	255	255	progression	26.1%	[22.6, 29.6]	0.00

Sources: `outputs/finish_pending_2026-09-27/<lane>/result.json` for each lane. Spend excludes the shared actor endpoint, which was billed to its own \$150 cap and is not broken down by lane. E5’s figure includes its dedicated actor GPU (\$15.21). The E2, E6 and E9 figures are list-price estimates and E1’s is a Modal sandbox estimate.

Table 8.D. Deviations from the fail-closed protocol, 27 September 2026. Condensed from the deviations field of each lane’s result file; the full text is in that file.

	Lane Deviation from reference protocol/scaffold	Recorded in
E1	Actor at 32,768-token context (65,536 on 12 Sept), per-task max _tokens cap; source-context collector re-implemented; thin runner run_remaining.py bypassing paperwork gates, native eval in Modal sandboxes; batches r06/r07 harvested from orphaned sandboxes (exec stdout/returncode lost); two driver relaunches reusing, never resampling, completed generations; context-overflow short-circuit	E1/result .json
E2	Per-task GCP VMs instead of native Azure shapes; thin bash/query_image/finish loop instead of the CORE-Agent scaffold (native 8,100 s budget, 150-turn cap, 26k-token window); query_image answered by the same actor; infrastructure aborts rerun from scratch, none because of score	E2/result .json
E5	Thin runner code/run_tau3.sh replaces the fail-closed e5_subprocessor27 controller; dedicated 65k-context actor copy; OpenAI roles through a spend-capped relay	E5/result .json
E6	GCP VM instead of the AWS AMI; thin runner code/e6_driver.py mirroring native run.py; max_tokens 384, few-shot system turns mapped to user/assistant; retired native judge swapped to openai/gpt-4.1, which had no credit (108 tasks ungraded, counted as failures); 1,500 s per-task timeout; last 103 shopping-admin tasks sharded over three containers of the official image	E6/result .json
E9	GCP instead of AWS; backported one-line OpenHands context-window fix; loopback relay forcing non-thinking and restoring dropped tool-call text; max_input_tokens 28,000; one run per task	E9/result .json
E13	Non-thinking template (original run had thinking on); NetHack capped at 2,000 steps (never bound); macOS arm64 instead of the Linux x86 image, dynamics not bit-verified; killed or hung episodes rerun from scratch with new seeds	E13/result t.json

Sources: *outputs/finish_pending_2026-09-27/<lane>/result.json*, *field deviations*; *outputs/finish_pending_2026-09-27/AUTHORIZATION.json*.

The six figures are on six different benchmarks with six different metrics, and they are not averaged, pooled with the same lane’s original-contract result, or compared with a

base model. Five lanes (E2, E5, E6, E9, E13) now have a native score over their full replacement scope. E6's is a lower bound: 108 tasks needing an LLM judge could not be graded, and the true figure lies between 0.111 and 0.244. E1 has two separately reported scores, 1/190 and 4/110 (the earlier waves, served at a different context), that together cover all 300 tasks but are not combined. E5's pass¹ over the 72 tasks without an infrastructure error, $8/72 = 0.111$, is a sensitivity figure only, and E13's 26.1 per cent is not compared with the 26-episode small-scale E13 rows of Tables 8.A and 8.B. The lanes spent \$44.25 in total against caps of \$270, not counting the shared actor endpoint. Per-lane narratives are in Appendix D.

8.5 The E4 rerun as a diagnostic

A fresh full BankerToolBench rerun, authorised separately after the lane closed CLOSED_PARTIAL on 19 September, ran the base model with a zero-initialised LoRA snapshot through the Modal bridge /v1/responses path. It completed 100 of 100 trials with one NonZeroAgentExitCodeError and a mean reward of exactly 0.0. Agents stopped after 2 to 14 steps without producing deliverables, and twenty-seven or more trajectories ended in user user assistant ... role-token degeneration. The bridge's /v1/responses path passed no stop sequences, so generation ran past the end-of-turn token into invented turns; with a stop sequence set, the same base model scored 0.41 on one of six tasks in the small-scale rerun of §8.7. The 0.0 is therefore attributed to the serving bridge, not to the model's tool use or finance reasoning. It is not set against the original single-task outcome, and no full-suite BankerToolBench score exists for this actor family. Spend was \$15.07 against a \$55.91 cap. The full audit is in Appendix D (Figure 8.5).

8.6 Externally blocked lanes and terminal states

Six lanes have no local or paid path and were formally closed as externally blocked under the finish-all directive of 19 September 2026, whose verbatim text and interpretation are recorded in outputs/PES_Phase2_Review_2026-09-12/finish/ROOT_DIRECTIVE_FINISH_ALL_2026-09-19.json at 13:50Z. The six are E3 (SDAB private bundle), E7 (BinaryAudit private payload), E8-original (the LifeSciBench package), E10-original (AgentHarm private tasks), E12 (AppBench deployment) and E14-original (the FrontierMath hosted evaluation). Closure records were written for each, with an index, under outputs/PES_Phase2_Review_2026-09-12/finish/external_closures_2026-09-19/; each records "no provider response" and specifies the conditions un-

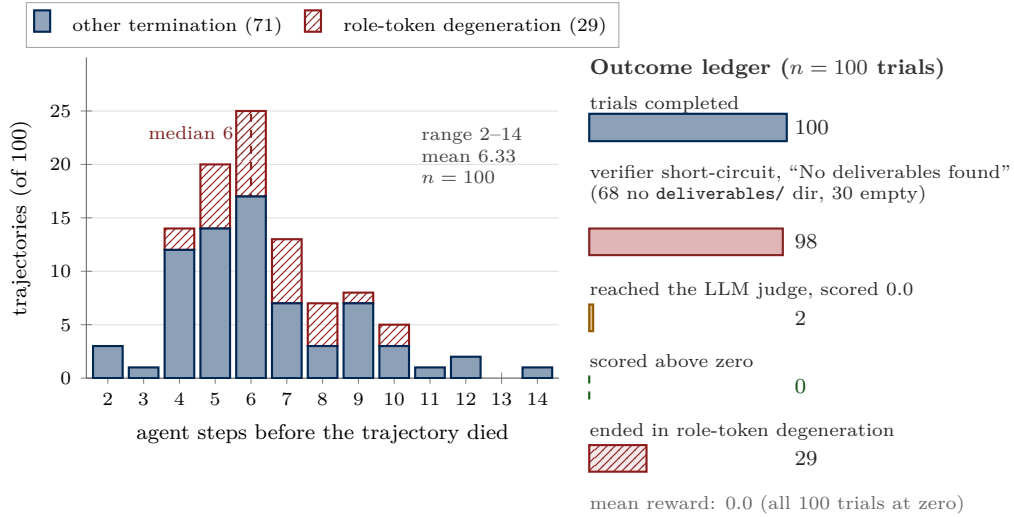


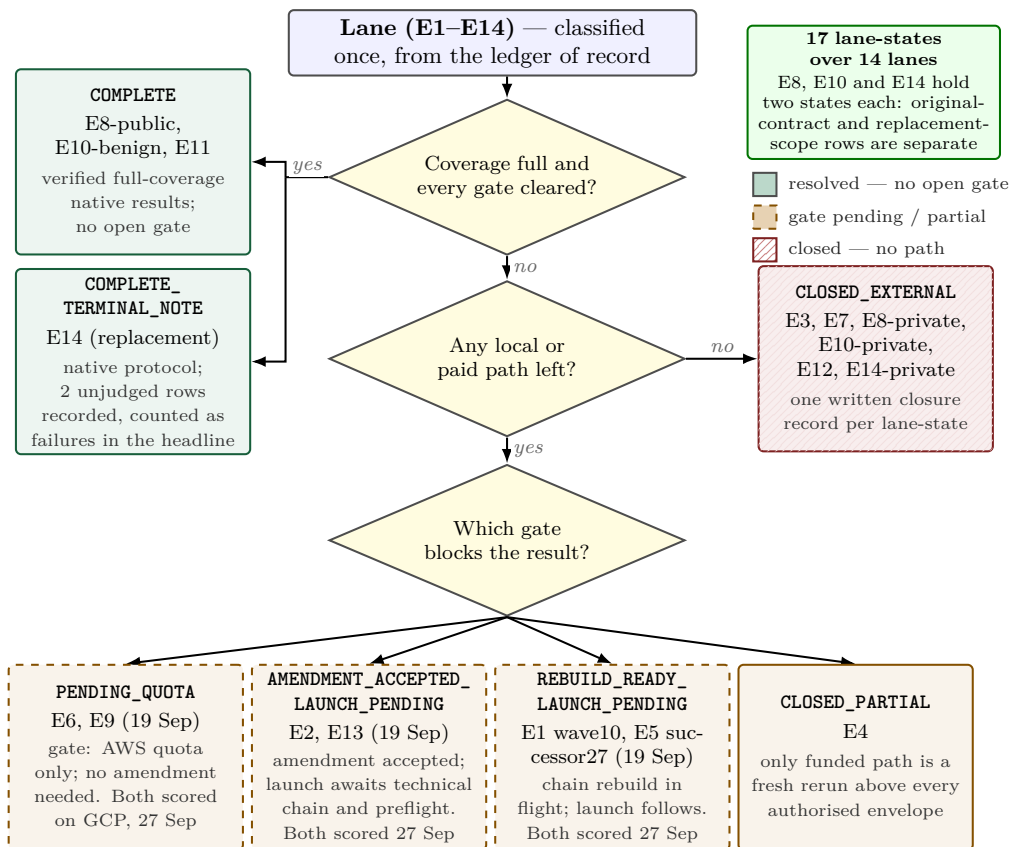
Figure 8.5: The E4 rerun diagnostic. Across 100 trials agents terminate at a median of roughly six steps without producing deliverables, and more than a quarter end in role-token degeneration, consistent with generation running past the end-of-turn token because the serving bridge passed no stop sequences. The 0.0 is not a measurement of finance reasoning. Left: per-trajectory step counts (`final_metrics.total_steps`), stacked by whether the final agent message carries at least two concatenated role tokens (29/100; the ledger records “27+”). Right: where the verifier chain stopped (`verifier/test-stdout.txt`, `reward.json`), all bars on one scale. Base-model rerun of 2026-09-21, all 100 trial directories under `outputs/e4_banker_toolbench/`.

der which the lane could be reopened. A seventh original contract, E13’s OpenReward held-out games, is also blocked on external access and is listed with the six in the ledger, but it has no closure record because the lane continues through its BALROG replacement scope (§8.4). Access requests, the deterministic ledger checks and the role of the model-based triage harness are recorded in Appendix D (Figure 8.6).

8.7 Small-scale reruns of all fourteen lanes

This section adds a separate, smaller set of runs, made on 26 September 2026, to give every one of the fourteen lanes a measured number under one fixed protocol, including the lanes that never produced a score. None of these figures is pooled with, averaged against, or substituted for a result reported earlier in this chapter.

The protocol fixes one actor family, a fixed item-selection seed (20260926), temperature 0 and a non-thinking chat template, and scores each lane with its native grader where one could be run. Lanes whose original benchmark is externally blocked were run on a public substitute, and each substitute’s gap from the original suite is recorded in its lane file. Items that errored or timed out are counted as failures in the denominator.



Source: finish/Pending_Experiments.md, “Terminal-state vocabulary”, 19 Sep 2026 — the seven states shown are that table’s full documented vocabulary, with the lane assignments of that date. The six lanes in the three pending leaves were all run and scored on 27 Sep 2026 under the lead’s receipt (Chapter 8); the vocabulary itself is unchanged.

Figure 8.6: Terminal-state taxonomy used to close every lane. Each state has a defined meaning and a named reopen condition where one exists. The seven states are the ledger’s full vocabulary; 17 lane-states cover 14 lanes because E8, E10 and E14 carry separate original and replacement rows. States classify lanes, not results: an unrecognised state is recorded as blocked rather than absorbed into a percentage, and original-contract and replacement-scope numbers are never pooled. The completion gate’s PARTIAL_EXACT/PARTIAL_RECOVERY labels are a separate evidence-class vocabulary (Chapter 5). Lane assignments are those of 2026-09-19; the six lanes in the three pending leaves were all run and scored on 2026-09-27 (Chapter 8).

Two arms were run. The first is the base model, Qwen/Qwen3.6-35B-A3B with no adapter, sampled on Tinker (Table 8.A). The second is a paired comparison of the trained actor against the same base model, both served by one vLLM configuration on Modal, on identical items (Table 8.B). The pairing exists because the adapter’s effect on individual token probabilities was measured to be about the same size as the numerical difference between the two serving engines on the same base weights, so a trained-on-vLLM against base-on-Tinker comparison would confound the adapter with the engine, and none is made.

Table 8.A. Base model on Tinker, small-scale subsets.

Lane	Benchmark run	Scope	Metric	Result	95% CI
E1	SWE-bench Pro public test split	original-public-subset	resolved rate	0/10 = 0.000	[0.00, 0.28]
E2	EffiBench	substitute	pass@1	23/30 = 0.767	[0.59, 0.88]
E3	Terminal-Bench v2.0	substitute	task success rate	4/8 = 0.500	[0.22, 0.78]
E4	BankerToolBench (official Harbor tasks)	original-public-subset	mean verifier reward	0.068 (n = 6; one nonzero task, 0.409)	— (not estimable)
E5	tau2-bench	substitute	pass^1	3/10 = 0.300	[0.11, 0.60]
E6	MiniWoB++	substitute	episode success rate	15/36 = 0.417	[0.27, 0.58]
E7	BinaryAudit upstream cbd86c7	original-public-subset	task success rate	0/6 = 0.000	[0.00, 0.39]
E8	LAB-Bench public MCQ	substitute	accuracy	26/80 = 0.325	[0.23, 0.43]
E9	MLE-bench lite/low-complexity split	original-public-subset	any-medal rate	1/5 = 0.200	[0.04, 0.62]
E10	AgentHarm test_public	original-public-subset	harmful refusal rate	20/30 = 0.667	[0.49, 0.81]
E11	VerilogEval	original-public-subset	pass@1	35/50 = 0.700	[0.56, 0.81]
E12	AppBench public tasks	original-public-subset	rubric-item pass rate	122/151 = 0.808	[0.74, 0.86]
E13	BALROG	substitute	mean progression (%)	23.1%	—
E14	Omni-MATH test	substitute	accuracy	38/100 = 0.380	[0.29, 0.48]

Table 8.B. Trained actor versus base model on the same vLLM engine, identical items.

Lane	Items	Trained	Base	Trained – base	Test
E1	10	0.000	0.000	0.000	McNemar p = 1 (b/c 0/0)
E2	30	0.700	0.800	-0.100	McNemar p = 0.25 (b/c 0/3)
E3	8	0.625	0.375	0.250	McNemar p = 0.5 (b/c 2/0)
E4	6	0.066	0.093	-0.027	one nonzero pair (-0.160); no interval
E5	10	0.500	0.200	0.300	McNemar p = 0.38 (b/c 4/1)
E6	36	0.444	0.472	-0.028	McNemar p = 1 (b/c 1/2)
E7	6	0.167	0.000	0.167	McNemar p = 1 (b/c 1/0)
E8	80	0.312	0.312	0.000	McNemar p = 1 (b/c 6/6)
E9	5	0.000	0.000	0.000	McNemar p = 1 (b/c 0/0)
E9 (above median)	5	0.000	0.200	-0.200	McNemar p = 1 (b/c 0/1)
E10 (refusal rate)	30 harmful, 30 benign	0.700	0.667	0.033	McNemar p = 1 (b/c 3/2)
E10 (harm score)	30 harmful, 30 benign	0.247	0.236	0.011	95% CI [-0.053, 0.078]
E10 (benign score)	30 harmful, 30 benign	0.780	0.794	-0.015	95% CI [-0.050, 0.012]
E11	50	0.660	0.660	0.000	McNemar p = 1 (b/c 2/2)
E12	151	0.748	0.781	-0.033	McNemar p = 0.3 (b/c 5/10); app-level sign-flip p = 0.25 (n = 6)
E13	26	25.510	23.630	1.875	95% CI [-2.73, 7.09]
E14	100	0.430	0.440	-0.010	McNemar p = 1 (b/c 5/6)

These are small samples. Most lanes score between five and eighty items, so a single item moves a lane’s rate by several percentage points and the confidence intervals are wide. In Table 8.B, b and c count items solved only by the trained actor and only by the base model respectively, and the CI is a paired bootstrap interval on the difference. Four lane-specific caveats matter. For E12 the 151 rubric items come from only six generated applications, so its McNemar p-value overstates the evidence; the exact sign-flip test over the six per-app differences gives p = 0.25, and the grader is an LLM judge that is the base model itself, so the E12 rates are judge-dependent. For E4, only btb-19b3361c scores above zero in either arm (trained 0.397, base 0.557; 0.409 in Table 8.A), so no interval is valid. For E13, the 1.875 difference is the mean of per-environment differences computed from unrounded values (recomputed 1.876), not 25.510 – 23.630. For E9, the

one competition that earned a medal in the Tinker run (nomad2018) has a 46.8k-token prompt that exceeds the 32,768-token vLLM limit and fails identically in both vLLM arms, so the vLLM medal rates are not comparable with Table 8.A. The paired differences are not combined across lanes, and none of the seventeen paired tests excludes zero. At these sizes that is an inconclusive result, not evidence of no effect: with 5 to 10 items, an exact McNemar test needs at least six discordant items, all in one direction, to reach $p < 0.05$. No equivalence test was run. Fuller E12 detail and the principal per-lane caveats from each result file are in Appendix D.

8.8 Summary of the campaign at the point of writing

By the 19 September ledger the campaign had established two full-suite results and one complete replacement scope with a score: E1 at $2/731 = 0.274$ per cent on SWE-bench Pro, E11 at $129/312 = 41.35$ per cent pass@1 (Wilson 95% CI [36.0, 46.9]) across its two framings, and E8 at $450/1967 = 0.2288$ ([0.211, 0.248]) on the LAB-Bench public split, alongside a complete benign-utility evaluation of E10 at 88 of 97 episodes (0.907, [0.833, 0.950]), a terminal-complete E14 replacement scope at $2,271/4,428 = 51.29$ per cent accuracy, and five original-contract partials (E2, E4, E5, E7, E9), none of which is a suite score. On 27 September six further replacement scopes were completed under a self-issued override whose deviations are recorded per lane (Tables 8.C and 8.D). Seven original contracts remain externally blocked, and no lane now waits on a launch or a cloud quota. The campaign did not establish a multi-seed estimate, a held-out security result, a BankerToolBench suite score, a graded result for the 108 WebArena tasks that need an LLM judge, or a baseline comparison for the campaign lanes themselves; the only trained-versus-base comparison, Table 8.B, is inconclusive in every lane. The evidence chain and reproduction path for each number are given in Appendix B; the next chapter draws the cross-study synthesis and conclusions.

Part IV

Synthesis

Chapter 9

Cross-Study Synthesis, Threats to Validity, Conclusions and Future Work

9.1 Cross-Study Synthesis

This report has presented two bodies of work built with different instruments. The first is the P1–P8 corpus, which probes GRPO-style post-training across libraries, model scales, group sizes and datasets and ends in a reporting standard, a machine-readable registry and a controller audit. The second is the E1–E14 campaign, a governance-bound evaluation of one Tinker-trained actor against fourteen agentic and reasoning suites that were not part of its training data. They are the same methodological commitment applied twice — once to *what does GRPO do*, once to *what does a GRPO-trained actor score*. That commitment is attribution: every comparative claim holds the entire training stack fixed except the one factor under test, and where that was unachievable the confound is reported alongside the number rather than absorbed.

The corpus’s own synthesis, recorded at the close of Phase 1, is that on this compute scale the *measurement* layer is useful while most *algorithmic* levers are noise-limited. The dominant measured phenomenon is advantage-signal starvation, with the Zero-Variance Fraction at approximately 0.72–0.77 on the Phase-1 main configuration, and it is easy-driven: the collapsed groups are mostly prompts the model has already solved. That rules out the most tempting remedy, because re-baselining the advantages merely converts the signal into a difficulty proxy with no learning content, and the remedy that remains — a curriculum that removes the wasted rollouts — returned a null. The narrow systems observation survived better than any algorithmic one: per-step telemetry separated the available method traces more cleanly than reward-only features, though the classifier’s integrity generalisation was never tested.

The Semester 4 internal reports sharpen those threads rather than reversing them. P2’s

diagnostics are recomputable from stored raw tensors rather than read back from logged summaries: three stored GSM8K group tensors reproduce ZVF at 0.130, 0.190 and 0.155, pooling to 0.1583, and the correlation between ZVF and final held-out outcome, $\rho = 0.27$, is not significant (95% CI $[-0.37, 0.88]$, $n = 23$ pooled cells). ZVF is therefore offered as a diagnostic and not a predictor. Once mean reward is regressed out, the pooled measured residual is $r = +0.21$ (95% CI $[-0.31, +0.62]$, $n = 17$), so ZVF is not an incremental predictor either (Chapter 6 §6.3.4). P4’s matched comparison of GRPO against Dr. GRPO finds the two held-out gains indistinguishable, with pooled mean completion length drifting from 194.4 to 183.5 tokens under a 200-token cap that bounds what the comparison can say. P4’s companion error-bar audit grades the single P4-facing headline as DECISIVE: audit H7, the iter-136 arithmetic late-training-efficiency contrast, paired $n = 5$ seeds, $d_z = +2.68$. Re-examined, it is *suggestive* only (Chapter 6 §6.5.4). Its “one-sided parametric $p = 0.031$ ” is the floor of an exact one-sided Wilcoxon test at $n = 5$. The two-sided exact sign-flip p is 0.0625, even though all five pairs point the same way. The direction was one of eight hypotheses in its family, and Holm correction across those eight raises the family’s p to 0.25. The audit’s other three DECISIVE grades do not survive either. Two (H1, H2) rest on an interval that excludes its own point estimate and on four reconstructed budgets (§6.4.2; Table E.1), and one (H6) is an AUROC computed on a synthetic and imputed panel (§6.3.4). Of the seven audited headlines, then, none is decisive, one is suggestive and the rest are null or hypothesis-generating. What survives is a consistent-direction late-training efficiency contrast on a training metric, not a capability claim.

P3 and P7 both treat group size as a lever and are best read as a pair, because they disagree in emphasis. P3 finds that under saturation the retention ratio between $G = 2$ and $G = 16$ is approximately 1.00. That ratio is of last-10 training reward, and a seed-level TOST ($n = 3$, margin ± 0.02) does not establish $G = 16$ as equivalent to $G = 2$ ($p = 0.056$; Chapter 6 §6.4.1). P3 also finds that the effective signal-to-noise ratio reaches only about 52% of the $\sqrt{G}$ ideal, and that a 505-task conditional utility audit selects $G = 4$ over $G = 5$ by 0.00177 with a 95% bootstrap interval of $[0.00082, 0.00270]$ — positive, small, and explicitly cohort- and objective-conditional. P7 models usable signal per prompt as $S = p(1 - p)(1 - h_G(p))$ with $h_G(p) = p^G + (1 - p)^G$, derives three falsifiable consequences, and checks them: a 368-run Weights & Biases audit across seven models reproduces the predicted U-shape in accuracy, with ZVF 0.95–0.97 at the low-reward extreme and 0.87–0.96 at the high-reward extreme and 0.25–0.29 at mid-range rewards of 0.35–0.50; a model $\times$ G grid confirms the monotone effect at interior accuracy, Qwen3-32B moving $0.33 \rightarrow 0.18 \rightarrow 0.08$ across $G = 4/8/16$; and a toy-scale open-backward-pass experiment finds $\text{corr}(\|\nabla\|, p(1-p)) = +0.71$ on a 0.5B model, which the P7 report labels directional only. Where P3’s sweep could not find an optimum, P7 sup-

plies a mechanism under which a shallow one should exist. That is a reconciliation, not a resolution: the mechanism is checked in telemetry, and its causal link to the gradient remains toy-scale.

The same report is clear about the control it does not have. A micro-jitter falsification collapses batch ZVF from 0.158 to 0.000 while leaving the pairwise-contrast density unchanged, which is why PCD rather than ZVF is promoted as the control signal. In a four-arm open-trainer audit, adaptive group size matches the best fixed-recipe held-out gain, +0.575, at 186 rollouts, while DAPO-style dynamic sampling drives ZVF to 0.00 at +45% rollout cost — an accuracy-versus-compute trade, not a controller win. In the larger 2,560-observation controller trace, 1,723 of 1,867 escalation events, 92.3%, occur on all-correct groups and only 144 on all-wrong, so a symmetric ZVF trigger spends most of its budget on prompts already solved; deleting those fires is a counterfactual only, because performance preservation was not measured. Adaptive group size is therefore not promoted to a recommended policy.

P5 and P6 supply the corpus’s methodological claim, and they contain its sharpest internal tension. P5’s headline exhibits are a nominally matched-configuration backend audit whose final training rewards span 5.0% to 85.6% — a 17× same-label swing the P5 report declines to assign to the backend, because the managed stack also silently pinned a different base checkpoint (Qwen3-8B-Base on Tinker against the instruction-tuned Qwen3-8B on TRL), and that undisclosed bundling is itself part of the exhibit — and a pair of runs carrying the same “DAPO” label in which an open trainer with true dynamic sampling produced mean ZVF 0.00 while a closed stack running an asymmetric-clip surrogate produced mean ZVF 0.58. Alongside them sits a third exhibit: a 12-cell head-to-head of four labelled algorithms on one fixed stack, landing within a 0.034 band with mean last-10 reward 0.710–0.744, a spread the P5 report calls comparable to seed-level variation. P5’s framing is that a label-only comparison can hide stack-level variation, and that the converse also holds: when the stack is genuinely fixed, the labels stop separating. P6 reads the same exhibit in the reverse direction, using it to motivate a schema while explicitly declining to re-claim it as novel empirical science; on the twenty seed entries the MIN-REPORT auditor badge spans 43–96, and *stackdiff* flags the open-versus-closed “DAPO” pair as an R5 label-flip risk from manifests alone.

Two gaps should be named. The first is that P7’s controller audit and P5’s stack-attribution programme have not been joined: the controller is evaluated on open trainers where the update rule is auditable, while the standard is motivated by a closed-stack divergence that cannot be audited. The second is P8, the credit-card fraud side-probe, absorbed here as a measurement-discipline case study and nothing more. Its two AUCs are not a like-for-like ranking: XGBoost’s 0.7955 and the Qwen3.5-4B SFT arm’s 0.48268 on a positive-enriched 500-row held-out evaluation sit on different held-out splits by

construction.

The E1–E14 campaign is a different instrument and should be read as one. A single Tinker-trained actor — base Qwen/Qwen3.6-35B-A3B at commit 995ad96e..., with adapter arvindr4/pavlov-portfolio-qwen36-seed809-stepfinal-tinker-cf0ad8c1... at commit 64444133..., in bfloat16 — was evaluated against fourteen suites that were not part of its training data, each through its own native evaluator at a pinned revision. Three scopes closed strictly complete: two replacement scopes, LAB-Bench public at 1,967/1,967 across eight categories and AgentDojo benign utility with 97/97 episodes completed and utility $88/97 = 0.907$ (Wilson 95% CI [0.833, 0.950]), and one original-contract suite, VerilogEval, at 312/312 with $129/312 = 41.35\%$ pass@1 canonical (Wilson 95% CI [36.0%, 46.9%]). Omni-MATH closed terminal-complete under its native protocol, with all 4,428 dispositions recorded, 4,426 accepted, and an accuracy of $2,271/4,428 = 51.29\%$ with the two unjudged rows counted as failures (the official scorer reproduces 51.31% over the 4,426 judged rows). Two suites carry full-coverage scores under their original contracts: SWE-bench Pro at $2/731 = 0.274\%$ pass@1 (Wilson 95% CI [0.08%, 0.99%]) and VerilogEval, the same E11 result counted once. Six further replacement scopes were run in full on 27 September 2026 (Table 8.C): CORE-Bench hard at $27/45 = 0.600$, Tau3 at $8/97 = 0.082$, WebArena at $90/812 = 0.111$ (a lower bound, with 108 judge-dependent tasks ungraded), ML-Dev-Bench at $10/34 = 0.294$, BALROG at 26.1% progression over 255 episodes, and SWE-bench Multilingual at 1/190 on the tasks not attempted before, kept separate from the earlier 4/110. Lane scope and status are tabulated once, in Table C.3. The rest are partial or externally blocked, each with a terminal state and a named gate.

What the campaign establishes is descriptive, and its scope matters. For *this* actor, at *this* revision, under *each suite's own* evaluator and denominator, these are the scores. It is not a post-training delta: no per-suite base-model comparator appears in the campaign ledger at the campaign's own scopes, so no campaign number, high or low, is evidence about what the GRPO step contributed. The one paired comparison of trained actor and base model is the 26 September small-scale rerun (Table 8.B): 14 lanes, 5 to 151 identical items each, one serving engine. It finds no significant difference in any lane, which at those sizes is inconclusive rather than evidence of no effect. The separately authorised E4 BankerToolBench base-model rerun returned a mean reward of 0.0 across 100/100 completed trials. That zero is attributed to the serving bridge, which passed no stop sequences, so agents ran into invented turns and ended after 2–14 steps with no deliverables/ output (Section 8.5; source: outputs/e1_e14_small_scale_2026-09-26/E4/BRIDGE_STOP_TOKEN_FINDING.md). It measures the harness, not the model.

What governance bought is visible in the ledger itself: eleven of eleven deterministic

checks passing on 2026-09-19, and a historical snapshot recording E7 as “1/28 errored” sitting visibly beside the superseding table’s “1/46 attempted” rather than being quietly reconciled.

9.2 Rigour and Threats to Validity

What rigour was exercised. Diagnostics are recomputable from stored raw tensors, and headline structural claims were re-verified by deterministic code checks on 2026-09-19, 11/11 PASS, covering exact divisions, receipt presence, sealed-artifact hash equality and lost-source absence probes. Held-out n is reported alongside every gain, and failed runs are counted rather than hidden: approximately \$30–35 of Tinker spend went to failed runs, disclosed because concealing it would understate reproduction cost by roughly 25% (Figure 9.1).

Statistical power: single-seed Tinker experiments. Most Tinker configurations ran once, because cost precluded repetition, and those single-seed runs carry no variance estimates or significance tests. The exceptions are stated where they occur. They are the three-seed-per-arm managed head-to-head (Chapter 7 §7.1), the five-seed Qwen3-8B held-out control ($p = 0.26$, Chapter 6 §6.5.4) and the Qwen3-8B scale extension. Henderson et al. (2018) recommend at least ten seeds, and no result family reported here reaches that bar. The seed counts are: one seed for most Tinker runs and all P1 anchors; three for the P3 sweep, the P4 GSM8K-CoT and scale-extension arms and the managed head-to-head; five for the same-stack PPO/GRPO contrast, the P4 arithmetic arms and the Qwen3-8B pre/post control; and eight for the variant audit (P11). The limitations document says the ten-seed bar was met for the TRL baseline and the GSM8K evaluation and then that the evaluation was cut to five seeds; the run records in Appendix A show five seeds for both the TRL Modal campaign (Table A.3) and the GSM8K pre/post evaluation. The 30B MoE scaling analysis uses three seeds; the bootstrap confidence intervals assume independent observations, so any positive correlation from shared initialisation or data ordering would make the reported intervals too narrow, and the true intervals wider; and secondary analyses carry uncorrected p -values. Quantitative conclusions therefore come from the open-source side, where every hyperparameter is auditable, while Tinker figures are descriptive.

Horizons of 30–50 steps. The Tinker runs are early-training snapshots; long-horizon effects such as reward hacking, catastrophic forgetting and late-stage policy collapse are unlikely to manifest, and the source conclusion is direct: “Asymptotic claims are unsupported”. This bounds results rather than vitiating them: P1’s failure to recover a scaling law across roughly 2.4 orders of magnitude is a bounded negative, and P4’s drift

threat to validity applied mitigation, paired by the rule at left		
Internal validity	Construct validity	External validity
single-seed runs · short horizons · lost archive	denominators · no aggregate · scope · sampler	closed platform · withheld third-party access
T1. Single-seed Tinker runs. Each Tinker configuration ran once; cost precluded repetition, so no variance estimate and no significance test exists on Tinker data at all. Henderson et al. (2018) recommend at least ten seeds. → Mitigation. Quantitative conclusions are drawn only from runs with repeated seeds — the TRL baseline and the held-out GSM8K evaluation — while Tinker figures are reported descriptively and marked †. No significance claim rests on a single-seed comparison.	T4. Native denominators are not comparable. Every result sits on its own suite's denominator — 2/731 SWE-bench Pro, 90/812 WebArena tasks, 255 BALROG episodes, 40/75 MLE-bench competitions, 4,428 OmniMATH dispositions — which differ in size and in what they count and exclude. → Mitigation. Results are never pooled, within lanes or across them (E1's two scopes, E2's, E5's, E9's). A 41.35% and a 0.274% on different denominators license no comparison, average or ranking.	T8. Closed Tinker internals. The SDK exposes a custom loss path, a limited optimiser surface and standard LoRA hyperparameters, but not the server-side GRPO loss formulation, the reward normalisation or baseline-subtraction scheme, minibatch construction, hardware configuration or system telemetry. Tinker results therefore measure the <i>platform's</i> implementation of GRPO, not an abstract algorithmic specification; the SDK is also a moving target, with Phase 1 pinned at <code>tinker=0.16.1</code> and the September 2026 bridge work requiring <code>tinker=0.30.0</code>. → Mitigation. No backend numerical-parity claim is made. Serving conditions and the pinned SDK version are recorded per lane, and the stack is reported as part of the result: a label without its stack is not a reproducible experimental specification.
T2. Horizons of 30–50 steps. The runs are early-training snapshots. Reward hacking, catastrophic forgetting and late-stage policy collapse need longer horizons to manifest, so asymptotic claims are unsupported. → Mitigation. Every claim is scoped to the observed horizon: the scaling-law null is a bounded negative at this horizon, not a statement about GRPO's asymptotics, and the 200-token cap bounds the length-bias drift. No “converges to” language is used.	T5. No cross-suite aggregate. No mean, median or composite index over suites appears anywhere in this work, so the campaign has no single headline number. → Mitigation. A position, not an omission: an aggregate over native denominators would be a number with no referent. The four complete scopes and the two original-contract scores are the honest maximum.	T9. Third-party access withheld. Six original contracts could not obtain their private bundles or hosted evaluations — E3 SDAB, E7 BinaryAudit, E8 LifeSciBench, E10 AgentHarm, E12 AppBench and E14 FrontierMath — and the access requests remain unanswered, so those scores are not producible outside the provider. → Mitigation. Each lane carries a terminal close-out record with a named reopen condition. The replacement scope is reported under its own name and never under the original's, and no substitute number is reported where the original was withheld.
T3. A missing pass3/pass16 archive. The E4 recovery's per-task artefacts do not survive: zero of six verify, and only TINKER_BRIDGE.json does. → Mitigation. The surviving 0.3115 is reported as a receipt-only verifier recovery metric on one of 100 tasks, labelled CLOSED_PARTIAL, never as a suite score or as the raw count of 37 of 128 verifier criteria met (0.289).	T6. The E10 scope measures benign utility only. E10's completed evidence is 97/97 tasks under AgentDojo's benign scope; its original contract, private AgentHarm tasks, is externally blocked, so the result says nothing about adversarial robustness. → Mitigation. The benign scope is named at every reference to the result; no adversarial-robustness claim is made for the trained actor.	
	T7. The retained E11 result came from the original sampler. VerilogEval's 129/312 = 41.35% was sampled through the Tinker managed sampler, as the receipt records. That route is gone — the run is purged — and the recovery path is a vLLM serve of the HuggingFace mirror. → Mitigation. The lane is receipt-bound, with provenance recorded down to the sampler path. No backend numerical-parity claim is made, and a re-run would be a same-weights, different-sampler comparison whose divergence is not measured.	

Figure 9.1: Threats to validity grouped as internal, construct and external, each with the mitigation applied in this work. The pairing is mostly a reduction in what is claimed rather than a repair of the evidence: where the evidence is partial the claim is narrowed to what it supports. † marks figures and summary statistics drawn from Tinker data only; independent replication of them requires Tinker API access (LIMITATIONS_AND_IMPACT.md, Sec. 6.3). In T3, 37/128 is the number of verifier criteria met on the single E4 task; the reported 0.3115 is the verifier's own recovery metric, not that fraction.

from 194.4 to 183.5 mean completion tokens is bounded by the 200-token cap.

Closed Tinker internals. The Tinker SDK exposes a custom loss path, a limited optimiser surface and standard LoRA hyperparameters, but not the exact server-side GRPO loss formulation, the reward normalisation or baseline-subtraction scheme, minibatch or gradient-accumulation strategy, hardware configuration, or system-level telemetry. This applies unevenly: the custom-loss Tinker runners, including the one that trained the campaign actor, define their loss in client code (REINFORCE with a group-mean baseline, Table 4.2), whereas the runners that use Tinker’s built-in `importance_sampling` loss leave its server-side details unrecorded. The consequence is best quoted: Tinker results measure “the **platform’s** implementation of GRPO, not an abstract algorithmic specification”. The illustrative case in the limitations document compares a pooled last-10 training reward of 99.9% for Tinker GRPO with 73.4% for TRL GRPO on GSM8K. Those figures are training rewards, not held-out accuracy, and they pool Tinker runs on 4B–235B models against TRL on Qwen2.5-0.5B, so model size and implementation are confounded; the source itself attributes part of the gap to the model difference. The only same-model pair, Qwen3-8B, used the Base checkpoint on Tinker and the Instruct checkpoint on TRL (Table A.2), so it does not remove the confound either. The threat propagates into the campaign, whose actor was trained on Tinker. The SDK is also a moving target: Phase 1 pinned `tinker==0.16.1`, while the September 2026 bridge required `tinker==0.30.0` after the older SDK produced a no-longer-supported error.

Per-suite native denominators are not comparable. Every E1–E14 result is expressed on its own suite’s denominator, and those denominators differ in size, in what they count and in what they exclude. SWE-bench Pro is 2/731, whose denominator retains 14 generation failures and 4 artifact losses alongside 713 native evaluations; APEX-Agents is 7/480; E9 MLE-bench is 40/75 competitions natively graded, stated as 53.33% coverage with a null suite score, because coverage and score are different quantities; E1’s replacement SWE-bench Multilingual is 1/190 on one contract and 4/110 on another, never summed to a 300-task figure; E6 WebArena is 90/812 with 108 tasks ungraded, so its denominator counts items that were never graded; E13 BALROG is a mean of six per-environment means rather than a rate over episodes; and E14’s Omni-MATH records 4,428 dispositions of which 4,426 were accepted, with the headline accuracy computed over all 4,428 (the official scorer’s 51.31% uses the 4,426 judged rows). A 41.35% on one denominator and a 0.274% on another do not license a comparison, an average or a ranking, and none is offered; the rule holds within lanes too, and the ledger states it flatly: “N/A means unverified; no pooled results”.

No cross-suite aggregate. No mean, median or composite index over suites appears anywhere in this report or in the campaign materials it draws on: an aggregate over native denominators would be a number with no referent, mixing suites that measure

different capabilities at different difficulties under different scoring rules. The cost is that the campaign has no single headline; the three strictly complete scopes, the terminal-complete Omni-MATH scope, the six replacement scopes completed on 27 September and the two original-contract scores are the most the record supports.

The E10 benign-utility scope says nothing about adversarial robustness. The E10 lane’s completed evidence is a benign-utility evaluation of 97/97 completed tasks with utility $88/97 = 0.907$ under AgentDojo’s benign scope; its original contract, private AgentHarm tasks, is externally blocked and carries a closure record with a named reopen condition. The completeness result therefore carries no adversarial-robustness measurement, and this report makes no such claim for the trained actor. The lane has a dual-use dimension, though the limitations analysis finds the marginal uplift of the released artefacts small, because the base model’s capability is overwhelmingly pre-existing: the pre-RL Qwen3-8B-Instruct checkpoint already scores 82.0% on held-out GSM8K, and full GRPO adds only +1.3 points to 83.3%, not significant at $p = 0.26$. The 82.0% is the same Instruct checkpoint before RL, not a separate base model.

The retained E11 result came through the original sampler. The receipt of the 129/312 full run records `model.sampler_path = tinker://cf0ad8c1-1f1b-5ff3-8bd7-2a0bf232657b:train:0/sampler_weights/seed809_final`, so the scores come from sampling the trained adapter through the Tinker managed sampler rather than from a locally served copy of the weights. That route no longer exists: the sampler status record of 2026-09-20 reports a bridge deployment booting to a sampler-weights 404, `list_checkpoints` on the `cf0ad8c1` run returning 404 with the run purged, no `last_sampler_checkpoint` on any of twenty training runs, and the status string `TINKER_SAMPLER_WEIGHTS_DELETED_E9_AND_BRIDGE_AFFECTED`; the recorded recovery path is to serve the HuggingFace mirror `arvindcr4/pavlov-portfolio-qwen36-seed809-stepfinal` at commit 64444133... through vLLM on Modal. The threat is not that the number is wrong — it is receipt-bound, provenance recorded — but that the sampling path is not re-runnable, so a re-run would be a same-weights, different-sampler comparison whose divergence is unmeasured. The full receipt’s `outstanding_blockers` field is empty. The blocker that the decontamination receipt ID does not exist and that the split-manifest validator still fails belongs to the separate four-task step-40 pilot, whose receipt records the `step_seed809_40` sampler path (Chapter 8 §8.1; source: `outputs/e11_verilog_eval/e11_trained_step40_receipt.json`).

Known evidence loss in the finish-era recovery source. The record is explicit: the `.codex-run` deletion recorded in `finish/STATE_RECONCILIATION_2026-09-19.md` removed the execution sources of the finish-era recovery and build code — the E1 wave-10 controller, the E9 restored dependencies and the E6 validator — while sealed requests, validation receipts and test logs survive with matching hashes, and the replace-

ment lanes’ native runners survive under `zvf-program/flagship/`. The lost-source policy was applied rather than waived: a faithful re-implementation was written into a tracked path under `zvf-program/e1_wave10/`, where it verifies 10/10 targeted regressions offline (`test_process_group.py` 4/4, `test_cleanup_regressions.py` 6/6). Two consequences follow: a sealed request without its execution source is not a rerun, so the E9 MLDevBench runtime recipe was never reconstructed (the 27 September run built a fresh environment on Google Cloud instead); and the policy response — execution sources henceforth live under tracked paths — cannot restore what was already lost. A related item: the Jev (TypeSafe) judgment harness lost reliability from roughly 13:10Z on 2026-09-19, both blocker-classification batteries returning degenerate distributions and a controlled yes-probe falling from about 1.0 to 0.3; the receipts are retained as incident evidence and the batteries must be re-run once the health check passes, so the ledger’s first-uncleared-blocker column stands on document evidence rather than judgment output.

Scope and confound inventory. Several Phase-1 threats remain live. Many early runs logged train-set rewards only, so time-series comparisons spanning old and new runs must account for the instrumentation change. Initial phases used LoRA at ranks 8–64 exclusively; historical baseline comparisons use default hyperparameters from the Tinker cookbook and the TRL documentation; and no full learning-rate $\times$ rank $\times$ batch $\times$ group-size grid was swept. Tinker, Modal, Colab and the PES A100 node use different accelerators, memory hierarchies and training stacks, so observed differences in reward dynamics are not purely algorithmic, and the work was not pre-registered. The model-scale range is 0.6B–1T, with the anchors above 235B single-seed and descriptive (Chapter 6 §6.1); the limitations record’s “reliable” range of 0.6B–235B is the multi-run part of that roster, so dense-70B and trillion-parameter MoE extrapolations are unreliable; benchmark coverage is narrow and biased toward verifiable rewards such as exact match, unit tests and schema validity, with no human preference data, so findings should not be extended to reward-model-based RLHF; and the work was carried out at one institution in Bengaluru, so benchmark choices and evaluation design reflect that context. The resource accounting is itself a threat. The recorded out-of-pocket spend was \$130–140, with roughly 60 A100 GPU-hours of in-kind PES University compute valued at approximately \$270 and projected carbon of about 776 kWh and about 296 kg CO₂-eq; the dominant uncertainty is the Tinker GPU-hour count, inferred because the platform exposes no hardware telemetry, giving a pessimistic Tinker contribution of about 540 kg against an optimistic one of about 130 kg. No offsets were purchased; the mitigation offered is reproducibility by artefact.

9.3 Conclusions

This report set out to build a multi-framework training harness for GRPO-style post-training of large language models and to study that post-training under a single methodological rule: hold the stack fixed, change one factor, report what happens. The harness dispatches one configuration to several back-ends, but only Tinker and TRL produced completed runs, on different base checkpoints, so it is not a completed multi-framework benchmark. The four research questions of §1.4 have the following answers.

- **RQ1 (ZVF).** Yes, it is recomputable: the three stored Qwen3-8B tensors reproduce 0.130, 0.190 and 0.155 exactly. Its information about outcome is limited: its reported association with catastrophic collapse is stronger than with final held-out accuracy, and the latter is not distinguishable from zero (§6.3; Table 1.1). Its magnitude is regime-dependent, from 0.16 to 0.84 across the measured runs.
- **RQ2 (algorithm labels on a fixed stack).** Unanswered at this budget. PPO versus GRPO, GRPO versus Dr. GRPO and $G \in \{2, 4, 8, 16\}$ all sit at evaluation accuracy 0.98–0.99 on 0.5B two-digit arithmetic, and nulls at ceiling are not evidence of equivalence: they cannot separate no effect from no headroom. The unsaturated GRPO/Dr. GRPO comparisons (Qwen2.5-1.5B GSM8K-CoT; the eight-seed variant audit of §1.2) are inconclusive and establish no equivalence either (§6.2.3, §6.4, §6.5; Table 1.1). The same-stack rerun on the Qwen2.5-1.5B GSM8K-CoT configuration set out in §9.4 is what would answer it.
- **RQ3 (manifest-level flip risk).** Yes, for the one pair tested: `stackdiff` flags the open-versus-closed “DAPO” pair as R5 from manifests alone (§7.2). Whether it flags risk that matters on outcome was not tested beyond that pair.
- **RQ4 (governance-first evaluation).** It yields receipt-bound per-suite scores and an explicit terminal state for every lane (Table C.3). It does not yield a post-training delta: the actor had 40 training steps, and the paired Table 8.B comparison is inconclusive in every lane.

On the analytical side the contribution is a measurement layer and a set of bounded negatives. The Zero-Variance Fraction is a reproducible diagnostic of advantage-signal starvation, recomputable from stored tensors, regime-dependent in magnitude (0.72–0.77 on the Phase-1 main configuration, 0.16 to 0.84 across the measured runs; Table 1.1), with an association with final held-out outcome that is not distinguishable from zero ($\rho = 0.27$, 95% CI $[-0.37, 0.88]$, $n = 23$). It is descriptive, not predictive. Its theory has been made falsifiable, and two of its three consequences reproduced: the U-shape in accuracy across reward levels and the monotone group-size effect; the gradient link remains toy-scale and directional. Group size therefore has a candidate mechanism, but the

measured three-seed sweep found its arms statistically indistinguishable and establishes no optimum (Chapter 6 §6.4). Length bias is not established: the roster-level flag that marks four of eleven GRPO runs — peak before 65% of training, terminal reward below 90% of peak — has post hoc thresholds, is not specific to length, and fires on most runs in the wider trace pool, so it is not evidence of length bias; in the controlled comparison the same rule fires on both GRPO and Dr. GRPO (five of six GSM8K-CoT runs) while no run shows length growing, completion length drifts shorter, and the two are indistinguishable on held-out gain (Chapter 6 §6.5). No claim is made that Dr. GRPO is uniquely free of the pattern. The levers we could test — a naive curriculum, a matched-token-budget intervention, an integer group-size sweep — did not produce reliable held-out gains. The same-stack de-confounds (PPO versus GRPO, GRPO versus Dr. GRPO, and group size) were all measured on a saturated 0.5B arithmetic task with evaluation accuracy near 0.99, so their nulls say that no difference was detectable there, not that the methods are equivalent; the one unsaturated configuration (Qwen2.5-1.5B on GSM8K-CoT, +5 to +6 points pre-to-post) was not used for a PPO/GRPO or group-size contrast. The strongest original contribution remains P5, the reporting standard, whose status the record states precisely: a targeted search of the GRPO, model-reporting and provenance literature through July 2026 did not identify an equivalent GRPO-specific machine-readable run datasheet with rollout provenance, and this is “an evidence-bounded novelty claim, not proof of non-existence”. The earlier second bet — predictive layer freezing from step-1 behaviour — is weakened rather than withdrawn: it held at 1.5B and collapsed to chance at 3B across four seeds, at a mean of 0.083 ± 0.048 .

The E1–E14 campaign is an evaluation-governance contribution, not evidence about GRPO: the actor received 40 steps of critic-free policy-gradient training (§8.1.1), so its scores describe a checkpoint rather than a training method. It delivered what a governance-first campaign can deliver: complete, receipt-bound results on three suites (two replacement scopes and one original contract), with Omni-MATH terminal-complete; full-coverage original-contract scores on two (E1 and E11); and an explicit, auditable terminal state for every remaining lane, with a named gate and a named reopen condition instead of an indefinite pending. Its discipline is negative, which is why it is credible: original-contract and replacement-scope numbers are never pooled, no cross-suite aggregate is computed, and no claim of improvement over a baseline or over the base model is made anywhere in this work (the one paired trained-versus-base comparison, Table 8.B, is inconclusive in every lane). The strongest complete positive result, $129/312 = 41.35\%$ pass@1 on VerilogEval at full 312/312 coverage, is an end-to-end harness score (150 of its 183 failures are extraction failures) and is reported as a measurement of one actor on one suite, with its provenance recorded down to the sampler path.

The report’s overall conclusion is about method rather than algorithms. On this compute scale, and with these instruments, the measurements are informative while the algorithmic levers are noise-limited; the stack is a material part of the result, so a label without its stack is not a reproducible experimental specification; and the most durable artefacts produced here are the ones that make other people’s results checkable — the ZVF/PCD diagnostic layer, the eight-item minimum reportable stack (seven manifest fields and one evaluation item), the registry and its CLI, and a campaign ledger in which every number is bound to a receipt and every gap has a gate.

9.4 Future Work

The work leaves three kinds of open question: evaluation lanes whose measurement is incomplete, analytical claims that the present evidence could not settle, and methodological tools that have been built but not yet tested at scale.

Closing the remaining evaluation gaps. The six replacement lanes that were launch-pending or quota-blocked on 21 September were run to completion on 27 September 2026 for \$44.25 in lane spend (Chapter 8 §8.4). Two measurement gaps remain in them. The 108 WebArena tasks that need an LLM judge are ungraded because the judge account had no credit, and the native judge model has since been retired, so a re-grade would record a judge substitution. SWE-bench Multilingual has been attempted on all 300 tasks, but under two serving contracts, so a single 300-task figure would need the first 110 tasks rerun under the 27 September contract (32,768-token context and the re-implemented source collector). Six original-contract scopes are recorded CLOSED_EXTERNAL with written closure records: E3 (SDAB private bundle), E7 (BinaryAudit private payload), E8-original (LifeSciBench package), E10-original (AgentHarm private tasks), E12 (AppBench deployment) and E14-original (FrontierMath hosted evaluation). A seventh original contract, E13’s OpenReward held-out games, is also blocked on external access but has no closure record, because the lane continues through its BALROG replacement scope (Table C.3). Each reopens only on a provider response, not on an internal milestone. A full BankerToolBench score for E4 would be a new-runtime experiment with a grader-only lower bound of approximately \$59.45, and the 100/100-trial base-model rerun at mean reward 0.0 provides its calibration point.

Turning the campaign into a post-training measurement. The campaign reports scores for one actor, not the contribution of the training step. The paired same-engine comparison of §8.7 (Table 8.B) is the first base-model arm; scaling it from 5–151 items per lane to the full native scopes, with more than one training seed and a longer training run than the actor’s 40 steps, would turn each lane from a descriptive score into a post-

training delta. Separately, the retained E11 result was sampled through a Tinker sampler route that no longer exists. Re-serving the published adapter mirror and re-running E11 would measure the same-weights, different-sampler divergence that is currently unmeasured.

Settling the analytical questions. Five Phase-1 directions remain open, each answering a threat identified above:

- adequately powered evaluation, moving from the Phase-1 probes' held-out n of 8–20 (the multi-seed controlled comparisons already use 200–500 items, Chapter 1 §1.5) to the full GSM8K test split or a large stratified subset at five or more seeds;
- a token-budget-optimal curriculum with staleness bounds, tested multi-seed;
- testing whether the mid-to-late band in the per-layer gradient profile supports a FLOP-saving freeze schedule, even though step-1 predictability did not survive scaling;
- hardening the Phase-1 method-trace classifier on grouped run-level and temporal splits rather than row-wise cross-validation;
- consolidating the diagnostic findings and the reporting standard for publication.

Three more follow from the Chapter 6 results. The first is a length-confounded regime, with the completion cap lifted and longer horizons, in which a Dr. GRPO length advantage would be expected to appear if it exists. The second is the head-to-head protocol between ZVF and the pairwise-contrast density, to test whether PCD meets its target association. The third is a harder, unsaturated task on which algorithm and group-size effects could be detected at all: rerunning the same-stack PPO/GRPO contrast and $G \in \{2, 8\}$ on the Qwen2.5-1.5B GSM8K-CoT configuration, with five paired seeds and a disjoint evaluation split, and reporting the difference with its interval against a stated equivalence margin in accuracy points.

Testing the methodological tools. Three extensions come from the Semester-4 work:

- a pre-registered controller bakeoff against static $G = 16$ and naive boundary heuristics, the test P7 declines to claim in advance of;
- a cross-framework divergence study that runs an identical task, reward, decoding and seed configuration across TRL, veRL, OpenRLHF and Tinker, turning P5's attribution argument from an audit of existing runs into a designed experiment;
- dogfooding the registry schema on the campaign's own runs, so that every future benchmark result ships with a verifiable provenance record.

Two commitments carry forward, because the threats above are structural rather than incidental. The first is the never-pool rule: original-contract and replacement-scope

numbers stay separate, and per-suite native denominators are never averaged. The second is that failures are reported with their mechanism, as the E11 sampler path, the E4 zero, the lost execution sources, the seed-count discrepancy and the failed-run spend are here.

References

- [1] Madhu Kumara L, Sandhya Jeyaraj, Mohammad Rafi, Arumugam Chetty K, Arvind C R, and Dhruva N Murthy, “Literature survey — RL for LLMs: A systematic GRPO scaling study,” PES University, Bengaluru, M.Tech Semester-3 Capstone Literature Survey (Group 6), 2026.
- [2] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, arXiv:1706.03741.
- [3] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano, “Learning to summarize with human feedback,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 3008–3021.
- [4] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, arXiv:2203.02155.
- [5] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving, “Fine-tuning language models from human preferences,” 2019. [Online]. Available: <https://arxiv.org/abs/1909.08593>
- [6] N. Jaques, S. Gu, D. Bahdanau, J. M. Hernández-Lobato, R. E. Turner, and D. Eck, “Sequence tutor: Conservative fine-tuning of sequence generation models with KL-control,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, arXiv:1611.02796.
- [7] N. Jaques, A. Ghandeharioun, J. H. Shen, C. Ferguson, À. Lapedriza, N. Jones, S. Gu, and R. Picard, “Way off-policy batch deep reinforcement learning of implicit human preferences in dialog,” arXiv preprint arXiv:1907.00456, 2019.
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv preprint arXiv:1707.06347, 2017.

- [9] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *International Conference on Learning Representations (ICLR)*, 2016, arXiv:1506.02438.
- [10] R. Zheng, S. Dou, S. Gao, Y. Hua, W. Shen, B. Wang, Y. Liu, S. Jin, Q. Liu, Y. Zhou *et al.*, “Secrets of RLHF in large language models part i: PPO,” arXiv preprint arXiv:2307.04964, 2023.
- [11] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models,” arXiv preprint arXiv:2402.03300, 2024.
- [12] A. Ahmadian, C. Cremer, M. Gallé, M. Fadaee, J. Kreutzer, O. Pietquin, A. Üstün, and S. Hooker, “Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 12 248–12 267, arXiv:2402.14740.
- [13] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi *et al.*, “DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning,” *Nature*, vol. 645, no. 8081, pp. 633–638, 2025, preprint: DeepSeek-AI, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning”, arXiv:2501.12948.
- [14] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, arXiv:2305.18290.
- [15] M. G. Azar, M. Rowland, B. Piot, D. Guo, D. Calandriello, M. Valko, and R. Munos, “A general theoretical paradigm to understand learning from human preferences,” arXiv preprint arXiv:2310.12036, 2023.
- [16] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela, “KTO: Model alignment as prospect theoretic optimization,” arXiv preprint arXiv:2402.01306, 2024.
- [17] J. Hong, N. Lee, and J. Thorne, “ORPO: Monolithic preference optimization without reference model,” arXiv preprint arXiv:2403.07691, 2024.
- [18] Y. Meng, M. Xia, and D. Chen, “SimPO: Simple preference optimization with a reference-free reward,” arXiv preprint arXiv:2405.14734, 2024.

- [19] Y. Zhao, R. Joshi, T. Liu, M. Khalman, M. Saleh, and P. J. Liu, “SLiC-HF: Sequence likelihood calibration with human feedback,” arXiv preprint arXiv:2305.10425, 2023.
- [20] Y. Wu, L. Ma, L. Ding, M. Li, X. Wang, K. Chen, Z. Su, Z. Zhang, C. Huang, Y. Zhang *et al.*, “It takes two: Your GRPO is secretly DPO,” arXiv preprint arXiv:2510.00977, 2025.
- [21] Z. Liu, C. Chen, W. Li, P. Qi, T. Pang, C. Du, W. S. Lee, and M. Lin, “Understanding R1-Zero-like training: A critical perspective,” in *Conference on Language Modeling (COLM)*, 2025, arXiv:2503.20783.
- [22] Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu *et al.*, “DAPO: An open-source LLM reinforcement learning system at scale,” *arXiv preprint*, 2025, arXiv:2503.14476.
- [23] C. Zheng, S. Liu, M. Li, X.-H. Chen, B. Yu, C. Gao, K. Dang, Y. Liu, R. Men, A. Yang *et al.*, “Group sequence policy optimization,” arXiv preprint arXiv:2507.18071, 2025.
- [24] Y. Yue, Y. Yuan, Q. Yu, X. Zuo, R. Zhu, W. Xu, J. Chen, C. Wang, T. Fan, Z. Du *et al.*, “VAPO: Efficient and reliable reinforcement learning for advanced reasoning tasks,” arXiv preprint arXiv:2504.05118, 2025.
- [25] H. Zheng, Y. Zhou, B. R. Bartoldson, B. Kailkhura, F. Lai, J. Zhao, and B. Chen, “Act only when it pays: Efficient reinforcement learning for LLM reasoning via selective rollouts,” arXiv preprint arXiv:2506.02177, 2025.
- [26] Z. Lin, M. Lin, Y. Xie, and R. Ji, “CPPPO: Accelerating the training of group relative policy optimization-based reasoning models,” arXiv preprint arXiv:2503.22342, 2025.
- [27] G. Nan, S. Chen, J. Huang, M. Lu, D. Wang, C. Xie, W. Xiong, X. Zeng, Q. Zhou, Y. Li *et al.*, “NGRPO: Negative-enhanced group relative policy optimization,” arXiv preprint arXiv:2509.18851, 2025.
- [28] X. Zhang, S. Wu, Y. Zhu, H. Tan, S. Yu, Z. He, and J. Jia, “Scaf-GRPO: Scaffolded group relative policy optimization for enhancing LLM reasoning,” in *International Conference on Learning Representations (ICLR)*, 2026, arXiv:2510.19807.
- [29] Y. Ichihara, Y. Jinnai, T. Morimura, M. Sakamoto, R. Mitsuhashi, and E. Uchibe, “MO-GRPO: Mitigating reward hacking of group relative policy optimization on multi-objective problems,” arXiv preprint arXiv:2509.22047, 2025.

- [30] K. Yang, Z. Chen, Y. Wang, Z. Li, N. Cheng, S. Wang, and J. Xiao, “SSPO: Subsentence-level policy optimization,” arXiv preprint arXiv:2511.04256, 2025.
- [31] T.-L. V. Le, M. Jeon, K. Vu, V. Lai, and E. Yang, “No prompt left behind: Exploiting zero-variance prompts in LLM reinforcement learning via entropy-guided advantage shaping,” in *International Conference on Learning Representations (ICLR)*, 2026, arXiv:2509.21880.
- [32] Z. Zhang, Z. Han, C. Mavromatis, Q. Zhu, Y. Zhang, S. Guan, D. Wang, X. Zhou, S. Wang, S. Adeshina *et al.*, “Train less, learn more: Adaptive efficient rollout optimization for group-based reinforcement learning,” arXiv preprint arXiv:2602.14338, 2026.
- [33] K. Zhang, Y. Hong, J. Bao, H. Jiang, Y. Song, D. Hong, and H. Xiong, “GVPO: Group variance policy optimization for large language model post-training,” in *Advances in Neural Information Processing Systems*, 2025, arXiv:2504.19599.
- [34] Y. Wang, J. Zhao, C. Zhao, S. Guan, G. Penn, and S. Liu, “ λ -GRPO: Unifying the GRPO frameworks with learnable token preferences,” arXiv preprint arXiv:2510.06870, 2025.
- [35] Y. Kim, “MC-GRPO: Median-centered group relative policy optimization for small-rollout reinforcement learning,” arXiv preprint arXiv:2601.22582, 2026.
- [36] Y. Guo, W. Deng, Z. Cheng, and X. Tang, “G²RPO-A: Guided group relative policy optimization with adaptive guidance,” arXiv preprint arXiv:2508.13023, 2025.
- [37] F. Zhang, X. Ma, Z. Dong, X. Leng, J. Zhao, X. Sun, Y. Yang, and G. Jiang, “Revisiting reinforcement learning with verifiable rewards from a contrastive perspective,” arXiv preprint arXiv:2605.12969, 2026.
- [38] I. Mahrooghi, A. Lotfi, and E. Abbe, “Goldilocks RL: Tuning task difficulty to escape sparse rewards for reasoning,” arXiv preprint arXiv:2602.14868, 2026.
- [39] X. He, Q. Sun, A. Cheng, X. Li, X. Ji, H. Lu, R. Huang, and Q. Hu, “Advantage collapse in group relative policy optimization: Diagnosis and mitigation,” in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, arXiv:2605.21125.
- [40] N. Lambert, J. Morrison, V. Pyatkin, S. Huang, H. Ivison, F. Brahman, L. J. V. Miranda *et al.*, “Tulu 3: Pushing frontiers in open language model post-training,” *arXiv preprint*, 2024, arXiv:2411.15124.

- [41] J. Hu, X. Wu, W. Shen, J. K. Liu, Z. Zhu, W. Wang, S. Jiang, H. Wang, H. Chen, B. Chen *et al.*, “OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework,” arXiv preprint arXiv:2405.11143, 2024.
- [42] G. Sheng, C. Zhang, Z. Ye, X. Wu, W. Zhang, R. Zhang, Y. Peng, H. Lin, and C. Wu, “HybridFlow: A flexible and efficient RLHF framework,” arXiv preprint arXiv:2409.19256, 2024.
- [43] Z. Yao, R. Y. Aminabadi, O. Ruwase, S. Rajbhandari, X. Wu, A. A. Awan, J. Rasley, M. Zhang, C. Li, C. Holmes *et al.*, “DeepSpeed-Chat: Easy, fast and affordable RLHF training of ChatGPT-like models at all scales,” arXiv preprint arXiv:2308.01320, 2023.
- [44] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, arXiv:2309.06180.
- [45] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, “SGLang: Efficient execution of structured language model programs,” arXiv preprint arXiv:2312.07104, 2023.
- [46] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-Awareness,” arXiv preprint arXiv:2205.14135, 2022.
- [47] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022, arXiv:2106.09685 (2021).
- [48] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, “Training verifiers to solve math word problems,” arXiv preprint arXiv:2110.14168, 2021.
- [49] J. Vendrow, E. Vendrow, S. Beery, and A. Madry, “Do large language model benchmarks test reliability?” arXiv preprint arXiv:2502.03461, 2025.
- [50] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, “Measuring mathematical problem solving with the MATH dataset,” in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2021, arXiv:2103.03874.

- [51] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, “Let’s verify step by step,” in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2305.20050 (2023).
- [52] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, “Model cards for model reporting,” in *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, 2019, pp. 220–229.
- [53] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, “Datasheets for datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [54] E. M. Bender and B. Friedman, “Data statements for natural language processing: Toward mitigating system bias and enabling better science,” *Transactions of the Association for Computational Linguistics (TACL)*, vol. 6, pp. 587–604, 2018.
- [55] M. Pushkarna, A. Zaldivar, and O. Kjartansson, “Data cards: Purposeful and transparent dataset documentation for responsible ai,” in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT ’22)*, 2022, pp. 1776–1826.
- [56] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [57] C. Colas, O. Sigaud, and P.-Y. Oudeyer, “A hitchhiker’s guide to statistical comparisons of reinforcement learning algorithms,” *arXiv preprint*, 2019, arXiv:1904.06979.
- [58] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. Bellemare, “Deep reinforcement learning at the edge of the statistical precipice,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 29 304–29 320, arXiv:2108.13264; supplementary materials at <https://agarwl.github.io/rliable/>.
- [59] S. M. Jordan, A. White, B. Castro da Silva, M. White, and P. S. Thomas, “Position: Benchmarking is limited in reinforcement learning research,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, arXiv:2406.16241.
- [60] J. Dodge, S. Gururangan, D. Card, R. Schwartz, and N. A. Smith, “Show your work: Improved reporting of experimental results,” in *Proceedings of the 2019*

Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 2185–2194.

- [61] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d’Alché Buc, E. Fox, and H. Larochelle, “Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program,” *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021. [Online]. Available: <https://jmlr.org/papers/v22/20-303.html>
- [62] E. Miller, “Adding error bars to evals: A statistical approach to language model evaluations,” arXiv preprint arXiv:2411.00640, 2024.
- [63] S. Biderman, H. Schoelkopf, L. Sutawika, L. Gao, J. Tow, B. Abbasi, A. F. Aji, P. S. Ammanamanchi, S. Black, J. Clive *et al.*, “Lessons from the trenches on reproducible evaluation of language models,” arXiv preprint arXiv:2405.14782, 2024.
- [64] A. Hochlehnert, H. Bhatnagar, V. Udandaraao, S. Albanie, A. Prabhu, and M. Bethge, “A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility,” in *Conference on Language Modeling (COLM)*, 2025, arXiv:2504.07086.
- [65] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar *et al.*, “Holistic evaluation of language models,” *Transactions on Machine Learning Research*, 2023, arXiv:2211.09110.
- [66] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” arXiv preprint arXiv:2001.08361, 2020.
- [67] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark *et al.*, “Training compute-optimal large language models,” arXiv preprint arXiv:2203.15556, 2022.
- [68] J. Hilton, J. Tang, and J. Schulman, “Scaling laws for single-agent reinforcement learning,” *arXiv preprint*, 2023, arXiv:2301.13442.
- [69] L. Gao, J. Schulman, and J. Hilton, “Scaling laws for reward model overoptimization,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 10 835–10 866, arXiv:2210.10760 (2022).

- [70] R. Rafailov, Y. Chittepudi, R. Park, H. Sikchi, J. Hejna, B. Knox, C. Finn, and S. Niekum, “Scaling laws for reward model overoptimization in direct alignment algorithms,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, arXiv:2406.02900.
- [71] D. Nimmatouri, V. Bhargava, R. Ghosh, J. George, and D. Dutta, “Predictive scaling laws for efficient GRPO training of large reasoning models,” *arXiv preprint*, 2025, arXiv:2507.18014; identifies three consistent training phases: slow start, rapid improvement, plateau.
- [72] Z. Tan, H. Geng, X. Yu, M. Zhang, G. Wan, Y. Zhou, Q. He, X. Xue, H. Zhou, Y. Fan, Z. Li, Z. Zhang, G. Zhang, C. Zhang, Z. Yin, P. Torr, and L. Bai, “Scaling behaviors of LLM reinforcement learning post-training: An empirical study in mathematical reasoning,” *arXiv preprint arXiv:2509.25300*, 2025.
- [73] C. Snell, J. Lee, K. Xu, and A. Kumar, “Scaling LLM test-time compute optimally can be more effective than scaling model parameters,” *arXiv preprint arXiv:2408.03314*, 2024.
- [74] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, arXiv:2305.14314.
- [75] S. Mukherjee, L. Yuan, D. Hakkani-Tur, and H. Peng, “Reinforcement learning finetunes small subnetworks in large language models,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025, arXiv:2505.11711.
- [76] P. Wang, L. Li, Z. Shao, R. X. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui, “Mathshepherd: Verify and reinforce LLMs step-by-step without human annotations,” *arXiv preprint arXiv:2312.08935*, 2023.
- [77] M. Song, Z. Su, X. Qu, J. Zhou, and Y. Cheng, “PRMBench: A fine-grained and challenging benchmark for process-level reward models,” *arXiv preprint arXiv:2501.03124*, 2025.
- [78] L. Luo, Y. Liu, R. Liu, S. Phatale, M. Guo, H. Lara, Y. Li, L. Shu, Y. Zhu, L. Meng *et al.*, “Improve mathematical reasoning in language models by automated process supervision,” *arXiv preprint arXiv:2406.06592*, 2024.
- [79] S. Xia, X. Li, Y. Liu, T. Wu, and P. Liu, “Evaluating mathematical reasoning beyond accuracy,” *arXiv preprint arXiv:2404.05692*, 2024.

- [80] L. Yuan, W. Li, H. Chen, G. Cui, N. Ding, K. Zhang, B. Zhou, Z. Liu, and H. Peng, “Free process rewards without process labels,” arXiv preprint arXiv:2412.01981, 2024.
- [81] T. Zeng, S. Zhang, S. Wu, C. Classen, D. Chae, E. Ewer, M. Lee, H. Kim, W. Kang, J. Kunde *et al.*, “VersaPRM: Multi-domain process reward model via synthetic reasoning data,” arXiv preprint arXiv:2502.06737, 2025.
- [82] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Advances in Neural Information Processing Systems*, 2023, arXiv:2305.10601.
- [83] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2203.11171.
- [84] J. Zhang and C. Zuo, “GRPO-LEAD: A difficulty-aware reinforcement learning approach for concise mathematical reasoning in language models,” arXiv preprint arXiv:2504.09696, 2025.
- [85] P. Aggarwal and S. Welleck, “L1: Controlling how long a reasoning model thinks with reinforcement learning,” in *Conference on Language Modeling (COLM)*, 2025, arXiv:2503.04697.
- [86] X. Li, H. Zou, and P. Liu, “LIMR: Less is more for RL scaling,” arXiv preprint arXiv:2502.11886, 2025.
- [87] B. Jin, H. Zeng, Z. Yue, J. Yoon, S. Arik, D. Wang, H. Zamani, and J. Han, “Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning,” arXiv preprint arXiv:2503.09516, 2025.
- [88] Y. Zuo, K. Zhang, L. Sheng, S. Qu, G. Cui, X. Zhu, H. Li, Y. Zhang, X. Long, E. Hua *et al.*, “TTRL: Test-time reinforcement learning,” arXiv preprint arXiv:2504.16084, 2025.

Part V

Appendices

Appendix A

Complete Run Registry

This appendix is the auditable index of every training and evaluation run on which the claims of this report rest. It does not present results — those live in Chapters 6–8 — but makes each claim traceable to the artefacts that produced it: run identifier, library and platform, base model, algorithm, group size, dataset, seed, steps, and terminal status. Where a claim cannot be traced through this appendix to a run, the project’s evidence rules require it to be withdrawn rather than stated.

Table A.9 at the end of this appendix reconciles every run total quoted in the report. **The registry covers 1,708 runs: 1,662 in the machine-readable ledger and 46 more recorded only in the Semester-4 opening artefacts of §A.3.1.** The ledger, `platform_hybrid/experiments/results/experiment_ledger.tsv`, holds one row per run with the columns `source`, `project`, `run_id`, `model`, `task`, `algo`, `G`, `steps`, `created`, `headline_metric`. Its 1,662 data rows decompose into **844 runs inventoried from the Tinker training service** and **818 runs inventoried from Weights & Biases**. The Tinker inventory was built by paginating `RestClient.list_training_runs` and `list_checkpoints`; the W&B inventory was built per project and validated against the live W&B API. The ledger’s provenance document, `EXPERIMENT_LEDGER.md`, records the same grand total and notes that the W&B workspace holds fourteen projects, not the thirteen previously believed (`tinker-new-research` was the omission).

Two other totals appear in the repository and are deliberately *not* used here: the identity-audit manifest records 949 Tinker and 940 W&B runs, and `tinker_runs.txt` reports “20 training runs (963 more not shown)”. These are snapshots of the same platforms at different times and are not reconcilable without re-querying them. The ledger is the only artefact carrying per-run identifiers for both platforms in one file, so it is the figure used throughout for ledgered runs. The 46 runs outside it are counted separately, from the per-run name and mode fields of the artefacts that recorded them, and are named

individually in §A.3.1.

A.1 Composition and temporal split

Semester 3 ends at the tag `capstone-final-2026-04-25`, boundary commit `21a99ef7` dated 23 April 2026. Because every ledger row carries a creation timestamp, the two eras separate mechanically rather than by assertion: **806 runs were created before 2026-04-23 and 856 on or after it.**

Registry composition by logging project.

Table A.1: Registry composition by logging project, with temporal split and distinct base models.

Project	Source	Runs	Era	Distinct base models	Algorithm	G	Dataset	Status
tinker (training- service inventory)	Tinker API	844	387 S3 / 457 S4	31	—	—	—	all terminal at snapshot
tinker-rl -lab-world- d-class	W&B	174	Semester 3	14	GRPO	—	gsm8k	logged
tinker-rl -scaling	W&B	88	Semester 3	—	—	—	—	logged
tinker-st ructural- ceiling	W&B	72	Semester 3	—	—	—	—	logged
tinker-rl -webarena	W&B	40	Semester 3	—	—	—	WebArena	logged
webarena- eval	W&B	26	Semester 3	—	—	—	WebArena	logged
tinker-ag entic-smo ke	W&B	10	Semester 3	—	—	—	—	smoke
huggingfa ce	W&B	3	Semester 3	—	—	16	—	logged
skyr1-tin ker	W&B	3	Semester 3	Qwen/Qwen3-grpo 8B	—	8	—	logged
tinker-rl -zvf-coun terfactua l	W&B	9	3 S3 / 6 S4	—	—	—	—	logged
zvf-audit	W&B	368	Semester 4	3	—	4, 8	llama3, qwen3_instruct	logged
zvf-colab -experime nts	Colab → W&B	16	Semester 4	—	—	—	—	logged
tinker-ne w-researc h	W&B	9	Semester 4	—	—	—	—	logged

The ledger snapshot covers runs created from 8 January to 4 July 2026. Runs after that

cut-off, including the August 2026 training run of the E1–E14 actor and every campaign evaluation, are not in the 1,662 ledger rows; they are indexed by their own receipts (§A.4). The split is not clean with respect to authorship — a single Tinker account was used throughout, with 387 Tinker rows before the boundary and 457 after — which is why the inherited-versus-individual statement in Appendix B rests on commit history rather than run counts.

A.2 Semester-3 shared benchmark runs

The Semester-3 contribution was the multi-framework benchmark infrastructure: a common harness, a fixed evaluation configuration, and integrations for several independent RL post-training libraries. These runs are the shared baseline measurements, and they are the ones for which the registry is *most* complete, because each was produced under a declared configuration.

Cross-framework comparison at the fixed benchmark configuration. Configuration for all four rows: model Qwen/Qwen3-8B (Tinker row uses Qwen/Qwen3-8B-Base), algorithm GRPO, group size $G = 8$, learning rate $1e-5$, dataset GSM8K[:500], 30 steps, seed 42. Metric is `last10_avg`, the mean of the final ten reported training rewards. Source: `platform_hybrid/experiments/results/framework_comparison.json`, generated by `aggregate_framework_comparison.py`.

Table A.2: Semester-3 cross-framework shared benchmark runs.

Run identifier	Stack / platform	Model	Algo	G	Dataset	Seed	Steps	Outcome / status
<code>campaign_v2_w1_qwen3-8b-base</code>	Tinker	Qwen/Qwen3-GRPO 8B-Base		8	<code>gsm8k-500</code>	42	30	<code>last10 = 0.85625 — real run</code>
<code>modal_trl_trl_qwen3_8b</code>	TRL on Modal H100	Qwen/Qwen3-GRPO 8B		8	<code>gsm8k-500</code>	42	30	<code>last10 = 0.05, 735.7 s — real run</code>
<code>verl-dryrun</code>	verl	Qwen/Qwen3-GRPO 8B		8	<code>gsm8k-500</code>	42	30	<code>last10 = 0.5528 — seeded dryrun, not a training run</code>
<code>openrlhf-dryrun</code>	OpenRLHF	Qwen/Qwen3-GRPO 8B		8	<code>gsm8k-500</code>	42	30	<code>last10 = 0.4785 — seeded dryrun, not a training run</code>

The two dryrun rows are a documented threat to validity, not a result: the generator falls back to a deterministic seeded trace from `verl.VERLTrainer / OpenRLHFTrainer` when the framework is absent from the sandbox, and the JSON carries the note “Seeded deterministic fallback ... (verl not installed in sandbox)”. Any comparison reading them as measurements of verl or OpenRLHF is invalid. The Tinker row also initialises from the *base* checkpoint while the open-framework rows use the instruction-tuned one, so even the two real rows are not a clean framework contrast.

Modal multi-framework campaign, 4 frameworks $\times$ 5 seeds = 20 runs. Source: `modal_results_all.json` (repository root). All rows: model Qwen/Qwen2.5-0.5B, dataset arithmetic/math, 125 training steps, NVIDIA L4, one run per seed. Seeds are 42, 123, 456, 789, 1024. Group size is not recorded in the artefact and is therefore shown as “—”.

Table A.3: Semester-3 Modal campaign runs.

Run identifier	Stack / platform	Model	Algo	G	Dataset	Seed	Steps	Outcome / status
trl_grpo_math	TRL / Modal L4	Qwen/Qwen2.5-0.5B	GRPO	—	math	42, 123, 456, 789, 1024	125	final acc 0.735 / 0.810 / 0.620 / 0.740 / 0.765; 139.8–183.7 s
sb3_ppo_math	Stable-Baselines3 / Modal L4	Qwen/Qwen2.5-0.5B	PPO	—	math	42, 123, 456, 789, 1024	125	final acc 0.003 / 0.014 / 0.011 / 0.012 / 0.010; steps_to_95 null for all five
cleanrl_ppo_math	CleanRL / Modal L4	Qwen/Qwen2.5-0.5B	PPO	—	math	42, 123, 456, 789, 1024	125	final acc 0.010 / 0.008 / 0.014 / 0.009 / 0.004; 25.4–32.4 s
tianshou_ppo_math	Tianshou / Modal L4	Qwen/Qwen2.5-0.5B	PPO	—	math	42, 123, 456, 789, 1024	125	final acc 0.011 / 0.002 / 0.005 / 0.002 / 0.009; 1.67–3.30 s

Group-size sweep, shared-harness runs. Source: `platform_hybrid/experiments/results/groupsize_zvf_sweep.tsv` (per-run data in `groupsize_zvf_sweep.j`

son). Model Qwen/Qwen2.5-0.5B on a synthetic arithmetic-correctness task, 3 seeds per group size (42, 123, 456; 12 runs), 40 steps, 16 prompts per step, one NVIDIA A10 on Modal, with held-out accuracy on 200 fresh problems per run. An earlier version of this table labelled these runs Qwen3-8B / Tinker / GSM8K. The raw per-run records contradict that label, and the held-out figures below match the raw records exactly. Held-out accuracy is the mean $\pm$ SE over seeds; mean ZVF is the mean zero-variance fraction.

Table A.4: Semester-3 group-size sweep runs.

Run identifier	Stack / platform	Model	Algo	G	Dataset	Seed	Steps	Outcome / status
groupsize_g2	Modal A10 (shared harness)	Qwen/Qwen2.5-0.5B	GRPO	2	arithmetic, 200 held-out	3 seeds	40	held-out acc 0.9817 $\pm$ 0.0044; mean ZVF 0.8380
groupsize_g4	Modal A10 (shared harness)	Qwen/Qwen2.5-0.5B	GRPO	4	arithmetic, 200 held-out	3 seeds	40	held-out acc 0.9883 $\pm$ 0.0017; mean ZVF 0.7635
groupsize_g8	Modal A10 (shared harness)	Qwen/Qwen2.5-0.5B	GRPO	8	arithmetic, 200 held-out	3 seeds	40	held-out acc 0.9900 $\pm$ 0.0029; mean ZVF 0.6906
groupsize_g16	Modal A10 (shared harness)	Qwen/Qwen2.5-0.5B	GRPO	16	arithmetic, 200 held-out	3 seeds	40	held-out acc 0.9783 $\pm$ 0.0060; mean ZVF 0.6312

The remaining Semester-3 runs are the exploratory blocks of Table A.1: tinker-rl-lab-world-class (174 runs over fourteen base models from Llama-3.2-1B to Qwen3-235B-A22B), tinker-rl-scaling (88, the material behind the cross-scale study), tinker-structural-ceiling (72), tinker-rl-webarena (40) plus the separate webarena-eval scoring pass (26), the agentic smoke block (10), three SkyRL-on-Tinker runs, and three Hugging Face runs. These are indexed by identifier in the ledger but given here in aggregate, because for most of them fewer than half the required columns have values.

A.3 Semester-4 P1–P8 study runs

The P1–P8 series is the individual Semester-4 contribution, and its runs are indexed by *claim* rather than by project. The audit ledger at `platform_hybrid/experiments/results/claim_to_run/` maps each published claim to its supporting run identifiers,

the W&B URLs where they can be inspected, whether the model named in the paper matches the model actually trained, the seed and step counts, the held-out metric, and an evidence tier. It holds **18 claims** carrying **713 claim-to-run links** — a count of links, not of distinct runs, and the 368 links on P7-C1 are the `zvf-audit` project count rather than 368 separable experiments.

P1–P8 claim-to-run ledger. Tier vocabulary: A = ≥ 5 seeds and ≥ 100 steps; B = 3–4 seeds and 50–99 steps; C = completed/descriptive; R = deterministic resource or schema claim; X = unlinked, contradicted, or provenance-conflicted.

Table A.5: Semester-4 P1-P8 study runs: each claim mapped to its linked runs, stack, model and algorithm.

Claim	Linked runs (representative)	Stack	Model	Algo	G	Dataset	Seeds / steps	Outcome	Tier
P1-C1	20 — <code>tinker:51a8ef9e...</code>	Tinker (managed) + W&B	5 GSM8K anchors: Qwen3.5-4B, Qwen3-8B, Llama-3.1-8B-Instruct, Nemotron-120B, DeepSeek-V3.1	—	—	GSM8K	1 per anchor (seed 42) / 20–30	No held-out metric on the exact-linked runs; the claim rests on training-reward traces	C
P1-C2	1 — <code>tinker:657a920a...</code> (HF-arbitrated)	Tinker (managed) + HF adapter	nvidia/NVIDIA-Nemotron-3-Super-120B-A12B-BF16 (MoE, A12B active)	—	—	<code>frontier_gsm8k</code>	1 (seed 42) / 20	Peak 0.875, last-10 0.1625, zero-reward step fraction 0.55 (11/20); no held-out metric	C
P2-C1	80 local rows pooled to 23 cells	Local analysis pool (no W&B)	Heterogeneous; Qwen3-8B at G = 8 among the listed rows	—	—	GSM8K	up to 3 per source row / 30–600	Mean ZVF versus collapse: Spearman 0.56, point-biserial 0.62 (n = 23)	C

Claim	Linked runs (representative)	Stack	Model	Algo	G	Dataset	Seeds / steps	Outcome	Tier
P2-C2	80 (as P2-C1)	as P2-C1	as P2-C1	—	—	GSM8K	as P2-C1	Spearman $\rho = 0.27$, bootstrap 95% CI $[-0.37, 0.88]$ (n = 23)	C
P3-C1	12 — wandb:rlvr-openings/122x3tca	W&B-managed open trainer; no Tinker identity	Qwen/Qwen3.5-4B	—	2, 4, 8, 16	— (saturated arithmetic regime; dataset not recorded)	3 per G (seeds 0, 1, 2) / 8	Mean held-out Δ : G=2 +0.0000, G=4 +0.0000, G=8 -0.0625, G=16 +0.0000; evaluation size not logged	C
P3-C2	5 — wandb:zvfaudit-v2/6snvz1qd	W&B, sampling only	— (G = 32 cells)	—	32	—	5 cells / 0 optimizer steps	No matched G = 4/G = 32 training replication; the comparison cannot be executed	X
P4-C1	6 — local:drgrpo_gsm8k_cot:grpo:seed42	Local	Qwen/Qwen3.5-7.2B-Instruct as mapped (abstract says Qwen3-8B) — conflict	GRPO / Dr.GRPO	—	GSM8K CoT	3 per arm (42, 123, 456) / 30	GRPO 0.2017→0.2633 (Δ +0.0617); Dr.GRPO 0.2050→0.2550 (Δ +0.0500); n = 200/seed	X
P5-C1	2 — wandb:tinker-rl-lab-world-class/xmot42ot,.../w83mv3ok	Tinker-managed versus TRL backend	Qwen/Qwen3-8B-Base versus Qwen/Qwen3-8B — con-founded	GRPO / TRL-GRPO	8	GSM8K	1 per backend (seed 42) / 30	Final/last-10 training reward 0.050→0.856; no held-out metric	X

Claim	Linked runs (representative)	Stack	Model	Algo	G	Dataset	Seeds / steps	Outcome	Tier
P5-C2	2 — zvf-colab-experiment s/6c7p198f, .../15m91qij	Open trainer versus managed stack	Qwen2.5-0.5B-Instruct versus Qwen3.5-4B (declared non-consistent exhibit)	DAPO label versus asymmetric-clip surrogate	—	—	2 open versus 3 managed / not logged (open), 15 (managed)	Mean ZVF 0.00 versus 0.58; no comparable held-out metric	C
P5-C3	1 — zvf-colab-experiment s/15m91qij	Managed summary	Qwen/Qwen3.5-4B	Four algorithm labels	—	—	3 per arm (42, 123, 456) / 15	Last-10 reward band 0.710–0.744 (width 0.034); no held-out metric	C
P6-C1	48 registry entries	Registry (schema validation)	N/A	N/A	N/A	N/A	N/A	44/48 entries pass schema validation	R
P6-C2 (new audit)	82 — tinker:0dfe749...	Tinker ↔ W&B ↔ HF, three-way	N/A (identity audit)	N/A	N/A	N/A	as logged per underlying run	33/41 exact IDs model-consistent; 5 HF-arbitrated mislinks; 3 unresolved	R
P7-C1	368 — wandb:zvf-audit/fzforbvg	W&B only (0 exact IDs)	Qwen3-4B-Instruct-2507 (113), Llama-3.2-3B (112), Llama-3.2-1B (107), 3 further models (36)	—	4, 8, 16 (also 6)	— (prompt/token families llama3, qwen3_instruct recorded, not datasets)	10 logged (0–9) / 2–12	ZVF U-shape reproduced; no held-out metric in the W&B summaries	C
P7-C2	1 — zvf-colab-experiment s/ds83rymc	W&B (Colab)	Qwen/Qwen2.5-0.5B-Instruct	—	—	—	3 (0, 1, 2) in one aggregate / 30	Gradient magnitude versus p(1–p): Pearson r = +0.71	C

Claim	Linked runs (representative)	Stack	Model	Algo	G	Dataset	Seeds / steps	Outcome	Tier
P7-C3	1 — zvf-colab-experiment s/6c7p198f	W&B (Colab)	Qwen/Qwen2.5-72B-Instruct	Adaptive-G versus fixed recipes	—	—	1 per arm / not logged	Adaptive-G held-out $\Delta +0.575$; mean ZVF 0.23; 186 rollouts	C
P8-C1	1 — loca1:qp8_fr aud: xgboost: seed 42	Local (XGBoost)	Gradient-boosted trees (200 estimators, depth 6)	—	N/A	Synthetic card-fraud: 50,000 rows; 10,000-row held-out split	random_state 42 / 200 estimators	AUC 0.7955 on the 10,000-row held-out split	C
P8-C2	1 — wandb:tinker-new-research/ek1b2cxn	Tinker (managed SFT)	Qwen/Qwen3-72B	32 row serialisation (no KL, no reference policy)	N/A	Row-to-text serialisations; 500-row positive-enriched held-out subset	1 (seed 0) / 63 mini-batches	AUC 0.482675, accuracy 0.792 (n = 500) — near chance	C
P8-C3	0 — UNLINKED	—	Proposed sensor / scribe / triage architecture	—	—	—	0 / 0	No execution; operational benefit over XGBoost alone not measured	X

Three facts here are load-bearing. First, **no empirical headline row in the P1–P8 ledger reaches Tier A or B** under the statistical-rigor thresholds; the rectified tally is C = 12, R = 2, X = 4, and the registry’s only Tier A block belongs to P11, an extension described below. Second, the four Tier X rows are X for specific, checkable reasons: **P3-C2** rests on G = 32 *sampling* cells with zero optimizer steps, so no matched G = 4/G = 32 training replication exists; **P4-C1**’s abstract names Qwen3-8B while every mapped run is Qwen/Qwen2.5-72B-Instruct; **P5-C1** compares Qwen/Qwen3-8B-Base against Qwen/Qwen3-8B, so the base checkpoint changes together with the backend and the pair must not be cited as a backend-only causal effect; and **P8-C3** is a proposed architecture with no execution, hence zero linked runs. Third, of 41 exact Tinker identifiers in the identity audit, 33 are model-consistent with their W&B counterpart and 8 conflict; five conflicts were resolved as W&B mislinks by three-way arbitration against the Hugging Face adapter listing, and three remain unresolved (0ef59237, Qwen3.5-4B recorded against a Nemotron run; 78e2d35b, Qwen3.5-27B; 0dfee749, Llama-3.1-8B-Instruct

recorded against a DeepSeek V3.1 run). Those three model fields are contested.

Two column caveats travel with the table. The dataset cell for P7-C1 is – because the ledger’s only dataset-adjacent field for those 368 runs is a prompt/tokenizer family (llama3, qwen3_instruct, gpt_oss_low_reasoning, nemotron3_disable_tinking), not a dataset name. And the P3 sweep here is **not** the Semester-3 sweep of Table A.4: it is a different project (rlvr-openings), a different model (Qwen3.5-4B), and it reports deltas of 0.0000 rather than held-out accuracies near 0.99. The two share a group-size grid and nothing else. Chapters 4 and 6 cite the Table A.4 sweep (Qwen2.5-0.5B) whenever they give the 0.838 $\rightarrow$ 0.631 ZVF decline or the 0.98–0.99 held-out figures. They do not cite P3-C1.

The extension ledger added in August 2026 carries five rows besides P11: P10-C1 is Tier R (an algebraic identity, not an empirical result), P10-C2 and P12-C2 are Tier X (placeholder figures and unlinked prospective claims, zero runs), and P12-C1 is Tier R as a lemma and Tier X as an empirical headline.

P11 is the only block in the registry meeting the report’s strictest evidence bar: 40 arm–seed units at `local:zvf-program/audit/results/full/{grpo,dapo,gsdp,drgrpo,aero}-seed-{11,23,37,53,71,89,107,131}.json`, five arms $\times$ eight seeds on one stack (Qwen3-8B, GSM8K, 30 steps, Colab A100, held-out $n = 500$). Its headline contrast, DAPO versus GRPO, is $\Delta = +0.001$, percentile-bootstrap CI $[-0.0045, +0.00675]$ (paired-t 95% CI $[-0.0063, +0.0083]$, $n = 8$ seeds; the bootstrap undercovers at this n), $MDE_{80} = 0.01012$, all raw $p > 0.21$, with Benjamini–Hochberg rejecting nothing. The mechanism is visible in the diagnostic: DAPO’s mean ZVF is 0.000 against GRPO’s 0.693, bought with $3.61\times$ the rollouts (1,734 versus 480) and $1.44\times$ the wall clock. It is the only place in the work where a claim rests on five or more seeds over a hundred or more steps, and it supports a null.

A.3.1 Opening and null-result runs

Chapter 6 reports four null families — a mixed-variance curriculum, a matched token budget, hard-prompt filtering, and step-1 layer selection (§6.4.3) — and each rests on runs that are not in the ledger. They are absent for a checkable reason: the string `rlvr-openings` occurs nowhere in the ledger, and no file in the fourteen-project `wandb_inventory/` covers it, although every Tinker-managed block below is logged to that project under the groups named in the table. The rows are counted from the per-run name (or mode) fields of each artefact’s JSON — `results.json` for the curriculum, token-budget, hard-prompt and loss-formulation blocks, `result.json` and its siblings for P1 — not from an inventory API, which is why this cohort is reported as its own total. Three neighbouring blocks add nothing to it: the P3 group-size artefacts are al-

ready indexed — `groupsize_zvf/results.json` reproduces the per-group deltas of Table A.5’s P3-C1 row exactly, and the second sweep file is discussed below — and the P2, P7 and P8 “openings” (`p2_openings/collapse_baseline_analysis.json`, `p7_controller/sim.json`, `p8_openings/*.json`) are zero-compute re-analyses that launch no runs.

Semester-4 opening and null-result runs (46 runs). Model is Qwen/Qwen3.5-4B and dataset GSM8K unless a row says otherwise. All rows are Tinker-managed except the P1 layer-freeze block, which ran on Colab for per-layer gradient access. “Curriculum” means training only on non-collapsed (mixed-variance) groups. Group sizes marked *default* are the launch defaults of the scripts in `platform_hybrid/experiments/openings/`; only the campaign artefact records a group size per run. Δ is the change in held-out accuracy.

Table A.6: Semester-4 opening and null-result runs: the curriculum, token-budget, hard-prompt, loss-formulation and layer-freeze blocks.

Run identifier	Stack / platform	Model	Algo	G	Dataset	Seed	Steps	Outcome / status
campaign/ baseline- G4-s0...s5	Tinker → W&B rlvr -openings (campaign)	Qwen/Qwen3.5-4B	GRPO	4	GSM8K	0–5	8	6 runs; mean Δ +0.0111 (0.0000–0.0333)
campaign/ curriculum- G4-s0...s5	Tinker → W&B rlvr -openings (campaign)	Qwen/Qwen3.5-4B	GRPO + curriculum filter	4	GSM8K	0–5	8	6 runs; mean Δ +0.0167 (–0.0667 to +0.0667); oversample 3.3–7.2×; zero-loss fraction 0.0
campaign/ baseline- G2-s0	Tinker → W&B rlvr -openings (campaign)	Qwen/Qwen3.5-4B	GRPO	2	GSM8K	0	8	1 run; Δ 0.0000; zero-loss fraction 0.7
campaign/ baseline- G8-s0	Tinker → W&B rlvr -openings (campaign)	Qwen/Qwen3.5-4B	GRPO	8	GSM8K	0	8	1 run; Δ –0.1333
campaign/ baseline- G16-s0	Tinker → W&B rlvr -openings (campaign)	Qwen/Qwen3.5-4B	GRPO	16	GSM8K	0	8	1 run; Δ +0.0333
curriculum_ m_opening/ baseline	Tinker → W&B rlvr -openings (ungrouped)	Qwen/Qwen3.5-4B	GRPO	4 <i>default</i>	GSM8K	0	8	1 run; Δ +0.0500; held-out 20; zero-loss fraction 0.50

Run identifier	Stack / platform	Model	Algo	G	Dataset	Seed	Steps	Outcome / status
curriculum_opening/ curriculum	Tinker → W&B rlvr -openings (ungrouped)	Qwen/Qwen3.5-72B	GRPO + curriculum filter	4 default	GSM8K	0	8	1 run; Δ +0.0500; held-out 20; oversample $4.81\times$; zero-loss fraction 0.00; superseded, see below
token_budget/base line-b3000-s0...s2	Tinker → W&B rlvr -openings (token-budget)	Qwen/Qwen3.5-72B	GRPO	4 default	GSM8K	0–2	30k-token budget	3 runs; mean Δ +0.0278; no groups skipped
token_budget/curri culum-b3000-s0...s2	Tinker → W&B rlvr -openings (token-budget)	Qwen/Qwen3.5-72B	GRPO + curriculum filter	4 default	GSM8K	0–2	30k-token budget	3 runs; mean Δ +0.0278; 11–20 groups skipped per seed
hard_curriculum/ba seline-hard-s0...s2	Tinker → W&B rlvr -openings (hard-curriculum)	Qwen/Qwen3.5-72B	GRPO	6 default	GSM8K hard-learnable pool (40 of 300 probed)	0–2	10	3 runs; mean Δ +0.0278; zero-loss fraction 0.1–0.2
hard_curriculum/cu rriculum-hard-s0...s2	Tinker → W&B rlvr -openings (hard-curriculum)	Qwen/Qwen3.5-72B	GRPO + curriculum filter	6 default	GSM8K hard-learnable pool (40 of 300 probed)	0–2	10	3 runs; mean Δ 0.0000; zero-loss fraction 0.0
p4_surprise/{sum, mean,surprise}-s0...s1	Tinker → W&B rlvr -openings (p4-surprise)	Qwen/Qwen3.5-72B	GRPO, loss variant	4 default	GSM8K	0–1	8	6 runs; mean Δ +0.1250 (sum) / 0.0000 (mean) / +0.0417 (surprise)
p1_layerfreeze/res ult.json	Colab L4	Qwen/Qwen2.5-7.2B-Instruct	(gradient probe, no optimizer steps)	—	—	—	5	1 run; step-1 predicts final top-layer set at overlap 1.0; top-25% concentration 0.476

Run identifier	Stack / platform	Model	Algo	G	Dataset	Seed	Steps	Outcome / status
p1_layerfreeze/scaled_4seed_result.json	Colab L4	Qwen/Qwen2.5-72B-Instruct	GRPO	4	GSM8K (24 problems)	0-3	10	4 runs; overlap 0.0833 ± 0.0481 — approximately chance; concentration 0.3908
p1_layerfreeze/freeze_flop_result.json	Colab L4	Qwen/Qwen2.5-72B-Instruct	GRPO (step-1-frozen arm)	—	—	0-1	—	4 runs, full and frozen arms; frozen arm gains 0.0000 against 0.0625 at 60.7% of the parameter count
p1_layerfreeze/merge_ergence_result.json	Colab L4	Qwen/Qwen2.5-72B-Instruct	GRPO (step-1 layer selection)	—	—	0-1	—	2 runs; mean step-1 overlap 0.643

Three provenance notes belong with this table, because two of its blocks carry superseded siblings and one carries a refuted claim. First, `campaign/FINDINGS.md` describes a nine-run, three-seed version of the campaign on a twelve-prompt held-out set with arm means of $+0.028$ and -0.028 ; `campaign/results.json` holds fifteen runs over six seeds on a thirty-prompt held-out set ($0.8667 = 26/30$), and the table follows the per-run file. The FINDINGS warning that `zero_loss_frac` was invalid (an empty-metrics-dict bug, “since fixed”) belongs to the nine-run batch, not to that file. Second, the P1 layer-freeze block has the same shape: the two-seed 3B pass (overlap 0.1111 ± 0.0000) is superseded by the four-seed pass (0.0833 ± 0.0481), and both sit near chance, so the toy-scale overlap of 1.0 must not be read as a transferable layer-selection signal. Third, the two single-seed training blocks flagged SUSPECT by independent verification (kimi, 2026-07-06) are superseded by the multi-seed blocks above: the curriculum opening’s $+0.05$ (campaign curriculum and baseline arms separate by 0.006 over six seeds, at $3.3\text{--}7.2\times$ oversample cost), and a single-seed $G = 4$ gain of $+0.125$ in `p3_groupsize/sweep_results.json` — the campaign’s G4 seed-0 baseline gains 0.0000, not 0.125, and that sweep file retains only two of its twelve entries, so the W&B group is its record.

A.4 E1–E14 evaluation campaign lanes

The fourteen evaluation lanes are not training runs; they are evaluations of a single frozen actor, and they carry the registry’s strictest receipt discipline. The actor is base Qwen/Qwen3.6-35B-A3B (commit 995ad96e...) with adapter arvindcr4/pavlov-portfolio-qwen36-seed809-stepfinal-tinker-cf0ad8c1... (commit 64444133...), bfloat16, trained on Tinker, run seed 809 taken from the adapter name. The adapter was trained for 40 steps at $G = 4$ and learning rate 1×10^{-5} by REINFORCE with a group-mean baseline (no KL, no clipping), as recorded in the local W&B run directory for run bsv8vx04 and the run receipt checkpoints/grpo/pavlov_portfolio_api_swegym_qwen36_20260809_seed809.json (Chapter 8 §8.1.1). Those are properties of the single actor rather than of any lane, so they are shown as “—” in the table rather than repeated. Terminal states are from the ledger of record, outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md; all scores are from outputs/E1_E14_FINAL_RESULTS_2026-09-19.md, except the six rows marked “finish 2026-09-27”, which are from outputs/finish_pending_2026-09-27/<lane>/result.json.

E1–E14 campaign lanes.

Table A.7: E1-E14 evaluation campaign lanes with their run identifiers, stacks, platforms, models, algorithms and group sizes.

Lane / run identifier	Stack / platform	Model / actor	Algo	G	Dataset / suite	Seed	Steps	Outcome / terminal state
E1 SWE-bench Pro	native harness, Tinker actor	Qwen3.6- 35B-A3B + adapter	—	—	SWE-bench Pro	809	—	2/731 = 0.274% pass@1 (97.54% coverage, 713/731 native evals); originals retained
E1 replacement (finish 2026-09-27)	native swbench 5.0.2, Modal sandboxes	as above	—	—	SWE-bench Multilingual	809	—	1/190 resolved (57 graded); earlier waves 4/110 kept separate — COMPLETE
E2 Frontier- SWE	native harness	as above	—	—	FrontierSWE	809	—	1/17 tasks; replay normalized 0.8628

Lane / run identifier	Stack / platform	Model / actor	Algo	G	Dataset / suite	Seed	Steps	Outcome / terminal state
E2 replacement (finish 2026-09-27)	native grader, GCP VM per capsule	as above	—	—	CORE-Bench hard	809	—	27/45 = 0.600 task accuracy — COMPLETE
E3 SDAB	—	as above	—	—	SDAB (private bundle)	809	—	no local or paid path — CLOSED_EXTERNAL
E4 Banker-ToolBench	Tinker→Modals bridge	as above	—	—	BankerToolBench	809	—	1/100 tasks, recovery 0.3115 — CLOSED_PARTIAL
E4 base-model rerun	Modal /v1/ responses	base model, zero-init LoRA	—	—	BankerToolBench	809	0	100/100 trials, mean reward 0.0, \$15.07 of \$55.91 cap; zero traced to bridge passing no stop sequences (\$8.5)
E5 APEX-Agents	native harness	as above	—	—	APEX-Agents	809	—	7/480 native-scored; prefix mean 0.050505
E5 replacement (finish 2026-09-27)	native tau2, Modal 65k actor	as above	—	—	Tau3 banking_knowledge	809	—	8/97 = 0.082 pass^1 (25 infra errors counted as failures) — COMPLETE
E6 WebArena (finish 2026-09-27)	native evaluator, GCP VM	as above	—	—	WebArena	809	—	90/812 = 0.111, lower bound; 108 ungraded (judge unavailable) — COMPLETE
E7 BinaryAudit	native harness	as above	—	—	BinaryAudit (private payload)	809	—	1/46 attempted, verifier reward 0.0, no grade — CLOSED_EXTERNAL
E8 LAB-Bench (public)	native harness	as above	—	—	LAB-Bench public, 8 categories	809	—	1967/1967 COMPLETE
E8 LifeSciBench (original)	—	as above	—	—	LifeSciBench private	809	—	CLOSED_EXTERNAL

Lane / run identifier	Stack / platform	Model / actor	Algo	G	Dataset / suite	Seed	Steps	Outcome / terminal state
E9 MLE-bench	Modal streaming arm	as above	—	—	MLE-bench	809	—	40/75 natively graded (53.33%); suite score null
E9 replacement (finish 2026-09-27)	OpenHands CodeActAgent, GCP VM	as above	—	—	ML-Dev-Bench	809	—	10/34 = 0.294 task success — COMPLETE
E10 AgentDojo (benign)	native harness	as above	—	—	AgentDojo benign utility	809	—	97/97 COM- PLETE
E10 AgentHarm (original)	—	as above	—	—	AgentHarm private	809	—	CLOSED_EXTERNAL
E11 VerilogEval	native harness	as above	—	—	VerilogEval (both framings)	809	—	312/312 = 129/312 pass@1 (41.35%); 67/156 completion + 62/156 spec-to-RTL CLOSED_EXTERNAL
E12 AppBench	—	as above	—	—	AppBench deployment	809	—	255/255 episodes; 26.1% progression — COMPLETE
E13 BALROG (finish 2026-09-27)	native BALROG evaluator, local	as above	—	—	BALROG	809	—	externally blocked, no closure record
E13 Open-Reward (original)	—	as above	—	—	OpenReward held-out games	809	—	4426/4428 accepted (99.95%); accuracy 2271/4428 = 51.29% (official scorer 51.31% on 4426 judged rows) — COMPLETE_TERMINAL
E14 Omni-MATH	native scorer	as above	—	—	Omni-MATH	809	—	CLOSED_EXTERNAL
E14 FrontierMath (original)	—	as above	—	—	FrontierMath hosted	809	—	CLOSED_EXTERNAL

Two source documents disagree on two lanes, and both observations are retained rather than resolved into one, because the disagreement is itself evidence: the historical snapshot records E7 as “1/28 errored, no grade” against the terminal-state table’s “1/46 attempted” — both real observations at different points in the lane’s life. Likewise E11’s canonical figure is $129/312 = 41.35\%$; a 129/311 sensitivity figure survives in non-canonical receipts and is not used as the headline.

A.5 Completeness limitations

The registry is complete as an *index* and incomplete as a *dataset*, and the distinction governs how it may be used. Coverage is measured over the 1,662 ledger rows only — the 46 opening runs of §A.3.1 are logged per run in their own artefacts and are not part of this table:

Field coverage across the 1,662 ledgered runs.

Table A.8: Field coverage across the 1,662-run ledger: rows carrying a value against rows blank or UNKNOWN.

Field	Rows with a value	Rows blank or UNKNOWN	Notes
model	1,632	30	all 30 gaps are W&B rows
task (dataset)	544	1,118	only 188 are real dataset names; 356 are tokenizer-family labels (llama3 225, qwen3_instruct 131)
algo	139	1,523	only 139 rows name an algorithm at all
G	626	1,036	distribution where known: G=8 → 270, G=4 → 181, G=16 → 151, G=6 → 10, G=32 → 6, G=64 → 5, G=2 → 3
steps	628	1,034	bimodal: 5 steps → 318 rows, 30 steps → 207 rows, then a long tail
headline_metric	28	1,634	98.3% of runs carry no recorded outcome value

The largest gap is structural: **all 844 Tinker-source rows record task, algo, G and steps as UNKNOWN**. The Tinker pagination API returns training-run identity, base model, LoRA rank and checkpoint metadata but not the training configuration, so those runs’ configurations cannot be recovered from the surviving inventory, and every algorithm label in the registry comes from the W&B half of the ledger. The raw snapshot `tinker_runs.txt` shows the same boundary differently: run ID, base model truncated to “Qwen...”, LoRA rank 4, timestamp and status “OK” — no configuration at all.

Four further limits apply.

1. **Aggregate presentation.** This appendix names 105 runs individually: the 4 cross-framework runs of Table A.2, the 20 Modal campaign runs of A.3, the 12 sweep runs (in 4 rows) of A.4, the 46 opening runs (in 16 rows) of A.6, and the 23 E-lane rows of A.7 — plus the 18 claim rows of Table A.5, which index claims rather than runs. The remaining ~1,600 runs are indexed by identifier and project in `experiment_ledger.tsv`, presented here in aggregate because for most of them fewer than half the required columns have values.
2. **Non-runs inside the count.** Three entries are not training results: `ver1-dryrun` and `openr1hf-dryrun` are seeded deterministic fallbacks, and the E4 base-model rerun performs zero training steps; to these add §A.3.1’s P1 gradient probe, which takes no optimizer steps at all. They are counted because they are runs in the registry’s sense, and flagged so no reader treats them as measurements.
3. **Identity conflicts.** Eight Tinker identifiers were linked to W&B runs whose recorded model did not match; five were resolved against the Hugging Face adapter listing, three remain unresolved, and those `model` fields are contested.
4. **Snapshot disagreement.** The 844/818 split used here, the 949/940 split in the audit manifest, and the “983 runs” implied by `tinker_runs.txt` are three observations of the same platforms at three times. No reconciliation was attempted and none should be inferred.

None of this changes a reported result, because every result in Chapters 6–8 is bound to named artefacts rather than to the registry as a whole. It does bound what the registry can be used for: it establishes provenance and identity, shows that the run population is far larger than the reported claims, and shows that those claims rest on a small, explicitly tiered subset — one Tier A block, a majority Tier C, and four Tier X rows that must not be cited as evidence.

The run totals that appear in different chapters count different corpora. Table A.9 reconciles them.

Run totals quoted in this report, reconciled. Each total is a different corpus, not a different count of the same one.

Table A.9: Run totals quoted in this report and the corpus each one counts.

Corpus	Count	What it is	Used in
Registry total	1,708	1,662 ledger rows plus 46 opening runs recorded only in their own artefacts (§A.3.1)	Appendix A

Corpus	Count	What it is	Used in
Experiment ledger	1,662	844 Tinker-service runs and 818 W&B runs, snapshot through 4 July 2026	Appendix A
P1 cross-scale roster	70+	Runs across seven RL libraries and five model families; 12 of them are the fitted anchors	Chapter 6 §6.2
Instrumented roster (P8 workshop source)	79	The instrumented roster of <code>pa_per_P8_workshop.tex</code> , 0.6B to ~671B	Chapter 4 §4.1
<code>zvf-audit</code> W&B project	368	Logged runs in one W&B project, a subset of the ledger's W&B half (Table A.1)	Chapter 4 §4.3; Chapter 7
P5 audit corpus	790	The corpus from which the 368-run audit is drawn (<code>p5_st_ack.tex</code>); its relation to the ledger is not recorded	Chapter 7 §7.1
Mega-campaign cells	98	Sampling-only cells of <code>mega_20260704</code>	Chapter 7 §7.1
E1–E14 actor training	1	The 40-step <code>pavlov_portfo</code> lio run of August 2026, after the ledger cut-off	Chapter 8 §8.1.1

Appendix B

Reproducibility and Provenance

B.1 What this appendix establishes

This appendix records three things that should not be confused with one another. The first is what a reviewer can independently re-run: a pinned container, a pinned dependency set, a headline training command, and the tolerance within which a rerun is expected to land. The second is what the repository verifies about itself without re-running any model: an audit suite driven by a single entry point, and a deterministic ledger check that recomputes the reported arithmetic directly from surviving evidence files. The third is provenance: which parts of this work were produced by the Semester 3 group, which by the individual contribution reported here, and what was lost along the way.

B.2 The pinned runtime

The container definition is `Dockerfile`, built from `nvidia/cuda:12.4.1-cudnn-devel-ubuntu22.04`. It installs Python 3.12 through the `deadsnakes` archive rather than the distribution default, with the in-file comment that Python 3.12 “matches the frozen runtime validated 106/106”. It sets `PYTHONHASHSEED=42` and a fixed `LANG/LC_ALL` of `C.UTF-8`; stamps the build commit into an OCI `org.opencontainers.image.revision` label and a `.build_info` file; fixes `SEED=42`, `WANDB_MODE=offline`, `HF_HUB_DISABLE_TELEMETRY=1` and `OMP_NUM_THREADS=8`; self-checks by importing the core stack at build time; and runs under `tini`.

The authoritative pin set is `requirements.txt`, headed as “pinned to the frozen E1 runtime lock ... validated 106/106 on this stack”. Its exact pins include `torch==2.7.1`, `numpy==2.2.6`, `transformers==5.5.4`, `trl==1.2.0`, `datasets==4.8.4`, `accelerate==1.13.0`, `peft==0.19.1`, `huggingface-hub==1.11.0` and

wandb==0.21.0, with bounded ranges for scipy, safetensors, tokenizers, rliable, math-verify, latex2sympy2-extended and pytest, alongside the framework clients tinker, tinker-cookbook and atroposlib.

The 106/106 figure is the recorded outcome of the frozen-runtime validation, which `execution-notes.md` states as passing the complete pinned pilot/preregistration suite on Python 3.12.13 with the pinned versions listed above, and the focused amendment, protocol, launcher and verifier suite passing 55/55. The same record binds five artefacts by SHA-256 — protocol document, screening manifest, launch manifest, launch fingerprint and source binding — and states that a mismatch in package, Python, A100, FLOP, token or gradient conditions fails closed rather than proceeding.

B.3 The reviewer’s entry point

REPRODUCE.md is the reviewer-facing document. Its fastest path is the structural smoke test at `platform_modal/scripts/smoke_test.sh`, which runs in about two minutes offline and about eight when a live Tinker key is supplied so that the wire protocol is exercised; the offline path needs no GPU and no account.

The headline result is GRPO on Qwen3-8B over GSM8K for 30 steps at LoRA rank 32, group size 8 and seed 42, recorded as a peak per-step training accuracy (the binary reward) of 62.5 % with a last-10 average of 34.4 %; these summarise the training curve of the seed-42 run, not a held-out evaluation, and Table 1.1 lists them beside the other Qwen3-8B GSM8K figures they are sometimes confused with. Reviewer-side reruns are accepted within ± 5 percentage points on the last-10 figure and ± 10 points on peak, justified in the same document by the residual non-determinism discussed in §B.10. Per-seed reference values and W&B run identifiers are tabulated for seeds 42, 123, 456 and 789, with seed 1024 marked as a reviewer-only rerun; the canonical multi-seed set is {42, 123, 456, 789, 1024, 2048, 4096, 8192, 16384, 32768}. The group-size ablation is expected to reproduce a peak of 50.0 % at G=2, 75.0 % at G=4, 100 % at G=8 and 71.9 % at G=16, with last-10 values of 37.5 %, 52.1 %, 84.4 % and 38.0 %. Five-seed aggregation runs through `utils/stats.py` with rliable intervals and 10,000 bootstrap samples; `utils/verify_results.py` checks a rerun against the recorded expectations at those tolerances.

The cost is stated plainly: about five GPU-hours on a single A100-80GB for the headline run, against roughly 446 GPU-hours for a full paper reproduction. Dataset provenance is tabulated with pinned revisions — GSM8K at 740312a with 7,473 training and 1,319 test rows, plus MATH-500, HumanEval, ArenaHard Auto, NoRobots and OpenThoughts3 each at a named snapshot — with exact shas in ARTIFACT.md §3.4.

B.4 Where the documentation has drifted from the code

Four discrepancies exist between the repository’s prose and its live tree, and a reviewer following the prose literally would hit them. The training driver has moved: REPRODUCE.md and ARTIFACT.md both invoke `grpo_gsm8k_base.py` at the repository root, and the file now lives at `platform_tinker/grpo_gsm8k_base.py`, though the documented command still works because the CLI in `platform_tinker/tinkerrl/grpo_cli.py` carries a compatibility entry point, `legacy_main`, that translates the documented flag spellings onto the current ones — what changed is the path a reviewer must type, not the flags the documents specify. ARTIFACT.md names the smoke test at `scripts/smoke_test.sh` rather than at `platform_modal/scripts/smoke_test.sh`, though REPRODUCE.md gives the correct path. The pins quoted in prose are stale: REPRODUCE.md §9 states that `requirements.txt` pins `transformers` `>=4.56.2, <4.60.0` and a `datasets` `<3.2` bound, whereas the live file pins `transformers` `==5.5.4` and `datasets` `==4.8.4`, and ARTIFACT.md §3.3 quotes a third, older set; similarly, ARTIFACT.md describes the environment as Python 3.10 from the Ubuntu 22.04 archive, whereas the Dockerfile installs Python 3.12 via `deadsnakes` and the frozen-runtime record validates on Python 3.12.13. The ARTIFACT.md items are explained by that document’s frozen status — it records a Semester 3 artefact at commit `9019b5c946f17ffe4761b5f4eac13b86d1e51cc0`, dated 19 April 2026 — but REPRODUCE.md §9 is live guidance, not a frozen record, and carries the same defect.

B.5 The audit apparatus

The integrity suite is driven by a single entry point, `platform_local/run_all_audits.py`, which imports a shared result interface from `utils/audit_utils.py` and registers its audits as a tuple of name-and-function pairs. As it stands in this tree, that tuple contains **nine** audits: `claim_issues`, `sync_issues`, `anon_issues`, `strength_issues`, `package_issues`, `workflow_issues`, `export_guard_issues`, `caveat_issues` and `scientific_issues`. Each is implemented by a corresponding `platform_local/` module carrying a `__main__` block, so each can also run on its own; `tests/test_audit_runner.py` asserts that the registered modules and the `platform_local/*_audit.py` files are the same set.

Some repository documents give different counts, and the difference is historical. At the blind-review sweep the suite was thirteen audits, registered as a list of script filenames at the repository root and executed as subprocesses; the committed log of that run records METRIC `audits_total=13` with ten passing and three failing, and the same sweep states that all seventeen `*_audit.py` scripts then at the root were executed, with the

suite covering thirteen of them. The registry now holds nine audits, all passing: a pruning commit removed obsolete audits, and a later restructuring replaced the subprocess runner with the in-process interface, so the seventeen-script description has narrowed by the same route. No script now prints a metric line directly; the line is constructed in one place, `AuditResult.metric_line()` in `utils/audit_utils.py`, and emitted by the nine audit modules plus the runner. A repository-wide scan finds several dozen files matching `*_audit.py`, but that population is not a suite: it includes unrelated analyses, test modules and a third-party file in the virtual environment.

The output contract is uniform. `AuditResult.metric_line()` emits `METRIC <name>=<issue_count>`, so every audit reports one machine-readable line whose right-hand side is the number of issues it found, and an audit passes exactly when that count is zero. `render_suite` prints each audit's metric line and issue codes, then three suite-level lines — `METRIC suite_issues`, `METRIC audits_total` and `METRIC audits_passing` — before either listing the failing audits by name or stating that all audits are passing. The entry point exits 0 when the suite passes and 1 otherwise, which makes it usable as a gate; the packaging step runs the suite before exporting and refuses to package if it fails. The runner's own test asserts this rendering against a synthetic two-audit suite, checking for `METRIC audits_total=2` and `METRIC suite_issues=1`.

B.6 The deterministic ledger check

Where the suite above audits the submission package's internal consistency, a second instrument audits the campaign's reported figures. `outputs/verification/LEDGER_CODE_CHECK_2026-09-19.json`, under schema `ledger-code-check-v1`, records eleven checks of which eleven passed and none failed. Each recomputes a claim from the underlying evidence rather than trusting a summary of it, and the four classes of check prove four different things.

Table B.1: The deterministic ledger check by class, listing each specific check and what it establishes.

Class of check	Specific checks	What it proves
Exact divisions	E1 2/731; E11 129/312; E11 67+62=129; E14 442 6/4428 and accuracy 0.5 131043831902395 (which is 2271/4426, the scorer’s judged-row denominator; the reported headline counts the 2 unjudged rows as failures, 2271/4428 = 0.5129); E9 40 of 75	That each reported ratio is the quotient of the two integers the report claims, and that a partition sums to its whole
Receipt presence	E8 eight categories summing to 1967 with every evaluated == expected_total; E9 graded-row counts; the strict-completion set	That every claimed unit of coverage has a surviving graded row behind it, and that the set of strictly complete suites is exactly [E8, E10, E11]
Sealed-artefact hash equality	E1 v6 sealed request and fixture receipt	That the sealed request and the validation receipt are byte-identical to the artefacts recorded at sealing time
Lost-source absence probes	codex_run_execution_sources_absent	That four .codex-run paths are genuinely absent, so a later re-creation cannot be mistaken for the lost original

The distinction matters under questioning. An exact division proves the arithmetic is not a rounding convenience or a transcription error; it cannot prove the underlying evidence is sound. A receipt-presence check proves coverage claims are backed by artefacts; it cannot prove those artefacts were produced correctly. A hash-equality check proves identity between two artefacts — here, between what was sealed and what was

later validated — and is the strongest of the four because it does not depend on re-running anything. An absence probe proves a negative: it prevents a later reconstruction from being passed off as the original. Two instances show the classes in use. The E1 records show the sealed request’s SHA-256 beginning 7adc6e2616f61bc7 matching `validation_v6.json`, with the fixture receipt matching on the same basis. The E9 records show 41 graded rows resolving to 40 unique competitions against a denominator of 75 in the streaming arm and exactly one valid grade in the merged-VLLM arm, which is the check that confirms the two serving arms were never pooled.

B.7 The receipt model

A receipt in this campaign is a durable JSON artefact binding an outcome to the inputs that produced it, and the binding carries the evidential weight rather than the file’s existence. The clearest statement of the principle is the receipt-bundle auditor’s docstring, which records that the campaign and the receipt builders are kept deliberately separate from the auditor, and that the auditor “never imports a provider SDK, contacts a service, launches a job, or treats `status/online/a URL/ID` alone as provenance”. The auditor reconstructs contract coverage, receipt predicates, canonical hash rules, budget caps and cross-artefact bindings, carrying its expectations as constants rather than reading them from what it checks — suite counts of 12 training, 14 primary-evaluation and 26 total, and budget constants of 18.00 USD maximum, 16.50 USD operational cap and a 1.50 USD safety reserve — so a receipt set that quietly loses a suite fails the audit rather than redefining it.

Receipts bind four dimensions of a run: protocol, runtime, source and event order. The frozen-runtime record applies that discipline to the runtime dimension alone, binding five artefacts — protocol document, screening manifest, launch manifest, launch fingerprint and source binding — by SHA-256 and failing closed on any mismatch. The judgment layer for subjectively scored questions follows the same pattern: one durable receipt per judgment binds the state bytes, the question text and the raw answer, and battery execution is gated on a health check rather than the service’s availability, so a degraded answer cannot be laundered into a valid one. Where a lane could not be scored, its terminal state is recorded in the ledger of record with a named reopen condition, so the absence of a result is itself receipted.

B.8 Evidence loss in the finish era

One loss of evidence occurred and is recorded rather than concealed. During post-review disk cleanup the untracked `.codex-run/` working tree was deleted. Four paths were confirmed absent by the absence probes described above: `finish_20260912/e1_recover_v6/bounded_sdk.py`, holding the E1 wave10 repair code; `finish_20260912/e9_completion/restored/`, holding 1,613 hash-verified restored dependency sources from the OpenHands and Composio forks used to build the MLDevBench runtime; `finish_20260912/e6_continuation/checks_v1.py`, an offline receipt validator the reconciliation record describes as structure-only and of low impact; and `public_native_improve_v2/e1/metadata_wave08.py`, the driver for the E1 multilingual run that graded 35 of 300 tasks.

The loss does not invalidate any figure reported in this thesis, for three reasons that hold independently. First, none of the four paths is the source of any reported score. The E1 and E9 replacement lanes were eventually run on 27 September with new thin runners rather than the lost code (§8.4). The loss still bears on E1: the source-context collector had to be re-implemented for that run and was checked byte-for-byte against the original on only three recorded tasks, which is one reason the 1/190 and 4/110 figures are not pooled. Second, the sealed requests, validation receipts, regression fixtures and the 10-of-10 targeted-test log from the lost E1 code all survive, so a faithful re-implementation is constrained by surviving fixtures rather than free to invent behaviour; that re-implementation has since landed under a tracked path and passes the lost suite’s recorded regressions, four process-group and six cleanup. Third, the loss is confined to finish-era iteration code: the replacement-lane native runners survive in tracked locations, including a 515-line multilingual runner carrying no reference to the deleted tree. The policy adopted in response — that execution sources live in tracked repository paths and never in untracked scratch trees — is the durable outcome of the incident.

B.9 The Semester 3 and Semester 4 provenance boundary

The boundary between inherited and individual work is a git tag. `capstone-final-2026-04-25` resolves to commit `21a99ef766543a1ca86bfbad188445d3596ea73c`, dated 23 April 2026, whose subject line records the final metric-separation rewrite that segregated training rewards, held-out capability and proxy diagnostics. Two hundred and fifty commits precede that tag. A reviewer can reproduce the state at the boundary by adding it as a worktree, `git worktree add`

../tinker-rl-lab-sem3 capstone-final-2026-04-25, which materialises the Semester 3 tree without disturbing the current checkout.

The Semester 3 group comprised six students — Arvind C R, Sandhya Jeyaraj, Madhu Kumara L, Mohammad Rafi, Dhruva N Murthy and Arumugam Chetty K — under guides Anwesh Reddy Paduri and Narayana Darapaneni. Their work is the multi-framework benchmark infrastructure and the initial GRPO baseline experiments. Semester 4 was carried out individually under Ramesh Prakash Guledgudd, and the declaration facing this report assigns to that contribution the research direction, the experimental expansion, the statistical analyses, the P1–P8 paper series and the E1–E14 campaign. The solo-authorship commit 282e6b4f0f158de11b3f5bb4656e2d871f68f5c0, dated 7 July 2026, records that division at paper level. The provenance record states that 584 commits and 5,050 changed files separated the boundary from the tree as it then stood, and that the eight paper PDFs were compiled at commit 006e90a38e4dabe2d0b4778be01dc7b27f2d3462 on 9 July 2026, one recompiled the following day after a findings-ledger rename. Those counts carry an as-of date rather than describing the current tree, so a reviewer recomputing them will obtain a larger number.

One consequence of this history shapes the repository’s structure: the implementation remains shared, with experiment drivers, result tables, figures, LaTeX sources and audit scripts referencing one another across the boundary, and the provenance record is explicit that this is why the code is not split into separate trees. What the boundary separates is authorship of the research contribution, not access to the machinery. Two reporting rules follow and are enforced throughout this report: original-contract and replacement-scope results are never pooled, with no cross-suite aggregate, which the ledger check independently verifies through the arm-separation and component-split checks; and no replacement benchmark’s score is presented as the score of the benchmark it replaces, with every substitution named as such at its point of use.

B.10 Residual non-determinism and the limits of reproduction

Not everything here is reproducible in the strict sense, and the tolerances exist because of it. Three sources of residual non-determinism survive even with fixed seeds and deterministic cuDNN settings: remote LoRA training is not bit-deterministic across scheduler decisions, with runs replayed months apart drifting by two to four percentage points on GSM8K last-10, documented through three reruns of the same seed-123 configuration; some matmul operations on H100 and A100 hardware introduce non-determinism through CUDA and cuBLAS; and sampling at temperature 0.8 with top-p 0.95 executes

in a remote virtual machine, so a fixed seed does not fix the sampled trajectory. The record states the consequence itself: run-to-run variance on this setup is of the same order as the effect reported in the paper’s PPO-versus-GRPO comparison, which is why that comparison is reported as statistically indistinguishable rather than as a difference.

Three limits remain. Lanes blocked on external access cannot be reproduced from this repository, because the blocking resource is not in it. The six lanes completed on 27 September can be rerun from the code in their lane directories under `outputs/finish_pending_2026-09-27/`, but only with cloud accounts, a served copy of the actor and, for E6, a judge account with credit; CORE-Bench’s answer keys are deliberately kept out of the repository. The evidence-loss paths cannot be reproduced in their original form, only re-implemented against surviving fixtures, and that distinction should be preserved in any future citation. And figures originating at the Semester 3 boundary describe the tree at that tag, so reproducing them requires the worktree checkout rather than the current HEAD.

Appendix C

Notation, Symbols and Abbreviations

C.1 Scope and convention

This appendix records the symbols and abbreviations used across this report and the papers behind it, as the sources actually use them rather than as a house style would prefer. Where a symbol carries more than one meaning across documents, both are listed and the collision flagged, because a reader carrying one definition into another chapter will misread a figure.

C.2 Symbols

Table C.1: Notation: symbols used in this report, their meaning, and any divergence in how the underlying sources use them.

Symbol	Meaning	As used in the sources
G	Group size: completions sampled per prompt	Headline runs $G = 8$; sweeps $G \in \{2, 4, 8, 16\}$
K	Rollouts per group in the ZVF formalisation; separately , successes within a group	Elsewhere $K = \sum_i R_i \sim \text{Binomial}(G, p)$
$B_t, \mathcal{P}_t$	Batch at optimizer step t ; prompts at step t	Competing denominators of ZVF

Symbol	Meaning	As used in the sources
$p, \hat{p}$	Latent success probability; sample rate K/G	Bernoulli(p) model; $\hat{p}$ centres the advantage
r_i, R_i	Scalar reward of completion i ; binary reward	Verifiable reward; derivations appendix
$h_G(p)$ ZVF_t, GU_t	$p^G + (1 - p)^G$ Zero-Variance Fraction: share of identical-reward groups; Gradient Utilization $1 - ZVF_t$	Bernoulli null for ZVF Principal diagnostic and its complement
A_i	Advantage of completion i	Unnormalised $R_i - \hat{p}$; normalised $(r_i - \mu_g)/(\sigma_g + \varepsilon)$; clipped Dr.GRPO
μ_g, σ_g	Group mean and standard deviation of reward	$\mu_g = \frac{1}{G} \sum_j r_j$; σ_g in population form ($1/G$) throughout, as every GRPO-style runner computes it (§4.2)
$\pi_\theta, \pi_{\theta_{\text{old}}}, \pi_{\text{ref}}$	Trained, sampling and frozen reference policy; per-token importance ratio $\rho_{i,t}$	Ratio against the sampling policy; KL penalty against the reference
$\mathcal{L}(\theta)$	Policy loss	Original GRPO: clipped surrogate plus KL penalty. In this project no runner has a KL term; the Tinker runners are REINFORCE with a group-mean baseline (Table 4.2)
β	KL coefficient; separately , a regression slope	TRL and veRL 0.04; OpenRLHF 0.02; Tinker managed; also a slope in M1/M2

Symbol	Meaning	As used in the sources
ε	Three distinct meanings — see §C.3	Clip range 0.2; ZVF tolerance 10^{-6} ; Adam term 10^{-8}
γ, λ, T	Discount factor; GAE parameter; sampling temperature	0.99; 0.95; canonical 0.8
SI, PTD	Stability index; peak-to-tail drift	Reported in place of direct KL
PCD	Pairwise-contrast density	$\mathbb{E}[\hat{p}(1 - \hat{p})] = \frac{G-1}{G} p(1 - p)$
pass@1	Pass rate under the suite’s native scorer	E11 129/312 = 41.35 %; E1 2/731 = 0.274 %
r, ρ	Correlation coefficients, Pearson and rank	$r = -0.769$, $p = 0.0008$; $\rho \approx 0.65$

C.3 Four collisions worth flagging

The symbol ε carries three unrelated meanings. In the hyperparameter mapping it is the PPO/GRPO clip range, default 0.2, swept over $\{0.05, 0.1, 0.2, 0.3, 0.5\}$. In the ZVF definition it is a numerical tolerance, 10^{-6} , deciding whether a group’s sample variance counts as degenerate. In the optimiser configuration it is the Adam epsilon, 10^{-8} . It also appears additively in the advantage denominator, $\sigma_g + \varepsilon$.

K is used two ways: in the formal treatment of ZVF it denotes the rollouts in a group, a synonym for the group size elsewhere called G , whereas in the derivations appendix G is the group size and K is the number of successes — a design parameter in the one case, an outcome in the other. The letter r is similarly the reward, the LoRA rank (swept over $\{4, 8, 16, 32, 64\}$) and the Pearson correlation.

Finally, ZVF is expanded two ways in the sources. The formal definition and the abstract both read “Zero-Variance Fraction”, the expansion used here; one figure caption reads “Zero-Value-Filtered”. The caption variant is noted only so that a reader meeting it is not misled.

C.4 Abbreviations

Table C.2: Abbreviations and their expansions, with names that no source expands recorded as identities rather than glossed.

Abbreviation	Expansion or identity
GRPO	Group Relative Policy Optimization — critic-free, group-relative
PPO	Proximal Policy Optimization
DPO	Direct Preference Optimization
RLHF	Reinforcement learning from human feedback
IS Loss	Importance-sampling loss, the distillation objective
SFT	Supervised fine-tuning
LoRA	Low-rank adaptation of large language models
KL	Kullback–Leibler divergence; never spelled out in the sources
CoT	Chain-of-thought; spelled out in prose, written “GSM8K CoT” for the variant
ZVF, GU	Zero-Variance Fraction; Gradient Utilization
PCD, SI, PTD	Pairwise-contrast density; stability index; peak-to-tail drift
Dr.GRPO	GRPO with the per-response length normalisation and the division by the group standard deviation removed
GAE	Generalised advantage estimation; $\lambda = 0.95$
MoE	Mixture-of-experts
BF16	bfloat16, the serving precision for the LAB-Bench, AgentDojo and Omni-MATH lanes
IQM	Interquartile mean, reported via <code>rliable</code>
CI	Confidence interval; 95 %, 10,000 bootstrap resamples

Abbreviation	Expansion or identity
W&B	Weights & Biases, the experiment-tracking service
MLE-bench	The suite forming lane E9’s original contract; MLE never appears standalone
MATH-500	The mathematics evaluation suite
GSM8K	The grade-school arithmetic suite; 7,473 training and 1,319 test rows
HumanEval	The code-generation suite used for the code arm
ArenaHard, NoRobots, OpenThoughts3	Dataset names used as-is, each at a pinned revision
MiniWoB	Browser micro-task suite used for the proxy smoke test
SWE-bench (Pro, Multilingual)	The repository-repair suite forming lane E1
LAB-Bench, AgentDojo, VerilogEval, Omni-MATH, BALROG, WebArena, APEX-Agents, CORE-Bench, Tau3, MLDevBench, SDAB, BankerToolBench, BinaryAudit, LifeSciBench, AgentHarm, WebBench, AppBench, OpenReward Games, FrontierSWE, FrontierMath	Suite names used as names, with no expansion in any source; see §C.5
Tinker	The remote training and sampling API used to train the adapter
TRL	HuggingFace’s LLM post-training library, in both framework rosters
SkyRL, verl (veRL), OpenRLHF, Atropos	Launchers in the cross-launcher roster; only Tinker and TRL produced completed runs, the veRL and OpenRLHF entries being dry-run placeholders

C.5 The E1–E14 lane identifiers, scopes and statuses

Each lane’s short code names one original benchmark contract and is not a rank or a score. Several lanes hold two states at once — an original contract and a replacement scope — so the code alone does not say which scope a figure comes from. Table C.3 is the single record of lane scope, headline figure, terminal state and receipt; the chapters point here rather than restating it. “orig” is the original contract and “repl” a declared replacement scope. Terminal states are from `outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md`; the E1, E2, E5, E6, E9 and E13 replacement scopes were completed on 2026-09-27, after that ledger, so their states are assigned in this report from each lane’s `result.json` (§8.4); the result files themselves carry no state field. Receipt paths are relative to `outputs/`.

Table C.3: The E1–E14 lanes: original contract, replacement scope, headline figure, terminal state and receipt.

Lane	Original contract	Replacement scope	Headline figure (scope)	Terminal state	Receipt
E1	SWE-bench Pro	SWE-bench Multilingual	2/731 = 0.274% pass@1 (orig, full suite); 1/190 resolved on the tasks not attempted before, 4/110 earlier, not pooled (repl)	orig: complete, no ledger label; repl: COMPLETE on coverage (all 300 attempted, two contracts)	<code>modal_e1_e14/2026-08-16/e1_swe_bench_pro_full/seed1818/receipt.json</code> ; <code>finish_pending_2026-09-27/E1/result.json</code>
E2	FrontierSWE	CORE-Bench	1/17 tasks, replay 0.8628 (orig, partial); 27/45 = 0.600 task accuracy (repl, complete)	orig: partial; repl: COMPLETE	<code>e2_frontier_swe/e2_terminal_attempt_receipt_2026-08-22.json</code> ; <code>finish_pending_2026-09-27/E2/result.json</code>
E3	SDAB (private bundle)	—	none	CLOSED_EXTERNAL	<code>PES_Phase2_Review_2026-09-12/finish/external_closures_2026-09-19/E3_sdab.json</code>
E4	BankerToolBench	—	1/100 tasks, recovery metric 0.3115 (orig, partial)	CLOSED_PARTIAL	<code>modal_e1_e14/2026-08-16/e4_recovery_pass16_receipt.json</code>
E5	APEX-Agents	Tau3	7/480 natively scored (orig, partial); 8/97 = 0.082 pass [^] 1 (repl, complete)	orig: partial; repl: COMPLETE	<code>e5_apex_agents/e5_exact_sequential_prefix_aggregate_2026-08-22.json</code> ; <code>finish_pending_2026-09-27/E5/result.json</code>

Lane	Original contract	Replacement scope	Headline figure (scope)	Terminal state	Receipt
E6	WebBench	WebArena	90/812 = 0.111 task success, lower bound; 108 judge-dependent tasks ungraded (repl, complete in coverage)	orig: no original-contract result (ledger N/A); repl: COMPLETE on coverage; 108 ungraded	finish_pending_2026-09-27/E6/result.json
E7	BinaryAudit (private payload)	—	1/46 attempted, verifier reward 0.0, no grade (orig)	CLOSED_EXTERNAL	e7_binaryaudit/2026-08-22_e7_paid_attempt_receipt.json
E8	LifeSciBench (private package)	LAB-Bench public split	450/1,967 = 0.2288 (repl, complete; end-to-end harness score)	repl: COMPLETE; orig: CLOSED_EXTERNAL	public_portfolio_2026-09-05/labbench_native_receipt.json
E9	MLE-bench	MLDevBench	40/75 competitions graded, suite score null (orig, partial); 10/34 = 0.294 task success (repl, complete)	orig: partial; repl: COMPLETE	e9_mle_bench/modal_streaming/; finish_pending_2026-09-27/E9/result.json
E10	AgentHarm (private tasks)	AgentDojo benign utility	88/97 = 0.9072 utility, 97/97 completed (repl, complete)	repl: COMPLETE; orig: CLOSED_EXTERNAL	public_portfolio_2026-09-05/agentdojo_native_receipt.json
E11	VerilogEval	none (two native framings)	129/312 = 41.35% pass@1 (orig, complete; end-to-end harness score)	COMPLETE	modal_e1_e14/2026-08-16/e11_full_receipt.json
E12	AppBench (deployment)	—	none	CLOSED_EXTERNAL	PES_Phase2_Review_2026-09-12/finish/external_closures_2026-09-19/E12_appbench.json
E13	OpenReward Games (held-out games)	BALROG	26.1% BALROG progression, 255/255 episodes (repl, complete)	repl: COMPLETE; orig: externally blocked, no closure record	finish_pending_2026-09-27/E13/result.json
E14	FrontierMath (hosted)	Omni-MATH	2,271 correct of 4,428 dispositions (repl, terminal-complete; §8.2)	repl: COMPLETE_TERMINAL_NOTE; orig: CLOSED_EXTERNAL	public_portfolio_2026-09-05/native_finish_v5/e14_official_collector/official_result01/native_score.json

Counted from this table: three scopes are strictly complete (E8 and E10 replacement scopes, E11 original contract), E14 is terminal-complete, and six original contracts are CLOSED_EXTERNAL with written closure records (E3, E7, E8-original, E10-original, E12, E14-original). E13’s original contract is also blocked on external access but carries

no closure record, because the lane continues through its replacement scope. Original-contract full-coverage scores exist for E1 and E11.

Appendix D

Campaign Lane Detail

This appendix holds the lane-by-lane record behind Chapter 8: provenance of the training run, per-lane serving and scoping notes, the engineering history behind individual figures, the per-lane narratives of the six replacement scopes completed on 27 September 2026, the full E4 rerun audit, and the per-lane caveats of the small-scale reruns. None of it changes a figure reported in Chapter 8; every number here is quoted exactly as its source receipt records it, and the two reporting rules of Chapter 8 (no pooling of original-contract and replacement-scope results, no precision beyond the receipt) apply throughout.

D.1 Actor identity, inference conditions and scoping notes

The design consequence of evaluating one actor for seed variance is specific: every suite number reported in Chapter 8 comes from one actor, and where a lane sampled more than one completion per task it did so to satisfy the suite’s own protocol, not to estimate variance.

Inference conditions were frozen per lane and recorded in that lane’s own receipt rather than imposed globally, and the lanes differ widely. The E10 native run declared a temperature of 0, a 32,768-token context, 4,096 output tokens per request, 3,600 seconds of wall time and an \$8 envelope. Its receipt also declares `max_generation_requests = 1,967`, which is a ceiling on generation requests, not a task count: the lane has 97 episodes, and the run recorded 414 native requests (405 in the main run and 9 in the continuation). The collector is written explicitly not to admit shorter native conditions. The E11 run was sampled at temperature 0.2, top-p 0.95, 1,024 maximum output tokens and seed 1816, one sample per problem across all 312 prompts. The point of recording

them per lane is that the sampling regime is part of the result, not a detail beneath it.

Two scoping notes complete the picture of what the actor has actually been measured on. The E1 original-contract evaluation was a one-checkpoint evaluation whose 731 generations were produced by a mixed backend within a single run — 476 Tinker generations and 255 Modal vLLM generations — and that backend mix is recorded in the original receipt. Separately, two small probes exist at other parameter settings but are not suite results and are not reported as such: a base-model smoke run of the E11 harness measured zero scored tasks, and a four-task pilot on the step-40 checkpoint (the last training step, saved separately from `stepfinal`) returned four passes out of four with zero extraction failures while carrying an unresolved outstanding blocker — the decontamination receipt identifier for that pilot does not exist, and the split-manifest validator still fails on it.

D.2 Training-run provenance

The adapter’s configuration survives in three places: the local Weights & Biases run directory for run `bsv8vx04` (`wandb/run-20260809_140744-bsv8vx04/files/`), whose `config.yaml`, `wandb-metadata.json` and `output.log` hashes match those recorded in `outputs/public_portfolio_2026-09-05/training_identity_verified.json`; the local run receipt checkpoints/`grpo/pavlov_portfolio_api_swe_gym_qwen36_20260809_seed809.json`; and the Hugging Face checkpoint branches. The two local files are excluded from version control by `.gitignore`, so they exist on the project workstation but not in the public repository.

Four things cannot be recovered, and they are listed so that no reader fills them in:

- **The exact training code.** The run records git commit `8fdade2f` with uncommitted changes that were never saved. The `pavlov_portfolio` preset and its dataset builder do not exist at that commit; they first appear in commit `72b9336d3` on 2026-08-14. The reward and data-pool code in Table 8.1 is therefore taken from the current tree, whose dataset hash does not match the recorded `bd1f0db6...`. It is the best reconstruction, not proof. The loss function and training loop are identical at `8fdade2f`.
- **Which 80 training prompts were drawn.** The draw used `random.Random(809)` over the 512-row pool, but the pool construction that ran is the unrecoverable code above.
- **The Tinker sampler weights.** They were purged with the run; the four checkpoints survive only on Hugging Face.

- **Any held-out training-time evaluation.** None was run (`evaluate_heldout` false).

The run also departs from `zvf-program/flagship/PAVLOV_EXPERIMENT_PROTOCOL_L_2026-08-09.md`. The protocol plans a learning-rate search and a twelve-suite curriculum with 1,200/128 prompt and response tokens. This run used two suites, 40 steps, 1,536/384 tokens and a learning rate of 1×10^{-5} , and the protocol does not mention it; only the budget matches.

D.3 Serving conditions: format limits and the scope of the stop-sequence mechanism

In Table 8.2, most E8 responses and nearly half the E11 responses yield no gradable answer, and two thirds of E14 responses hit the length cap. The single-turn Tinker generations of E1 show no role-token run-on, which suggests that the degeneration needs multi-turn agent context or the `/v1/responses` route. The E4 finding document states the mechanism more generally than this evidence supports.

D.4 Strictly complete scopes: lane detail

The ledger lists E11 both among its “replacement scopes” and among its original-contract results; Table 8.A lists it as original-public-subset. It is one run and one result, not two.

E8 — LAB-Bench, public split. Per-category accuracy, correct counts and coverage are: TableQA 0.7869 (192/244); FigQA 0.5138 (93/181); ProtocolQA 0.2500 (27/108); SuppQA 0.1829 (15/82); LitQA2 0.1759 (35/199); DbQA 0.1308 (68/520); SeqQA 0.0333 (20/600); and CloningScenarios 0 (0/33). No category is incomplete. Coverage per category is at or above 0.9878 everywhere, and the CloningScenarios coverage is exactly 1.0 with zero correct. The receipt also records zero parser errors, 1,960 responses marked “sure” against 7 marked “unsure”, and a completion profile of 1,086 length-truncated finishes against 881 stop finishes. Because the receipt’s correct and incorrect fields partition all 1,967 rows, the 1,259 unparsed responses are counted as incorrect in the published accuracy, and the figure is an end-to-end harness score rather than an accuracy over the subset of responses that yielded a parseable answer.

E10 — AgentDojo, benign utility scope. The receipt records `missing_evaluation_ids` empty and `native_handled_error_count` zero. The 97 episodes decompose as

banking 16, slack 21, travel 20 and workspace 40. The receipt carries the run manifest hash 00e4228f4b757dc32b69b74f05a88348151f13185b470c5f00e31fdaa830ef30 and fixes the protocol identity as `benign_task_utility v1.2.2` with attack, defence and security-evaluation fields all null and `heldout_claim` false.

An earlier attempt terminated with 94 of 97 episodes and score null, missing workspace tasks 37 through 39. The cause was a confirmed collector fault rather than a model failure: verified task receipts recorded absolute `/cache/. . .` trace paths, and the frozen summary recomputed those paths on the local host, so every completed receipt failed comparison after extraction. The repair was deliberately minimal — the original archive and every extracted receipt and trace byte are unchanged, and a narrowly scoped adapter rewrites only the freshly computed `task_result.native_trace_path` to the verified canonical remote path before the frozen summary compares it, leaving native task code, environment, prompt, result parsing, utility values and handled-error policy untouched. The revised collector passed 18 offline tests, independently re-run in 0.507 seconds, and the continuation run that closed episodes 37 through 39 was executed under reservation `public-20260905-agentdojo-continuation01` with the native wrapper hash 11a8b61631390d9b20decd4263008e5a4c6f648a033d314384bac9e5d2456d77.

E11 — VerilogEval. The upstream NVlabs/verilog-eval harness ran at revision c498220d0a52248f8e3fdffe279075215bde2da6, using Icarus Verilog 12.0 at revision 4fd5291632232f8e1ba49b2c26bb6b2bf1c6c9cf and Verilator 5.050. Pass verdicts were determined deterministically: for each pinned problem the single `sample01 sv-iv-test` log was read and marked PASS only when it contained a full-line “Mismatches: 0 in samples” verdict, with missing or non-matching logs marked FAIL. The upstream `sv-iv-analyze` summariser could not be used because it fails on a missing optional `langchain` dependency; that failure is recorded in the receipt and does not affect the aggregation, which reads the simulator logs directly. An extraction failure writes an empty candidate file that always fails.

The E11 lane also carries the campaign’s clearest example of a scoring decision taken for evidential rather than numerical reasons. The all-problems reference sweep — running the shipped reference implementation of every problem through both simulators — returned 311 of 312. One problem, `verilog_eval/spec-to-rtl/Prob099_m2014_q6c`, fails to elaborate against its own test bench under both simulators, with Icarus reporting that ports Y2 and Y4 are not ports of `good1`; no candidate implementation can pass that task, and the upstream defect biases `pass@k` downwards by at most 0.64 percentage points on the specification-to-RTL split (1/156) and by 0.13 percentage points overall (129/311 – 129/312). Excluding it yields $129/311 = 0.4148$. That corrected figure is *not* canonical. The recorded amendment of 2026-08-22 states the reason plainly: no immutable benchmark-level justification was captured for the exclusion, so the raw

129/312 result is canonical and the 129/311 value is retained only as a noncanonical sensitivity; no samples and no verifier artifacts were changed by the amendment. The ledger’s integrity check confirms that the component split sums correctly to the canonical total.

E14 — Omni-MATH replacement scope. The 4,426 accepted rows are 99.95 per cent of 4,428. The two rows not accepted are `omni-01195-a6cbdc4ee3c8694b` at row index 1,195 and `omni-02044-612ddacaa39d91fe` at row index 2,044. The terminal note of 2026-09-19 resolves them in a direction that does not flatter the lane: the Omni-Judge outputs for both rows are truncated before any “## Equivalence Judgement” section — row 1,195 ends mid numeric answer and row 2,044 ends mid sum expression — so the judge emitted no verdict at all, and the native scorer’s omission of rows lacking an equivalence judgement is correct official behaviour rather than a parser defect. The terminal note recorded the score impact as nil on the grounds that both rows already count in the denominator as not correct; that is not so for the official scorer, whose 0.5131 is computed over the 4,426 judged rows, so Chapter 8 counts the two rows as failures and reports $2,271/4,428 = 51.29$ per cent. The lane closes as terminal-complete under the native protocol with 4,428 of 4,428 dispositions recorded (4,426 accepted plus 2 recorded skip reasons), and no re-judge was performed because re-judging would deviate from the official scorer and constitute new evidence generation with no score effect.

D.5 Original-contract results: lane detail

E1 — SWE-bench Pro. The denominator is the complete 731-row test split of ScaleAI/SWE-bench_Pro at revision `7ab5114912baf22bb098818e604c02fe7ad2c11f`. Within that denominator the run produced 713 native evaluations, fourteen generation failures and four lost generation artifacts; those eighteen non-native outcomes are retained in the denominator rather than trimmed, so the figure is a rate over all 731 tasks and not over the subset that graded cleanly. The declared licence for the dataset is null in the lane’s own receipt. The evaluation harness is `scaleapi/SWE-bench_Pro-os` at commit `ca10a60a5fcae51e6948ffe1485d4153d421e6c5`, and the 731-task scoring run used the 731 official run scripts unmodified on native `linux/amd64` Modal workers. A 1,200-second timeout, against the official 120, was used only in an earlier single-task harness check that ran the gold patch under `arm64 QEMU` emulation. That check is recorded as not a model score, so the timeout deviation does not apply to the 2/731 figure.

E2 — FrontierSWE. Beyond the missing revision-bound authorisation, the original

checkpoint was unavailable on Tinker at the time; its Tinker sampler route was later confirmed purged, while the adapter weights are preserved on the Hugging Face branch.

E4 — BankerToolBench. The lane’s public artifacts were restored and 1,175 manifest checks pass, but the pass3/pass16 archive is genuinely missing — zero of six artifacts verify, and only `TINKER_BRIDGE.json` does.

E7 — BinaryAudit. The terminal ledger of record gives the denominator as 46 attempted; an earlier working snapshot, since superseded, recorded the same lane as “1/28 errored”. The two snapshots differ in the scope of what they counted as an attempt, and Chapter 8 reports the terminal figure while flagging that the superseded snapshot’s narrower count exists in the record.

E9 — MLE-bench. Coverage without complete grading is not a score, and the lane records it as such. Behind the coverage number sit 97 legacy receipts across 50 competition identifiers, of which 41 graded receipts cover 40 unique competitions; the ledger’s integrity check verifies precisely that decomposition. The separate merged-vLLM arm holds seven receipts, of which exactly one is a valid grade (H&M, 0.02132) and six are invalid submissions, and the integrity check confirms that arm is recorded separately with one graded row. One old Spooky Author submission is absent locally although the later run’s complete submission is retained.

D.6 Replacement-scope lane narratives, 27 September 2026

Three things changed in how the lanes were executed, and each is recorded as a deviation in the lane’s own result file. First, where a fail-closed controller blocked only on paperwork that the directive had superseded, the lane used a thin runner that calls the benchmark’s native loop and native grader directly. Second, compute moved off AWS: E2, E6 and E9 ran on Google Cloud virtual machines, E1’s grading ran in Modal sandboxes, and E13 ran locally. Third, every lane was served the same actor, `pavlov-public-portfolio-bf16` (the base model with the seed809 adapter merged, §8.1), through vLLM with the non-thinking chat template. No lane went through the Tinker bridge whose missing stop sequences produced the E4 zero (§8.1.2, §8.5), so that defect is not expected here, although the lanes’ result files do not record their stop settings. The shared endpoint served at a 32,768-token context. E5 used a dedicated 65,536-token copy of the same weights because its banking prompts exceed 32k.

E1 — SWE-bench Multilingual. This run covers the 16 wave-10 recovery tasks and the 174 tasks that the 12 September waves never reached, 190 tasks in all. The actor

resolved one (phpoffice__phpspreadsheet-3463), which is $1/190 = 0.53$ per cent. Only 57 of the 190 produced a patch that applied and could be graded: 120 failed at patch application or evaluation and 13 produced an empty patch. Over graded tasks alone the rate is $1/57 = 1.75$ per cent ($[0.3, 9.3]$). Thirteen tasks produced no generation and were scored as failures: eight exceeded the 32,768-token serving context and five hit provider errors. The earlier waves covered the other 110 tasks, with 4 resolved (3.64 per cent, $[1.42, 8.98]$). They used the same task universe, adapter and grader family, but served at a 65,536-token context, and their source-context collector was lost and had to be re-implemented for this run. Because the contract differs, the two runs are not combined, and the lane does not report a 300-task figure.

E2 — CORE-Bench, hard difficulty. The actor answered every question correctly on 27 of the 45 codeocean_hard capsules, a task accuracy of 0.600. At the question level it answered 53 of 79 correctly: 28 of 33 written questions and 25 of 46 vision questions. Thirty-two tasks ended with the agent calling finish, ten at the 150-turn cap and three at the 8,100-second time limit. The agent was a thin tool loop (bash, query_image, finish) rather than the benchmark’s reference CORE-Agent scaffold, and the same actor answered the vision queries. Thirty tasks never reached a machine because of a local DNS outage, and a driver crash then interrupted ten of their reruns. All thirty were rerun from scratch, and every aborted attempt is archived. Because a leak of the answers would inflate this figure, the lane checked for one. No agent command referenced any host that serves the answers. Each capsule’s results/ directory is stripped before the agent starts, and the answer keys are held off the agent’s machine. Two qualifications remain. Twelve of the 27 correct tasks left no files in the capsule’s results directory, which the native grader does not require. Some vision questions are categorical and could be guessed. With $n = 45$ the interval spans 27 percentage points.

E5 — Tau3. Eight of 97 tasks succeeded, a pass¹ of 0.082. Twenty-five tasks ended in infrastructure errors: fourteen exceeded the 65,536-token context and eleven returned an empty assistant message. Their cause is not established, and they count as failures. An accidental resume of the run overwrote 24 of those error records in the results file; their identifiers and causes were rebuilt from the run log. Over the 72 tasks without such an error the native pass¹ is $8/72 = 0.111$, reported as a sensitivity figure only. The 20 tasks cleaned before 19 September were rerun under the same protocol rather than carried over, and scored 1/20. The user simulator was gpt-4.1-2025-04-14 at temperature 0.

E6 — WebArena. All 812 tasks were run, and 90 succeeded, giving 0.111. The figure is a lower bound. Of the 114 tasks whose native evaluator calls an LLM judge, 108 could not be graded, because the judge account had no credit for the whole run; the other six were resolved to zero by a non-judge check. Forty-one tasks failed with a

harness-level browser timeout. Both groups count as failures. The graded-only rate is $90/704 = 0.128$ ([0.105, 0.155]). If every ungraded task had passed, the rate would be $198/812 = 0.244$. With the 41 harness errors counted as failures, the true figure lies between 0.111 and 0.244, and nearer the lower end if the ungraded tasks succeed at the graded rate. The native judge model (gpt-4-1106-preview) is retired, so a later regrade would substitute a different judge. Most failures are early stops by the native loop: the same action three times (237), the same typing three times (103), the step limit (101), and three consecutive parse failures (98). The agent was limited to 384 output tokens per step, the few-shot examples were passed as user and assistant turns because the chat template rejects mid-conversation system messages, and sites were not reset between tasks, as in the native runner. The last 103 shopping-admin tasks were split across three containers of the same official site image to finish in time.

E9 — ML-Dev-Bench. Ten of the 34 released tasks passed their native validators, 0.294. The agent was the benchmark’s OpenHands CodeActAgent, limited to 50 iterations. No task errored or timed out, but only after one harness repair: OpenHands 0.21.0 hangs when it truncates its context window, and a one-line upstream fix was backported without touching prompts, validators or tasks. Thirty-nine of 1,045 model calls exceeded the 32,768-token context and were absorbed by that truncation path. A relay between the agent and the actor forced the non-thinking template and restored tool-call text that the server’s parser had dropped. An earlier four-task batch that hung before the repair is excluded.

E13 — BALROG. All 255 native episodes were played and graded, and the BALROG score is 26.1 per cent progression (standard error 1.77; a stratified bootstrap gives [22.7, 29.5]). BALROG’s score is the mean of six per-environment means, not a mean over episodes. By environment the actor reached BabyAI 82.0 per cent ($n = 50$), Baba Is AI 25.0 ($n = 120$), Crafter 22.7 ($n = 10$), TextWorld 16.9 ($n = 30$), MiniHack 10.0 ($n = 40$) and NetHack 0.0 ($n = 5$). Sampling followed the recorded protocol at temperature 1.0, but with thinking off where the original run had it on. A 2,000-step cap was placed on NetHack for cost; all five NetHack episodes ended on their own before it, so the cap never took effect. Episodes stopped by an operational stop or a hung worker were restarted from scratch with fresh seeds. The result file records 262 episode attempt records for the 255 scored episodes, of which 7 are marked killed for restart (`n_killed_for_restart_records`); its caveat text instead counts 11 restarts (one NetHack and five Baba Is AI episodes at the 01:40 UTC stop, plus five hung workers), and the file does not reconcile the two counts. If hangs correlate with episode difficulty, that is a mild selection risk. The run used macOS arm64 builds of the pinned sources rather than the Linux image. The 26.1 figure is on a different scope from the 26-episode small-scale E13 rows of Tables 8.A and 8.B and is not compared with them.

The launch-pending and quota labels of 19 September no longer describe any lane.

D.7 E4 rerun: closure and full audit

The E4 lane closed `CLOSED_PARTIAL` on 19 September 2026: the `pass3/pass16` archive is genuinely missing, only one of six artifacts verifies, and the 0.3115 recovery metric stands as receipt-only evidence because unlimited budget removes a spending cap but cannot reconstruct model outputs that no longer exist. The only funded path recorded at closure was a fresh full rerun whose grader-only lower bound was estimated at approximately \$59.45, which sits outside every envelope then authorised. That path was subsequently authorised separately and executed, and its result is reported as a diagnostic rather than as a score, because that is what its design makes it.

The rerun was configured on the base model with a zero-initialised LoRA snapshot — the lane’s record states this is equivalent to base sampling, with no training steps — executed through the Modal bridge `/v1/responses` path at `--n-concurrent 2` under a cap of \$55.91445263. It completed 100 of 100 trials with one `NonZeroAgentExitCodeError`, and returned a mean reward of exactly 0.0, with all one hundred trials in the 0.0 bucket, over a runtime of 3 hours 29 minutes.

The value of this run lies entirely in the failure mechanism, which was audited across all one hundred trajectories. Agents terminate after 2 to 14 steps, with a median of about 6, and produce no `deliverables/` directory; the verifier consequently short-circuits to 0.0 on the message “No deliverables found”. Twenty-seven or more of the trajectories end in role-token degeneration of the form `user user assistant ... repetition`. The record at the time read this as the base model being unable to sustain the `opcode` tool-use loop. A later check does not support that reading. The bridge’s `/v1/responses` path passed no stop sequences and decoded with special tokens removed, so generation ran past the end-of-turn token into invented `user`, `tool_response` and `assistant` turns, which is exactly the role-token degeneration observed. Two implications follow, and they cut in opposite directions. The positive implication is that the harness behaves correctly at the boundary — it fails closed, produces a legible mechanism for every zero, and does not manufacture partial credit. The negative implication is that no full-suite `BankerToolBench` score exists for this actor family; the only other measurement is the six-task subset of Tables 8.A and 8.B. The same bridge served E2, E5, E7 and the E9 streaming arm, so Table 8.2 marks those lanes as exposed to the same defect. In particular, the rerun’s 0.0 must not be set against the original single-task outcome as though the two were a controlled pairing: they are different configurations under different runtime conditions, and the campaign makes no comparison between them.

Spend on the rerun was \$15.07 charged against the \$55.91 cap, comprising 16.35 million prompt tokens and 2.18 million completion tokens.

D.8 Externally blocked lanes: access requests and verification

Access was requested through each provider’s public channel: a GitHub issue for E7 (QuesmaOrg/BinaryAudit#22), a Hugging Face dataset discussion for E10, and direct requests to the maintainers of E3, E12 and E14. No grant had been received at the time of writing, and delivery of the direct requests could not be confirmed from the sending side.

Two further items complete the terminal record. First, verification: the deterministic checks of 19 September passed 11 of 11 with zero failures, covering exact divisions, receipt presence, sealed-artifact hash equality and absence probes for the lost execution sources. Second, the judgment layer: a model-based triage harness was used only for classification and value ordering, never for arithmetic; it became unreliable during a service degradation on 19 September, so the per-lane blocker column in the terminal ledger rests on document evidence rather than on model judgment.

D.9 Small-scale reruns: detail and per-lane caveats

Each lane’s items, raw outputs and grader logs are kept so that every figure in Tables 8.A and 8.B can be recomputed.

For E12 the 151 rubric items come from only six generated applications, so they are not independent. The cluster-level test is an exact sign-flip test over the six per-app pass-rate differences (−0.042, +0.030, 0.000, −0.160, 0.000, −0.042), and it gives $p = 0.25$. The E12 grader is an LLM judge that is the same Qwen/Qwen3.6-35B-A3B base model as the base arm, so self-preference could favour the base arm. Two judge versions disagree widely (0.536 against 0.808 on the Tinker base run; E12/result.json). The E12 rates and difference are therefore judge-dependent, and no independent judge was run. For E4, all six tasks score 0 in both arms except btb-19b3361c; a Wilson interval is not valid for a mean of fractional [0, 1] rewards, and a bootstrap over one nonzero difference in six is degenerate. For E13, in 22 of the 26 episodes the progression is identical in the two arms. No lane-level difference is interpreted as evidence that the adapter improves or degrades performance unless its test excludes zero, and none is extrapolated to the full suite.

The principal caveat for each lane, as recorded in its result file, is listed below; the full list is in each lane's `result.json`.

- **E1.** New base-model arm; not comparable to or poolable with the original 2/731 (Tinker sampler route for the seed809 adapter purged, weights preserved on the Hugging Face branch; temperature 0.2).
- **E2.** Substitute benchmark; never pool with or present as the FrontierSWE lane score (original: 1/17 tasks, replay 0.8628; the seed809 adapter's Tinker sampler route has since been purged, its weights are preserved on the Hugging Face branch).
- **E3.** New base-model arm; never pool with or present as the original SDAB/seed809 lane score.
- **E4.** Small $n=6$ of 100; one nonzero task, so no interval is reported (the result file's Wilson95 on summed fractional reward is not valid for a $[0,1]$ mean).
- **E5.** $n=10$, not the requested 20: the E4+E5 6M Tinker-token cap was exhausted (E4 1.56M, abandoned APEX attempt 2.33M, tau2 2.05M).
- **E6.** Tasks drawn by `random.Random(20260926).sample` from all 128 sorted miniwob env ids; episode seeds 0-2.
- **E7.** New base-model arm; never pool with or present as the original lane score.
- **E8.** New base-model arm; not comparable to or poolable with the seed809-adapter 1967/1967 run (22.88%), whose Tinker sampler route is purged though its weights are preserved.
- **E9.** Competition pool: 6 low-split competitions with small non-image-heavy data (detecting-insults, jigsaw-toxic, leaf-classification, nomad2018, random-acts-of-pizza, spooky); 5 drawn by `random.Random(20260926).sample`. $n=5$ so the CI is very wide.
- **E10.** New base-model arm on AgentHarm itself; the seed809-adapter lane figure (AgentDojo benign utility $88/97 = 90.72\%$; Tinker sampler route purged, weights preserved) is a different benchmark and not comparable.
- **E11.** New base-model arm; not comparable to the retained adapter receipt $129/312 = 41.35\%$ (that run used thinking-default template, temperature 0.2, all 312).
- **E12.** Public tasks, not held-out. Label mandatory. New base-model arm; never pool with the original lane.
- **E13.** New base-model arm; not comparable to or pooled with the original seed809-LoRA E13 plan (21-episode BabyAI admission set).
- **E14.** New base-model arm; the prior $2271/4428 = 51.29\%$ figure (51.31% on the official scorer's 4426 judged rows) is not comparable (different actor, full split).

Appendix E

Corrections to Internal Source Documents

The numbers in this thesis are drawn from the project’s internal source documents: the unpublished P1–P8 reports and their section files, audit summaries and result files. Some of those documents contain errors — a wrong sign, a test applied to the wrong unit of analysis, an interval that does not describe its own estimate, or a label that does not match the rows it covers. The chapters report only the corrected value and mark it “(corrected; see Table E.1)”. This appendix records, for each correction, what the source states, what the thesis uses instead, and the basis for the change, so that a reader who opens the source can reconcile it with the thesis. Caveats about the evidence itself, as opposed to errors in a source document, stay in the chapters. Paths are relative to the repository root; “sections/” is `platform_hybrid/paper/sections/` and “results/” is `platform_hybrid/experiments/results/`.

Corrections applied to internal source documents.

Table E.1: Corrections applied to internal source documents. Each row’s corrected value is the one used in the chapter named in the first column.

Location in thesis	Source document	What the source states	Corrected value / reading	Basis
§1.2 (variant audit)	<code>zvf-program/audit/STATISTICAL_REANALYSIS.md</code>	Percentile-bootstrap 95% intervals over the eight seeds, e.g. Dr. GRPO −0.00200, [−0.00950, +0.00725]	Paired- t 95% intervals, e.g. Dr. GRPO −0.00200, [−0.01272, +0.00872], $p = 0.673$; DAPO +0.00100, [−0.00632, +0.00832] (also in Table 1.1)	A percentile bootstrap undercovers at $n = 8$; recomputed from <code>zvf-program/audit/results/full/*.json</code>
§4.1, §6.2.3 (same-stack PPO vs GRPO)	<code>sections/frontier_synthesis_scaling.tex</code>	Reports paired $\Delta = -0.002$, $p = 0.374$ without naming the model or task	<code>Qwen/Qwen2.5-0.5B</code> , two-digit addition, 40 steps, five paired seeds, 200 fresh draws per seed	<code>results/samestack_ppo_grpo.json</code> ; <code>platform_hybrid/experiments/modal/modal_samestack_ppo_grpo.py</code>

Location in thesis	Source document	What the source states	Corrected value / reading	Basis
§6.3.2 (60-run ZVF ranges)	sections/p2_results_intro.tex	The 60 runs behind the late-phase and early-phase ZVF ranges and the lag-1 autocorrelation of ≈ 0.94 are “measured”	45 of the 60 are five-seed variance-mitigation projection rows; only the remainder, which includes the 12-run group-size sweep, is measured	The nine-row panel comes from the dry-run file <code>experiments/results/variance_mitigation.tsv</code> (results/zvf_by_library.tsv)
§6.3.3 (ZVF against the Bernoulli null)	sections/group_size.tex	Closed form evaluated at each arm’s held-out accuracy (0.964, 0.954, 0.923 and 0.704 at $G = 2, 4, 8, 16$), giving negative residuals	Per-step-average null 0.828, 0.709, 0.597, 0.522 against measured 0.838, 0.764, 0.691, 0.631; residuals +0.010, +0.055, +0.094, +0.109, positive at every G	ZVF is a training-batch quantity and held-out accuracy is not; recomputed from results/groupsize_zvf_sweep.json; the per-step-average baseline is the one named in sections/appendix_zvf_formalization.tex
§6.3.4 (naive ZVF–accuracy correlation)	sections/zvf.tex	$r = 0.22$ over $n = 23$ pooled rows, bootstrap 95% CI $[-0.29, 0.94]$	$n = 17$ per-experiment rows; Fisher-z 95% CI $[-0.29, +0.63]$	The residual table on disk, results/zvf_iter26_residual.tsv, holds 17 rows; a bootstrap upper bound near +0.95 at this n indicates an unstable bootstrap
§6.3.4 (residualised ZVF)	sections/zvf.tex	After regressing out mean reward, residualised ZVF remains positively associated with last-10 accuracy, $r = +0.80$, 95% CI $[+0.54, +1.00]$	That slice is the $n = 9$ variance-mitigation rows, a simulation projection; the measured pooled residual is $r = +0.205$, Fisher 95% CI $[-0.31, +0.62]$, $p = 0.43$, $n = 17$	results/zvf_iter26_residual.tsv; projection status in §6.3.2
§6.4.1 (group-size TOST)	sections/group_size_iter27.tex	TOST p-values against $G = 2$ of $p < 10^{-9}$ at $G = 4$ and $G = 8$ and $p < 10^{-4}$ at $G = 16$, with accompanying intervals	Seed-level paired TOST (margin ± 0.02 , $n = 3$): $p = 0.012$ ($G = 4$), 0.043 ($G = 8$), 0.056 ($G = 16$); equivalence not established at $G = 16$; the source intervals are withdrawn	p-values that small cannot come from three paired seeds, so the source does not use the seed as the unit; recomputed per seed
§6.4.1 (retention asymptote)	sections/group_size_iter27.tex	Asymptote interval $[0.712, 0.745]$ for $R(T) = 0.729 + 0.247 e^{-T/5.76M}$	No confidence interval; the fit is illustrative only	A three-parameter curve through four reconstructed points leaves one residual degree of freedom
§6.4.2 (algorithm axis of the factorization audit)	results/berkeley/unpacking/dpo_ppo_factorization.json	“DECISIVE” equivalence of GRPO and PPO	Equivalence not established; paired t $p = 0.374$ is the primary test, exact sign-flip $p = 0.625$	A failure to reject is not equivalence; paired TOST at ± 0.005 gives $p = 0.104$ (§6.2.3)

Location in thesis	Source document	What the source states	Corrected value / reading	Basis
§6.4.2 (H1, gradient-utilisation ratio)	results/berkeley/adding_errorBars_summary.json; sections/p3_conclusion.tex	Graded DECISIVE, with interval [4.16, 4.82]	Held at hypothesis-generator status; point estimate 5.03, no interval reported	The interval excludes its own point estimate; it is taken over $n = 4$ budgets around their mean of 4.456, and the audit’s own note says “ $n=4$ budgets ... cannot support CI on the ratio”
§6.4.2 (H2, retention-versus-token slope)	results/berkeley/adding_errorBars_summary.json; sections/p3_conclusion.tex	Graded DECISIVE, with interval $[-0.237, -0.038]$	Held at hypothesis-generator status	The slope is fitted to the four reconstructed budgets of the token-budget grid, not to trained arms
§6.5.3 (sliding-window trap rate)	P4 source, sections/length_bias.tex	The 6/16 (37.5%) sliding-window rate is an upper bound on trap prevalence in the controlled data	A point estimate; Wilson 95% CIs [5.7%, 51%] (2/10), [30%, 90%] (4/6), [18.5%, 61%] (6/16)	A point estimate is not a bound
§6.5.3 (iteration-20 mechanism regression)	sections/length_bias.tex	Arithmetic OLS slope “reduced by 0.43” with 95% CI $[-0.58, -0.28]$, paired $p = 0.036$; Theil–Sen “reduced by 1.11” with CI $[-1.59, -0.63]$, $p = 0.047$	Dr. GRPO minus GRPO, paired by seed, $n = 5$: OLS +0.43 [+0.18, +0.68], paired t $p = 0.009$; Theil–Sen +1.11 [+0.33, +1.88], $p = 0.017$; permutation $p = 0.0625$ for both	With mean OLS slopes of -3.65 (GRPO) and -3.22 (Dr. GRPO), Dr. GRPO minus GRPO is positive, so the printed intervals have the wrong sign, and the source’s CIs and p-values do not come from the same test; recomputed from results/length_bias_mechanism_per_run.tsv
§6.5.4 (GSM8K-CoT pre-to-post gains)	sections/p4_results_intro.tex	McNemar exact $p = 1.1 \times 10^{-4}$ (GRPO) and 7.6×10^{-4} (Dr. GRPO), pooled over seeds	Seed-level one-sample t on the three per-seed deltas: $p = 0.0051$ (GRPO), $p = 0.0099$ (Dr. GRPO)	The pooled test treats the 600 seed-item pairs as independent, but the same 200 items recur across seeds; recomputed from results/drgro_gsm8k_cot.json
§6.5.4 (iteration-136 late-efficiency contrast)	results/berkeley/adding_errorBars_summary.json; sections/p4_conclusion.tex	Graded DECISIVE, with “one-sided parametric $p = 0.031$ ”	Suggestive: $p = 0.031$ is an exact one-sided Wilcoxon signed-rank p at its $n = 5$ floor (1/32); paired $t(4) = 5.96$, two-sided $p = 0.004$; two-sided sign-flip $p = 0.0625$; Holm across the eight-hypothesis family gives 0.25	Recomputed from length_bias_iter136_step_coupling.tsv
§7.1 (variance decomposition, group-size axis)	sections/p5_evidence.tex	$\eta^2 = 1.0000$ for the group-size axis, and group-size-to-algorithm ratios of $22\times$ (ZVF) and $133\times$ (reward)	No group-size η^2 and no ratio reported	Computed on groupsize_zvf_sweep.tsv, which holds one row per G level (4 cells, $df = 0$ within), so $\eta^2 = 1$ by construction

Location in thesis	Source document	What the source states	Corrected value / reading	Basis
§7.1 (98-cell decomposition)	sections/p5_evidence.tex	Stack-to-seed variance ratios of $503\times$ to $96,128\times$	Not quoted as effect sizes	The seed axis rounds to 0.000, so the ratios divide by a near-zero quantity
§7.3 (hindsight-optimal shrink rule)	sections/p7_controller.tex	The 0.797 cost ratio of the per-prompt hindsight optimum is a ceiling on rollout spend	The best cost ratio under a no-shrinking-of-saturated-prompts constraint, not a lower bound on cost for any adaptive rule	Dualformer-Auto's observed 0.66 to 0.68 on the four-bin rule is cheaper

Appendix F

Applied Side Study: Credit-Card Fraud Detection (P8)

Scope statement. This appendix reports a side study. It uses synthetic data and sits outside the RL contribution of this thesis. The dataset is generated with scikit-learn’s `make_classification`; the language-model arm is trained by supervised fine-tuning, not by reinforcement learning; and no result in this appendix is used to support any GRPO, PPO, ZVF, group-size or campaign claim elsewhere in the report. The study is retained, in condensed form, as the P8 slot of the internal report series and as one worked example of measuring a claimed capability rather than accepting its label.

The study compares a gradient-boosted tree with a fine-tuned language model on credit-card fraud detection. Its conclusion is that the tree keeps the real-time scoring seat, and that the language model’s plausible roles lie in tasks the tree cannot perform at all.

F.1 Dataset, splits, and the two arms

The dataset is synthetic: `make_classification` (seed 42), 50,000 transactions, 20 anonymised numeric features V1–V20 (10 informative, 2 redundant), two clusters per class, class separation 0.8 and `flip_y=0.01`, with a realised fraud rate of 1.44%. Four per-row aggregates (mean, standard deviation, maximum, minimum) are appended, giving 24 features. The data reproduce class imbalance and anonymised feature shape, and none of the temporal drift, verification latency or adversarial adaptation of real card-fraud data.

Table F.1: The two arms of P8.

Arm	Configuration	Evaluation split
XGBoost (scorer)	200 estimators, depth 6, lr 0.05, subsample/colsample 0.8, scale_pos_weight=7; no search	80/20 stratified split, 10,000 test rows, 144 positives
Qwen3.5-4B SFT	TabLLM-style row serialisation; answer-token cross-entropy, no reference policy, no KL; 63 steps at batch 32	500-row positive-enriched held-out subset at 20% fraud, disjoint from training

Sources: `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_setup.tex`;
`platform_hybrid/experiments/results/quick_20260704/qp8-fraud-sft_manifest.json`;
`platform_hybrid/experiments/results/quick_20260704/qp8-fraud-sft.tsv`.

F.2 The head-to-head result

Table F.2: Head-to-head result on the synthetic card-fraud set, by arm and evaluation split.

Arm	Eval split	n	Accuracy	ROC-AUC
XGBoost (scorer)	full test	10,000	n/a	0.7955
Qwen3.5-4B SFT	positive-enriched test	500	0.7920	0.4827

Source: `platform_hybrid/experiments/results/quick_20260704/qp8_fraud.tsv`.

The two AUCs sit on different held-out splits, so this is not a like-for-like ranking. The language model’s AUC of 0.4827 is chance-level ranking: its Hanley–McNeil 95% CI, with 100 positives and 400 negatives, is [0.42, 0.55] and includes 0.5. Its accuracy of 0.792 is slightly *below* the 0.800 that a constant “legitimate” answer would score at 20% prevalence, so it is not evidence of discrimination. The tree’s AUC interval is [0.75, 0.84], with 144 positives among 10,000 test rows. The tree’s precision 0.723 (34/47, Wilson 95% CI [0.58, 0.83]), recall 0.236 (34/144, [0.17, 0.31]) and F1 0.356 at the

default threshold come from the released script’s own run (AUC 0.7942, 0.49 s training, about 6 ms inference per 10,000 rows), which uses a slightly different configuration from the 0.7955 artefact. An older record of 21 June 2026 (XGBoost AUC 0.975, language model 0.948) could not be reconstructed and is not used.

F.3 Why the scoring seat stays with the tree

Four operational arguments hold even at AUC parity:

- **Latency.** Card authorisation needs a score within tens of milliseconds; the tree takes about 6 ms per 10,000 rows, against roughly 300 ms for a language-model pass.
- **Cost.** The released cost table puts the synchronous language-model path at about 35 times the per-transaction cost of the tree path.
- **Calibration.** Downstream threshold and expected-loss rules consume the score as a probability; verbalised language-model confidences are poorly calibrated.
- **Prompt injection.** A model that reads attacker-controlled text fields in a synchronous decision loop opens an injection channel that a tree scoring numbers does not have.

F.4 Supporting measurements

Table F.3: Supporting P8 measurements. Bracketed ranges are paired-bootstrap 95% intervals as reported by the source. Source files are under `platform_hybrid/experiments/results/` and `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/`.

Check	Result	Source
Aggregate-only tree (stand-in for a language-model “sensor”) vs 24-feature tree	Worse on every axis: AUC -0.0245 $[0.0174,$ $0.0320]$, accuracy -0.0135 $[0.0109, 0.0160]$, Brier $+0.0109$ $[0.0099,$ $0.0120]$	p5p8/p8_headline_cis.tsv

Check	Result	Source
Adding the four aggregates to the 20 raw features	AUC +0.0002 [−0.0002, 0.0007], within noise	p5p8/p8_headline_cis.tsv
Leakage caveat	Tables reporting AUC 0.998–0.999 fit on fraud_data.csv and score a subset of it; they are within-protocol relative comparisons only. The leakage-free split gives 0.7955	p8_evidence.tex
Cost per fraud caught (\$0.50/alert, \$100 missed-fraud loss)	Raw features \$2.68; sensor-augmented \$3.28; sensor-only \$14.92; the sensor never pays for itself in the 25-cell (σ , L) grid	p8_evidence.tex
Sensor-noise break-even	\$5.29 (oracle), \$26.44 ($\sigma \leq 0.02$), \$36.40 ($\sigma = 0.05$)	p8_noisy_sensor.tex

F.5 Proposed roles for the language model

The study argues, without measuring, that the language model’s useful roles are tasks the tree cannot enter: document and image fraud through a vision-language model, drafting compliance narratives (Suspicious Activity Reports under 31 C.F.R. § 1020.320; adverse-action reasons under 12 C.F.R. § 1002.9), cold-start triage of new fraud typologies before labels exist, and post-score investigation of the alert queue. The proposed architecture keeps XGBoost as the scorer and uses the language model as a sensor (typed features, no authority) and a scribe (narratives conditioned on the tree’s own attributions, with a human signature). The “85 times cheaper than an analyst” triage figure is an internal estimate from June-2026 token prices and assumed investigation times, and has not been validated.

F.6 Limitations

The dataset is synthetic and single-generator, so no number here is a production expectation, and there is no cross-institution or public-benchmark replication. The challenger is one model family, one serialisation and one fine-tuning budget. Of the proposed roles, only the scorer comparison carries a measurement of the study’s own; the others rest on external literature and regulatory text, and nothing in the compliance discussion is legal advice.

Appendix G

Evidence Map

Every factual claim in the body of this report was written against a named source file in the project repository. To keep the running text readable those source notes are collected here, section by section, in the order the claims appear. Paths are relative to the repository root. Table sources are printed under their tables and are not repeated here.

G.1 Chapter 1: Introduction

1.1 Background: critic-free RL post-training and the group-relative estimator

- `platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex`
- `platform_hybrid/paper/sections/group_size.tex`
- `platform_hybrid/paper/sections/zvf_iter122.tex`
- `platform_hybrid/paper/main_eai_body.tex`

1.2 A Motivating Example

- `platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex`
- `platform_hybrid/paper/main_eai_body.tex`
- `platform_hybrid/paper/sections/_shared_methods.tex`
- `platform_hybrid/experiments/results/samestack_ppo_grpo.json`
- `platform_hybrid/experiments/modal/modal_samestack_ppo_grpo.py`
- `autoresearch/deli-neurips-tmlr-260802/audits/statistical_refutation.md`

1.3 Problem Statement

- platform_hybrid/paper/main_eai_body.tex
- platform_hybrid/paper/sections/_shared_methods.tex
- platform_hybrid/experiments/results/samestack_ppo_grpo.json
- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex

1.6 Contribution of the Candidate

- platform_hybrid/sem 4 work/EXPERIMENTS.md
- platform_hybrid/sem 4 work/README.md
- platform_hybrid/sem 4 work/PROVENANCE.md
- outputs/E1_E14_FINAL_RESULTS_2026-09-19.md
- outputs/e1_e14_small_scale_2026-09-26/E4/BRIDGE_STOP_TOKEN_FINDING.md
- autoresearch/deli-neurips-tmlr-260802/audits/statistical_refutation.md

G.2 Chapter 2: Literature Survey

- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex

2.1 From RLHF and PPO to Group-Relative Methods

- capstone-literature-survey/section_2_2_rlhf.tex
- capstone-literature-survey/section_2_3_ppo.tex
- capstone-literature-survey/section_2_5_grpo.tex
- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex
- capstone-literature-survey/section_2_6_deepseek_r1.tex

2.2 Direct Preference Optimisation as a Contrast

- capstone-literature-survey/section_2_4_dpo.tex
- platform_hybrid/paper/sections/related_work_v2.tex
- platform_hybrid/paper/sections/p7_intro.tex

2.3 Fixing GRPO’s Biases and Collapse

- platform_hybrid/paper/sections/p4_intro.tex
- platform_hybrid/paper/sections/p6_intro.tex
- platform_hybrid/paper/sections/related_work_v2.tex
- platform_hybrid/paper/sections/p7_intro.tex
- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex
- platform_hybrid/paper/sections/variance_mitigation_comparison.tex
- platform_hybrid/experiments/results/drgrp0_vs_grp0.json
- platform_hybrid/experiments/results/drgrp0_gsm8k_cot.json

2.4 Verifiable Rewards, Frameworks and Reporting Practice

- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex
- platform_hybrid/paper/sections/related_work_v2.tex
- capstone-literature-survey/section_2_8_mathematical_reasoning_benchmarks.tex
- platform_hybrid/paper/sections/p5_related.tex
- platform_hybrid/experiments/results/p5p8/p5_iter109_minreport_coupling.tsv
- platform_hybrid/experiments/results/p5p8/p5_iter109_crossref_verify.tsv
- platform_hybrid/experiments/results/p5p8/p5_iter109_bib_audit.tsv
- platform_hybrid/paper/sections/p6_intro.tex
- platform_hybrid/paper/sections/p6_related_resources.tex

2.5 Scaling and Layer-Adaptation Context

- platform_hybrid/paper/sections/related_work_v2.tex
- platform_hybrid/paper/sections/p1_intro.tex
- platform_hybrid/paper/sections/p2_intro.tex
- platform_hybrid/paper/sections/p3_intro.tex
- platform_hybrid/paper/sections/p4_intro.tex
- platform_hybrid/paper/sections/frontier_synthesis_scaling.tex
- platform_hybrid/paper/sections/p1_conclusion.tex
- capstone-literature-survey/section2_7_peft.tex
- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex

2.6 Process-, Tree- and Stability-Aware Variants

- platform_hybrid/paper/sections/related_work_v2.tex
- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex
- platform_hybrid/paper/sections/p7_intro.tex

2.7 Positioning the Portfolio

- platform_hybrid/paper/sections/p6_related_resources.tex
- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex
- outputs/E1_E14_FINAL_RESULTS_2026-09-19.md
- outputs/e1_e14_small_scale_2026-09-26/E4/BRIDGE_STOP_TOKEN_FINISHING.md
- platform_hybrid/paper/sections/related_work_v2.tex

G.3 Chapter 3: System Requirements Specification

3.1 Overview and Actors

- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex
- zvf-program/flagship/e1_e14_completion_gate.py
- outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md

3.4 Test-Case Specification

- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex
- platform_local/run_all_audits.py
- utils/audit_utils.py
- platform_local/claim_strength_audit.py
- platform_local/paper_sync_audit.py
- platform_local/export_guard_audit.py
- platform_local/scientific_audit.py
- zvf-program/flagship/e1_e14_completion_gate.py
- platform_hybrid/paper/paper_P6_registry.tex

- platform_hybrid/registry/query.py

3.7 Software Requirements

- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex
- platform_local/run_all_audits.py
- utils/audit_utils.py
- zvf-program/flagship/e1_e14_completion_gate.py
- platform_hybrid/registry/query.py
- platform_local/scientific_audit.py
- outputs/PES_Phase2_Third_Review_2026-09-24/thesis/assemble_the_sis.py
- outputs/E1_E14_FINAL_RESULTS_2026-09-19.md

3.8 Constraints and Assumptions

- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex
- platform_hybrid/paper/paper_P7_zvf_controller.tex
- platform_hybrid/paper/sections/_shared_methods.tex
- platform_hybrid/experiments/results/framework_comparison.json
- platform_hybrid/paper/paper_P1_scaling.tex
- outputs/E1_E14_FINAL_RESULTS_2026-09-19.md
- platform_hybrid/paper/paper_P6_registry.tex

G.4 Chapter 4: Proposed Methodology

4.1 The Design Commitment to Attribution

- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex
- platform_hybrid/sem 4 work/submissions/mtech-final-review/build_review_pptx.py
- slide 7
- platform_hybrid/experiments/results/samestack_ppo_grpo.json
- platform_hybrid/paper/sections/_shared_methods.tex
- its status is recorded in §6.2.3

- platform_hybrid/paper/neurips_2026_variants/paper_P8_workshop.tex
- §6.1

4.2 Per-Step Telemetry

- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex
- platform_hybrid/paper/sections/p2_abstract.tex

4.3 Formalising the Zero-Variance Fraction

- platform_hybrid/paper/sections/appendix_zvf_formalization.tex
- platform_hybrid/paper/neurips_2026_variants/paper_P8_workshop.tex
- platform_hybrid/paper/sections/zvf_pipeline_spec.tex
- platform_hybrid/paper/sections/p2_abstract.tex
- results in Chapter 6 §6.3.3
- platform_hybrid/paper/sections/p7_abstract.tex
- results in Chapter 6 §6.3.4
- platform_hybrid/paper/sections/zvf.tex
- §6.3.2

4.4 The Eight-Study Portfolio

- platform_hybrid/sem 4 work/README.md
- platform_hybrid/paper/sections/p1_abstract.tex
- platform_hybrid/paper/sections/p2_abstract.tex
- platform_hybrid/paper/sections/p3_abstract.tex
- platform_hybrid/paper/sections/p4_abstract.tex
- platform_hybrid/paper/sections/p5_abstract.tex
- platform_hybrid/paper/sections/p6_abstract.tex
- platform_hybrid/paper/sections/p7_abstract.tex
- platform_hybrid/sem 4 work/submissions/mtech-final-review/README.md
- platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_abstract.tex
- platform_hybrid/paper/PAPERS_README.md

4.5 The Design of the E1–E14 Held-Out Evaluation Campaign

- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md`
- `checkpoints/grpo/pavlov_portfolio_api_swegym_qwen36_20260809_seed809.json`
- `outputs/public_portfolio_2026-09-05/agentdojo_native_receipt.json`
- `outputs/modal_e1_e14/2026-08-16/e11_full_receipt.json`
- `zvf-program/flagship/`
- `outputs/public_portfolio_2026-09-05/decontamination/README.md`
- `zvf-program/flagship/PAVLOV_EXPERIMENT_PROTOCOL_2026-08-09.md`
- `outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md`
- `outputs/UNBLOCK_CARRYOUT_2026-09-21.md`

G.5 Chapter 5: Implementation Details

5.1 Two Rosters, One Dispatch Surface

- `platform_hybrid/paper/sections/_shared_methods.tex`
- `platform_tinker/atropos/`
- `platform_tinker/atropos/README.md`

5.2 The Adapter Architecture: What versus Where

- `platform_local/unified/backends/base.py`
- `platform_local/unified/matrix.json`
- `platform_local/unified/launcher.py`
- `platform_hybrid/experiments/framework_config_dumps/README.md`

5.3 The Tinker GRPO Loop

- `platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex`
- `platform_tinker/tinkerrl/grpo.py`
- `make_grpo_loss_fn`
- `platform_tinker/tinkerrl/grpo_cli.py`

5.4 The Parallel Runner and a Real Logging Defect

- `platform_hybrid/experiments/results/p3_groupsize/FINDINGS.md`

5.5 The Colab White-Box Path (Per-Layer Gradient Profile)

- `platform_hybrid/experiments/results/p1_layerfreeze/FINDINGS.md`
- `platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex`
- `platform_hybrid/experiments/results/p1_layerfreeze/scaled_4seed_result.json`
- `platform_hybrid/experiments/results/p1_layerfreeze/FREEZE_FINDINGS.md`

5.6 Software Modules

- `platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex`
- `platform_local/unified/`

5.7 The E1–E14 Evaluation Harness

- `zvf-program/flagship/e1_e14_completion_gate.py`
- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md`

5.8 The Bridge and Sampler Path

- `zvf-program/flagship/modal_tinker_openai_bridge.py`
- `zvf-program/e1_wave10/README.md`

5.9 Recovery Re-implementations under `zvf-program/`

- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md`
- `zvf-program/e2_core/launch_gate.py`

G.6 Chapter 6: Results I: Core GRPO Studies (P1–P4)

6.1 Scope, Sources and Evidence Boundary

- `platform_hybrid/paper/sections/p1_abstract.tex`
- `platform_hybrid/paper/sections/zvf.tex`
- `platform_hybrid/paper/sections/p3_intro.tex`
- `platform_hybrid/paper/sections/length_bias.tex`

- `platform_hybrid/paper/sections/_shared_methods.tex`

6.2 P1 — Cross-Library and Cross-Scale GRPO Behaviour

- `platform_hybrid/paper/sections/p1_abstract.tex`

6.2.1 The anchor pool and the fitted saturation forms

- `platform_hybrid/paper/sections/p1_abstract.tex`
- `platform_hybrid/paper/sections/scaling_laws.tex`

6.2.2 The cross-scale test and its nulls

- `platform_hybrid/paper/sections/scaling_laws.tex`

6.2.3 Where the geometric template fails

- `platform_hybrid/paper/sections/p1_abstract.tex`
- `platform_hybrid/paper/sections/_shared_methods.tex`
- Chapter 1 §1.2

6.2.4 A withdrawn architectural contrast

- `platform_hybrid/paper/sections/scaling_laws.tex`
- provenance in `autoresearch/deli-neurips-tmlr-260802/state/nemotron_architecture_resolution.md`
- Table `tab:scaling-iter17-kappa`

6.3.1 Definition and measurement

- `platform_hybrid/paper/sections/zvf.tex`
- `platform_hybrid/paper/sections/p2_abstract.tex`
- `platform_hybrid/paper/sections/zvf_pipeline_spec.tex`

6.3.2 Measured magnitudes across libraries

- `platform_hybrid/paper/sections/zvf.tex`
- `platform_hybrid/experiments/results/zvf_by_library.tsv`
- `platform_hybrid/paper/sections/p2_results_intro.tex`

6.3.3 The all-correct wall and the mastery-incapacity aliasing

- `platform_hybrid/paper/sections/zvf.tex`
- `platform_hybrid/paper/sections/p2_conclusion.tex`
- `platform_hybrid/paper/sections/frontier_synthesis_zvf.tex`
- `platform_hybrid/paper/sections/appendix_zvf_formalization.tex`

6.3.4 Prognostic value

- `platform_hybrid/experiments/results/zvf_iter26_residual.tsv`
- `corrected`
- `see Table E.1`
- `platform_hybrid/paper/sections/zvf.tex`
- `platform_hybrid/paper/neurips_2026_variants/paper_P8_workshop.tex`
- `platform_hybrid/paper/sections/appendix_zvf_formalization.tex`
- `platform_hybrid/paper/sections/p2_results_intro.tex`
- `Appendix A`
- `claim P2-C2`
- `platform_modal/scripts/zvf_diagnostic_iter130.py`
- `platform_hybrid/paper/sections/p2_conclusion.tex`
- `platform_hybrid/paper/sections/frontier_synthesis_zvf.tex`

6.4.1 The measured sweep

- `platform_hybrid/paper/sections/group_size.tex`
- `platform_hybrid/experiments/results/group_size_effect_dpo_checks.tsv`
- `platform_hybrid/paper/sections/group_size_iter27.tex`
- `platform_hybrid/paper/sections/p3_abstract.tex`

6.4.2 The same-stack four-axis audit and the reconstructed budgets

- `platform_hybrid/experiments/results/berkeley/unpacking_dpo_ppo_factorization.json`
- `as summarised in platform_hybrid/paper/sections/p3_conclusion.tex`
- `platform_hybrid/experiments/results/berkeley/adding_errorBars_summary.json`
- `platform_hybrid/experiments/results/group_size_effect.tsv`

- platform_hybrid/paper/sections/group_size.tex
- platform_hybrid/paper/sections/group_size_iter23.tex
- platform_hybrid/paper/sections/p3_conclusion.tex
- platform_hybrid/paper/sections/p3_abstract.tex

6.4.3 The curriculum and matched-token null results

- platform_hybrid/experiments/results/curriculum_opening/FINDING S.md
- as summarised in platform_hybrid/paper/sections/robustness_smallscale_nulls.tex
- platform_hybrid/paper/sections/robustness_smallscale_nulls.tex
- platform_hybrid/experiments/results/p1_layerfreeze/scaled_4seed_result.json

6.5.1 The controlled null

- platform_hybrid/experiments/results/drgrpo_vs_grpo.json
- platform_hybrid/paper/sections/length_bias.tex
- platform_hybrid/experiments/results/drgrpo_gsm8k_cot_full.json
- platform_hybrid/paper/sections/p4_abstract.tex
- platform_hybrid/paper/sections/p4_results_intro.tex

6.5.2 The roster-level verbosity-trap audit

- platform_hybrid/paper/archive/absorbed/U01_main_compendium/main.tex
- the same rates appear in platform_tinker/reports/combined/combined_capstone.tex
- where the trap rates are given as GRPO 36% and PPO 71%
- platform_tinker/reports/combined/combined_capstone.tex
- platform_hybrid/paper/sections/length_bias_iter48.tex

6.5.3 Elevated analyses

- platform_hybrid/paper/sections/length_bias.tex

6.5.4 Held-out generalisation and the scale extension

- platform_hybrid/paper/archive/absorbed/U01_main_compendium/main.tex

- platform_hybrid/paper/sections/p4_results_intro.tex
- platform_hybrid/paper/sections/p4_intro.tex
- platform_hybrid/paper/sections/p4_abstract.tex
- platform_hybrid/paper/sections/length_bias.tex
- platform_hybrid/experiments/results/berkeley/adding_errorBars_summary.json
- platform_hybrid/paper/sections/p4_conclusion.tex

G.7 Chapter 7: Results II: Reporting, Registry and Controller (P5–P7)

7.1 A Minimum Reporting Standard for Stack-Conditioned Results (P5)

- platform_hybrid/paper/sections/p5_intro.tex
- platform_hybrid/paper/sections/p5_evidence.tex
- Table tab:p5-h2h
- platform_hybrid/paper/sections/p5_stack.tex
- §Item 3
- platform_hybrid/paper/sections/p5_abstract.tex
- Exhibits 5 and 7
- platform_hybrid/experiments/results/groupsize_zvf_sweep.tsv
- Table tab:p5-minreport
- Table tab:p5-ushape
- Table tab:p5-gsize
- §Item 7
- §Item 8
- Exhibit 8
- Exhibits 6 and 18
- platform_hybrid/paper/sections/p6_registry_health.tex
- Table tab:p6-minreport-subfield
- reported for the same corpus
- Table tab:p5-discriminative-entropy
- Table tab:p5-field-suff
- Table tab:p5-manifest-self-suff
- §Placebo-Replacement Bound
- §Live-Corpus Coupling

7.2 The GRPO-Registry (P6)

- `platform_hybrid/paper/sections/p6_abstract.tex`
- `platform_hybrid/paper/paper_P6_registry.tex`
- `platform_hybrid/paper/sections/p6_schema.tex`
- `platform_hybrid/paper/sections/p6_population.tex`
- `platform_hybrid/paper/sections/p6_registry_health.tex`
- `platform_hybrid/experiments/results/p5p8/registry_stress_per_run.tsv` and `registry_stress_summary.json`
- §Stress Test
- `platform_hybrid/paper/sections/p6_measured_evidence.tex`
- Table `tab:p6-measured-deltas`
- §Contrastive Yield and Anti-Herding
- `platform_hybrid/paper/sections/p6_query_auditor.tex`

7.3 From Diagnostic to Controller (P7)

- `platform_hybrid/paper/sections/p7_abstract.tex`
- `platform_hybrid/paper/sections/p7_theory.tex`
- Tables `tab:p7-modelG` and `tab:p7-gsweep`
- `platform_hybrid/paper/sections/p7_controller.tex`
- Table `tab:p7-e3`
- Tables `tab:p7-cf-sweep` and `tab:p7-cf-granularity`
- §Bayesian Refinement
- §Posterior-Predictive Contrast Restoration and §Statistically Validated Pareto Frontier
- §Exact Finite-Pool Correction
- `platform_hybrid/paper/sections/p7_design_rules.tex`

G.8 Chapter 8: The E1–E14 Held-Out Evaluation Campaign

8.1 The evaluated actor and the one-actor design

- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md`
- the same identity pair is recorded independently in the E10 native receipt and in the E11 full receipt
- which also preserves the training sampler path `tinker://cf0ad8c1-1f1b-5ff3-8bd7-2a0bf232657b:train:0/sampler_weights/seed809_final:sources outputs/public_portfolio_2026-09-05/agentdojo_native_rec`

- eipt.json and outputs/modal_e1_e14/2026-08-16/e11_full_receipt.json
- outputs/TINKER_SAMPLER_STATUS_2026-09-20.json
- outputs/e1_e14_small_scale_2026-09-26/ADAPTER_AVAILABILITY_CHECK_2026-09-26.json

8.2 Suites with strictly complete scopes

- outputs/verification/LEDGER_CODE_CHECK_2026-09-19.json
- check_strict_completed_suites_are_E8_E10_E11
- outputs/public_portfolio_2026-09-05/labbench_final_diagnostics.json
- outputs/public_portfolio_2026-09-05/agentdojo_native_receipt.json
- outputs/modal_e1_e14/2026-08-16/e11_full_receipt.json
- outputs/e11_verilog_eval/e11_verilog_eval_rerun_receipt.json
- outputs/e11_verilog_eval/lane_status_2026-08-09.md
- outputs/PES_Phase2_Review_2026-09-12/finish/e14_terminal_note_2026-09-19.json

8.3 Original-contract results

- outputs/modal_e1_e14/2026-08-16/e1_swe_bench_pro_full/seed1818/receipt.json
- outputs/e1_e14_results_2026-09-05/E1_E14_Results.md
- outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md
- outputs/E1_E14_FINAL_RESULTS_2026-09-19.md
- outputs/verification/LEDGER_CODE_CHECK_2026-09-19.json

8.4 Replacement-scope lanes completed on 27 September 2026

- outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md
- outputs/UNBLOCK_CARRYOUT_2026-09-21.md

8.5 The E4 rerun as a diagnostic

- outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md

- which cites the receipt at `outputs/e4_banker_toolbench/official_repo_f6db552/jobs/btb-banking-tasks-tinker-bridge/result.json`
- `outputs/e1_e14_small_scale_2026-09-26/E4/BRIDGE_STOP_TOKEN_FINAL.md`

8.6 Externally blocked lanes and terminal states

- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md`
- `outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md`

8.7 Small-scale reruns of all fourteen lanes

- `outputs/e1_e14_small_scale_2026-09-26/PROTOCOL.md`
- `outputs/e1_e14_small_scale_2026-09-26/TRAINED_ACTOR_ENDPOINT.md`

G.9 Chapter 9: Cross-Study Synthesis, Threats to Validity, Conclusions and Future Work

9.1 Cross-Study Synthesis

- `platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex`
- `platform_hybrid/paper/sections/p4_conclusion.tex`
- `platform_hybrid/experiments/results/berkeley/adding_errorBars_summary.json`
- `platform_hybrid/paper/sections/p7_abstract.tex`
- `platform_hybrid/paper/sections/p5_abstract.tex`
- `platform_hybrid/paper/sections/p6_abstract.tex`
- `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_abstract.tex`
- `outputs/verification/LEDGER_CODE_CHECK_2026-09-19.json`
- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md`
- `outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md`

9.2 Rigour and Threats to Validity

- `outputs/verification/LEDGER_CODE_CHECK_2026-09-19.json`
- `LIMITATIONS_AND_IMPACT.md §3`
- `LIMITATIONS_AND_IMPACT.md §7.1`
- `Table 1.1`
- `LIMITATIONS_AND_IMPACT.md §6.2`
- `LIMITATIONS_AND_IMPACT.md §6.1`
- `platform_tinker/reports/combined/combined_capstone.tex`
- `outputs/TINKER_SAMPLER_STATUS_2026-09-20.json`
- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md`
- `outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md`
- `LIMITATIONS_AND_IMPACT.md §2.2`
- `§2.3`
- `platform_hybrid/paper/archive/absorbed/U01_main_compendium/main.tex`
- `outputs/modal_e1_e14/2026-08-16/e11_full_receipt.json`
- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md §6`
- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md §9.5`
- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md §8`
- `LIMITATIONS_AND_IMPACT.md §7.2`
- `§7.3`
- `§4`

9.3 Conclusions

- `platform_hybrid/paper/sections/p7_abstract.tex`
- `LIMITATIONS_AND_IMPACT.md §2.2`
- `platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex`
- `platform_hybrid/experiments/results/p1_layerfreeze/scaled_4seed_result.json`

9.4 Future Work

- `outputs/finish_pending_2026-09-27/E6/result.json`
- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md §5`
- `outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md`
- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md §9.7`

- outputs/TINKER_SAMPLER_STATUS_2026-09-20.json
- platform_tinker/reports/esa_phase1/Phase1_Project_Report_ZVF.tex

G.10 Appendix D: Campaign Lane Detail

D.1 Actor identity, inference conditions and scoping notes

- outputs/public_portfolio_2026-09-05/agentdojo_native_receipt.json
- outputs/public_portfolio_2026-09-05/agentdojo_terminal_readiness.md
- outputs/modal_e1_e14/2026-08-16/e11_full_receipt.json
- outputs/e1_e14_results_2026-09-05/E1_E14_Results.md
- outputs/e11_verilog_eval/e11_smoke_receipt.json
- outputs/e11_verilog_eval/e11_trained_step40_receipt.json

D.4 Strictly complete scopes: lane detail

- outputs/public_portfolio_2026-09-05/labbench_final_diagnostics.json
- outputs/public_portfolio_2026-09-05/agentdojo_native_receipt.json
- outputs/public_portfolio_2026-09-05/agentdojo_score_snapshot_fast02_terminal_v3.json
- outputs/public_portfolio_2026-09-05/agentdojo_terminal_readiness.md
- outputs/public_portfolio_2026-09-05/agentdojo_continuation01_result.json
- outputs/public_portfolio_2026-09-05/agentdojo_continuation01_terminal_evidence.json
- outputs/modal_e1_e14/2026-08-16/e11_full_receipt.json
- outputs/e11_verilog_eval/e11_verilog_eval_rerun_receipt.json
- outputs/e11_verilog_eval/lane_status_2026-08-09.md
- outputs/verification/LEDGER_CODE_CHECK_2026-09-19.json
- check E11_component_split_sums_67_plus_62_equals_129
- outputs/PES_Phase2_Review_2026-09-12/finish/e14_terminal_note_2026-09-19.json

D.5 Original-contract results: lane detail

- `outputs/modal_e1_e14/2026-08-16/e1_swe_bench_pro_full/seed1818/receipt.json`
- `outputs/e1_swe_bench_pro/lane_receipt_2026-08-09.json`
- `outputs/e1_swe_bench_pro/preflight_2026-08-09.json`
- `outputs/e1_e14_results_2026-09-05/E1_E14_Results.md`
- `outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md`
- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md`
- `outputs/verification/LEDGER_CODE_CHECK_2026-09-19.json`

D.6 Replacement-scope lane narratives, 27 September 2026

- `outputs/finish_pending_2026-09-27/E1/result.json`
- `outputs/finish_pending_2026-09-27/E2/result.json`
- `outputs/finish_pending_2026-09-27/E5/result.json`
- `outputs/finish_pending_2026-09-27/E6/result.json`
- `outputs/finish_pending_2026-09-27/E9/result.json`
- `outputs/finish_pending_2026-09-27/E13/result.json`

D.7 E4 rerun: closure and full audit

- `outputs/PES_Phase2_Review_2026-09-12/finish/Pending_Experiments.md`
- which cites the receipt at `outputs/e4_banker_toolbench/official_repo_ff6db552/jobs/btb-banking-tasks-tinker-bridge/result.json`
- `outputs/e1_e14_small_scale_2026-09-26/E4/BRIDGE_STOP_TOKEN_FINDING.md`

D.8 Externally blocked lanes: access requests and verification

- `outputs/UNBLOCK_CARRYOUT_2026-09-21.md`
- `outputs/e1_e14_results_2026-09-05/E1_E14_Results.md`
- `outputs/verification/LEDGER_CODE_CHECK_2026-09-19.json`
- `outputs/E1_E14_FINAL_RESULTS_2026-09-19.md`

D.9 Small-scale reruns: detail and per-lane caveats

- `outputs/e1_e14_small_scale_2026-09-26/PROTOCOL.md`

G.11 Appendix F: Applied Side Study: Credit-Card Fraud Detection (P8)

- `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_intro.tex`

F.1 Dataset, splits, and the two arms

- `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_setup.tex`
- `fraud_data.csv`
- `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_imitations.tex`

F.2 The head-to-head result

- `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_abstract.tex`
- `xgboost_results.json`
- `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_setup.tex`
- `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_imitations.tex`

F.3 Why the scoring seat stays with the tree

- `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_scorer.tex`
- `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_evidence.tex`

F.5 Proposed roles for the language model

- `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_taxonomy.tex`
- `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_compliance.tex`
- `platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_architecture.tex`

F.6 Limitations

- platform_hybrid/paper/archive/absorbed/P08_fraud/sections/p8_1imitations.tex